Year 12

RPG game

Conor Desouza

Sir John Deane's Sixth Form College

Year 12

Center number: 6229

Candidate number: 40417

TABLE OF CONTENTS

Analysis: 4

Background: 4

Description of the current system: 5

Analysis of similar systems: 7

Likes and dislikes about similar systems: 10

What I like 10

What I dislike 10

Different algorithms and logic: 11

Terrain generation 11

Cellular automaton 11

Perlin noise 11

Simplex noise 11

Random walk 11

Prim's algorithm 11

Pathfinding 11

Dijkstra's algorithm 12

A* search algorithm 12

Breadth-first search 12

Depth-first search 12

Maximum flow 12

Minimum spanning tree 12

sorting 12

Bubble sort 12

Merge sort 13

Selection sort 13

Insertion sort 13

Bucket sort 13

Quicksort 13

interview: 14

objectives: 19

Modelling: 26

Main room: 26

Entity structure: 26

Ray casting: 26

Boss and Npc room: 27

Inventory and UI: 27
Start and finish screens: 28
character creator and finish screen: 28
Technical solution: 28
Bars_class: 29
Base_attack: 33
Base_enemy: 37
Boss_class: 43
Buttono_class: 54
Character_creator: 55
Door_class: 62
Finish_screen: 63
Goblin_enemy: 67
Instruction_menu: 68
Item_class: 70
Form1: 72
Map_maker: 86
Npc: 92
Player_interface: 95
Player_class: 103
Ray_cast: 118
Sheild_class: 125
Text_class: 128
Utilities: 131
Wall_class: 135
Warlock_enemy: 140
Design: 142
Overview: 142
Prototype: 143
Flowchart: 145
Code: 146
Game loop: 148
class diagram: 149
relationship diagram: 150
class definitions: 151
Graphical user interface: 154
Start screen: 154
Instruction screen: 154

Character creator screen: 154
Create leader board name screen: 155
Leader board screen's 156
Class explanations: 157
Reasons for the reduced use of encapsulation: 157
Window classes: 157
Main parent classes 163
UI classes: 165
Enemy classes: 168
Helper classes: 173
Main classes 177
Player classes: 179
Primary Algorithms: 185
Cellular automata: 185
A* pathfinding: 192
blocker maker in ray casting: 199
Ray casting: 207
spider boss limbs: 210
map creation: 214
saving and LOADING: 218
Quick sort and leaderboard creation: 223
Secondary Algorithms: 227
Euclidean Vector Normalization for Constant Velocity: 227
The "Perpendicular Vector" Positioning Algorithm: 229
Image Processing (Manual Pixel Manipulation): 230
Radial Spawning: 232
Data structures: 233
Arrays: 233
Lists: 234
Dictionaries: 234
Priority queue: 235
Graphs: 235
Text files: 235
Testing: 239
Objective testing: 239
Robustness testing: 247
Evaluation: 248

ANALYSIS:

BACKGROUND:

Throughout my time in coding, I have been deeply interested in complex algorithms and how to optimise them. Hence, to be able to explore this, I've decided to create an RPG (role-playing game). Making RPGs is a great pastime of mine, having created two in Python and one in C++, which has given me experience in making them and helped me understand which areas may be the hardest to code. Algorithms I have considered using include A* pathfinding, which involves nodes and trees, cellular automata for map generation, and ray casting to determine whether an enemy can see the player.

Whilst making RPGs, pixel art images have been the main focus in terms of visuals; therefore, I have decided to do the same in this game, loading images from separate files stored on the computer to make it easier to upload new art. However, due to using PictureBoxes (a Visual Basic Forms inbuilt system that allows creating objects that move, collide, and can have different images), the background is not transparent. This is how they are made; as a result, all images will be box-like and have no white space (empty space).

One concept I have come across is "speedrunning," which involves finishing a game in as short a time as possible and being ranked with other players according to this time achieved. Consequently, my game will be created around the idea of trying to finish it quickly, with the end screen showing a leaderboard of times achieved (I am not able to show times of all players, as using servers is beyond the scope of this project).

I am fascinated by how programs can be optimised. Therefore, my game will use different methods for optimization. Furthermore, as I will be using PictureBoxes, a lot of lag is created due to the large amounts of data they contain. This means optimization will not only be something I want to do but is necessary for a reasonable refresh rate.

DESCRIPTION OF THE CURRENT SYSTEM:

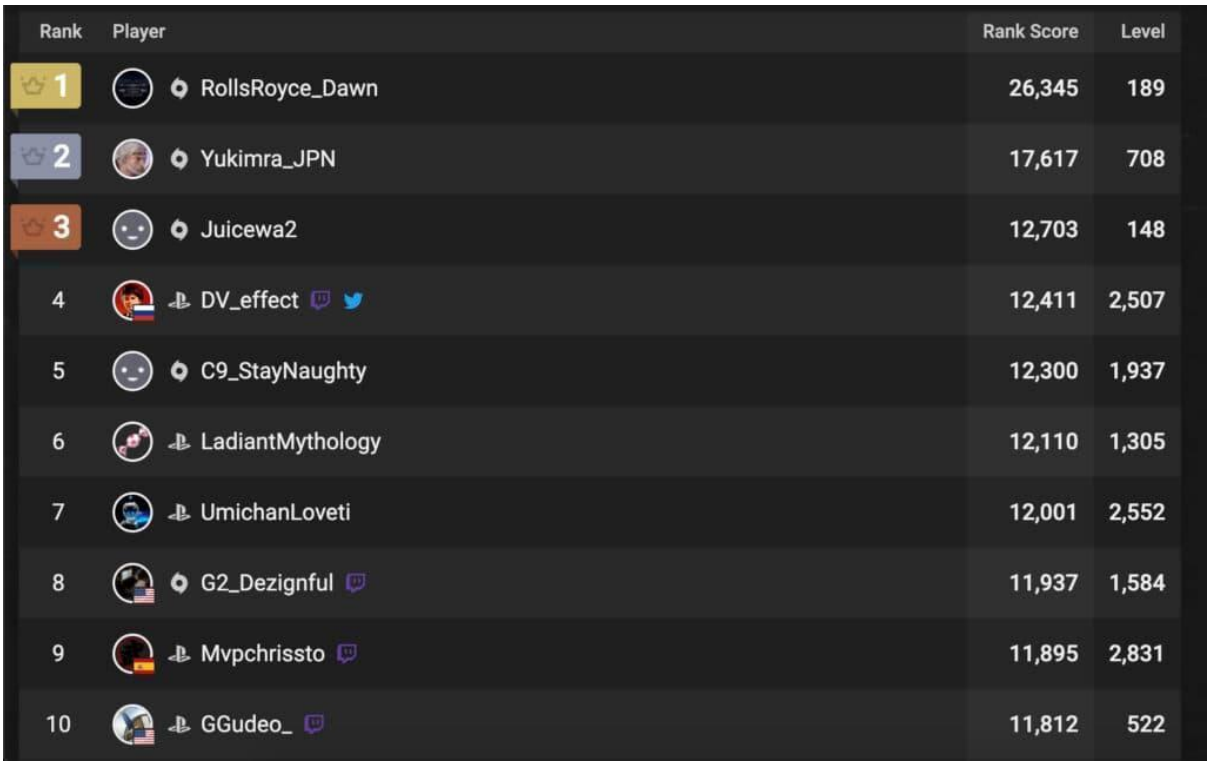
The RPG will have an entry screen with a Start, Exit, and Instruction button. If the Instruction button is pressed, it will go to a page that gives information about the game, similar to the instruction screens found in other RPGs or video games. This page will also have an Exit button that returns the user to the main screen. If the Start button is pressed, the user will be sent to a screen that allows them to customise their player by changing their colour and stats. A Done button will also be present so the game can begin. When this button is pressed, the user will enter the main game with their chosen customisation and stats.

In the main game, there will be different rooms with different enemies, and each room is randomly generated to allow for replayability and the use of more complex algorithms. For room generation, cellular automata will be used alongside set rules to create a box-like layout and to load enemies into the space. A* will also be used to create paths between the player and different doors.

Combat will be very quick-paced, involving clicking the mouse to attack and using the mouse wheel to choose between different attacks. Dodging will also be a component, whether avoiding fireballs from enemies or using a shield to absorb blows and push enemies back. Some attacks will be magical and use mana, while others will not. There will also be cooldown times so attacks cannot be spammed, making the game more challenging. This is a key feature in most RPGs, as it makes attacks feel more impactful and prevents them from blending into a stream of rapid button presses.

Bosses will also be a major part of the game, being more powerful and unique than normal enemies. They will have a separate health bar and drop unique items. Defeating the final boss will complete the game, dropping the player's eye (the story being that the player is searching for their eye). Both enemies and bosses will use ray-casting in their code to check whether they can see the player and therefore move towards them.

At the end of the game, once the eye has been equipped, a final screen will appear allowing the user to input their name. This will be saved to a text file along with the time taken to finish the game. Afterwards, a leaderboard will appear ranking previously achieved times and the names associated with them. This system is widely used across gaming to create competitiveness and encourage players to replay the game to beat their previous times.



Rank	Player	Rank Score	Level
1	RollsRoyce_Dawn	26,345	189
2	Yukimra_JPN	17,617	708
3	Juicewa2	12,703	148
4	DV_effect	12,411	2,507
5	C9_StayNaughty	12,300	1,937
6	LadiantMythology	12,110	1,305
7	UmichanLoveti	12,001	2,552
8	G2_Dezignful	11,937	1,584
9	Mvpchrissto	11,895	2,831
10	GGudeo_	11,812	522

Figure 1 :Apex legends leader board ranking players

This leaderboard ranks players across all systems and platforms, creating pride in being at the top and giving players a target to reach. Creating a multiplayer leaderboard requires servers, which cannot be implemented in this project as they cost money to

run. As a result, I have decided for leaderboards to be stored locally on each machine that runs the RPG.



Figure 2 :Real-time combat in a similar 2d RPG

As you can see, in this real-time (not turn-based; all actions made by players and enemies happen at the same time) RPG, the player can attack other entities and has health (hit points) they can track alongside other stats. This game is quite old and is called “The Final Dungeon.”

ANALYSIS OF SIMILAR SYSTEMS:

There is a wide variety of RPGs out there, coming in many different forms: 2D, 3D, pixel animation, turn-based, real-time, and so much more. As a result, I also have to make my game unique like all the others to give it extra depth; this will also increase the challenge I have for coding it. Therefore, I have decided I will make a fast-paced, real-time RPG where movement (this includes combat) is the key to finishing the game. However, as so many have been made by large teams of people over many years, I can use their expertise and take some examples from their games.

For example:
A similar system found is “The Final Dungeon,” which is a real-time, 2D role-playing game (RPG). It involves the player navigating through different rooms, which contain a wide range of unique enemies and items for them to use. Combat also plays a major role, being real-time and requiring quick decision-making and agile reflexes from the user, allowing them to feel immersed and also challenged by the game. Reflexes are important because dodging, attacking, and doing various abilities are all major parts of the system. Another system uncovered in “The Final Dungeon” is the leaderboard, which encourages replayability through fun competitiveness. The way this is ranked is through performance, which can be interpreted as either user times, scores, or kills. Furthermore, procedurally generated rooms are another system utilized in their game, creating fun and different encounters for users each time they play the game, which I will be able to recreate in my game.

“Enter the Gungeon” is another RPG with fast-paced combat and randomized rooms; however, a new system they add is bosses, which create a greater challenge for users and a possible greater reward. As chests and similar approaches to collecting items reduce how fast-paced most systems are able to be, I will try to avoid them. Instead, I will have enemies either automatically give items once defeated or drop them, allowing the user to decide if they want to sacrifice some time to collect them.

Another instance we can uncover in current RPGs using similar approaches is in “The Legend of Zelda: A Link to the Past,” where the user interface and inventory play a key role in giving the user information on how their game is going. Showing items collected, player health, player mana, and items equipped all give a greater insight into how their game is going. Although, due to the need for a smaller window size making coding in Visual Basic easier, these systems will need to be scaled down so the user is not confused or overstimulated. This will be done by either making images appear smaller or removing them altogether.

Figure 4: Ash World 2D RPG, the inventory system
This RPG is a 2D real-time game that involves a lot of pixel art. Unlike mine, it is set in a more futuristic era; however, the use of bars to represent health and other stats is something I would really like to implement in my game, providing a more visual representation of information to the user. As well as this, an HP bar for the boss can be seen in the centre, therefore giving that enemy an extra level of importance. This is another way in which information can be represented to the player without the use of explicit wording, making them feel much more immersed.

In “Ash World,” there is also the use of a grid-like inventory system, something I would like to take a lot of inspiration from to make my own inventory for the user. By using images and not only text, it is much quicker to tell what things do and makes them much more appealing. It also gives a much larger clickable space so the item can be utilized by the player. Although the system Ash World uses does not allow items to be stacked (having multiple of the same item), this is unlike the system I would like to create. Consequently, I’d like to use numbers to represent how much of each item there is.

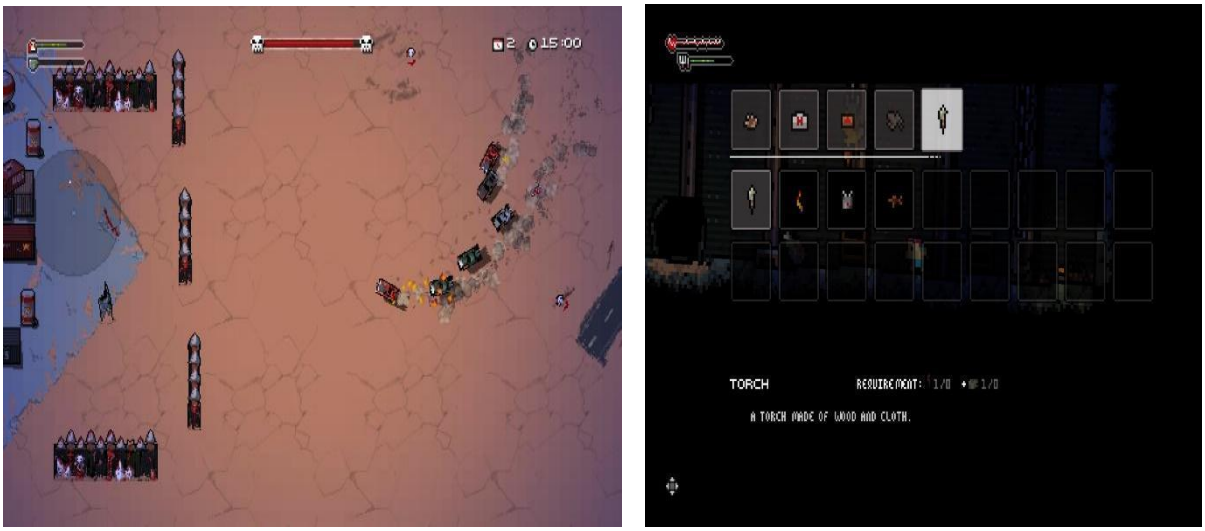


FIGURE 4 :ASH WORLD 2D RPG, INVENTORY SYSTEM

This RPG is a 2D real-time game that involves a lot of pixel art. Unlike mine, it is set in a more futuristic era; however, the use of bars to represent health and other stats is something I would really like to implement in my game, giving a more visual representation of information to the user. As well as this, an HP bar for the boss can be seen in the centre, therefore giving that enemy an extra level of importance. This is another way in which information can be represented to the player without the use of explicit wording, making them feel much more immersed.

In "Ash World," there is also the use of a grid-like inventory system, something I would like to take a lot of inspiration from to make my own inventory for the user. By using images and not only text, it is much quicker to tell what things do and makes them much more appealing. It also gives a much larger clickable space so the item can be utilized by the player. Although the system Ash World uses does not allow items to be stacked (having multiple of the same item), this is unlike the system I would like to create. Consequently, I'd like to use numbers to represent how much of each item there is.

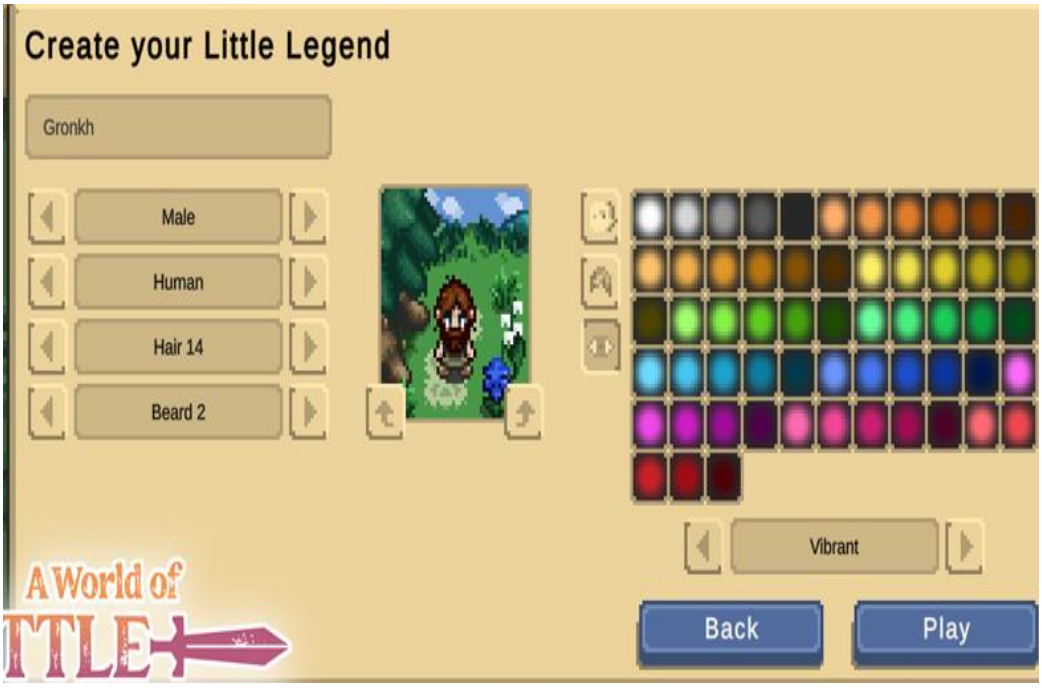


Figure 5 :A world of little legends-RPG, 2d, user character customization page



Figure 6 :A world of little legends-NPC communication

As you can see, the customization page allows the user to change the colour of their character; this is something I want to include. In contrast, my game will only allow the user to input their name at the end of the game, accompanied by the time they have achieved to finish the game. This is where “A World of Little Legends” differs greatly from my own RPG, as it is a much slower-paced game. Although saying that, it includes very similar systems that I want to appear in my own project.

In Fig. 6, we can observe the use of NPCs (non-player characters) to communicate with players, provide possible missions, and explain rules. This is better than just creating text to tell the user these missions and rules at the start of the game. Despite this being a feature, I would like to include, my NPCs will only be able to “talk” to the player when they go close to them, and will only be able to accept missions by clicking on the entity, making them despawn (be removed from the game).

LIKES AND DISLIKES ABOUT SIMILAR SYSTEMS:

WHAT I LIKE	***WHAT I DISLIKE***
Unique characters that can be made by the player	How overcomplicated these systems can be, making the player confused and meaning a lot of extra effort must be put into designing their character
A large variety of enemies and attacks so the player will not get bored quickly and can have different experiences throughout the game	Having too many attacks and enemies may overstimulate the player having to remember different enemies, reducing their enjoyment of the game as they have to spend time remembering an increasing number of things.
Being able to save my progress and load games that I have already saved, making the player feel as though they are working towards something each time they play.	Having to allocate memory, increasing the overhead making it less attractive to download and use.
Random generation of maps and spawn locations, giving a more unique experience each time, the game is played	Occasionally map can be like mazes not having a direct path to key locations, like doors or enemies
A diversity of items which are usable by the player creating different combinations of ways of playing the game	In many RPG's items feel as if they are all the same with little difference and noticeable impact on gameplay
Having the ability to collect and store items in an inventory as you journey along the game	Being restricted by the weight of items and having to get rid of certain things gives the feeling that progress has been lost
Having a start menu, with an instructions page giving the player an introduction before abandoning them in the game	Instructions can be quite vague and have many words making them boring to read
An armour system making the player feel more powerful as they progress through the game, giving them the feeling of advancement	In some games the right combinations of armour and weapons can make the player invincible and do great deals of damage making the game unfair and easy
Side missions that allow the player to decide what route they want to take and making the game seem larger	Missions do not feel as if the reward they give is linked to the mission, or a fair reward
Bosses, if defeated, give a very unique reward creating the sense of completion and satisfaction	In more well-known games (like the dark soul or Elden ring) bosses can be extremely difficult taking a large amount of time to beat, making the player feel as if their progressive has been stunted.

DIFFERENT ALGORITHMS AND LOGIC:

In most RPG's a variety of problems relating to the logic of the game must be solved and due to the fact that I am aiming for a game that has infinite replay ability most systems like terrain generation, entity spawning and entity type will have to be unique therefore random each time they are loaded.

N = Number of values in a list

T = Time complexity (number of cycles done)

V = Vertices

E = Edges

TERRAIN GENERATION

There is a multitude of ways to create random terrain/rooms with each way being used under different circumstances.

Name	Complexity to write	Randomness	Efficiency	Type
CELLULAR AUTOMATON	Medium	High, large variety of shapes	O(N*T)	Cave/room generation
PERLIN NOISE	High	Medium, some variety of different landscapes	O(2^N)	Landscape generation
SIMPLEX NOISE	Medium	Low, very isotropic landscapes	O(N^2)	Landscape generation
RANDOM WALK	Low	High, can be extremely random	O(N*T)	Cave/room generation
PRIM'S ALGORITHM	Medium	Medium, many corridors	O((V+E) logV)	Maze generation
Kruskal's algorithm	Medium	Medium, many corridors	O(V)	Maze generation

I shall use cellular automata as it has medium complexity with a high randomness, it is also used for room generation which I am aiming for.

PATHFINDING

Pathfinding is necessary for making sure all objects/entities are reachable for the player, using it for movement by NPC's is unnecessary when rooms are so small.

Name	Complexity to write	precision	Efficiency
DIJKSTRA'S ALGORITHM	Medium	Quite precise always finding a path but not in the most optimised	O(E + V(log(V)))
A* SEARCH ALGORITHM	High	Extremely precise always finding the most optimised	O(ElogV)
BREADTH-FIRST SEARCH	Easy	Not precise, just finds a path	O(V+E)
DEPTH-FIRST SEARCH	Easy	Not precise, just finds a path	O(V+E)
MAXIMUM FLOW	Medium	Not precise, used to check if there are multiple paths, as if liquid were flowing through them	O(VE)
MINIMUM SPANNING TREE	Medium	Quite precise, but instead of finding a direct path it finds a path connecting all nodes in the tree.	O(V^2)

I shall use the A* search algorithm as I must check each path for each entity for each map which must be done as quickly as possible as it is used whilst loading rooms.

SORTING

Sorting is used to order a list many different types of algorithms are used as list need to be sorted in a variety of ways.

Name	Complexity to write	Efficiency	Type
BUBBLE SORT	low	O(N^2)	Lowest to highest sort iteration
MERGE SORT	Medium	O(Nlogn)	Lowest to highest sort Iteration, recursion
SELECTION SORT	Low	O(N^2)	Lowest to Highest sort iteration
INSERTION SORT	Low	O(N^2)	Lowest to highest sort iteration
BUCKET SORT	Medium	O(N)	Elements sorted into groups
QUICKSORT	Medium	O(NlogN)	Lowest to highest sort Iteration, recursion

I shall use Quicksort to order all times in a list as it will need to be quick due to the player waiting for the finishing scoreboard to load which depends on the sorting algorithm. There will not be many values as the scores are only taken from the local device, hence quicksort's best time complexity will be most common being O(NlogN).

INTERVIEW:

1.how would you like levels to be designed:

Levels Generated randomly for variety and replay ability with boss rooms, Npc rooms and many more so give the feeling of a journey. Randomised spawning of entities meaning dogging will be a large part of the game.

This answer allows me to create some kind of random terrain/room generation that will involve cellular automaton, a* pathfinding and a 1D text map made up of strings giving it a 2D array feel, whilst saving space and making it quicker to loop through, as well as this I have created a way to choose which wall image will be painted onto side walls next to areas the player can walk allowing them to check around themselves and decide the image accordingly.

Sub objectives:

. create an ordered way to show the text map to the player as a full map.

. create a class to randomly make terrain for each level, to make it more enclosed (class called mapmaker).

. create a way to randomly spawn entities e.g. players, doors and enemies, with navigable paths to each other.

2.what do you find in a common role-playing game:

enemies have difficulty and variety e.g. different attack types, different forms. Magic is also a key characteristic of most RPG's giving the player mana and magic attacks.

This answer allows me to create a base class and inherited classes that use polymorphism for more different and specialised enemies. It also means I have to create a base class for attacks that not only my enemies will use but also my player making it a very dynamic system. The way it will read the stats and image locations for each attack is by accessing a text file. Different attacks will move and not move meaning either I'll have to make a go to player function or a circle that creates points for attacks to go to. I will also have to store the maximum mana and the mana at each instant in time to create a progression bar for it making attacks have an underlying tone of magic.

Sub objectives:

. create a way to kill attacks after a while (TTL) as a way to decrease lag.

. create a way to check if the player is close so attacks can be used.

. create a way to make certain enemies use attacks at different times, another way to decrease lag and create randomness.

.Create a yellow mana progression bar

3.what features should be different then an average RPG:

More side quests, with high difficulty and reward (no fixed path)

I need to make some bosses not mandatory and have small quests (e.g. looking for a key to a chest) to make it feel less controlled and more up to the player, I will also need a boss class that can inherit from the base enemy class, I will also need to add special variants to differentiate them from the normal enemies. I may also create npc's that says text to you that can help u on your journey, I can use another text file that can do this and also a search algorithm that will find each text for each npc.

Sub objectives:

. The bosses will need larger health bars and a different ui to make them stand out.

. The npc's will need their own class and own functions so speech will be enabled.

. The player will need to select missions, and store them so they know which ones are active

. The player will need to be able to keep track of their kills so missions can be completed and rewards gained

4.In modern RPG'S what do you dis-like:

Customization is too complicated so: make a simplified customisation feature

This answer means I will have to create some type of customization UI before the player enters the game. The system will have to be simple but informative to allow the creation of a quick yet responsive character customisation scheme. It also means I will have to create a window that allows inputs (in the form of buttons) to change certain stats and characteristics.

Sub objectives:

. use of a simple UI system, made of player interactable buttons and progression bars.

. Once the character is made it deletes UI and starts the game.

. A variety of different stats will be needed, for the player limb colours and certain characteristics like strength, wisdom and dexterity.

5.How interactable should the game be:

To make the user feel more immersed, add more ways you can utilise the mouse e.g. attacks, shield, choosing attacks

The answer given prompts me to create a way to scroll the mouse for new attacks, left click to use the attack and right click to use the shield if its equipped. I will also have to show this to the user animating different abilities and allowing them to see which one is in use, animating the shield into the game and

creating an image for it that matches the image of the shield that is equipped. As well as this I will have to vary abilities so they don't seem monotonous to the user, meaning my attack class will have to be dynamic and easy to make varied attacks.

Sub objectives:

. A way to store different attacks and abilities they may have, using text files.

. Give the shield push-back and health to make it functional and make it follow the mouse.

. I also have to allow abilities to move from the player to the mouse to increase intractability.

6.do you think non-interactable characters are good:

Yes, as it gives the world more life after you die and when your exploring.

The answer given, means I have to make NPC's (Non-player characters) in the game, which will have their own class allowing me to create different text and images for the character. I will also have to make a way to randomly generate NPC's that makes sense, this can be done by manually placing them or randomly placing then in chosen rooms.

Sub objectives:

. NPC's may give items to you.

. Create a way to add NPC's to the game by spawning in random locations.

. Create a way to assign certain rewards to different NPC's.

. Use a text file to store all possible missions the NPC's can have and any extra data that come with those missions e.g. reward type, reward amount, enemy to kill, amount of enemies to kill and so on.

7.How powerful is your device:

It is not powerful at all I have a 2 gb's of ram and an Intel 4004(very old)

Optimise as many algorithms as possible including simple ones e.g. creating maps (only done once at the start of each map), warlock enemies shoot at different time (reduces lag calculating different fireball positions at once). These strategies can reduce lag created by using picture boxes and not visual basic graphics that involves drawing straight to the screen which can be more complicated and does not provide as many features that picture boxes do (collisions, movement, clearing the screen).

Sub objectives:

. The best way to optimise the game is reduce collision check with wall that aren't physically touchable (just create images).

. To make attacks happen at different times give warlock enemies different unique ID's and spawn fireball in accordingly.

. Creating a flooring system that allows more creativity on the floors create lag as a 50 by 50 picture box must be created for each cell instead just have a common background colour (no lag created).

. Only create rooms once the player reaches them, so rooms aren't created unnecessarily if the player dies before accessing them.

8.What time of gameplay to you expect e.g. Fast paced, retro:

I expect a fast-paced game that involves movement and quick reaction times.

Movement will have to be quick, visual basic does not allow multiple key presses to be registered at once, as a result velocity will have to be used instead of left and right movement going straight to a location. Meaning diagonal movement will be allowed in the game giving more possible movement but increasing the amount of collision's checks for the player must be taken into count when adding this.

. Reaction times, attacks will have to also have movement e.g. Fireballs, webs, invisible teleportation orbs.

. Another way reaction times will be assessed is through the shield also having movement allowing pushback and absorbing attacks.

. Enemies must also have some sort of movement aspect about them to keep the theme of fast-paced gameplay real, by either making them quicker or having more accurate player location finding and reaching. This will be done using ray tracing to track whether the enemy can see the player, or if it blocked by walls.

9.How do you like to store items and equipment:

I prefer to keep my collectables in an inventory type system that also shows the items I have equipped.

This response shows the player is looking for a UI system where consumables and items can be stored and updated depending on uses. I will have to create a way to open the inventory without making it seem overbearing in screen presence and also a way to use consumables updating amounts accordingly. I will also have to create a way to store the stats of different consumable.

Sub objectives:

. To store stats of different items, including images, I can use a text file that stores all item data, which can be updated easily and then reading it in runtime to use the information provided.

. To make the inventory responsive I will have to create a way to delete items when their total is zero and increase their count when more than one is collected.

. To show the items equipped (armour, weapons, shield) I will have to create a new UI that allows you to see them and change them accordingly.

. To show armour has an effect on the player a small bar in the bottom left of the screen changes depending on the amount of armour used and its type.

OBJECTIVES:

Objects:

1. **UI objects that allow more interactable inputs from the user widely used systems**
    A. **Progression bar objects that can change capacity depending on how different variables compare to one another**
    B. **Button objects that can be clicked on by the user creating an input method that is interactable**
    C. **Text objects that have different colours, sizes and looks depending on how they have been customized by the programmer**
2. **Utility objects that reduce the amount of repetitive code need to systems used in a variety of different classes**
    A. **A UI object that loads and saves to text files and generates random numbers, both systems that are widely used by different entities.**
    B. **A Ray Casting object that is inputted as a parameter into different enemies to check if they can see the player behind obstacles**
3. **The form objects that are the actual interface for the user**
4. **A player object that takes in a variety of user inputs and giving the player object as a parameter into other objects to find where they are or what they are doing**
5. **A base enemy class that contains all widely used systems, used by entities that inherit from the class**
    A. **Inherited by both the goblin and warlock enemy objects which look different and attack differently but both need collisions, pathfinding, entity limbs and similar variables**
    B. **Also inherited by the boss object to create more overwhelming enemies with more dramatized stats**
6. **A base block class that nearly all tangible objects on the screen inherit from**
7. **A base attack that can be greatly modified for different attacks**
    A. **An object orientated approach is taken to reduce the sheer volume of different attack classes needed to be programmed but allow great variety between attacks using the same class**

Entry screen:

1. **An exit button object to stop the game deleting the form and all other objects from memory.**
2. **A start Button object that begins character customisation exiting the entry screen**
3. **An instruction menu that allows the user to read the rules and key-binding of the game**
    A. **A one button object so the user can exit the instruction menu and return back to the start screen**
    B. **The ability to scroll through instructions so the user can read the entire text**
4. **The title of the game, to show the user what application their using**

Customisation menu:

1. 3 indictors for strength, dexterity and wisdom
    A. Strength affects the player health and the amount of damage physical attacks do
    B. Dexterity affects the player speed allowing for more dodges and shorter run time
    C. Wisdom affects the player mana and the amount of damage spell attacks do
2. Corresponding progression bar objects to each that show where attribute points have been allocated
3. An increase button object that allows points to be added to certain stats
4. A decrease button object that allows the user to remove previously allocated points
5. A piece of text that shows how many attribute points are available
    A. Once all the points are used make it so the user can no longer add points
6. An outlined box that contains the player character
7. 3 button objects that are assigned an +R or -R for G and B that dictates how much of each colour appears in each limb
8. 2 Button objects that allow the user to switch between limbs and change each one's colour variable
    A. A piece of text that informs the user which limb they are on
    B. The ability to loop through limbs so the user can always press forward and backwards
9. A single button object that has the word Done on it, when pressed it starts the game with the desired attributes

Instruction menu:

1. State each instruction using text objects which are large so also incorporate a scrolling bar
2. Explaining the layout of the game (5 enemy rooms, 2 boss rooms, 1 Npc village)
3. Showing which keys correspond to which instruction (W, A, S, D for movement)
4. Explaining how the game is replayed, to beat your time, to kill the final boss.
5. A button object that allows the Instruction menu to be exited back to the main screen
6. Explain to the user how to interact with consumable e.g. (health and mana potions, item objects)

Map generation:

1. Maps are represented as text map where "b" corresponds to a wall or "e" corresponds to a goblin object etc, the text map is stored as an array of strings.
    A. Add a character for the boss "B"
    B. Add a character for the NPC" N"
    C. Add characters for different types of enemies, goblin "e" and warlock "w"
2. The text map is iterated through with each letter being read and spawns whatever is assigned to the screen.
3. Cellular automata are used to create a random-ordered environment each time a room is loaded, where starting cells are randomly positioned.
4. Rules are used (50 or more empty spaces) to make rooms playable by the user

5. **A* pathfinding is used to spawn the player and door to exit he room with a direct path of to one another**
6. **To reduce lag caused by each entity checking every wall, only walls next to empty spaces are add to a wall list and are check for collision so the others are just images.**
    A. **Use the same "real" walls by the ray casting to get the line equations and reduce the amount of lines needed to check.**

Player movement and attacks:

1. **The W, A, S, D keys are inputted into by the user to give movement to the player**
2. **As visual basic does not register multiple keys at once, velocity is used instead so a key must only be pressed once so the character moves in that direction indefinitely**
3. **If a key is pressed twice the player object moves faster in that direction**
4. **Keys can't be pressed infinitely to make the player go faster and faster they, at most, can only pressed twice.**
5. **When the left click is pressed the chosen attack is used**
    A. **It follows the user's mouse giving it direction**
    B. **Some attacks can move towards the mouse**
6. **A wide range of attacks e.g. Fireball, lightning and slash each with different effects**
7. **Use the scroll wheel to choose between attacks, with the image in the top right using attack chose attribute in the player class showing which attack is being used**
8. **Range for each attack and a cooldown bar object that can be seen under the image in the top right, so players can't attack constantly.**
    A. **Attack cooldown times are stored in a text file**
    B. **Attack damage and images are store in the same file**

Ray Casting:

1. **Create a map in memory that uses all the generated walls in the text map to create lines(blockers) that stop rays from travelling**
    A. **Create blockers using two coordinates, the start coordinates and end coordinate which are found by checking runs of wall characters (in rows and columns)**
    B. **At corners create a blocker so rays cannot slip through**
2. **Rays are used by enemy objects to check whether the player is in their sights**
3. **Use aggregation so only one blocker map must be generated for all entities to use**
4. **Have a timer that means enemies wills continue travelling towards the player object for a time even if the player is out of sight**

Collisions:

1. **Collisions needed for the player object and the walls (base block object) so they cannot leave the designated area.**
2. **Collisions for the enemy objects with the walls and the player object, so they cannot leave and to cause any physical damage to them, also when the player object is touched pushback is caused to them. Changing the enemy objects their x and y attribute.**
3. **Doors to the next room have collisions to allow traversal of the levels**
    A. **Door objects are linked to the room they connect to**
    B. **Their collisions are turned off until all enemy objects are killed**
4. **Boss objects have collisions with parts of their body also pushing them back and applying damage to the player.**

Inventory/ game UI:

1. **When the (I) key is pressed the inventory object is either opened, or closed if already opened**
    A. **Some UI pieces are move for the inventory (the health bar and mana bar)**
2. **2 bars objects can be seen when the inventory is closed in the top right representing the player objects health property and mana property each with the percentage left of each attribute.**
3. **When the inventory is opened a grid is drawn where items can be found that the user has picked up**
    A. **The stackable items show the number of them there are in the bottom corner of their image**
    B. **Armour and weapons are not stackable so just add another one to the inventory**
4. **If the user clicks on these items they can consume them or equip them each of which has a count of how many there are**
5. **A separate column of 7 boxes can also be seen when the inventory object is opened showing the item objects that are equipped(armour/shield)**
6. **Attributes for the in-game item objects are stored in a text file loaded into the game each time it is run as a dictionary, storing the image path and the different attributes of the item objects**

Shield:

1. **After beating the spider boss object, the shield object is dropped as an item object and can be used is picked up and equipped**
2. **The shield object has its own bar object that uses its own health property so after multiple collisions it is destroyed, and is removed from the inventory**
    A. **Delete the shield from the inventory and from the player's view when is disappears**
    B. **When the shield is not equipped hide the bar**
3. **If the enemy collides with it, they are pushed back, if magic attacks collide with it they are destroyed**
4. **The shield follows the user's cursor direction but is attached to the player character**

5. **Can be turned on and off if equipped by the player using the right click.**
   A. **Make is so the inventory can be opened when the shield is not being used**
6. **If the shield is equipped then the inventory cannot be opened**

Enemy:

1. **Enemy objects are represented as different characters in the map maker**
   A. **Make sure the enemy doesn't spawn in the player or is stuck on a wall**
   B. **Different types of enemy will be represented by different characters (goblin=" e" and warlock = "w")**
2. **Different types of enemies will be made to create diversity**
3. **Enemies will have the ability to attack, with ranged attacks and by colliding with the player**
   A. **The goblin will only be able to collide with the player which in turn does damage**
   B. **The warlock will spray fireballs around themselves which will go outwards as ranged attacks, they too can collide with the player causing damage**
4. **A base class for the enemy will be used to create a starting template for all different enemy types and certain entities**
5. **Enemies will have id's so they can be differentiated between**
   A. **To reduce lag the ranged attack by the warlock involving fireballs must be done at different times by different warlocks, hence id's will be used to separate these time**
6. **They will have limbs meaning they may look a bit less cube like**
7. **Each enemy will have the ray caster connected to them by aggregation so I only have ray cast and generate the blockers once, being more efficient**
8. **If there are no obstacles between the enemy and player the enemy will simply travel in a straight line towards the player**
9. **Each time an enemy is destroyed they will drop an item, depending on how rare the enemy is it will dictate the uniqueness of the item.**

Bosses:

1. **In the map generation their will have to be a setting for only boss objects to be spawned and not normal enemy objects**
   A. **A special character to show where the boss should be spawned**
2. **Once the player object enters a boss room a bar object appears showing the bosses health attribute and the percentage that is left signified by a separate number (as a text object)**
   A. **This bar object is removed when the player object leaves the boss room**
   B. **The bar object has a border and is larger making it feel more important**
   C. **The bar object is in the centre of the screen so it cannot be missed by the user**
3. **There are 2 types of boss objects a spider and an invisible boss.**
4. **when the player object leaves the room, the boss object will be deleted on the screen and all their body parts too**
5. **The spider has 6 femurs, with 3 black picture boxes in each surrounding the main body and moving with the body, it has 6 thighs attached to these femurs which every few second anchor to a new position closer to the main body, simulating a spider crawling.**

    A. Make it so these anchor positions are in a circular pattern around the spider boss object
6. The invisible boss object has a main body surrounded by white picture boxes to simulate a circle out of cubes.
7. The invisible boss object has a swarm like cloud of small picture boxes they can absorb magic attack objects from the player object and do damage (taking points from the player objects health attribute) when they have contact with it
8. The invisible boss object also shoots a ray (using the ray casting object) towards the player of purple orbs that if touched teleport the player into the swarm causing a massive amount of damage.
9. As predicted by the name at certain intervals he boss will become invisible only its cubes can be seen by the user
    A. Attack objects cannot be created by the invisible boss object whilst invisible
    B. If the player object lands an attack the invisible boss lights up grey for a short amount of time
10. When killed (boss objects health attribute is less than or equal to 0) both boss objects drop more valuable item objects with the spider dropping the shield object and the invisible boss dropping the player eye item object
    A. The eye item that finishes the game

NPC'S:

1. Make a separate class for NPC's allowing them to be spawned in on the map
2. Allow NPC's to offer missions to players
    A. Store the different missions in a text file
    B. Make sure each NPC has a different mission by only spawning the necessary amount
3. Make the NPC's show the mission their offering in a speech bubble above them
4. Allow the user the click on them to accept the mission
    A. Delete the NPC once it has been clicked on
5. Write all the missions to a text file with their accompany attributes e.g. reward and what to kill, when read it gives each mission to the NPC's in order
6. Create a room only for NPC's and make them spawn randomly but accessible to the user
7. Give a reward to the user if the mission is finished, therefore track progress to finishing the mission and show it to the user
    A. Turn a mission green when it has been finished
    B. Keep the mission green until it has been clicked on by the user
8. When the inventory is opened create a array of missions chosen, showing progress of the missions accepted first and get rid of the missions when the inventory is closed.

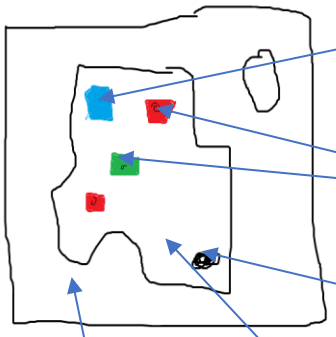
End screen:

1. When the final boss is killed the player eye item object is dropped onto the floor (they do not have it throughout the game)
2. When the player eye item object is equipped the game is finished
    A. The enemy count text object changes pink and says the game has finished.

3. **When the game finishes the main form closes and a new form appears**
4. **This new form takes the users name and displays their time next to the top times from other users, on their device**
5. **This will have to be done by saving times and names to a text file and using a timer for how long the player is in game.**
6. **The user will enter their name and press submit when it done adding their time to the file, so I will need to use a text file**
7. **Other button objects will be a restart button and exit button**
8. **Previous game times are shown and sorted using quicksort to create a leader board**
   A. **All previous times are stored in a text file with their name below the time**
   B. **Only the top 10 times are shown to save space on the screen**
   C. **The user's time is shown in a red box in the top right corner with their time and where they ranked**
9. **A quicksort algorithm will take place to order these times**

MODELLING:

MAIN ROOM:



The player spawns in the top corner of each room

An assortment of different types of enemies

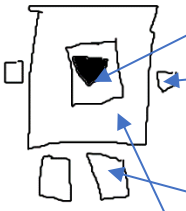
The door to the next room spawns in the bottom corner

Random room generation, usually creating a cavernous space for quick movement

The walls next to empty space will be real, all others will just be images, different walls use different images

This will be the layout for each room the player accesses, however they will be random using cellular automata creating a more unique experience. Around 3 enemies will spawn in rooms with a different assortment each time to create a new experience. All rooms will also be connected by doors, each map storing where they lead on to

ENTITY STRUCTURE:



The eye that nearly all entities have except the player, made of a picture box and a circular image drawn on top of it.

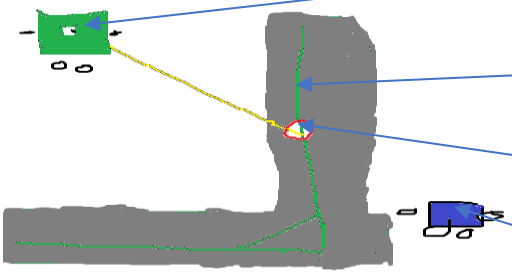
The arms which nearly all entities have except for the bosses, which have more unique body parts, these can move in certain ways to give more life to the object.

The legs, another part which nearly all entities have except for the bosses, these move one after the other to simulate. walking

The main body which all entities have this dictates the movement of all other limbs being their origin. As well as this it is the only object to experience collisions meaning all other limbs may look like they're in the wall but are not.

This structure is used by all entities giving them a common layout and reducing the amount of code that must be written it also gives a constant look for different entities making them feel of the same genre

RAY CASTING:



The enemy shoots out a ray towards the player all done mathematically (cannot be seen by the player

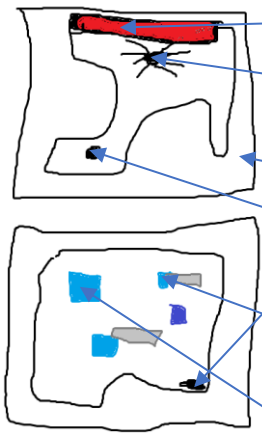
Instead of a blocker for ray for each side of a wall, one line is created for all walls in a row, being much quicker computationally

If the ray collides with a blocker the program is notified and the enemy cannot see the player, these checks use line-line intersection math

The player does not have any ray casting but their location is known by the ray casting algorithm to make it so only one ray per entity must be processed to check if they can "see" the player character

This ray casting method is done by all entities, using the same blocker map meaning the game is greatly optimised in this way.

BOSS AND NPC ROOM:



The boss's health bar, also with a number in the bottom left that indicates the percentage of health it has left

The boss itself, there will be a few bosses all being different in looks and abilities, they will drop a unique item on being killed

The boss room, which will be connected to rooms so players can decide on killing it

The doors to each type of room allowing them to be access and exited from

When the player is within range the NPC will show what mission, they hold in a text box above themselves, if the text box is in view they can be clicked on and the mission will be selected by the player

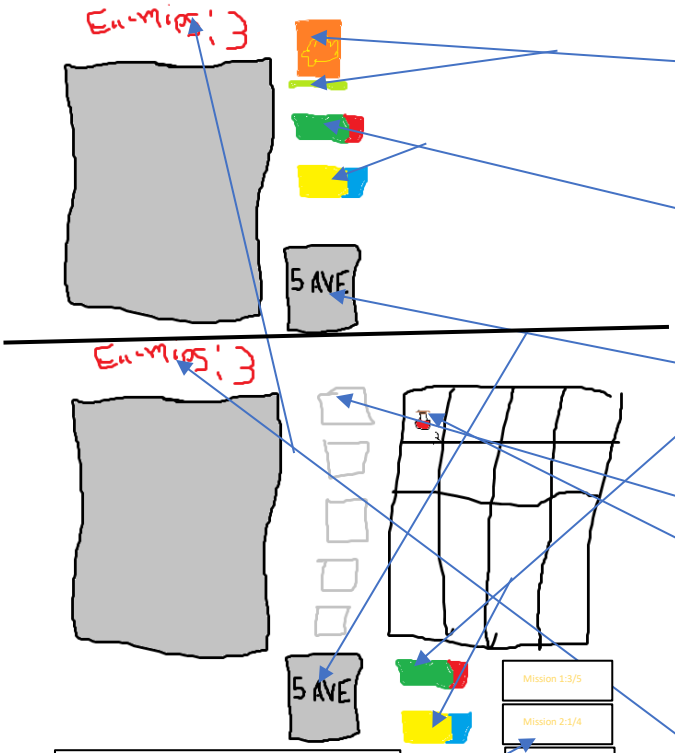
The NPC, it can give missions if the player is close, it does not move or do is aggressive in any way however is the same colour as the player-to show they are related

Both of these rooms have different entities in them Hence the spawning must be changed

As well as this due to the boss bar at the top A progression bar will have to be hard coded for this room so it can be as informatic to the user. On top of this to spawn different bosses a randomiser may be used or they may be hard coded to reduce the chance of the same boss being spawned twice

In the NPC room, NPC's will have to be created and assigned missions dynamically so the game can tell which missions have been chosen.

INVENTORY AND UI:



Chosen attack image, giving the player insight into what attack they have chosen. This is accompanied by the attack cooldown bar showing the player how long they have for each attack

Progress bars used to visualize health points and mana to the player. Depending on whether the inventory is open they move so the player can always see them

This button, when pressed saves the map and stats of the player to a text file, only one thing can be saved at once to save space

Shows the armour and weapons equipped if something is already equipped and something new is added the old item is overwritten

The inventory, which is a grid, stores all the items that have been picked. If an item is already present it stacks it and shows the amount under the items unless it is armour or a weapon which is un-stackable and just adds it to the inventory by itself

Shows the number of enemies left in a room to the player and changes colour if that number is 0. Once 0 the door unlocks so the player can go to the next room.

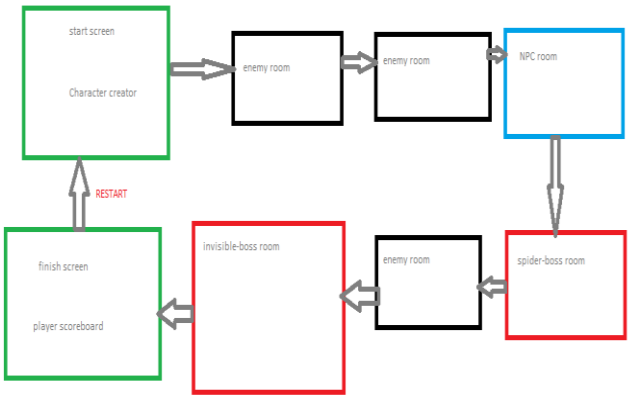
This is where missions are stored after being selected by the player, they track how close the player is to finishing them. If they have been completed they turn green meaning they can be clicked on and their reward can be collected

The inventory allows for the storing of different items that can be used, the UI is used to show the player important information about their character

Items in the inventory can be clicked on to either be equipped or used, there will also be a shield that can be equipped in this way, however when being used the inventory cannot be opened to make the game a little harder.

The UI shall update the entire time it can be seen unlike the inventory where certain things only happen when it is opened and closed. However, when the inventory Is opened some things in the UI are closed like the chosen attack, meaning the player cannot just keep it open the entire game making it more interactive.

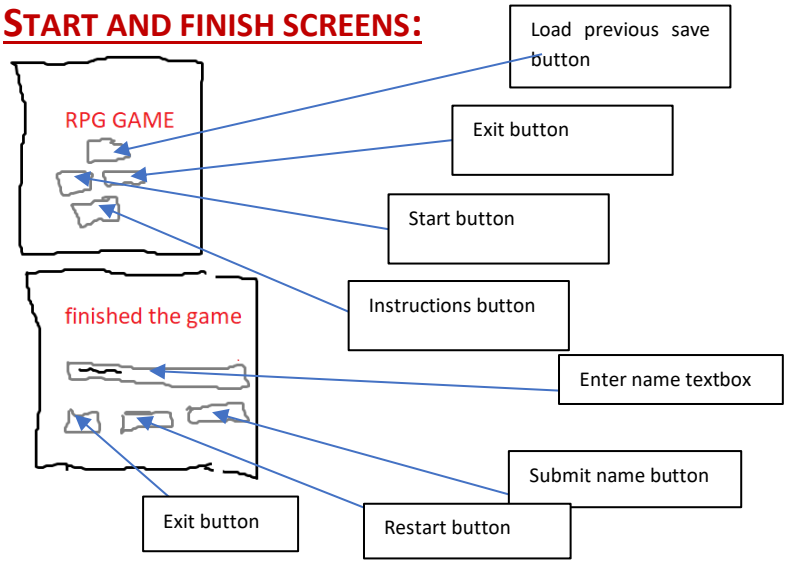
Main game loop:



This Is the basic format of the game creating a replayable loop.

The player can start the game by 2 ways, either: load a previous save or start the game from scratch. The game cannot be saved in a boss room.s

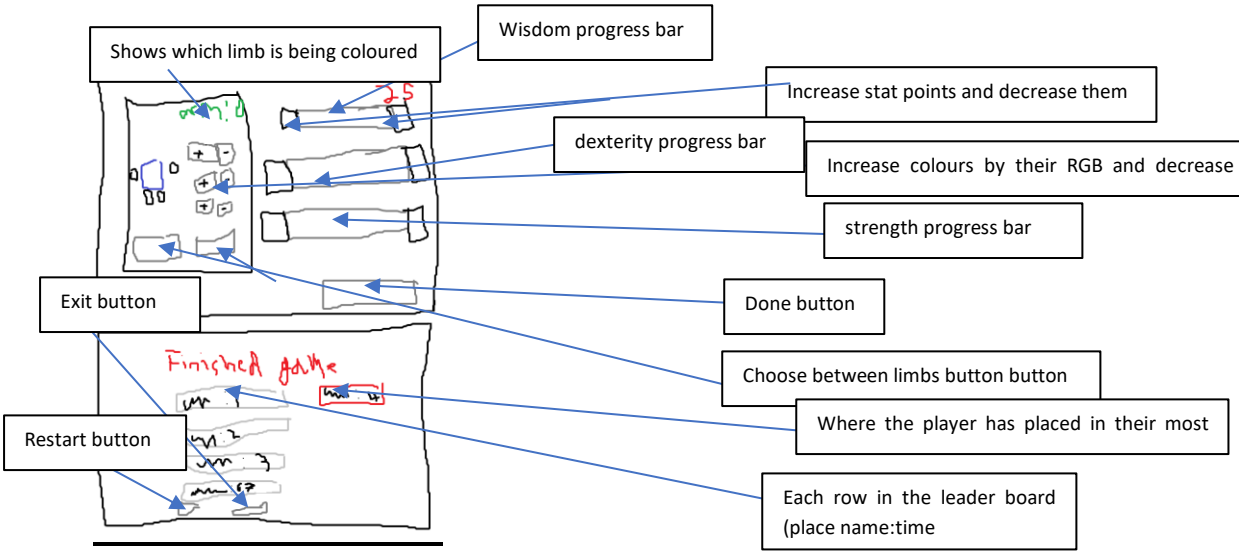
START AND FINISH SCREENS:



The start screen is the window with RPG game on it and is first seen when entering the game, it allows the player to go right in, load a previous save from a text file or look at the instructions of the game.

The finish screen has the title of finished the game and is first screen when the final boss is killed and the player eye is equipped. The player can enter their name and submit it so it can be ranked against other times they achieved

CHARACTER CREATOR AND FINISH SCREEN:



The top image is that of the character creator, allowing to customise the character as the player pleases and customize the stat points in each attribute. This makes each run of the game more unique

The bottom image shows the leader board at the end of the game, giving the player to see their time ranked against other times they have achieved creating progression. It also allows for the game to be restarted creating the game loop.

TECHNICAL SOLUTION:

BARS_CLASS:

```
Public Class bar
    Inherits base_block

    Private fullness As Double = 1
    Public topbox As New PictureBox

    Public bottom_col As Color
    Public top_col As Color

    Public border As Integer

    Public stat_text As text

    Public txt_alive As Boolean = False
    Public txt_size As Integer = -1

    Public bar_alive As Boolean = False

    Public Sub New(ByVal game As Form, ByVal width As Integer, ByVal height As Integer, ByVal x As
Integer, ByVal y As Integer, ByVal bottom_cl As Color, ByVal top_cl As Color, Optional ByVal border
As Integer = -1, Optional ByVal txt As Boolean = False, Optional ByVal txt_size As Integer = -1)
        MyBase.New(game, height, width, x, y)


        Me.bottom_col = bottom_cl
        Me.top_col = top_cl

        Me.box.BackColor = Me.bottom_col
        Me.box.Left = x
        Me.box.Top = y

        Me.border = border
        Me.txt_alive = txt
        Me.txt_size = txt_size
    End Sub

    Public Sub load_all(Optional ByVal num As Integer = 1)
        'create the picture boxes, load them in to the game
        Me.box.Size = New Size(Me.width, Me.height)

        Me.box.Top = Me.y
        Me.box.Left = Me.x

        If Me.txt_size <> -1 Then
            Me.stat_text = New text(Me.game, Me.x, Me.y + (Me.height - Me.width / 12), Me.txt_size,
"", Color.Black)
        Else
            Me.stat_text = New text(Me.game, Me.x, Me.y + (Me.height - Me.width / 12), Me.width /
12, "", Color.Black)
        End If
        '------------------------------------------------

        'if a border is wanted (border=1) it will make the back box larger
        If Me.border = -1 Then
            Me.topbox.Size = New Size(Me.width, Me.height)
            Me.topbox.Top = Me.y
            Me.topbox.Left = Me.x
        Else
            Me.topbox.Size = New Size(Me.width - (Me.width * 0.16), Me.height - (Me.height * 0.4))
            Me.topbox.Top = Me.y + (Me.height * 0.2)
            Me.topbox.Left = Me.x + (Me.width * 0.08)
        End If
```

```
        '-------------------------------------------------------------
        Me.box.BringToFront()
        Me.topbox.BringToFront()

        If Me.txt_alive Then
            Me.stat_text.box.BackColor = Me.top_col
            Me.stat_text.load()
            Me.stat_text.box.Text = num
        End If

        Me.topbox.BackColor = top_col

        Me.game.Controls.Add(Me.topbox)
        Me.game.Controls.Add(Me.box)


    End Sub

    Public Sub destroy()
        ' remove the top picturebox and bottom picturebox from graphical mamory, keeps the object in
memory to be used for later
        Me.game.Controls.Remove(Me.topbox)
        Me.game.Controls.Remove(Me.box)
        '-----------------------------------------------

        If Me.txt_alive Then
            Me.stat_text.destroy()
        End If

    End Sub

    Public Sub reload_all(Optional ByVal num As Integer = 1)

        'reloads the object if its just been removed from graphical memory but is still stored in
memory
        Me.game.Controls.Add(Me.box)
        Me.game.Controls.Add(Me.topbox)

        Me.box.BringToFront()
        Me.topbox.BringToFront()

        If Me.txt_alive Then

            Me.stat_text.box.BackColor = Me.top_col
            Me.stat_text.box.Text = num

            Me.stat_text.load()

            Me.stat_text.box.BringToFront()
        End If
        '-------------------------------------------------------------------------------------------

    End Sub

    Public Sub new_size(ByVal num As Double)
        'change the size of the topbox to make it seem as if the bar is reducing
        Me.fullness = num
        Me.topbox.Size = New Size(Me.width * (Me.fullness), Me.height)

        Me.box.BringToFront()
        Me.topbox.BringToFront()

        If Me.txt_alive Then
            If num < 0 Then num = 0
```

```
            'gets rid of long doubles
            Me.stat_text.box.Text = CInt(num * 100) / 100

            Me.stat_text.box.BringToFront()
        End If

        '-------------------------------------------------------------------------
    End Sub

    Public Sub new_loc(ByVal new_x As Integer, ByVal new_y As Integer)

        Me.x = new_x
        Me.y = new_y

        Me.stat_text.box.Location = New Point(new_x, new_y + 20)

        Me.box.Location = New Point(new_x, new_y)
        Me.topbox.Location = New Point(new_x, new_y)

    End Sub
End Class
```

BASE_ATTACK:

```
Public Class base_attack
    Inherits base_block

    Public life_time As Integer = 0
    Public spawn_time As Integer = 0

    Private _typeofattack As String
    Public ReadOnly Property typeofattack
        Get
            Return _typeofattack
        End Get
    End Property

    Public base_block_obj As base_block

    Public dead As Boolean = False

    Private _damage As Integer = 1
    Public Property damage As Integer
        Get
            Return _damage
        End Get
        Set(ByVal value As Integer)
            _damage = value
        End Set
    End Property

    Public Sub New(ByVal game As Form, ByVal height As Integer, ByVal width As Integer, ByVal x As
Integer, ByVal y As Integer, ByVal life_time As Integer, ByVal typo As String, ByVal originalimage
As Bitmap, ByVal dmg As Integer, Optional ByVal angle As Double = 0)
        MyBase.New(game, height, width, x, y)

        Me.life_time = life_time

        Me._typeofattack = typo
        Me.damage = dmg

        Me.rotate_img(Me.box, originalimage, angle)

        Me.box.SizeMode = PictureBoxSizeMode.StretchImage

    End Sub

    Public Sub rotate_img(ByVal pic As PictureBox, ByVal original_image As Image, ByVal
angle_radians As Double)
        'gets the angle from the sent position to the target
        Dim angle_degrees As Double = angle_radians * (180 / Math.PI)

        'creates a bitmap of the dimensions of the original image
        Dim rotated_bmp As New Bitmap(original_image.Width, original_image.Height)

        'rotates the image by the angle specified
        Using g As Graphics = Graphics.FromImage(rotated_bmp)

            g.TranslateTransform(CSng(original_image.Width / 2), CSng(original_image.Height / 2))
            g.RotateTransform(CSng(angle_degrees))

            g.TranslateTransform(-CSng(original_image.Width / 2), -CSng(original_image.Height / 2))
            g.DrawImage(original_image, New Point(0, 0))

        End Using
```

```
        '--------------------------------------

        'applies image to the bitmap so it can be shown on screen
        pic.Image = rotated_bmp
    End Sub

    Function direction_to(ByVal mousex As Integer, ByVal mousey As Integer, ByVal entity As
base_block, ByVal speed As Integer)
        ' if the attack is moving towards the player(like firebal) it uses this

        'find the change in x and change in y
        Dim dx As Double = mousex - entity.x
        Dim dy As Double = mousey - entity.y
        Dim dist As Double = Math.Sqrt(dx ^ 2 + dy ^ 2)

        Dim end_coords(2) As Double

        ' Avoid division by zero
        If dx = 0 And dy = 0 Then
            end_coords(0) = 0
            end_coords(1) = 0
            Return end_coords

        ElseIf dx = 0 Then
            end_coords(0) = 0
            end_coords(1) = (dy / dist) * speed
            Return end_coords

        ElseIf dy = 0 Then
            end_coords(1) = 0
            end_coords(0) = (dx / dist) * speed
            Return end_coords
        Else
            end_coords(0) = (dx / dist) * speed
            end_coords(1) = (dy / dist) * speed
            Return end_coords

        End If

        'normalise and multiply by the speed

        '-----------------------------------------------------------------------
        Return end_coords
    End Function

    Public Sub move(ByVal mousex As Integer, ByVal mousey As Integer, ByVal entity As base_block,
Optional ByVal speed As Integer = 8)

        'chooses if an attack can move
        If Me.typeofattack = "fireball" Or Me.typeofattack = "web" Then

            Dim coords As Array = direction_to(mousex, mousey, entity, speed)

            Me.xvel = coords(0)
            Me.yvel = coords(1)

        End If
        '----------------------------
    End Sub

    Function check_alive(ByVal tick As Integer)
        'check if the attacks been alive too long
        If (tick - spawn_time > life_time) Or Me.dead Then
            Return True
        End If
```

```
        '----------------------------------------
        Return False
    End Function

    Sub update()

        'updates the coordinates in memory for usage
        Me.x += Me.xvel
        Me.y += Me.yvel

        'update the picturebox location on the screen
        Me.box.Left = Me.x
        Me.box.Top = Me.y

    End Sub

End Class
```

Base_block:

```
Public Class base_block

    Public box As New PictureBox

    Protected game As Form

    Public x As Integer = 0
    Public y As Integer = 0

    Public xvel As Integer = 0
    Public yvel As Integer = 0

    Public width As Integer
    Public height As Integer

    Private wall_img_pth As String

    Public Sub New(ByVal game As Form, ByVal height As Integer, ByVal width As Integer, ByVal x As
Integer, ByVal y As Integer)
        Me.game = game
        Me.x = x
        Me.y = y

        Me.width = width
        Me.height = height

    End Sub

    Public Overridable Sub load()
        'used by many classes to load the image of the entitiy to the screen
        Me.box.Size = New Size(Me.width, Me.height)

        Me.box.Top = Me.y
        Me.box.Left = Me.x

        Me.game.Controls.Add(Me.box)
        '----------------------------------------------------------------------
    End Sub

End Class
```

BASE_ENEMY:

```
Imports System.Drawing.Drawing2D
Imports System.Drawing

Public MustInherit Class base_enemy
    Inherits base_block

    Protected util As utilities

    Public type_enemy As String
    Public type_magic As Boolean = False

    Public colour As Color

    Protected Const see_radius As Integer = 800
    Protected sees_player As Boolean = False
    Protected see_timer As Integer = 0
    Protected plr1_see As Boolean = False

    Public dead As Boolean = False

    Protected playerx As Integer = 0
    Protected playery As Integer = 0
    Protected enemyx As Integer = 0
    Protected enemyy As Integer = 0

    Protected max_hp As Integer = 2
    Public hp As Integer = 2

    Public attack_dict As New Dictionary(Of String, List(Of String))
    Public live_attacks As New List(Of base_attack)
    Public dead_attacks As New List(Of base_attack)
    Public cubes As New List(Of base_block)

    Protected angle As Double = 0

    Protected leftLeg As New PictureBox()
    Protected rightLeg As New PictureBox()
    Protected leftArm As New PictureBox()
    Protected rightArm As New PictureBox()

    Public eye As New PictureBox()
    Public eyebox As New PictureBox()

    Protected legWidth As Integer = width \ 3
    Protected legHeight As Integer = height \ 4
    Protected armWidth As Integer = width \ 4
    Protected armHeight As Integer = height \ 3

    Public all_limbs As New List(Of PictureBox)
    Public leg_move As String = "R"

    Protected _id As Integer
    Public ReadOnly Property id As Integer
        Get
            Return _id
        End Get
    End Property

    Public raycast_obj As ray_cast
    Public rand As New Random()
```

```
    Sub New(ByVal game As Form, ByVal height As Integer, ByVal width As Integer, ByVal x As Integer,
ByVal y As Integer, ByVal id As Integer, ByVal hp As Integer, ByVal utility As utilities, Optional
ByVal magic As Boolean = False)
        MyBase.New(game, height, width, x, y)
        Me._id = id

        Me.max_hp = hp
        Me.hp = hp


        Me.type_magic = magic
        Me.util = utility

    End Sub



    Public Overridable Sub collisions(ByVal wallz As List(Of wall), ByVal plr As player, ByVal tick
As Integer)
        'wall collisions, push back so they dont get stuck, stop movement, re-checks if they can see
the player
        For Each wal In wallz
            If wal.box.Bounds.IntersectsWith(Me.box.Bounds) Then
                ' Calculate enemy and wall boundaries
                Dim enemy_left As Integer = Me.x
                Dim enemy_right As Integer = Me.x + Me.width
                Dim enemy_top As Integer = Me.y
                Dim enemy_bottom As Integer = Me.y + Me.height

                Dim wall_left As Integer = wal.x
                Dim wall_right As Integer = wal.x + wal.width
                Dim wall_top As Integer = wal.y
                Dim wall_bottom As Integer = wal.y + wal.height

                ' Determine overlap distances
                Dim overlap_left As Integer = enemy_right - wall_left
                Dim overlap_right As Integer = wall_right - enemy_left
                Dim overlap_top As Integer = enemy_bottom - wall_top
                Dim overlap_bottom As Integer = wall_bottom - enemy_top


                Dim min_overlap As Integer = Math.Min(Math.Min(overlap_left, overlap_right),
Math.Min(overlap_top, overlap_bottom))

                ' Push the enemy out depending on which side is smallest
                If min_overlap = overlap_left Then

                    Me.x -= overlap_left
                    Me.xvel = 0
                ElseIf min_overlap = overlap_right Then

                    Me.x += overlap_right
                    Me.xvel = 0
                ElseIf min_overlap = overlap_top Then

                    Me.y -= overlap_top
                    Me.yvel = 0
                ElseIf min_overlap = overlap_bottom Then

                    Me.y += overlap_bottom
                    Me.yvel = 0
                End If
                Me.see_timer = 0
            End If
```

```
        Next

        '-------------------------------------------------------------------------------------------
-------------

        'check collisions with attacks from the player
        For Each i In plr.live_attack

            If i.box.Bounds.IntersectsWith(Me.box.Bounds) Then

                Me.box.BackColor = Color.LightGray
                Me.hp -= i.damage

                If plr.attack_dict(i.typeofattack)(3) = 1 Then
                    i.dead = True

                End If

            End If

        Next
        '-----------------------------------------------

    End Sub

    Private Sub move_to_plr(ByVal plr As player, ByVal speed As Integer)
        'if the player can be seen move to them
        Dim dx As Integer = plr.x - Me.x
        Dim dy As Integer = plr.y - Me.y
        '-------------------------------------

        ' find distance
        Dim distance As Double = Math.Sqrt(dx * dx + dy * dy)
        If distance = 0 Then Exit Sub
        '-------------

        'get direction needed then multiply by speed
        Dim stepX As Integer = CInt((dx / distance) * speed)
        Dim stepY As Integer = CInt((dy / distance) * speed)
        '-----------------------------------------

        Me.x += stepX
        Me.y += stepY
    End Sub

    Protected Friend Sub limb_load()
        'load the limbs of the enemy
        'give the limb picturebox the desired attributes of each limb
        With Me.leftLeg

            .Width = legWidth
            .Height = legHeight

            .Left = x
            .Top = y + height

            .BackColor = Color.Black
        End With

        With Me.rightLeg

            .Width = legWidth
            .Height = legHeight

            .Left = x + (width - legWidth)
```

```
    .Top = y + height

    .BackColor = Color.Black
End With

With Me.leftArm

    .Width = armWidth
    .Height = armHeight

    .Left = x - armWidth
    .Top = y + armHeight \ 3

    .BackColor = Color.Black
End With

With Me.rightArm

    .Width = armWidth
    .Height = armHeight

    .Left = x + armWidth
    .Top = y + armHeight \ 3

    .BackColor = Color.Black
End With


'load in the eye image
With eye

    .Width = 12
    .Height = 12

    .Left = x + width \ 2 - .Width \ 2
    .Top = y + height \ 4 - .Height \ 2

    'draw eyeball using graphics, paint to a picturebox
    Dim graphic As New GraphicsPath()

    graphic.AddEllipse(0, 0, .Width, .Height)

    .Region = New Region(graphic)
    '------------------------------------------------

    .BackColor = Color.Black

End With
'------------------------

'load in the eyebox
With eyebox

    .Width = 20
    .Height = 20

    .Left = x + width \ 2 - .Width \ 2
    .Top = y + height \ 4 - .Height \ 2

    .BackColor = Color.White

End With
'---------------------

all_limbs.Add(eye)
```

```
        all_limbs.Add(eyebox)

        If Me.type_enemy <> "spider" And Me.type_enemy <> "invisi" Then

            all_limbs.Add(leftLeg)
            all_limbs.Add(rightLeg)

            all_limbs.Add(leftArm)
            all_limbs.Add(rightArm)

            game.Controls.Add(leftLeg)
            game.Controls.Add(rightLeg)

            game.Controls.Add(leftArm)
            game.Controls.Add(rightArm)
        End If

        game.Controls.Add(eyebox)
        game.Controls.Add(eye)

        eyebox.BringToFront()
        eye.BringToFront()

    End Sub

    Protected Sub update_eye()

        'lets eye and eyebox follow the enemy x and y , to make it seem that it is on it/connected
        Dim centerx As Integer = Me.x + 5 + (Me.width / 2)
        Dim centery As Integer = Me.y + 2 + (Me.height / 2)

        eye.Left = centerx - eye.Width
        eye.Top = centery - eye.Height

        eyebox.Left = Me.x + Me.width \ 2 - eyebox.Width \ 2
        eyebox.Top = (Me.y + Me.height \ 4 - eyebox.Height \ 2) + 10

    End Sub

    Protected MustOverride Sub update_arms(ByVal plr1 As player)

    Protected Sub update_legs(ByVal tick As Integer)

        'move the left leg , then right leg, creates a timer between both to simulate walking
        If tick Mod 2 = 0 Then

            If leg_move = "R" Then

                rightLeg.Left = x + (width - legWidth)
                rightLeg.Top = y + height

                Me.leg_move = "L"

            ElseIf leg_move = "L" Then

                leftLeg.Left = x
                leftLeg.Top = y + height

                Me.leg_move = "R"

            End If
        End If
        '-------------------------------------------------------------------------------
    End Sub
```

```
    Public Sub can_see_move(ByVal plr1 As player, ByVal tick As Integer, ByVal map As List(Of
String))

        'turn the player and enemy coordinates into a 1d array [x, y] so the ray caster can work on
it
        Dim plr_coords(2) As Integer
        Dim my_coords(2) As Integer

        plr_coords(0) = plr1.x
        plr_coords(1) = plr1.y

        my_coords(0) = Me.x
        my_coords(1) = Me.y
        '-----------------------------------------------------------------------------------------
---

        'checks if the enemy can see the player
        If check_in_range(plr1) And Not Me.raycast_obj.ray_blocked(plr_coords, my_coords) Then
            'change the enemies colour when they see the player

            Me.plr1_see = True
        Else
            Me.plr1_see = False

        End If
        '---------------------------------------


        'if the player can be seen a counter it set to 20 allowing the enemy to follow the player
        If Me.plr1_see Then
            Me.see_timer = 20

        ElseIf Me.see_timer > 0 And Me.plr1_see = False Then
            'change the enmies colour depending on how long they can see the player

            Me.see_timer -= 1

        Else
            Me.see_timer = 0

        End If
        '-----------------------------------------------------------------------------------------

        'if the counter is checked to be lsrger than zero, if so, the enemy goes towards them
        If Me.see_timer > 0 Then
            move_to_plr(plr1, 2)

        End If
        '-----------------------------------------------------------------------------------------

    End Sub

    Public Function check_in_range(ByVal plr1)
        'checks if the player is in a set radius
        If Math.Sqrt((Me.x - plr1.X) * (Me.x - plr1.X) + (Me.y - plr1.Y) * (Me.y - plr1.Y)) <
base_enemy.see_radius / 2 Then

            Return True

        End If
        '---------------------------------------
        Return False
    End Function
```

```
    Public Sub move_limbs(ByVal tick As Integer, ByVal plr1 As player)

        Me.update_legs(tick)
        Me.update_arms(plr1)
        Me.update_eye()

    End Sub

    Public Sub change_col(ByVal tick)
        'change colour back to normal after being hit
        If tick Mod 6 = 0 Then

            If Me.box.BackColor = Color.LightGray Then
                Me.box.BackColor = Me.colour

            End If

        End If
        '-----------------------------------------------------------------------
    End Sub

    Protected Friend Sub load_attacks_fromtxt(Optional ByVal which_atks As String = "plren")

        'load the atributes for each attack into a dictionary
        Me.attack_dict = Me.util.load_attacks(which_atks)
        '-------------------------------------------------

    End Sub



    Protected Friend Overridable Sub update(ByVal all_objects As List(Of wall), ByVal plr1 As
player, ByVal enemies As List(Of base_enemy), ByVal map As List(Of String), ByVal tick As Integer)

        move_limbs(tick, plr1)
        can_see_move(plr1, tick, map)
        change_col(tick)
        collisions(all_objects, plr1, tick)
        Me.x += Me.xvel
        Me.y += Me.yvel
        Me.box.Left = Me.x
        Me.box.Top = Me.y
    End Sub
End Class
```

BOSS_CLASS:

```
Public Class boss
    Inherits base_enemy
    '[angle as double] used throughout the code for movement, inherited from base_enemy

    'spider boss:
    Protected Friend thighs As New List(Of List(Of PictureBox))
    Protected Friend femur As New List(Of List(Of PictureBox))
    Protected Friend left As Boolean = False

    Protected Friend next_xer As New List(Of Double)
    Protected Friend next_yer As New List(Of Double)

    Public radian As Double = 0

    'invisi boss:
    Public point As New List(Of Double)
```

```
    Public atk_point As New List(Of Double)

    Public body_parts As New List(Of base_block)
    Public invisible As Boolean = False

    Private plr1 As player

    Public Sub New(ByVal game As Form, ByVal x As Integer, ByVal y As Integer, ByVal width As
Integer, ByVal height As Integer, ByVal type_of As String, ByVal maxhp As Integer, ByVal plr As
player, ByVal utility As utilities, Optional ByVal mgic As Boolean = True)
        MyBase.new(game, height, width, x, y, 0, maxhp, utility, mgic)


        Me.plr1 = plr

        Me.type_enemy = type_of

        'change the body depending on the type of boss
        If type_of = "spider" Then

            Me.box.BackColor = Color.Black
            Me.colour = Color.Black

        ElseIf type_of = "invisi" Then
            Me.box.BackColor = Color.White
            Me.colour = Color.Transparent

        End If
        '-------------------------------------------
    End Sub


    Public Sub update_spider(ByVal tick As Integer)


        'the spider legs attach themselves to an anchor point then every few ticks the hav a new
anchor closer to the main body set, simulating crawling

        'moves the upper leg with the spider body, as if its connected to it
        For i = 0 To 5

            angle = i * 60
            Dim rado As Double = angle * Math.PI / 180


            For j = 0 To 3

                Dim xero As Integer = (Me.x + 10) + (j + 3) * 10 * Math.Cos(rado)
                Dim yero As Integer = (Me.y + 10) + (j + 3) * 10 * Math.Sin(rado)

                Me.thighs(i)(j).Left = xero
                Me.thighs(i)(j).Top = yero
            Next
        Next
        '-----------------------------------------------------------------------

        'empties the next_list of y and x coords for new ones
        For i = 0 To 5
            Me.next_xer.Add(0)
            Me.next_yer.Add(0)
        Next
        '-------------------------------------------------------------

        'every 8 ticks either the left lower legs or right lower create new anchors to go to
        If tick Mod 8 = 0 Then
```

```
            If Not left Then

                For i = 0 To 2
                    angle = i * 60
                    radian = angle * Math.PI / 180

                    Me.next_xer(i) = (Me.x + 8) + 100 * Math.Cos(radian)
                    Me.next_yer(i) = (Me.y + 8) + 100 * Math.Sin(radian)
                Next

                Me.left = True

            Else

                For i = 3 To 5
                    angle = i * 60
                    radian = angle * Math.PI / 180

                    Me.next_xer(i) = (Me.x + 8) + 100 * Math.Cos(radian)
                    Me.next_yer(i) = (Me.y + 8) + 100 * Math.Sin(radian)
                Next

                Me.left = False

            End If

        End If
        '-------------------------------------------------------------------------------------------
--------------

        'use the new anchor created to simulate moving, if new anchors havent been made stretch the
lower leg fom body so the legs still looks alive and not still
        For i = 0 To 5

            angle = i * 60
            radian = angle * Math.PI / 180

            Dim xer As Double = (Me.x + 8) + 60 * Math.Cos(radian)
            Dim yer As Double = (Me.y + 8) + 60 * Math.Sin(radian)

            For j = 0 To 3

                Dim distx As Double = xer - Me.next_xer(i)
                Dim disty As Double = yer - Me.next_yer(i)

                Me.femur(i)(j).Left = Me.next_xer(i) + (distx / (j + 1))
                Me.femur(i)(j).Top = Me.next_yer(i) + (disty / (j + 1))

            Next

        Next
        '-------------------------------------------------------------------------------------------
-------------------------------------------------------------

        eyebox.BackColor = Color.Purple
    End Sub

    Protected Overrides Sub update_arms(ByVal plr1 As player)
        'empty sub as it inherits base enemy where all other classes use the UpdateArms sub
    End Sub

    Public Sub spider_atks(ByVal plr1 As player, ByVal tick As Integer)
        'shoot webs towards the player and create poison splotges to slow the player and protect
from ranged attacks
```

```
        'checks the range from the player
        Dim dist = (Me.x - plr1.x) ^ 2 + (Me.y - plr1.y) ^ 2

        'use these coordinates in the raycasting to make the sure the player can be seen
        Dim plr_coords(2) As Integer
        Dim my_coords(2) As Integer

        plr_coords(0) = plr1.x
        plr_coords(1) = plr1.y

        my_coords(0) = Me.x
        my_coords(1) = Me.y

        'create the spider web attack, thet moves towards the player

        If tick Mod 12 = 0 And Not Me.raycast_obj.ray_blocked(my_coords, plr_coords) Then

            Dim dx = Me.x - plr1.x
            Dim dy = Me.y - plr1.y

            Me.angle = Math.Atan2(dy, dx)

            Dim new_attack = New base_attack(Me.game, 25, 25, Me.x, Me.y, 40, "web", New
Bitmap(Me.attack_dict("web")(2)), Me.attack_dict("web")(1), angle)

            new_attack.spawn_time = tick
            new_attack.load()
            new_attack.move(plr1.x, plr1.y, Me)
            new_attack.box.BringToFront()

            Me.live_attacks.Add(new_attack)

        End If
        '------------------------------------------------------------

        'check if the player is in range and make the attack happen every 9 ticks

        If dist < 100 ^ 4 And tick Mod 9 = 0 Then

            Dim new_attack = New base_attack(Me.game, 25 + Me.util.Rand0(1, 100), 25 +
Me.util.Rand0(1, 100), (Me.x - 100) + Me.util.Rand0(1, 100) * 2, (Me.y - 100) + Me.util.Rand0(1,
100) * 2, 30, "stickyweb", New Bitmap(Me.attack_dict("stickyweb")(2)),
Me.attack_dict("stickyweb")(1), 0)

            new_attack.spawn_time = tick
            new_attack.load()

            Me.live_attacks.Add(new_attack)


        End If

        '------------------------------------------------------------------------

    End Sub

    Public Sub load_legs()
        'add upper leg peices to the spider
        For i = 0 To 5

            Dim new_list As New List(Of PictureBox)

            For j = 0 To 4
```

```
                Dim new_add As New PictureBox

                AddHandler new_add.MouseClick, AddressOf Me.plr1.click_handler

                new_add.Width = 15
                new_add.Height = 15
                new_add.BackColor = Color.Black



                new_list.Add(new_add)
                new_add.BringToFront()
                Me.game.Controls.Add(new_add)

            Next

            Me.thighs.Add(new_list)
        Next
        '-----------------------------------

        'add lower leg peices to the spider
        For i = 0 To 5

            Dim new_list As New List(Of PictureBox)

            For j = 0 To 3

                Dim new_add As New PictureBox

                new_add.Width = 10
                new_add.Height = 10
                new_add.BackColor = Color.Black

                new_list.Add(new_add)
                Me.game.Controls.Add(new_add)
                new_add.BringToFront()

            Next

            Me.femur.Add(new_list)
        Next
        '-----------------------------------
    End Sub


    Public Sub invisi_atks(ByVal plr1 As player, ByVal tick As Integer)

        Dim plr_coords(2) As Integer
        Dim my_coords(2) As Integer

        plr_coords(0) = plr1.x + 25
        plr_coords(1) = plr1.y + 25

        my_coords(0) = Me.x + 25
        my_coords(1) = Me.y + 25

        playerx = plr1.x
        enemyx = Me.x

        playery = plr1.y
        enemyy = Me.y


        If Not Me.invisible And Not Me.raycast_obj.ray_blocked(plr_coords, my_coords) And tick Mod
20 = 0 Then
```

```
            Dim dy = enemyy - playery
            Dim dx = enemyx - playerx

            If dx = 0 Then dx = 1


            'make a line of attacks (purple orbs) towards the player

            'change the math is the player is in front of or behing the enemy
            If playerx >= enemyx Then
                'player in front
                For new_x = enemyx To playerx Step 20

                    Dim new_y As Integer = (dy / dx) * (new_x - x) + enemyy

                    new_y += (-25 + Me.util.Rand0(1, 50))

                    Dim new_attack = New base_attack(Me.game, 20, 20, new_x, new_y, 30,
"invisi_ball", New Bitmap(Me.attack_dict("invisi_ball")(2)), Me.attack_dict("invisi_ball")(1), 0)

                    new_attack.load()
                    new_attack.spawn_time = tick

                    Me.live_attacks.Add(new_attack)

                Next

            Else
                'player behind
                For new_x = playerx To enemyx Step 20

                    Dim new_y As Integer = (dy / dx) * (new_x - x) + enemyy

                    new_y += (-25 + Me.util.Rand0(1, 50))

                    Dim new_attack = New base_attack(Me.game, 20, 20, new_x, new_y, 30,
"invisi_ball", New Bitmap(Me.attack_dict("invisi_ball")(2)), Me.attack_dict("invisi_ball")(1), 0)

                    new_attack.load()
                    new_attack.spawn_time = tick

                    Me.live_attacks.Add(new_attack)

                Next

            End If
            '-----------------------------------------------------------------

            'if they hit the player, it teleports the player into the cloud of cubes causing damage
        End If
    End Sub

    Public Sub update_cubes_bodyparts(ByVal tick As Integer)
        'the white cubes that make the circular body follow the main body, so they seem as one
        For i = 0 To 12
            angle = i * (360 / 12)
            Dim rado As Double = angle * Math.PI / 180

            Dim tox = Me.x + 11 + 35 * Math.Cos(rado)
            Dim toy = Me.y + 11 + 35 * Math.Sin(rado)

            Me.body_parts(i).x = tox
            Me.body_parts(i).y = toy
```

```
            Me.body_parts(i).box.Left = Me.body_parts(i).x
            Me.body_parts(i).box.Top = Me.body_parts(i).y

            Me.body_parts(i).box.BringToFront()
        Next
        '-------------------------------------------------------------------------------------



        'every 10 ticks, the cubes choose a new point to go to
        If tick Mod 10 = 0 Then

            'new angles to go towards
            Me.point.Clear()

            'create random angles for the floating cubes to go to, new angles each time
            For i = 0 To 20
                angle = i * 18
                Dim rado As Double = angle * Math.PI / 180

                Me.point.Add(rado)

            Next
            '-----------------------------------------------------

            For i = 0 To 20
                Dim rand_int As Integer = rand.Next(Me.point.Count())
                Dim rand_pos As Double = Me.point(rand_int)

                Dim to_x = Me.x + 25 + 160 * Math.Cos(rand_pos)
                Dim to_y = Me.y + 25 + 160 * Math.Sin(rand_pos)
                Dim now_x = Me.cubes(i).x
                Dim now_y = Me.cubes(i).y

                Me.cubes(i).xvel = (to_x - now_x) / 25
                Me.cubes(i).yvel = (to_y - now_y) / 25
            Next
        End If
        '-----------------------------------------------------

        'make it seem as if the cubes are floating towards a point not instantly appearing there
        For i = 0 To 20
            Me.cubes(i).x += Me.cubes(i).xvel
            Me.cubes(i).y += Me.cubes(i).yvel

            Me.cubes(i).box.Left = Me.cubes(i).x
            Me.cubes(i).box.Top = Me.cubes(i).y
            Me.cubes(i).box.BringToFront()
        Next
        '-------------------------------------------------------------------------------------------

    End Sub

    Public Sub load_cubes_bodyparts()
        'loads in orbiting cubes around the invisible boss

        'loads in the cubes-------------------------------------------------------------
        For i = 0 To 20
            Dim defense_or_ofense_cubes As Integer = 0
            Dim new_add = New base_block(Me.game, 10, 10, 500, 500)

            If Me.util.Rand0(1, 100) >= 50 Then
                defense_or_ofense_cubes = 1
            End If
```

```
            AddHandler new_add.box.MouseClick, AddressOf Me.plr1.click_handler

            If defense_or_ofense_cubes = 1 Then
                new_add.box.BackColor = Color.Red
            Else
                new_add.box.BackColor = Color.Black
            End If

            new_add.load()

            Me.cubes.Add(new_add)

        Next
        '-------------------------------------------------------------------------------

        'gives starting x and y of each cube
        For i = 0 To 20
            angle = i * 18
            Dim rado As Double = angle * Math.PI / 180

            Me.point.Add(rado)

            Me.cubes(i).x = Me.x + 25 + 120 * Math.Cos(rado)
            Me.cubes(i).y = Me.y + 25 + 120 * Math.Sin(rado)

        Next
        '-------------------------------------
        '-----------------------------------------------

        'add white cubes around the body to make it look circle like
        For j = 0 To 12
            Dim new_add As New base_block(Me.game, 25, 25, Me.x, Me.y)

            new_add.box.BackColor = Color.White
            new_add.load()

            Me.body_parts.Add(new_add)

        Next
        '-------------------------------------------------------------

    End Sub

    Public Sub update_invisi(ByVal tick As Integer)
        'updates the insibible boss so they can interchange between invisible and visible
        If Not Me.invisible Then

            Me.box.BackColor = Color.White
            Me.eyebox.BackColor = Color.DarkRed
            Me.eye.BackColor = Color.Black

            For Each i In Me.body_parts
                i.box.BackColor = Color.White
            Next

        ElseIf Me.invisible Then

            Me.box.BackColor = Color.Transparent
            Me.eyebox.BackColor = Color.Transparent
            Me.eye.BackColor = Color.Transparent

            For Each i In Me.body_parts
                i.box.BackColor = Color.Transparent
            Next
```

```
        End If
        '-------------------------------------------------------------------------------

        'sets a timer so the boss flips between invisible and visible
        If tick Mod 60 = 0 Then

            Me.invisible = Not Me.invisible

        End If
        '-----------------------------------------------------------

    End Sub


    Public Overrides Sub collisions(ByVal wallz As List(Of wall), ByVal plr As player, ByVal tick As
Integer)
        MyBase.collisions(wallz, plr, tick)

        'handles blocking player attacks using special skills

        For Each i In plr.live_attack

            If Me.type_enemy = "spider" Then

                For Each j In Me.live_attacks
                    If j.typeofattack = "stickyweb" And i.box.Bounds.IntersectsWith(j.box.Bounds)
Then
                        If Me.attack_dict(i.typeofattack)(3) = 1 Then
                            i.dead = True
                        End If
                    End If
                Next

                If i.box.Bounds.IntersectsWith(Me.box.Bounds) Then
                    'changes colour to show you have hit
                    Me.box.BackColor = Color.LightGray
                    Me.hp -= i.damage

                    Me.plr1.my_interface.boss_bar.new_size(Me.hp / Me.max_hp)

                End If

            ElseIf Me.type_enemy = "invisi" Then
                'checks if cubes orbiting invisible boss have blocked an attack
                For Each j In Me.cubes
                    If i.box.Bounds.IntersectsWith(j.box.Bounds) Then
                        i.dead = True
                    End If
                Next

                If i.box.Bounds.IntersectsWith(Me.box.Bounds) Then
                    'changes colour to show you have hit
                    Me.box.BackColor = Color.LightGray
                    Me.hp -= i.damage

                    Me.plr1.my_interface.boss_bar.new_size(Me.hp / Me.max_hp)

                End If

            End If

        Next
        '-----------------------------------------------------
```

```
    End Sub

    Public Sub update_attacks(ByVal Timer)
        'checks if an attack needs to be destroed as it has been alive too long
        For i = 0 To Me.live_attacks.Count() - 1

            If Me.live_attacks(i).check_alive(Timer) Or Me.live_attacks(i).dead Then

                dead_attacks.Add(live_attacks(i))
                live_attacks.RemoveAt(i)

                'only removes one attack per frame so they dont all dissapear at once making it seem
laggy and unatural
                Exit For

            End If

        Next
        '--------------------------------------------------------------------------
    End Sub

    Public Overrides Sub load()
        'use the load in the inherited class
        MyBase.load()

        If Me.type_enemy = "spider" Then
            Me.load_legs()

        ElseIf Me.type_enemy = "invisi" Then
            Me.load_cubes_bodyparts()

        End If


    End Sub

    Protected Friend Overrides Sub update(ByVal all_objects As List(Of wall), ByVal plr1 As player,
ByVal enemies As List(Of base_enemy), ByVal map As List(Of String), ByVal tick As Integer)

        If Me.type_enemy = "spider" Then

            Me.update_spider(tick)

            Me.spider_atks(plr1, tick)

        ElseIf Me.type_enemy = "invisi" Then

            Me.update_cubes_bodyparts(tick)

            Me.update_invisi(tick)

            Me.invisi_atks(plr1, tick)

        End If

        Me.update_attacks(tick)

        MyBase.update(all_objects, plr1, enemies, map, tick)

    End Sub

End Class
```

BUTTONO_CLASS:

```
Public Class buttono
    Private game As Form
    Public mybutton As New Button()

    Public x As Integer
    Public y As Integer

    Public width As Integer
    Public height As Integer

    'text in the button and wether it has been clicked
    Protected Friend text As String
    Protected Friend been_clicked As Boolean = False

    Public Sub New(ByVal game, ByVal x, ByVal y, ByVal width, ByVal height, ByVal txt)

        Me.x = x
        Me.y = y

        Me.width = width
        Me.height = height

        Me.text = txt
        Me.game = game


    End Sub

    Public Sub destroy()
        Me.game.Controls.Remove(Me.mybutton)
    End Sub

    Public Sub Me_OnClick(ByVal sender As Object, ByVal e As EventArgs)
        'uses functionality to check for clicks in the button picturebox
        Me.been_clicked = True
    End Sub

    Protected Friend Sub load()
        'adds the text to the button
        mybutton.Text = Me.text
        mybutton.Font = New Font("Microsoft Sans Serif", 20)

        mybutton.Size = New Size(Me.width, Me.height)
        mybutton.Location = New Point(Me.x, Me.y)

        ' Add to the form
        Me.game.Controls.Add(mybutton)
    End Sub

End Class
```

CHARACTER_CREATOR:

```
Public Class character_creator

    Public game As Form
    Private plr1 As player

    Private str_bar As bar
    Private str_txt As text

    Private dex_bar As bar
    Private dex_txt As text
    Private wis_bar As bar
    Private wis_txt As text

    Private bar_list As New List(Of bar)
    Private txt_list As New List(Of text)
    Private btn_list As New List(Of List(Of buttono))
    Private image_list As New List(Of PictureBox)
    Private lone_btn_list As New List(Of buttono)

    Private _donebtn As buttono
    Public Property donebtn As buttono
        Get
            Return _donebtn
        End Get
        Set(ByVal value As buttono)
            _donebtn = value
        End Set
    End Property

    Private next_limb As buttono
    Private bef_limb As buttono

    Private attribute_pnts As Integer = 25
    Private attri_txt As text

    'dictionary of each limb and wach rgb value that each one has
    Private colour_limbs As New Dictionary(Of Integer, List(Of Integer))

    Private color_btns As New List(Of buttono)
    Private rgb_vals As New List(Of String)
    Private limb_on As Integer = 0
    Private limb_on_txt As text

    Public created As Boolean = False

    Public ended_char_crea As Boolean = False

    Sub New(ByVal plr As player, ByVal mygame As Form)

        Me.game = mygame
        Me.plr1 = plr

    End Sub

    Private Sub make_bars()

        str_bar = New bar(Me.game, 200, 40, 600, 500, Color.DarkGray, Color.Black)
        dex_bar = New bar(Me.game, 200, 40, 600, 300, Color.DarkGray, Color.Black)
        wis_bar = New bar(Me.game, 200, 40, 600, 100, Color.DarkGray, Color.Black)

        str_txt = New text(Me.game, 600, 450, 20, "strength", Color.DarkGray)
        dex_txt = New text(Me.game, 600, 250, 20, "dexterity", Color.DarkGray)
```

```
        wis_txt = New text(Me.game, 600, 50, 20, "wisdom", Color.DarkGray)

        Me.plr1.all_stats(0) = 0
        Me.plr1.all_stats(1) = 0
        Me.plr1.all_stats(2) = 0


        bar_list.Add(str_bar)
        bar_list.Add(dex_bar)
        bar_list.Add(wis_bar)


        txt_list.Add(str_txt)
        txt_list.Add(dex_txt)
        txt_list.Add(wis_txt)

        For Each i In Me.bar_list
            i.load_all()
            i.new_size(0)
        Next

    End Sub

    Private Sub make_buttons()
        'loads in buttons-------------------------
        For butn_num = 0 To bar_list.Count() - 1

            Dim new_btn1 As New buttono(Me.game, bar_list(butn_num).x - 50, bar_list(butn_num).y -
10, 50, 50, "dec")
            Dim new_btn2 As New buttono(Me.game, bar_list(butn_num).x + bar_list(butn_num).width,
bar_list(butn_num).y - 10, 50, 50, "inc")

            Dim newli As New List(Of buttono)

            newli.Add(new_btn1)
            newli.Add(new_btn2)

            btn_list.Add(newli)

        Next

        Me.donebtn = New buttono(Me.game, 600, 600, 200, 50, "done")
        Me.lone_btn_list.Add(Me.donebtn)

        rgb_vals.Add("R")
        rgb_vals.Add("G")
        rgb_vals.Add("B")

        For clrbutn = 0 To 2

            Dim new_btn As New buttono(Me.game, 300, 200 + (clrbutn * 100), 100, 50, "+" &
Me.rgb_vals(clrbutn))
            Me.lone_btn_list.Add(new_btn)
            Me.color_btns.Add(new_btn)

            Dim new_btn1 As New buttono(Me.game, 400, 200 + (clrbutn * 100), 100, 50, "-" &
Me.rgb_vals(clrbutn))
            Me.lone_btn_list.Add(new_btn1)
            Me.color_btns.Add(new_btn1)

        Next

        Me.next_limb = New buttono(Me.game, 340, 480, 150, 50, "Next limb")
        Me.lone_btn_list.Add(Me.next_limb)
```

```
        Me.bef_limb = New buttono(Me.game, 140, 480, 150, 50, "Last limb")
        Me.lone_btn_list.Add(Me.bef_limb)

        For Each btn In Me.btn_list

            btn(0).load()
            btn(0).mybutton.Font = New Font("Microsoft Sans Serif", 10)

            btn(1).load()
            btn(1).mybutton.Font = New Font("Microsoft Sans Serif", 10)

            AddHandler btn(0).mybutton.Click, AddressOf btn(0).Me_OnClick
            AddHandler btn(1).mybutton.Click, AddressOf btn(1).Me_OnClick

        Next

        For Each btn In Me.lone_btn_list

            btn.load()

            AddHandler btn.mybutton.Click, AddressOf btn.Me_OnClick

        Next
        '-------------------------------------------------------
    End Sub

    Private Sub make_txt()

        Me.attri_txt = New text(Me.game, 800, 50, 20, Me.attribute_pnts, Color.Red)
        Me.txt_list.Add(attri_txt)

        Me.limb_on_txt = New text(Me.game, 375, 150, 20, "arms" & Me.limb_on, Color.Green)
        Me.txt_list.Add(limb_on_txt)

        For Each i In Me.txt_list
            i.load()
        Next
    End Sub

    Private Sub make_images()
        'draws a box around the player colour changing area (was told it looks pretty)
        Me.plr1.x = 200
        Me.plr1.y = 300

        Me.plr1.load()
        Me.plr1.limb_load()

        Dim pictbox As New PictureBox

        pictbox.Width = 500
        pictbox.Height = 500

        pictbox.Left = 100
        pictbox.Top = 100

        Dim bitmp As New Bitmap(pictbox.Width, pictbox.Height)
        Dim graphic As Graphics = Graphics.FromImage(bitmp)
        Dim pen As New Pen(Color.Black, 5)

        graphic.DrawLine(pen, 0, 0, 400, 0)
        graphic.DrawLine(pen, 0, 450, 400, 450)
        graphic.DrawLine(pen, 0, 0, 0, 450)
        graphic.DrawLine(pen, 400, 0, 400, 450)

        pictbox.Image = bitmp
```

```
        Me.image_list.Add(pictbox)

        For Each img In Me.image_list
            Me.game.Controls.Add(img)
        Next
        '-------------------------------------------------
    End Sub

    Private Sub colr_limbs()
        'load the starting values for the colour dictionary 1:[0, 0, 0], 2:[0, 0, 0]...
        For ind = 0 To 2
            Dim new_li As New List(Of Integer)

            new_li.Add(0)
            new_li.Add(0)
            new_li.Add(0)

            Me.colour_limbs.Add(ind, new_li)
        Next
        '-----------------------------------------------------------------------------
    End Sub

    Private Sub update_plr_clrs()
        'allows the user to choose wich limbs colour they want to change and only changes that one
        If Me.limb_on = 0 Then

            Me.plr1.leftArm.BackColor = Color.FromArgb(Me.colour_limbs(Me.limb_on)(0),
Me.colour_limbs(Me.limb_on)(1), Me.colour_limbs(Me.limb_on)(2))
            Me.plr1.rightArm.BackColor = Color.FromArgb(Me.colour_limbs(Me.limb_on)(0),
Me.colour_limbs(Me.limb_on)(1), Me.colour_limbs(Me.limb_on)(2))

            Me.limb_on_txt.update("arms: " & Me.limb_on)

        ElseIf Me.limb_on = 1 Then

            Me.plr1.leftLeg.BackColor = Color.FromArgb(Me.colour_limbs(Me.limb_on)(0),
Me.colour_limbs(Me.limb_on)(1), Me.colour_limbs(Me.limb_on)(2))
            Me.plr1.rightLeg.BackColor = Color.FromArgb(Me.colour_limbs(Me.limb_on)(0),
Me.colour_limbs(Me.limb_on)(1), Me.colour_limbs(Me.limb_on)(2))

            Me.limb_on_txt.update("legs: " & Me.limb_on)

        ElseIf Me.limb_on = 2 Then

            Me.plr1.box.BackColor = Color.FromArgb(Me.colour_limbs(Me.limb_on)(0),
Me.colour_limbs(Me.limb_on)(1), Me.colour_limbs(Me.limb_on)(2))

            Me.limb_on_txt.update("body: " & Me.limb_on)

        End If
        '-----------------------------------------------------------------------------------------------
    End Sub

    Private Sub finish_maker()
        'destroy all ui so the user can enter the game
        For Each br In Me.bar_list
            br.destroy()
        Next

        For Each btn In Me.btn_list
            btn(0).destroy()
            btn(1).destroy()
        Next
```

```
        For Each btn In Me.lone_btn_list
            btn.destroy()
        Next

        For Each txt In Me.txt_list
            txt.destroy()
        Next

        For Each img In Me.image_list
            Me.game.Controls.Remove(img)
        Next
        '----------------------------------------

        'change the player stats once finished, using percentages so e.g. 1 + 0.3 = 1.3 = 130%
increase
        Me.plr1.max_hp *= (1 + plr1.all_stats(0))
        Me.plr1.hp = Me.plr1.max_hp

        Me.plr1.max_mana *= (1 + plr1.all_stats(2))
        Me.plr1.mana = Me.plr1.max_mana
        '-------------------------------------------------------------------------------------
    End Sub

    Public Sub load()
        Me.make_bars()
        Me.make_buttons()
        Me.make_txt()
        Me.make_images()
        Me.colr_limbs()
    End Sub

    Protected Friend Sub update()
        'checks each pair of buttons that allters player stats and changes them accordingly
(inc/dec)
        For btnon = 0 To Me.btn_list.Count() - 1

            If Me.btn_list(btnon)(0).been_clicked Then

                If Me.plr1.all_stats(btnon) > 0 Then
                    Me.plr1.all_stats(btnon) -= 1
                    Me.attribute_pnts += 1
                    Me.bar_list(btnon).new_size(Me.plr1.all_stats(btnon) / 20)
                End If

                Me.btn_list(btnon)(0).been_clicked = False

            ElseIf Me.btn_list(btnon)(1).been_clicked Then

                If Me.plr1.all_stats(btnon) < 20 AndAlso Me.attribute_pnts > 0 Then
                    Me.plr1.all_stats(btnon) += 1
                    Me.attribute_pnts -= 1
                    Me.bar_list(btnon).new_size(Me.plr1.all_stats(btnon) / 20)
                End If

                Me.btn_list(btnon)(1).been_clicked = False

            End If
        Next
        '-------------------------------------------------------------------------------------

        'updates all button pairs in the colour changing area and changes the players limbs colours
accordingly
        For Each btn In Me.color_btns

            If btn.been_clicked And btn.text = "+R" Then
```

```
                Me.colour_limbs(Me.limb_on)(0) += 40
                btn.been_clicked = False
            ElseIf btn.been_clicked And btn.text = "+G" Then

                Me.colour_limbs(Me.limb_on)(1) += 40
                btn.been_clicked = False
            ElseIf btn.been_clicked And btn.text = "+B" Then

                Me.colour_limbs(Me.limb_on)(2) += 40
                btn.been_clicked = False
            ElseIf btn.been_clicked And btn.text = "-R" Then

                Me.colour_limbs(Me.limb_on)(0) -= 40
                btn.been_clicked = False
            ElseIf btn.been_clicked And btn.text = "-G" Then

                Me.colour_limbs(Me.limb_on)(1) -= 40
                btn.been_clicked = False
            ElseIf btn.been_clicked And btn.text = "-B" Then

                Me.colour_limbs(Me.limb_on)(2) -= 40
                btn.been_clicked = False
            End If

        Next
        '-------------------------------------------------------------------------------------------
-------------

        'checks each limb colour in the dictionary for limb colours, if the one of the RGB values is
to big/small it resets it to max/min respectively
        For Each limb In Me.colour_limbs

            For ind = 0 To 2

                If Me.colour_limbs(limb.Key)(ind) < 0 Then

                    Me.colour_limbs(limb.Key)(ind) = 0

                ElseIf Me.colour_limbs(limb.Key)(ind) > 255 Then

                    Me.colour_limbs(limb.Key)(ind) = 255

                End If

            Next

        Next
        '-------------------------------------------------------------------------------------------
-----------------------------------------------

        'changes the amount of attribute points available
        Me.attri_txt.update(Me.attribute_pnts)

        'checks and updates main buttons
        If Me.donebtn.been_clicked Then

            Me.donebtn.been_clicked = False
            Me.finish_maker()

        ElseIf Me.next_limb.been_clicked Then

            Me.limb_on += 1
            Me.next_limb.been_clicked = False
```

```
        ElseIf Me.bef_limb.been_clicked Then

            Me.limb_on -= 1
            Me.bef_limb.been_clicked = False

        End If
        '-----------------------------------

        'makes sure the user can only change the viable amount of limbs, makes it a circular
selection e.g. 2->1->0->2->1.....
        If Me.limb_on > 2 Then
            Me.limb_on = 0
        ElseIf Me.limb_on < 0 Then
            Me.limb_on = 2
        End If
        '-----------------------------------------------------------------------------------------
-------------------------

        Me.update_plr_clrs()

    End Sub
End Class
```

DOOR_CLASS:

```
Public Class door
    Inherits base_block

    'a simple class that just stores where the door leads to

    Private _goes_to As String

    Public ReadOnly Property goes_to
        Get
            Return Me._goes_to
        End Get
    End Property

    Private Const image_path As String = "\\cc4sjd8\2024 Intake\24026166\Visual Studio
2010\Projects\practice_courswork_platformer\door_img.png"

    Public Sub New(ByVal game As Form, ByVal height As Integer, ByVal width As Integer, ByVal x As
Integer, ByVal y As Integer, ByVal goes_to As String)

        MyBase.New(game, height, width, x, y)

        Dim originalImage As New Bitmap(door.image_path)

        Me.box.Image = originalImage
        Me.box.BackColor = Color.Transparent
        Me.box.SizeMode = PictureBoxSizeMode.StretchImage

        Me._goes_to = goes_to

    End Sub
End Class
```

FINISH_SCREEN:

```
Imports System.Windows.Forms
Imports System.IO

Public Class finish_screen
    Inherits Form

    ' the class used to represent the form that displays you time vs others and ranks them

    Public timera As New Timer()
    Public timer_num As Integer = 0

    Private txtInput As New text(Me, 20, 400, 15, "name here", Color.Black, True, 0)
    Private finiText As New text(Me, 300, 50, 30, "finished the game", Color.Red)
    Private all_text As New List(Of text)

    Private submitbtn As New buttono(Me, 600, 700, 200, 50, "submit")
    Private exitbtn As New buttono(Me, 100, 700, 200, 50, "exit")
    Private rstbtn As New buttono(Me, 350, 700, 200, 50, "restart")

    Private all_buttons As New List(Of buttono)

    Private show_all_text As Boolean = False

    Private leaderboard_dict As New Dictionary(Of String, String)

    Public util As New utilities()

    Private Sub Form2_Load(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
MyBase.Load

        'loads in the main screen
        Me.Size = New Size(900, 850)

        Me.all_buttons.Add(Me.submitbtn)
        Me.all_buttons.Add(Me.exitbtn)
        Me.all_buttons.Add(Me.rstbtn)

        Me.all_text.Add(Me.txtInput)
        Me.all_text.Add(Me.finiText)

        For Each txt In Me.all_text
            txt.load()
        Next

        For Each btn In Me.all_buttons
            btn.load()
            AddHandler btn.mybutton.Click, AddressOf btn.Me_OnClick
        Next

        AddHandler Me.timera.Tick, AddressOf Me.Timer2_Tick
        Me.KeyPreview = True

        Me.timera.Interval = 60
        Me.timera.Start()
        '------------------------

    End Sub

    Private Sub load_leaderboard()
        'loads lines from saved scores file
        Me.leaderboard_dict = Me.util.Load_scores(Me.leaderboard_dict)
```

```
        'sort the dictionary
        Me.sort_dict()

        Dim ind As Integer = 0
        'print all the rows in the sorted dictionary
        For Each scr In Me.leaderboard_dict

            'only show the first 10
            If ind < 11 Then

                Dim new_row As New text(Me, 350, 150 + (ind * 50), 15, ind + 1 & ". " & scr.Key & "
: " & Me.leaderboard_dict(scr.Key) & " seconds", Color.Black)

                new_row.border = True
                new_row.load()

            End If
            '---------------------

            'show the player score/time and where they rank
            If scr.Key = Me.txtInput.textbox.Text Then

                Dim player_score As New text(Me, 600, 150, 15, ind + 1 & ". " & scr.Key & " : " &
Me.leaderboard_dict(scr.Key) & " seconds", Color.Red)

                player_score.border = True
                player_score.load()
            End If
            '-----------------------------------------------

            ind += 1

        Next
        '-----------------------------------------------


    End Sub

    Private Sub sort_dict()

        Dim sortable_list As New List(Of Tuple(Of Integer, String))
        Dim new_dict As New Dictionary(Of String, String)

        'change the dictionary to a list thaat can be sorted
        For Each scr In Me.leaderboard_dict

            Dim val As Integer = 0
            Dim parts() As String = Me.leaderboard_dict(scr.Key).Split(":"c)

            'change hours and minutes to seconds, for ordering
            val = CUInt(parts(0)) * 3600 + CUInt(parts(1)) * 60 + CUInt(parts(2))

            sortable_list.Add(Tuple.Create(val, scr.Key))

        Next
        '-------------------------------------------------

        'sort the new list and store the result as a dictionary
        For Each scr In quick_sort(sortable_list)
            new_dict((scr.item2).ToString()) = scr.item1
        Next

        Me.leaderboard_dict = new_dict
        '-------------------------------------------------------
```

```
    End Sub

    Private Function quick_sort(ByVal my_lizt As List(Of Tuple(Of Integer, String)))
        'performs a quick sort to a list

        Dim length As Integer = my_lizt.Count()

        'makes sure the list is big enough : bigger then 1/not smaller then 2
        If length < 2 Then
            Return my_lizt
        End If


        'split the list in halves
        Dim pivot As Tuple(Of Integer, String) = my_lizt(length - 1)

        my_lizt.RemoveAt(length - 1)
        '---------------------

        'order the split list
        Dim items_greater As New List(Of Tuple(Of Integer, String))
        Dim items_lower As New List(Of Tuple(Of Integer, String))

        For Each scr In my_lizt

            If scr.Item1 > pivot.Item1 Then

                items_greater.Add(scr)
            Else

                items_lower.Add(scr)
            End If

        Next
        '----------------------

        'recursively order it
        Dim return_my_lizt As New List(Of Tuple(Of Integer, String))

        return_my_lizt.AddRange(quick_sort(items_lower))
        return_my_lizt.Add(pivot)
        return_my_lizt.AddRange(quick_sort(items_greater))
        '--------------------

        Return return_my_lizt
    End Function

    Private Sub Timer2_Tick(ByVal sender As System.Object, ByVal e As System.EventArgs)

        'adds functionality to the buttons
        If Me.submitbtn.been_clicked And Me.txtInput.textbox.Text.Count() > 0 Then

            'when the name entered is submitted, it stores the name and score in a text file under
all previouse game attempts
            Me.util.save_score(Me.txtInput.textbox.Text)

            Me.submitbtn.been_clicked = False
            Me.show_all_text = True

            Me.submitbtn.destroy()
            Me.txtInput.destroy()

        ElseIf Me.exitbtn.been_clicked Then

            Me.exitbtn.been_clicked = False
```

```
            Application.Exit()

        ElseIf Me.rstbtn.been_clicked Then

            Me.rstbtn.been_clicked = False

            Application.Restart()

        End If
        '----------------------------

        If Me.show_all_text Then

            Me.load_leaderboard()
            Me.show_all_text = False

        End If

    End Sub
End Class
```

GOBLIN_ENEMY:

```
Public Class goblin_enemy
    Inherits base_enemy

    Protected Overrides Sub update_arms(ByVal plr1 As player)

        Dim centerX As Integer = x + width \ 2
        Dim centerY As Integer = y + height \ 2

        'the goblin arms follow the player
        Dim dx As Integer = plr1.x - centerX
        Dim dy As Integer = plr1.y - centerY

        Dim length As Double = Math.Sqrt(dx * dx + dy * dy)
        If length = 0 Then length = 1

        ' Normalized direction vector
        Dim nx As Double = dx / length
        Dim ny As Double = dy / length

        ' Perpendicular vector (for arm spacing)
        Dim perpX As Double = -ny
        Dim perpY As Double = nx

        Dim reach As Integer = 40
        Dim separation As Integer = 10

        ' Final positions for each arm
        Dim armBaseX As Integer = centerX + CInt(nx * reach)
        Dim armBaseY As Integer = centerY + CInt(ny * reach)

        ' Left arm
        leftArm.Left = armBaseX + CInt(perpX * separation) - leftArm.Width \ 2
        leftArm.Top = armBaseY + CInt(perpY * separation) - leftArm.Height \ 2

        ' Right arm
        rightArm.Left = armBaseX - CInt(perpX * separation) - rightArm.Width \ 2
        rightArm.Top = armBaseY - CInt(perpY * separation) - rightArm.Height \ 2

    End Sub

    Sub New(ByVal game As Form, ByVal height As Integer, ByVal width As Integer, ByVal x As Integer,
ByVal y As Integer, ByVal id As Integer, ByVal hp As Integer, ByVal util As utilities)
        MyBase.New(game, height, width, x, y, id, hp, util)

        Me.type_enemy = "goblin"

        Me.box.BackColor = Color.Green
        Me.colour = Color.Green
    End Sub



End Class
```

INSTRUCTION_MENU:

```
Public Class instruction_menu
    Private game As Form

    'this menu gives instruction to the user how to play the game
    Private _created As Boolean = False

    Public Property created As Boolean
        Get
            Return _created
        End Get
        Set(ByVal value As Boolean)
            _created = value
        End Set
    End Property

    Private all_buttons As New List(Of buttono)
    Private done_btn As buttono

    Private all_text As New List(Of text)

    Private game_aim_txt As text

    Private move_keys As text
    Private inv_keys As text
    Private attack_keys As text
    Private scroll_keys As text
    Private sheild_key As text

    Private use_consumables As text

    Public Sub New(ByVal mygame As Form)
        Me.game = mygame
    End Sub

    Private Sub make_buttons()
        Me.done_btn = New buttono(Me.game, 600, 700, 200, 50, "Done")
        Me.all_buttons.Add(Me.done_btn)

        For Each btn In Me.all_buttons
            btn.load()
            AddHandler btn.mybutton.Click, AddressOf btn.Me_OnClick
        Next
    End Sub

    Private Sub make_text()
        Dim game_aim_lines As Integer = 2
        Dim game_aim_text As String = "aim of the game: beat all enemies in each room(level) and
beat the bosses"

        Dim move_key_lines As Integer = 2
        Dim move_keys_text As String = "movement: (WASD) keys , uses velocity so only need to press
once , double press for speed"

        Dim inv_keys_lines As Integer = 2
        Dim inv_keys_text As String = "player_inteface: press (i) to open the player_inteface and
(i) agian to cose the player_inteface, cant open when sheild is equipped"

        Dim attack_keys_lines As Integer = 2
        Dim attack_keys_text As String = "attack click: press(LEFT-CLICK) to shoot whatever attack
is in use"

        Dim scroll_keys_line As Integer = 4
```

```
        Dim scroll_keys_text As String = "change attack: scroll the wheel on the mouse and look at
icon in top right to" & vbNewLine & "change attacks , also a cooldown bar for attacks"

        Dim sheild_key_lines As Integer = 6
        Dim sheild_key_text As String = "sheild use: equippe the sheild by pressing on it in your
player_inteface and then" & vbNewLine & "press(RIGHT-CLICK)to use it and pushback enemies or absorb"
& vbNewLine & "attacks ,it does have health indicated by the grey bar that appears"

        Dim use_consumables_lines As Integer = 2
        Dim use_consumables_text As String = "use consumables: to heal up , increase mna or euipe
items you must open the player_inteface and click on them"

        Me.game_aim_txt = New text(Me.game, 100, 100, 15, game_aim_text, Color.Black, True,
game_aim_lines)
        Me.move_keys = New text(Me.game, 100, 200, 15, move_keys_text, Color.Black, True,
move_key_lines)
        Me.inv_keys = New text(Me.game, 100, 250, 15, inv_keys_text, Color.Black, True,
inv_keys_lines)
        Me.attack_keys = New text(Me.game, 100, 300, 15, attack_keys_text, Color.Black, True,
attack_keys_lines)
        Me.scroll_keys = New text(Me.game, 100, 350, 15, scroll_keys_text, Color.Black, True,
scroll_keys_line)
        Me.sheild_key = New text(Me.game, 100, 450, 15, sheild_key_text, Color.Black, True,
sheild_key_lines)
        Me.use_consumables = New text(Me.game, 100, 600, 15, use_consumables_text, Color.Black,
True, use_consumables_lines)

        Me.all_text.Add(Me.game_aim_txt)

        Me.all_text.Add(Me.move_keys)
        Me.all_text.Add(Me.inv_keys)
        Me.all_text.Add(Me.attack_keys)
        Me.all_text.Add(Me.scroll_keys)
        Me.all_text.Add(Me.sheild_key)

        Me.all_text.Add(Me.use_consumables)

        For Each txt In Me.all_text
            txt.load()
        Next

    End Sub

    Protected Friend Sub load()
        Me.make_buttons()
        Me.make_text()
    End Sub

    Protected Friend Sub update()
        'restart the game
        If Me.done_btn.been_clicked Then
            Application.Restart()
        End If
        '---------------
    End Sub
End Class
```

ITEM_CLASS:

```
Public Class item
    Inherits base_block

    Private _name As String

    Public Property name As String
        Get
            Return _name
        End Get
        Set(ByVal value As String)
            _name = value
        End Set
    End Property

    Public touched As Boolean = False

    Public img_path As String
    Public image As Bitmap

    Public amount As Integer = 1
    Public amount_text As text
    Public loaded_text As Boolean = False

    Public take_away As Boolean = False

    Public Sub New(ByVal game As Form, ByVal height As Integer, ByVal width As Integer, ByVal x As
Integer, ByVal y As Integer, ByVal name As String)
        MyBase.New(game, height, width, x, y)

        Me.name = name
        Me.img_path = "\\cc4sjd8\2024 Intake\24026166\Visual Studio
2010\Projects\practice_courswork_platformer\invent_imgs\" & name & ".png"
    End Sub

    Public Sub make_image()

        'makes the items on the floor and images
        Me.box.BackColor = Color.FromArgb(99, 107, 118)

        Dim orig_img As New Bitmap(Me.img_path)
        orig_img.MakeTransparent()

        Me.image = orig_img
        Me.box.Image = orig_img
        Me.box.SizeMode = PictureBoxSizeMode.StretchImage
        '---------------------------------------

    End Sub

    Public Sub show_amount()
        amount_text = New text(Me.game, 0, 0, 10, 0.ToString(), Color.White)
        amount_text.load()
    End Sub

    Public Sub destroy()
        Me.game.Controls.Remove(Me.box)
        If Me.loaded_text Then
            Me.amount_text.destroy()
        End If
    End Sub
```

```
    Public Sub item_MouseClick(ByVal sender As Object, ByVal e As EventArgs)

        'check if they've been clicked
        Me.touched = True

    End Sub

    Public Sub update()

        'check if there is text on the item and update it
        'e.g. floor items dont have text

        If Not (Me.amount_text Is Nothing) Then

            amount_text.my_text = amount.ToString()
            amount_text.box.Text = amount_text.my_text

            amount_text.box.Left = Me.x
            amount_text.box.Top = Me.y + 30


        End If
        '------------------------------------------------

    End Sub

End Class
```

FORM1:

```
Imports System.Diagnostics
Imports System.IO

Public Class Form1

    Private raycaster As ray_cast
    Private utiliser As New utilities()

    Private timera As New Timer()
    Private timer_num = 0

    Private start_time As Date
    Private end_time As Date

    Private finish_scrn As New finish_screen()

    Private plr1 As New player(Me, 40, 40, 0, 0, Me.utiliser)

    Private char_crea As New character_creator(plr1, Me)
    Private intru_men As New instruction_menu(Me)


    Private walls_list As New List(Of wall)
    Private usable_wallz_list As New List(Of wall)
    Private enemy_list As New List(Of base_enemy)
    Private boss_list As New List(Of boss)
    Private door_list As New List(Of door)
    Private floor_items As New List(Of item)
    Private NPC_list As New List(Of Npc)
    Private new_floor_items As New List(Of item)

    Private type_bos As String = "spider"

    Private dead_enemys As New List(Of base_enemy)

    Private map_generator As New map_maker(utiliser, 14, 16)

    Private text_map1 As New List(Of String)
    Private text_map2 As New List(Of String)
    Private NPC_village As New List(Of String)
    Private text_map3 As New List(Of String)
    Private boss_floor As New List(Of String)
    Private text_mapp_used As List(Of String)

    Private all_missions As New Dictionary(Of List(Of String), List(Of Integer))
    Private all_missions_txt As New List(Of List(Of String))

    Private mousex As Integer
    Private mousey As Integer

    Private all_buttons As New List(Of buttono)

    Private start_btn As New buttono(Me, 475, 350, 200, 90, "start")
    Private exit_btn As New buttono(Me, 250, 350, 200, 90, "exit")
    Private instruct_btn As New buttono(Me, 350, 450, 200, 90, "instructions")
    Private load_btn As New buttono(Me, 350, 250, 200, 90, "load previous save")

    Private save_btn As New buttono(Me, 770, 650, 100, 100, "save")

    Private finished_menu As Boolean = False
```

```
    Private start_text As New text(Me, 300, 100, 50, "rpg game", Color.Red)

    Private game_has_start As Boolean = False

    Private id_enemy As Integer = 1
    Private id_npc As Integer = 0

    Private enemy_count_txt As New text(Me, 250, 0, 25, "enemy count:0", Color.Red)



    Private Sub create_map()

        Dim enemy_health As New Dictionary(Of String, Integer) From {
                {"spider", 150},
                {"invisi", 200},
                {"goblin", 2},
                {"warlock", 4}
        }

        'create coordinates for walls for ray_casting, to make the ray casting more optimizes
instead of checking each wall seperately

        Me.raycaster = New ray_cast(Me.text_mapp_used)


        'loads the text map into visualized the player can interact with.
        'whilst also making terrain generation and adding in different structure randomly incredibly
easy

        For y_add = 1 To text_mapp_used.Count() - 1
            For x_add = 0 To text_mapp_used(y_add).Length() - 1
                If text_mapp_used((text_mapp_used.Count() - 1) - (y_add - 1))(x_add) = "p" Then
                    ' moves the already loaded player image to the starting locations, so i dont
need to re-load the player each new map
                    plr1.x = x_add * 50
                    plr1.y = 800 - (y_add * 50)

                ElseIf text_mapp_used((text_mapp_used.Count() - 1) - (y_add - 1))(x_add) = "e" Then

                    Dim new_goblin As New goblin_enemy(Me, 45, 45, x_add * 50, 800 - (y_add * 50),
Me.id_enemy, enemy_health("goblin"), Me.utiliser)

                    new_goblin.load_attacks_fromtxt()

                    enemy_list.Add(new_goblin)

                    Me.id_enemy += 1
                ElseIf text_mapp_used((text_mapp_used.Count() - 1) - (y_add - 1))(x_add) = "b" Then
                    Dim new_wall As New wall(Me, 50, 50, x_add * 50, 800 - (y_add * 50))
                    new_wall.text_mapp_used = Me.text_mapp_used

                    If new_wall.real_wall(y_add, x_add) Then

                        Me.usable_wallz_list.Add(new_wall)

                    Else

                        Me.walls_list.Add(new_wall)

                    End If

                ElseIf text_mapp_used((text_mapp_used.Count() - 1) - (y_add - 1))(x_add) = "w" Then
```

```
                    Dim new_warlock As New warlock_enemy(Me, 45, 45, x_add * 50, 800 - (y_add * 50),
Me.id_enemy, enemy_health("warlock"), Me.utiliser, True)

                    new_warlock.load_attacks_fromtxt()

                    enemy_list.Add(new_warlock)

                    Me.id_enemy += 1
                ElseIf text_mapp_used((text_mapp_used.Count() - 1) - (y_add - 1))(x_add) = "d" Then

                    Dim new_box As New door(Me, 50, 50, x_add * 50, 800 - (y_add * 50),
text_mapp_used(0))

                    door_list.Add(new_box)

                ElseIf text_mapp_used((text_mapp_used.Count() - 1) - (y_add - 1))(x_add) = "N" Then

                    Dim new_box As New Npc(Me, Me.plr1, x_add * 50, 800 - (y_add * 50), 50, 50,
Me.all_missions_txt(Me.id_npc), Me.utiliser)

                    Me.NPC_list.Add(new_box)

                    Me.id_npc += 1


                ElseIf text_mapp_used((text_mapp_used.Count() - 1) - (y_add - 1))(x_add) = "E" Then
                    'a boss entity
                    Dim new_boss As New boss(Me, x_add * 50, 800 - (y_add * 50), 45, 45,
Me.type_bos, enemy_health(Me.type_bos), Me.plr1, Me.utiliser)

                    new_boss.load_attacks_fromtxt("none")

                    Me.enemy_list.Add(new_boss)
                    Me.boss_list.Add(new_boss)

                End If


            Next
        Next
        '-------------------------------------------------------------------------------------------
------------------

    End Sub

    Private Sub plr_finished()
        If Me.plr1.equipped(0) = "player_eye" Then
            Me.plr1.finished_game = True
        End If

        'if the user has beaten the game
        If Me.plr1.finished_game Then

            'change the enemy count text
            Me.enemy_count_txt.my_text = "GAME FINISHED!!!!!!!"
            Me.enemy_count_txt.colour = Color.HotPink

            Me.enemy_count_txt.load()

            Me.plr1.finished_game = False

            If Me.timer_num Mod 20 = 0 Then
                'saves the time taken to finish the game , to a text file so all previous times can
be ranked and shown to the user, in the finish_screen
                Me.end_time = DateTime.Now
```

```
            Dim elapsed As TimeSpan = Me.end_time - Me.start_time
            Dim elapsedString As String = elapsed.ToString("hh\:mm\:ss")

            Me.utiliser.save_score(elapsedString)

            Me.game_has_start = False
            Me.finished_menu = True

        End If

    End If
    '-------------------------------------
    'checks if the player has died
    If Me.plr1.hp <= 0 Then

        Me.enemy_count_txt.my_text = "U DEAD!!!!!!!"
        Me.enemy_count_txt.colour = Color.Yellow

        Me.enemy_count_txt.load()

        'gives a delay so the user can recognize their death and so it isnt an instant change
        If Me.timer_num Mod 10 = 0 Then
            Application.Restart()
        End If

    End If
    '------------------------------
End Sub


Private Sub change_room()
    If plr1.touch_door And Me.enemy_list.Count() <= 0 Then

        Me.clear_map()

        Dim new_gen_room As New map_maker(Me.utiliser, 14, 16)

        'room for only NPC's
        Dim new_gen_village As New map_maker(Me.utiliser, 14, 16, "vil")

        'loads in a boss room
        Dim new_gen_bossrm As New map_maker(Me.utiliser, 14, 16, "bossrm")

        Me.plr1.boss_room = False

        'checks which room to go to, by looking at the first string in the text map
        If plr1.which_door.goes_to = "floor2" Then

            text_map2 = new_gen_room.load_map()
            text_map2(0) = "vil"

            Me.text_mapp_used = text_map2

        ElseIf plr1.which_door.goes_to = "vil" Then

            NPC_village = new_gen_village.load_map()
            NPC_village(0) = "spider_den"
            Me.type_bos = "spider"
            Me.text_mapp_used = Me.NPC_village

        ElseIf plr1.which_door.goes_to = "floor4" Then

            text_map3 = new_gen_room.load_map()
            text_map3(0) = "invisi_den"
```

```
            Me.text_mapp_used = text_map3

            Me.type_bos = "invisi"

        ElseIf plr1.which_door.goes_to = "spider_den" Then

            boss_floor = new_gen_bossrm.load_map(True)
            boss_floor(0) = "floor4"

            Me.plr1.boss_room = True

            Me.text_mapp_used = Me.boss_floor

        ElseIf plr1.which_door.goes_to = "invisi_den" Then

            boss_floor = new_gen_bossrm.load_map(True)
            boss_floor(0) = "invisi_den"

            Me.plr1.boss_room = True

            Me.text_mapp_used = Me.boss_floor

        End If
        '-------------------------------------------------------------------------------

        plr1.touch_door = False

        Me.plr1.my_interface.check_boss_bar(Me.plr1.boss_room)
        Me.load_game()

    End If
End Sub


Private Sub amount_of_en_txt()
    'update the text that shows the number of enemies
    If Me.enemy_list.Count() < 1 Then
        Me.enemy_count_txt.colour = Color.LightGreen

    Else
        Me.enemy_count_txt.colour = Color.Red

    End If
    '-----------------------------------------------

    Me.enemy_count_txt.update("enemy count:" & (Me.enemy_list.Count()).ToString)
End Sub


Private Sub update_flr_items()

    Dim item_id As Integer = plr1.collided_flooritem

    'checks if the player has collided with a floor item
    If item_id <> -1 Then

        'the player gives the item id which can be used to decide what to do with it
        Me.Controls.Remove(Me.floor_items(item_id).box)

        Me.floor_items.RemoveAt(item_id)

    End If
    '-------------------------------------------------
```

```
    End Sub

    Private Sub update_npcs()

        For Each Npc In Me.NPC_list
            Npc.update()
        Next

    End Sub

    Private Sub update_enemies()

        For Each enemy In enemy_list

            enemy.update(Me.usable_wallz_list, Me.plr1, Me.enemy_list, Me.text_mapp_used,
Me.timer_num)

            If enemy.hp <= 0 Then
                enemy.dead = True
            End If

            'update all attacks in use right now
            For Each atk In enemy.live_attacks
                atk.update()
            Next
            '---------------------------------

            If enemy.dead Then

                'check if an enemy has been killed and which one has been killed ,for missions
                If Me.plr1.killed_enemy_amounts.ContainsKey(enemy.type_enemy) Then
                    Me.plr1.killed_enemy_amounts(enemy.type_enemy) += 1

                Else
                    Me.plr1.killed_enemy_amounts(enemy.type_enemy) = 1

                End If
                '-------------------------------------------------------------------------------

                Dim new_item As item

                'drop an item or add one to the players player_inteface when killed
                If enemy.type_enemy <> "spider" And enemy.type_enemy <> "invisi" Then

                    If Me.utiliser.Rand0() > 50 Then
                        new_item = New item(Me, 35, 35, enemy.x, enemy.y, "hp_potion")
                        Me.new_floor_items.Add(new_item)

                    Else
                        new_item = New item(Me, 35, 35, enemy.x, enemy.y, "mana_potion")
                        Me.new_floor_items.Add(new_item)

                    End If


                ElseIf enemy.type_enemy = "spider" Then

                    new_item = New item(Me, 35, 35, enemy.x, enemy.y, "shield")
                    Me.new_floor_items.Add(new_item)

                    new_item = New item(Me, 35, 35, enemy.x, enemy.y, "helmet")
                    Me.new_floor_items.Add(new_item)

                ElseIf enemy.type_enemy = "invisi" Then
```

```
            new_item = New item(Me, 35, 35, enemy.x, enemy.y, "player_eye")
            Me.new_floor_items.Add(new_item)

        End If
        '-------------------------------------------------------------

        'add the new floor item to the main floor items list
        For Each item In Me.new_floor_items
            item.make_image()
            Me.floor_items.Add(item)

            item.load()
        Next
        Me.new_floor_items.Clear()
        '-----------------------------------------------

        Me.Controls.Remove(enemy.box)
        Me.dead_enemys.Add(enemy)

        For Each boss In boss_list

            If boss.type_enemy = "spider" Then

                For z = 0 To 5
                    For x = 0 To 3
                        Me.Controls.Remove(boss.thighs(z)(x))
                        Me.Controls.Remove(boss.femur(z)(x))
                    Next
                Next

                boss.thighs.Clear()
                boss.femur.Clear()
            ElseIf boss.type_enemy = "invisi" Then

                For Each cu In boss.cubes
                    Me.Controls.Remove(cu.box)
                Next

                For Each limb_cu In boss.body_parts
                    Me.Controls.Remove(limb_cu.box)
                Next

                boss.cubes.Clear()
                boss.body_parts.Clear()
            End If
        Next

        If enemy.type_magic Then

            For Each j In enemy.live_attacks
                enemy.dead_attacks.Add(j)
            Next

            enemy.live_attacks.Clear()
        End If

    End If

    If enemy.type_magic Then

        For Each spell In enemy.dead_attacks
            Me.Controls.Remove(spell.box)
        Next

        enemy.dead_attacks.Clear()
```

```
            End If

        Next

        For Each i In dead_enemys

            Me.enemy_list.Remove(i)
            For Each j In i.all_limbs
                Me.Controls.Remove(j)
            Next

            i.all_limbs.Clear()

        Next
        Me.dead_enemys.Clear()

    End Sub


    Private Sub Form1_Load(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
MyBase.Load
        'loads the start of the game, the entry page
        Me.Size = New Size(900, 850)

        'adds the buttons to their own list
        all_buttons.Add(Me.start_btn)
        all_buttons.Add(Me.exit_btn)
        all_buttons.Add(Me.instruct_btn)
        all_buttons.Add(Me.load_btn)

        'loads the missions and sets the possible player mission to the missions now stored in a
dictionary
        'the reason this is not done in the player_vlass is because the NPC class uses this data as
well

        Me.plr1.all_missions = Me.utiliser.load_missions()
        Me.all_missions_txt = Me.utiliser.add_missions(Me.plr1.all_missions)


        'loads the starting text at the top of the screen
        Me.start_text.load()

        'loads the buttons in the list
        For Each i In all_buttons
            'handles clicking
            AddHandler i.mybutton.Click, AddressOf i.Me_OnClick

            i.load()

        Next

        AddHandler Me.MouseMove, AddressOf Me.utiliser.Form1_MouseMove
        'turn a the timera sub into the main game loop
        AddHandler Me.timera.Tick, AddressOf Me.Timer1_Tick

        timera.Interval = 50
        timera.Start()

    End Sub

    Private Sub load_game()
        'loads all the sprites for each new map
        create_map()

        Me.BackColor = Color.FromArgb(99, 107, 118)
```

```
        'loads the enemy_count text
        Me.enemy_count_txt.load()

        'only initializes the player once in the first map, no need to do it again
        'the reason it says floor2 is because this points to the next map , not the one being used
        'This makes doors easy to create and use
        If Me.text_mapp_used(0) = "floor2" Or Me.utiliser.loaded_game Then

            Me.plr1.load_attacks()

            Me.plr1.my_interface = New player_inteface(Me, Me.utiliser)
            Me.plr1.attack_img = New Bitmap(Me.plr1.attack_dict("fireball")(2))

            Me.plr1.my_interface.load_bars()
            Me.plr1.my_interface.load_inventory()
            Me.plr1.my_interface.load_ui(plr1)
            Me.plr1.my_interface.load_armour_stats()


            Me.plr1.my_sheild = New sheild(Me, Me.plr1, 50, 50, 0, 0, 100)


            Me.plr1.my_sheild.sheild_hp_bar = New bar(Me, 100, 20, 755, 230, Color.Brown,
Color.LightGray)

            AddHandler Me.MouseClick, AddressOf Me.plr1.click_handler
            AddHandler Me.MouseWheel, AddressOf Me.plr1.scroll_wheel
            AddHandler Me.KeyPress, AddressOf Me.plr1.key_handler

            'if the user finishes the game their time is ranked on their local computer against
other scores saved
            Me.start_time = DateTime.Now

            Me.save_btn.load()

            AddHandler Me.save_btn.mybutton.Click, AddressOf Me.save_btn.Me_OnClick

            Me.utiliser.loaded_game = False

            Me.plr1.my_interface.add("helmet")

        Else

            Me.plr1.xvel = 0
            Me.plr1.yvel = 0

            Me.plr1.update(Me.usable_wallz_list, Me.timer_num, Me.mousex, Me.mousey, Me.enemy_list,
door_list, floor_items, NPC_list)
        End If
        '-------------------------------------------------------------------------------

        'starts the enemy killed counter again
        Me.plr1.start_level = False

        Const wall_img_path As String = "\\cc4sjd8\2024 Intake\24026166\Visual Studio
2010\Projects\practice_courswork_platformer\wall_imgs\tile_"
        'loads all fake walls
        For Each i In walls_list
            i.load()
            i.change_img(text_mapp_used, wall_img_path)
        Next

        'loads all real walls
```

```
        For Each i In Me.usable_wallz_list
            i.load()
            i.change_img(text_mapp_used, wall_img_path)
        Next

        'loads all NPC's and makes it clickable
        For Each npx In NPC_list

            AddHandler npx.box.MouseClick, AddressOf npx.click_handler
            AddHandler npx.eyebox.MouseClick, AddressOf npx.click_handler
            AddHandler npx.eye.MouseClick, AddressOf npx.click_handler

            npx.load()
            npx.limb_load()

            npx.eye.Top += 15
            npx.eyebox.Top += 15

            npx.eyebox.BringToFront()
            npx.eye.BringToFront()

        Next

        'loads each enemy, makes they clickable so if the mouse is over them attacks can still be
used

        For Each i In enemy_list

            AddHandler i.box.MouseClick, AddressOf Me.plr1.click_handler
            AddHandler i.eyebox.MouseClick, AddressOf Me.plr1.click_handler
            AddHandler i.eye.MouseClick, AddressOf Me.plr1.click_handler

            'instead of creating a wall map , and ray caster object for each enemy one is just
shared among the many
            i.raycast_obj = Me.raycaster

            i.load()

            i.limb_load()

        Next

        'loads each door
        For Each i In Me.door_list

            i.load()

        Next



        Me.DoubleBuffered = True
        Me.KeyPreview = True
    End Sub

    Private Sub clear_map()
        'clear all dead attacks incase attacks that are dead are still store in memory, remove them
from their list
        For Each atk In plr1.dead_attack
            Me.Controls.Remove(atk.box)
        Next
        '------------------------------------------------------------

        'clear all pickup items from graphics memory and their list
        For Each item In floor_items
```

```
    Me.Controls.Remove(item.box)
Next
floor_items.Clear()
'---------------------------------

'clears all the walls from graphics memory and their list
For Each wall In Me.walls_list
    Me.Controls.Remove(wall.box)
Next
Me.walls_list.Clear()
'-------------------------

'clear all collidable walls from graphics memory and their list
For Each u_wall In Me.usable_wallz_list
    Me.Controls.Remove(u_wall.box)
Next
Me.usable_wallz_list.Clear()
'-------------------------------------------------------------

'clear all enemies from graphics memory and from their list
For Each ene In Me.enemy_list
    Me.Controls.Remove(ene.box)
    'clear each enemies attacks
    For Each j In ene.live_attacks
        Me.Controls.Remove(j.box)
    Next
    '-------------------------

    'clear all the limbs of enemies
    For Each j In ene.all_limbs
        Me.Controls.Remove(j)
    Next
    '--------------------------

    ene.all_limbs.Clear()
    ene.live_attacks.Clear()
Next
enemy_list.Clear()
'-------------------------------------------------------------

'clear all NPC's in graphics memory
For Each npc In Me.NPC_list
    Me.Controls.Remove(npc.box)

    'clear their limbs
    For Each limb In npc.all_limbs
        Me.Controls.Remove(limb)
    Next

    npc.del_text_box()
    npc.all_limbs.Clear()
Next
NPC_list.Clear()
'---------------------------------

'clear all boss entities from graphics memory and clear from their list
For Each boss In boss_list

    If boss.type_enemy = "spider" And Not boss.dead Then
        'clear the spider legs
        For leg = 0 To 5
            For leg_part = 0 To 3
                Me.Controls.Remove(boss.thighs(leg)(leg_part))
                Me.Controls.Remove(boss.femur(leg)(leg_part))
            Next
```

```
            Next

            boss.thighs.Clear()
            boss.femur.Clear()

        ElseIf boss.type_enemy = "invisi" And Not boss.dead Then
            'clear the orbiting cubes
            For Each cube In boss.cubes
                Me.Controls.Remove(cube.box)
            Next

            'clear cubes around the main body , that make it seem circular
            For Each limb In boss.body_parts
                Me.Controls.Remove(limb.box)
            Next

        End If

        boss.cubes.Clear()
        boss.body_parts.Clear()

    Next
    boss_list.Clear()

    '----------------------------------------

    'clear all doors from graphical memory and their list
    For Each door In Me.door_list
        Me.Controls.Remove(door.box)
    Next
    door_list.Clear()
    '-------------------------------------------------

End Sub

Private Sub clear_start_menu()

    For Each but In all_buttons
        but.destroy()
    Next

    'removes the title of the game
    Me.start_text.destroy()

    all_buttons.Clear()
End Sub

Private Sub load_old_game()
    Dim loaded As List(Of List(Of String)) = Me.utiliser.load_map()

    Me.text_mapp_used = loaded(0)

    For ind = 0 To 2
        Me.plr1.all_stats(ind) = loaded(1)(ind)
    Next

    Dim body_cols() As String = loaded(1)(3).Split("\")
    plr1.box.BackColor = Color.FromArgb(body_cols(0), body_cols(1), body_cols(2))

    Dim arm_cols() As String = loaded(1)(4).Split("\")

    plr1.leftArm.BackColor = Color.FromArgb(arm_cols(0), arm_cols(1), arm_cols(2))
    plr1.rightArm.BackColor = Color.FromArgb(arm_cols(0), arm_cols(1), arm_cols(2))

    Dim leg_cols() As String = loaded(1)(5).Split("\")
```

```
        plr1.rightLeg.BackColor = Color.FromArgb(leg_cols(0), leg_cols(1), leg_cols(2))
        plr1.leftLeg.BackColor = Color.FromArgb(leg_cols(0), leg_cols(1), leg_cols(2))

        Me.game_has_start = True

        Me.utiliser.loaded_game = True
        Me.load_btn.been_clicked = False

        Me.plr1.load()
        Me.plr1.limb_load()
    End Sub

    Private Sub game_update()
        'update all entities
        Me.plr1.update(Me.usable_wallz_list, Me.timer_num, Me.mousex, Me.mousey, Me.enemy_list,
Me.door_list, Me.floor_items, Me.NPC_list)

        update_flr_items()
        update_enemies()

        amount_of_en_txt()
        change_room()
        update_npcs()

        plr_finished()
        'cannot save the game in a boss room
        If Me.save_btn.been_clicked And Not Me.plr1.boss_room Then

            Me.utiliser.save_map(Me.text_mapp_used, plr1)
            Me.save_btn.been_clicked = False

        End If

        Me.mousex = Me.utiliser.mousex
        Me.mousey = Me.utiliser.mousey

        Me.timer_num += 1
    End Sub

    Private Sub start_screen()
        'gives certain buttons for the enter and end screen functionality
        If Me.finished_menu Then
            'show the end screen, hide this screen
            Me.finish_scrn.Show()
            Me.Hide()

            Me.finished_menu = False

        ElseIf Me.start_btn.been_clicked Then
            clear_start_menu()

            If Not char_crea.created Then

                Me.char_crea.load()
                Me.char_crea.created = True

            End If
            'begin the game, load in the first room
            If Me.char_crea.donebtn.been_clicked Then

                text_map1 = map_generator.load_map()
                text_map1(0) = "floor2"
                Me.text_mapp_used = text_map1
```

```
                Me.game_has_start = True
                load_game()

            End If

            Me.char_crea.update()

        ElseIf Me.instruct_btn.been_clicked Then
            Me.clear_start_menu()

            If Not Me.intru_men.created Then

                Me.intru_men.load()
                Me.intru_men.created = True

            Else

                Me.intru_men.update()

            End If
        ElseIf Me.load_btn.been_clicked Then

            clear_start_menu()

            load_old_game()

            load_game()



        ElseIf Me.exit_btn.been_clicked Then
            Application.Exit()

        End If
        '-----------------------------------------------------------

    End Sub


    Private Sub Timer1_Tick(ByVal sender As System.Object, ByVal e As System.EventArgs)
        'if the game has started begin the main loop, else stay on the start screen/s
        If Me.game_has_start Then
            game_update()
        Else
            start_screen()
        End If

    End Sub


End Class
```

MAP_MAKER:

```
Public Class map_maker

    Private width As Integer = 16
    Private height As Integer = 14

    Private new_map As New List(Of String)

    Private type_room As String = "normal"

    Private util As utilities

    Public Sub New(ByVal utl As utilities, ByVal wid As Integer, ByVal hei As Integer, Optional
ByVal typrm As String = "normal")

        Me.type_room = typrm

        Me.width = wid
        Me.height = hei

        Me.util = utl


    End Sub


    Public Function generate_random_map() As List(Of List(Of String))

        Dim density As Integer = 60
        Dim iterations As Integer = 100 'higher the number is the less unique rooms may be

        Dim tempMap As New List(Of List(Of String))

        ' randomly add walls to the empty grid----------------
        For y As Integer = 0 To height

            Dim row As New List(Of String)
            For x As Integer = 0 To width

                'add walls depending on density requested:
                If Me.util.Rand0(1, 100) < density Then
                    row.Add("b")
                Else
                    row.Add(".")
                End If

            Next

            tempMap.Add(row)

        Next
        '-----------------------------------------------------

        'amount of iterations:
        For iteration As Integer = 0 To iterations - 1

            Dim newMap As New List(Of List(Of String))

            ' use cellular automata to make the grid look more room-like----------------------------
------------------
            For y As Integer = 0 To tempMap.Count - 1

                Dim newRow As New List(Of String)
```

```
                For x As Integer = 0 To tempMap(0).Count - 1

                    Dim wallCount As Integer = 0

                    'count how many neibouring cells are walls-------------------------------------
------------------
                    For dy As Integer = -1 To 1
                        For dx As Integer = -1 To 1

                            Dim ny As Integer = y + dy
                            Dim nx As Integer = x + dx

                            If ny >= 0 AndAlso ny < tempMap.Count AndAlso nx >= 0 AndAlso nx <
tempMap(0).Count Then
                                If (dy <> 0 OrElse dx <> 0) And tempMap(ny)(nx) = "b" Then
                                    wallCount += 1
                                End If
                            Else
                                wallCount += 1
                            End If

                        Next
                    Next
                    '-----------------------------------------------------------------------------
------------------

                    'add rules so only cells with enough neighbours live--------
                    If wallCount >= 5 Then
                        newRow.Add("b")
                    Else
                        newRow.Add(".")
                    End If
                    '------------------------------------------------------------
                Next

                newMap.Add(newRow)

            Next

            tempMap = newMap

        Next
        '-----------------------------------------------------------------------------------------
---------------

        Return tempMap
    End Function

    Public Function is_navig(ByVal map As List(Of List(Of String)), ByVal start_pos As Integer(),
ByVal end_pos As Integer())

        Dim path As List(Of Integer()) = find_path(start_pos(0), start_pos(1), end_pos(0),
end_pos(1), map)

        ' returns true if their is a path present and there are more than 0 steps:
        Return Not path Is Nothing AndAlso path.Count > 0
    End Function

    Public Function load_map(Optional ByVal bossrm As Boolean = False) As List(Of String)

        Dim validMap As List(Of List(Of String))
        Dim floorTiles As New List(Of Integer())

        Dim start_pos As Integer()
        Dim end_pos As Integer()
```

```
        Dim sum_of_enemies As Integer = 0
        Dim sum_of_npcs As Integer = 0

        Dim spawned As Boolean = False

        Do

            floorTiles = New List(Of Integer())


            spawned = False

            validMap = generate_random_map()

            sum_of_enemies = 0
            sum_of_npcs = 0

            spawned = False

            'adds the border-----------------
            For i As Integer = 0 To Me.width - 1
                validMap(1)(i) = "b"
                validMap(Me.height)(i) = "b"
            Next

            For j As Integer = 1 To Me.height - 1
                validMap(j)(0) = "b"
                validMap(j)(Me.width) = "b"
            Next
            '-------------------------------

            ' Gather all floor tiles----------------------------------

            For y As Integer = 1 To height - 1
                For x As Integer = 0 To width - 1
                    If validMap(y)(x) = "." Then


                        floorTiles.Add(New Integer() {x, y})


                    End If
                Next
            Next
            '------------------------------------------------------------


            ' adds doors (to bossroom or to next room)-------------------------------------------
-------

            'check if there is enough empty space, if there is not it re-does the loop:
            If floorTiles.Count < 2 Then Continue Do

            'checks if there is a path to the doors.
            start_pos = floorTiles(Me.util.Rand0(1, (floorTiles.Count - 1) * 0.5))
            end_pos = floorTiles(floorTiles.Count - 1)


            If is_navig(validMap, start_pos, end_pos) Then

                spawned = True

                validMap(start_pos(1))(start_pos(0)) = "p"
```

```
                validMap(end_pos(1))(end_pos(0)) = "d"

            End If

            If Not spawned Then Continue Do

            '-------------------------------------------------------------------------------------
------


            If Not bossrm Then

                If Me.type_room = "normal" Then
                    'spawn in goblins/warlocks-----------------------------------------------------
-------------------------------------------------------------------

                    While sum_of_enemies < 3

                        Dim en_location As Integer() = floorTiles(Me.util.Rand0(1, floorTiles.Count
- 1))

                        'choose between goblin enemies and warlocks and check for a path----------

                        If Not (en_location Is start_pos) AndAlso is_navig(validMap, start_pos,
en_location) And validMap(en_location(1))(en_location(0)) = "." Then

                            Select Case Me.util.Rand0(1, 100)
                                Case Is > 50
                                    validMap(en_location(1))(en_location(0)) = "w"

                                Case Is <= 50
                                    validMap(en_location(1))(en_location(0)) = "e"
                            End Select

                            sum_of_enemies += 1
                            '-----------------------------------------------------
                        End If

                    End While

                    '-------------------------------------------------------------------------------
-------------------------------------------------------------------
                Else
                    'spawn in NPC's----------------------------------------------------------------


                    While sum_of_npcs < 3

                        Dim npc_location As Integer() = floorTiles(Me.util.Rand0(1, floorTiles.Count
- 1))

                        If Not (npc_location Is start_pos) AndAlso is_navig(validMap, start_pos,
npc_location) And validMap(npc_location(1))(npc_location(0)) = "." Then
                            validMap(npc_location(1))(npc_location(0)) = "N"
                        Else
                            Continue While
                        End If

                        sum_of_npcs += 1

                    End While
                    '-----------------------------------------------------------------------------
                End If
            Else
```

```
                'spawn in the bosses-------------------------------------------------------------------
---------------------------

                Dim boss_location As Integer() = floorTiles(Me.util.Rand0(3, floorTiles.Count - 1))

                If Not (boss_location Is start_pos) AndAlso is_navig(validMap, start_pos,
boss_location) And validMap(boss_location(1))(boss_location(0)) = "." Then
                    validMap(boss_location(1))(boss_location(0)) = "E"
                    sum_of_enemies += 1
                End If


                '-------------------------------------------------------------------------------------
-------------------------
            End If

        Loop Until floorTiles.Count() > (Me.width * Me.height) / 4 And (sum_of_enemies > 0 Or
sum_of_npcs > 2)

        ' convert list of list of strings("char") -> list of strings("full string")--
        For Each row In validMap
            new_map.Add(String.Join("", row.ToArray()))
        Next
        '-------------------------------------------------------------------------

        Return Me.new_map
    End Function

    Public Function find_path(ByVal startx As Integer, ByVal starty As Integer, ByVal endx As
Integer, ByVal endy As Integer, ByVal map As List(Of List(Of String)))
        Dim openList As New List(Of Integer())

        Dim cameFrom As New Dictionary(Of String, Integer())
        'arrays to check whether cells have scores and if they are higher than the new scores
        Dim gScore As New Dictionary(Of String, Integer)
        Dim fScore As New Dictionary(Of String, Integer)
        '-------------------------------------------------------------------------------------
        Dim path As New List(Of Integer())
        Dim startKey As String = startx.ToString() & "," & starty.ToString()

        openList.Add(New Integer() {startx, starty})
        gScore(startKey) = 0
        fScore(startKey) = Math.Abs(startx - endx) + Math.Abs(starty - endy)

        While openList.Count > 0

            Dim current As Integer() = openList(0)
            Dim currentKey As String = current(0).ToString() & "," & current(1).ToString()

            'scores all cells that have not got a score but are being checked-------------------
            For num As Integer = 1 To openList.Count - 1
                Dim node As Integer() = openList(num)
                Dim nodeKey As String = node(0).ToString() & "," & node(1).ToString()

                If fScore.ContainsKey(nodeKey) AndAlso fScore(nodeKey) < fScore(currentKey) Then
                    current = node
                    currentKey = nodeKey
                End If

            Next
            '-------------------------------------------------------------------------------------

            'checks if the end node is in the the list of checked cells, so it ends the program and
finds the best path depending on scores
            If current(0) = endx AndAlso current(1) = endy Then
```

```
                path.Add(current)

                While cameFrom.ContainsKey(currentKey)
                    current = cameFrom(currentKey)
                    path.Insert(0, current)
                    currentKey = current(0).ToString() & "," & current(1).ToString()
                End While

                Return path
            End If
            '-------------------------------------------------------------------------------------------
-------------------------------------
            openList.Remove(current)
            'list of all neighbouring cels to one being checked:
            Dim offsets As Integer()() = {New Integer() {1, 0}, New Integer() {-1, 0}, New Integer()
{0, 1}, New Integer() {0, -1}}
            'iterates between all neighbours changing their scores----------------------------
            For j As Integer = 0 To offsets.Length - 1

                Dim nx As Integer = current(0) + offsets(j)(0)
                Dim ny As Integer = current(1) + offsets(j)(1)

                'checks the cell being checked is empty(not a wall) and in the array:
                If nx < 0 Or nx >= map(0).Count Or ny < 0 Or ny >= map.Count Then Continue For
                If map(ny)(nx) = "b" Then Continue For

                Dim nKey As String = nx.ToString() & "," & ny.ToString()
                Dim tentativeG As Integer = gScore(currentKey) + 1

                'changes the score of cells already been checked, or adds a score to cells that have
not been checked---
                If Not gScore.ContainsKey(nKey) OrElse tentativeG < gScore(nKey) Then
                    cameFrom(nKey) = current
                    gScore(nKey) = tentativeG
                    fScore(nKey) = tentativeG + Math.Abs(nx - endx) + Math.Abs(ny - endy)

                    Dim found As Boolean = False

                    For k As Integer = 0 To openList.Count - 1
                        If openList(k)(0) = nx AndAlso openList(k)(1) = ny Then
                            found = True
                            Exit For
                        End If
                    Next

                    If Not found Then
                        openList.Add(New Integer() {nx, ny})
                    End If

                End If
                '-------------------------------------------------------------------------------------------
--------------------
            Next
            '-------------------------------------------------------------------------------------
        End While

        Return Nothing
    End Function
End Class
```

NPC:

```
Public Class Npc
    Inherits base_enemy

    Private plr1 As player

    Public textbox_open As Boolean = False
    Public speach_mission_txt As text


    Public mission_txt_list As List(Of String)
    Public mission_stats As List(Of Integer)
    Public mission_txt As text

    Private _already_clicked As Boolean = False
    Public ReadOnly Property already_clicked As Boolean
        Get
            Return _already_clicked
        End Get
    End Property

    Private _done_mis As Boolean = False


    Public Property done_mis
        Get
            Return _done_mis
        End Get
        Set(ByVal value)
            _done_mis = value
        End Set
    End Property



    Public Sub New(ByVal game As Form, ByVal plr As player, ByVal x As Integer, ByVal y As Integer,
ByVal wd As Integer, ByVal ht As Integer, ByVal mstxt As List(Of String), ByVal util As utilities)
        MyBase.new(game, ht, wd, x, y, 0, 0, util)

        Me.game = game
        Me.plr1 = plr

        Me.mission_txt_list = mstxt
        Me.box.BackColor = plr.box.BackColor

        Me.speach_mission_txt = New text(Me.game, Me.x + 20, Me.y - 20, 20, Me.mission_txt_list(0),
Color.Black)
        Me.speach_mission_txt.border = True

        Me.speach_mission_txt.box.Width = Me.mission_txt_list(0).Count() * 20

    End Sub

    Protected Overrides Sub update_arms(ByVal plr1 As player)

        Dim centerX As Integer = x + width \ 2
        Dim centerY As Integer = y + height \ 2

        'the goblin arms follow the player
        Dim dx As Integer = plr1.x - centerX
        Dim dy As Integer = plr1.y - centerY

        Dim length As Double = Math.Sqrt(dx * dx + dy * dy)
```

```
    If length = 0 Then length = 1

    ' Normalized direction vector
    Dim nx As Double = dx / length
    Dim ny As Double = dy / length

    ' Perpendicular vector (for arm spacing)
    Dim perpX As Double = -ny
    Dim perpY As Double = nx

    Dim reach As Integer = 40
    Dim separation As Integer = 10

    ' Final positions for each arm
    Dim armBaseX As Integer = centerX + CInt(nx * reach)
    Dim armBaseY As Integer = centerY + CInt(ny * reach)

    ' Left arm
    leftArm.Left = armBaseX + CInt(perpX * separation) - leftArm.Width \ 2
    leftArm.Top = armBaseY + CInt(perpY * separation) - leftArm.Height \ 2

    ' Right arm
    rightArm.Left = armBaseX - CInt(perpX * separation) - rightArm.Width \ 2
    rightArm.Top = armBaseY - CInt(perpY * separation) - rightArm.Height \ 2

End Sub

Public Function player_near()
    'player in distance
    If Math.Sqrt((Me.x - plr1.x) ^ 2 + (Me.y - plr1.y) ^ 2) <= 200 Then
        Return True
    End If

    Return False
End Function

Public Sub creat_text_box()
    'make sure the text box hasnt been deleted, so can still be loaded from memory
    If Not Me.textbox_open And Not Me.dead Then
        Me.speach_mission_txt.load()
    End If

    Me.textbox_open = True
End Sub

Public Sub click_handler(ByVal sender As Object, ByVal e As System.Windows.Forms.MouseEventArgs)

    If e.Button = MouseButtons.Left And Not already_clicked And Me.textbox_open Then
        Me._already_clicked = True
    End If

End Sub

Public Sub destroy()

    Me.game.Controls.Remove(Me.box)
    Me.game.Controls.Remove(Me.eyebox)
    Me.game.Controls.Remove(Me.eye)

    For Each limb In Me.all_limbs
        Me.game.Controls.Remove(limb)
    Next

    Me.speach_mission_txt.destroy()
```

```
        Me.dead = True
    End Sub

    Public Sub del_text_box()
        'delete the text box
        If Me.textbox_open Then

            Me.speach_mission_txt.destroy()

            Me.textbox_open = False

        End If

    End Sub

    Protected Friend Overloads Sub update()

        'checks wether the player is near and what to do about it
        If Me.player_near() And Not Me.dead Then
            Me.creat_text_box()

        ElseIf Not Me.dead Then
            Me.del_text_box()

        End If
        '-------------------------------------------------------

    End Sub

End Class
```

PLAYER_INTERFACE:

```
Imports System.Drawing
Imports System.Drawing.Imaging

Public Class player_inteface
    Private game As Form

    'also controls how missions are used and how UI is controlled

    'inventory:
    Protected Friend all_items As New List(Of item)

    Protected Friend itemDict As New Dictionary(Of String, List(Of Integer))
    '----------------------

    'missions:
    Public missions_npcs As New List(Of Npc)
    '----------------------

    'UI:
    Protected Friend hp_bar As bar
    Protected Friend mana_bar As bar
    Protected Friend attack_bar As bar
    Protected Friend boss_bar As bar

    Protected Friend ui_lis As New List(Of PictureBox)
    Protected Friend all_equip_boxes As New List(Of base_block)

    Protected Friend chosen_attack As New PictureBox

    Protected Friend grid_box As New PictureBox
    '----------------------



    Public Sub New(ByVal form_used As Form, ByVal util As utilities)

        Me.game = form_used

        Me.load_equipped()
        Me.itemDict = util.load_player_inteface_items()

        'grid_box-for the grid in the player_inteface
        grid_box.Width = 360
        grid_box.Height = 1000
        grid_box.Left = 850
        grid_box.Top = 50

    End Sub



    'UI:

    Public Sub load_bars()

        hp_bar = New bar(Me.game, 100, 20, 760, 160, Color.Red, Color.Green, -1, True)
        mana_bar = New bar(Me.game, 100, 20, 760, 100, Color.Blue, Color.Yellow, -1, True)
        attack_bar = New bar(Me.game, 60, 5, 765, 90, Color.Black, Color.LightGreen)
        boss_bar = New bar(Me.game, 500, 30, 170, 80, Color.Black, Color.DarkRed, 9, True, 15)

        'create the boss bar but remove it from the screen so it is still stored in memory
        Me.boss_bar.load_all()
```

```
        Me.boss_bar.destroy()
        '-------------------------------------------------------------------------------

        Me.attack_bar.load_all()
        Me.hp_bar.load_all()
        Me.mana_bar.load_all()

    End Sub

    Public Sub check_boss_bar(ByVal boss_room As Boolean)

        'if in a boss room load the boss bar
        If boss_room And Not Me.boss_bar.bar_alive Then
            Me.boss_bar.load_all()
            Me.boss_bar.bar_alive = True

        ElseIf Me.boss_bar.bar_alive And Not boss_room Then
            Me.boss_bar.destroy()
            Me.boss_bar.bar_alive = False

        End If
        '---------------------------------

    End Sub

    Public Sub load_ui(ByVal plr As player)

        'load in the chosen attack image
        chosen_attack.BackColor = Color.Transparent
        chosen_attack.Size = New Size(50, 50)

        chosen_attack.Image = plr.attack_img

        chosen_attack.Left = 770
        chosen_attack.Top = 45
        '--------------------------

    End Sub

    Public Sub load_ui_again(ByVal plr As player, ByVal boss_room As Boolean)

        'change the image of the attack being used by the player
        chosen_attack.Image = plr.attack_img
        chosen_attack.SizeMode = PictureBoxSizeMode.StretchImage

        Me.game.Controls.Add(chosen_attack)
        '-----------------------------------------------------

        Me.attack_bar.load_all()

        Me.check_boss_bar(boss_room)

    End Sub

    Public Sub delete_ui()
        'move the hp and mana bars out of the way for the player_inteface
        Me.hp_bar.stat_text.box.Left = 800
        Me.hp_bar.stat_text.box.Top = 570

        Me.mana_bar.stat_text.box.Left = 800
        Me.mana_bar.stat_text.box.Top = 520
        '-------------------------------------------------------

        Me.attack_bar.destroy()
        Me.game.Controls.Remove(Me.chosen_attack)
```

```
End Sub

Public Sub load_equipped()
    'create a box for each part of the body that can have armour
    Dim helmet As New PictureBox
    Dim body As New PictureBox
    Dim gauntlets As New PictureBox
    Dim legs As New PictureBox
    Dim boots As New PictureBox
    Dim equipe_r As New PictureBox
    Dim equipe_l As New PictureBox
    '------------------------------------------------------------

    'addd these peices to a ui list
    ui_lis.Add(helmet)
    ui_lis.Add(body)
    ui_lis.Add(gauntlets)
    ui_lis.Add(legs)
    ui_lis.Add(boots)
    ui_lis.Add(equipe_r)
    ui_lis.Add(equipe_l)
    '---------------------------

    'creates a grey box for each item
    For i = 0 To ui_lis.Count() - 1
        ui_lis(i).BackColor = Color.LightGray
        ui_lis(i).Size = New Size(50, 50)

        ui_lis(i).Left = 765
        ui_lis(i).Top = (i * 55) + 10

        ui_lis(i).SizeMode = PictureBoxSizeMode.StretchImage
    Next
    '--------------------------------

End Sub

Public Sub show_equipped()

    'show the armour peices used
    For Each item In Me.ui_lis
        Me.game.Controls.Add(item)
    Next

End Sub

Public Sub load_armour_stats()
    For ind = 0 To 8
        Dim new_arm As New base_block(Me.game, 20, 20, 25 + (25 * ind), 750)

        new_arm.box.BackColor = Color.Black
        new_arm.load()

        Me.all_equip_boxes.Add(new_arm)

    Next
End Sub


'missions:

Public Sub load_mis()

    'loads all chosen missions into veiw under the player player_inteface
```

```
        For mis = 0 To Me.missions_npcs.Count() - 1

            'NOT(has finished the mission and claimed the reward)
            If Not Me.missions_npcs(mis).done_mis Then

                Me.missions_npcs(mis).mission_txt.y = 500 + (mis * 50)

                Me.missions_npcs(mis).mission_txt.border = True
                Me.missions_npcs(mis).mission_txt.load()

            End If

        Next
        '------------------------------------------------------------

    End Sub

    Public Sub delete_mis()

        'gets rid of all the missions so a different ui can be shown
        For mis = 0 To Me.missions_npcs.Count() - 1

            If Not Me.missions_npcs(mis).done_mis Then

                Me.missions_npcs(mis).mission_txt.destroy()

            End If

        Next
        '------------------------------------------------------------

    End Sub

    'inventory:

    Public Function darken_bitmap(ByVal original As Bitmap, ByVal darkenfactor As Single)
        'darken the item picture when clicked on, giving feedback to the user

        'the matrix to darken an image
        Dim col_mat As New ColorMatrix(New Single()() {
            New Single() {darkenfactor, 0, 0, 0, 0},
            New Single() {0, darkenfactor, 0, 0, 0},
            New Single() {0, 0, darkenfactor, 0, 0},
            New Single() {0, 0, 0, 1, 0},
            New Single() {0, 0, 0, 0, 1}
        })

        Dim img_atri As New ImageAttributes()
        img_atri.SetColorMatrix(col_mat)

        Dim darkbmp As New Bitmap(original.Width, original.Height)

        Using g As Graphics = Graphics.FromImage(darkbmp)

            g.DrawImage(original, New Rectangle(0, 0, darkbmp.Width, darkbmp.Height), 0, 0,
original.Width, original.Height, GraphicsUnit.Pixel, img_atri)

        End Using
        '------------------------------------------------------------------
        Return darkbmp
    End Function

    Public Function not_in(ByVal val1 As String, ByVal lisz As List(Of item))
        'check if an item is in a list
        For Each item In lisz
```

```
        If item.name = val1 Then
            Return False
        End If

    Next
    '----------------------------

    Return True

End Function

Public Sub load_inventory_imgs()
    'load all item images in the player_inteface
    Dim x_coord As Integer = 0
    Dim y_coord As Integer = 0

    For item = 0 To all_items.Count() - 1

        all_items(item).x = 855 + (x_coord * 60)
        all_items(item).y = (y_coord * 60) + 55

        If all_items(item).x > 1200 Then
            x_coord = 0
            y_coord += 1
            all_items(item).x = 855 + (x_coord * 60)
            all_items(item).y = (y_coord * 60) + 55
        End If

        x_coord += 1

        all_items(item).load()
        all_items(item).show_amount()

        all_items(item).loaded_text = True

    Next
    '-------------------------------------------
End Sub

Public Sub load_inventory()

    Me.load_inventory_imgs()

    'change location of the bars
    Me.hp_bar.new_loc(800, 550)
    Me.mana_bar.new_loc(800, 500)
    '----------------------------

End Sub

Public Sub draw_grid(ByVal cellSize As Integer, ByVal cols As Integer, ByVal rows As Integer)
    'draws the grid looking area for the player_inteface

    Dim bit_map As New Bitmap(Me.grid_box.Width, Me.grid_box.Height)

    Dim g As Graphics = Graphics.FromImage(bit_map)
    Dim pen As New Pen(Color.LightGray, 5)

    ' Draw vertical lines
    For col As Integer = 0 To cols

        Dim x As Integer = col * cellSize
        g.DrawLine(pen, x, 0, x, rows * cellSize)
```

```
        Next

        ' Draw horizontal lines
        For row As Integer = 0 To rows

            Dim y As Integer = row * cellSize
            g.DrawLine(pen, 0, y, cols * cellSize, y)

        Next

        Me.grid_box.Image = bit_map

        Me.game.Controls.Add(Me.grid_box)
        '-----------------------------------------------

    End Sub

    Public Sub delete_inventory()
        'delete all items in inventory
        For Each item In Me.all_items

            item.destroy()

        Next

        For Each i In Me.ui_lis
            Me.game.Controls.Remove(i)
        Next
        Me.game.Controls.Remove(Me.grid_box)

        'move hp and mana bar to their original location
        Me.hp_bar.new_loc(755, 160)
        Me.mana_bar.new_loc(755, 100)

        Me.delete_mis()

    End Sub

    Public Sub add(ByVal name As String)
        Dim where As Boolean = Me.not_in(name, Me.all_items)

        If where Or Not (Me.itemDict(name)(0) <> 2 And Me.itemDict(name)(0) <> 3) Then
            'if it is not in the inventory add it
            Dim new_item As New item(Me.game, 50, 50, 0, 0, name)

            new_item.make_image()

            'make it clickable
            AddHandler new_item.box.MouseClick, AddressOf new_item.item_MouseClick

            Me.all_items.Add(new_item)

        ElseIf Me.itemDict(name)(0) <> 2 And Me.itemDict(name)(0) <> 3 Then

            'if it is in the inventory stack it (if stackable)
            For Each item In Me.all_items

                If item.name = name Then
                    item.amount += 1
                End If

            Next

        End If
```

```
    End Sub

    Public Sub clear_inventory()
        'so the list of all_items doesnt change whilst being iterated through
        Dim getrifoff As New List(Of Integer)

        'if it is more then 1 then add it to the list
        For item = 0 To all_items.Count() - 1

            If all_items(item).amount < 1 Then
                getrifoff.Add(item)
            End If

        Next
        '------------------------------------------------------------

        For Each pic In getrifoff

            Me.game.Controls.Remove(all_items(pic).box)
            Me.game.Controls.Remove(all_items(pic).amount_text.box)

            all_items.RemoveAt(pic)
        Next

    End Sub



    Public Function update(ByVal tick As Integer, ByVal plr As player)
        Me.clear_inventory()

        chosen_attack.Image = plr.attack_img

        'update the armour slots depending on wther items are equipped
        For armour_slot = 0 To plr.equipped.Count() - 1

            If plr.equipped(armour_slot) <> Nothing And plr.equipped(armour_slot) <> "0" Then

                Dim armour_img_path As String = "\\cc4sjd8\2024 Intake\24026166\Visual Studio
2010\Projects\practice_courswork_platformer\invent_imgs\" & plr.equipped(armour_slot) & ".png"

                ui_lis(armour_slot).Image = New Bitmap(armour_img_path)

            ElseIf plr.equipped(armour_slot) = "0" Then

                'get rid of old slot
                ui_lis(armour_slot).Dispose()

                'add new slot
                Dim new_slot As New PictureBox()

                new_slot.BackColor = Color.LightGray
                new_slot.Size = New Size(50, 50)

                new_slot.Left = 765
                new_slot.Top = (armour_slot * 55) + 10

                new_slot.SizeMode = PictureBoxSizeMode.StretchImage

                ui_lis(armour_slot) = new_slot

            End If
```

```
        Next
        '-------------------------------------------------------------

        'check for items clicks
        For Each item In all_items

            item.update()

            If tick Mod 6 = 0 Then
                item.box.Image = item.image
            End If

            If item.touched Then

                'darken the image and return a full tuple

                item.box.Image = darken_bitmap(item.image, 0.6)
                item.touched = False

                Return Tuple.Create(item.name, Me.itemDict(item.name)(0), item)
            End If
        Next
        '------------------------

        'return the item name, how many there are, the item class
        Return Tuple.Create("", -1, New item(Me.game, 0, 0, 0, 0, "nothing"))
        '---------------------------------null item class

    End Function
End Class
```

PLAYER_CLASS:

```
Imports System.Drawing
Imports System.Drawing.Imaging


Public Class player
    Inherits base_block

    Private util As utilities

    Private _max_hp As Integer = 100

    Public Property max_hp
        Get
            Return _max_hp
        End Get
        Set(ByVal value)
            _max_hp = value
        End Set
    End Property

    Private _hp As Integer = 100

    Public Property hp
        Get
            Return _hp
        End Get
        Set(ByVal value)
            _hp = value
        End Set
    End Property

    Private _max_mana As Integer = 100

    Public Property max_mana
        Get
            Return _max_mana
        End Get
        Set(ByVal value)
            _max_mana = value
        End Set
    End Property

    Public _mana As Integer = 100

    Public Property mana
        Get
            Return _mana
        End Get
        Set(ByVal value)
            _mana = value
        End Set
    End Property

    Public max_armour As Integer = 100
    Public armour As Integer = 0

    Public all_stats(3) As Integer

    Public enemy_lisz As List(Of base_enemy)
    Public door_lisz As List(Of door)
    Public wall_lisz As List(Of wall)
    Public floor_items_lisz As List(Of item)
```

```
Public npc_lisz As List(Of Npc)

Public live_attack As New List(Of base_attack)
Public dead_attack As New List(Of base_attack)

Public attack_dict As New Dictionary(Of String, List(Of String))
Public chosen_attack As String = "fireball"

Public index_on_attack As Integer = 0

Public tick As Double

Public my_interface As player_inteface

Public open_inv As Boolean = False
Public opened_ui As Boolean = True
Public loaded_all As Boolean = True

Public equipped(8) As String
Public collided_flooritem As Integer = -1

Public my_sheild As sheild

Public touch_door As Boolean = False
Public which_door As door = Nothing

Public mousex As Integer = 0
Public mousey As Integer = 0


Public leftLeg As New PictureBox()
Public rightLeg As New PictureBox()
Public leftArm As New PictureBox()
Public rightArm As New PictureBox()

Public legWidth As Integer = width \ 3
Public legHeight As Integer = height \ 4
Public armW As Integer = width \ 4
Public armH As Integer = height \ 3

Public all_limbs As New List(Of PictureBox)
Public leg_move As String = "R"

Public attack_img As Bitmap

Public all_missions As Dictionary(Of List(Of String), List(Of Integer))

Public killed_enemy_amounts As New Dictionary(Of String, Integer)
Public start_level As Boolean = False

Private _boss_room As Boolean = False

Public Property boss_room As Boolean
    Get
        Return _boss_room
    End Get
    Set(ByVal value As Boolean)
        _boss_room = value
    End Set
End Property

Private _finished_game As Boolean = False

Public Property finished_game As Boolean
    Get
```

```
            Return _finished_game
        End Get
        Set(ByVal value As Boolean)
            _finished_game = value
        End Set
    End Property


    Sub New(ByVal game As Form, ByVal height As Integer, ByVal width As Integer, ByVal x As Integer,
ByVal y As Integer, ByVal util As utilities)
        MyBase.New(game, height, width, x, y)

        Me.box.BackColor = Color.Blue
        Me.util = util


        ' Legs
        legWidth = width \ 3
        legHeight = height \ 4
        'arms
        armW = width \ 4
        armH = height \ 3


    End Sub

    Public Sub key_handler(ByVal sender As Object, ByVal e As KeyPressEventArgs)

        Select Case e.KeyChar
            Case "w"c
                Me.yvel -= 3

            Case "s"c
                Me.yvel += 3

            Case "d"c
                Me.xvel += 3

            Case "a"c
                Me.xvel -= 3

            Case "i"c
                'open the player_inteface if the sheild is not being used
                If Not Me.my_sheild.sheild_use Then
                    'open ui when player_inteface closed and vice-versa
                    If Not open_inv Then
                        Me.open_inv = True
                        Me.opened_ui = False
                        loaded_all = True
                    Else
                        Me.open_inv = False
                        Me.opened_ui = True
                        loaded_all = True
                    End If

                End If

        End Select

    End Sub

    Public Function get_angle_to_mouse(ByVal mx As Double, ByVal msy As Double)

        Dim deltaX As Integer = mx - Me.x
        Dim deltaY As Integer = msy - Me.y
```

```
        Dim angle As Double = Math.Atan2(deltaY, deltaX)

        Return angle ' Angle in radians

    End Function

    Public Sub click_handler(ByVal sender As Object, ByVal e As System.Windows.Forms.MouseEventArgs)

        If e.Button = MouseButtons.Left Then
            'use attack atributes saved in a text file to create attack
            If Me.mana >= Me.attack_dict(chosen_attack)(6) And Me.attack_dict(chosen_attack)(5) <= 0
Then

                Me.attack_dict(chosen_attack)(5) = Me.attack_dict(chosen_attack)(4)
                Me.mana -= Me.attack_dict(chosen_attack)(6)

                Select Case chosen_attack
                    Case "fireball"
                        load_fireball()
                    Case "slash"
                        load_slash()
                    Case "lightning"
                        load_lightning()
                End Select

            End If


            'if the sheild is equipped and the player_inteface is closed
        ElseIf e.Button = MouseButtons.Right And Not Me.open_inv Then

            Me.my_sheild.loaded = False

            If Not Me.my_sheild.sheild_use And Me.equipped(6) <> Nothing Then

                Me.my_sheild.sheild_use = True

            ElseIf Me.my_sheild.sheild_use Then

                Me.my_sheild.sheild_use = False

            End If

        End If

    End Sub

    Public Sub scroll_wheel(ByVal sender As Object, ByVal e As System.Windows.Forms.MouseEventArgs)
        'use the scroll wheel to choose between attacks

        Me.index_on_attack += 1


        If Me.index_on_attack > attack_dict.Count() - 1 Then
            Me.index_on_attack = 0
        End If

        Me.chosen_attack = Me.attack_dict.Keys(Me.index_on_attack)

        If chosen_attack = "fireball" Then
            Me.attack_img = New Bitmap(Me.attack_dict(chosen_attack)(2))
        ElseIf chosen_attack = "slash" Then
            Me.attack_img = New Bitmap(Me.attack_dict(chosen_attack)(2))
        ElseIf chosen_attack = "lightning" Then
```

```
            Me.attack_img = New Bitmap(Me.attack_dict(chosen_attack)(2))
        End If
        '-----------------------------------------------
    End Sub


    Public Sub update_bars()

        Me.my_interface.mana_bar.new_size(mana / max_mana)
        Me.my_interface.hp_bar.new_size(Me.hp / Me.max_hp)
        Me.my_interface.attack_bar.new_size((Me.attack_dict(Me.chosen_attack)(4) -
Me.attack_dict(Me.chosen_attack)(5)) / Me.attack_dict(Me.chosen_attack)(4))

    End Sub

    Public Sub update_attacks()
        'make attacks cooldown after being used
        For Each atk In Me.attack_dict



            If Me.attack_dict(atk.Key)(5) > 0 And Me.tick Mod 2 = 0 Then
                Me.attack_dict(atk.Key)(5) -= 0.25
            End If

        Next
        '----------------------------------------

        'chek all live attacks so they dont live too long
        For i = 0 To Me.live_attack.Count() - 1

            If Me.live_attack(i).check_alive(Me.tick) Then

                dead_attack.Add(live_attack(i))

                Me.game.Controls.Remove(Me.live_attack(i).box)

                live_attack.RemoveAt(i)

                Exit For

            End If

        Next
        '-----------------------------------------------

        Me.dead_attack.Clear()

    End Sub

    Public Sub update_invent_or_ui()
        'if the player eye object has been equipped end the game


        'if an item in the player_inteface has been clicked activate it
        Dim new_clicked As Tuple(Of String, Integer, item) = Me.my_interface.update(Me.tick, Me)

        If new_clicked.Item1 <> "" Then

            If Me.my_interface.itemDict(new_clicked.Item1)(0) = 2 Or
Me.my_interface.itemDict(new_clicked.Item1)(0) = 3 Then
                'add armour peice

                Me.equipped(Me.my_interface.itemDict(new_clicked.Item1)(1)) = new_clicked.Item1
```

```
                new_clicked.Item3.amount -= 1


            ElseIf Me.my_interface.itemDict(new_clicked.Item1)(0) = 1 Then

                If Me.my_interface.itemDict(new_clicked.Item1)(1) <> 0 Then
                    'If hp change, add hp, make it the max hp, if hp is already full dont change
anything
                    If Me.hp >= Me.max_hp Then
                        new_clicked.Item3.take_away = False

                    ElseIf Me.hp + Me.my_interface.itemDict(new_clicked.Item1)(1) < Me.max_hp Then
                        Me.hp += Me.my_interface.itemDict(new_clicked.Item1)(1)
                        new_clicked.Item3.take_away = True

                    ElseIf Me.hp + Me.my_interface.itemDict(new_clicked.Item1)(1) > max_hp Then
                        Me.hp = Me.max_hp
                        new_clicked.Item3.take_away = True

                    End If

                    If new_clicked.Item3.take_away = True Then
                        new_clicked.Item3.amount -= 1
                        new_clicked.Item3.take_away = False

                    End If

                End If

                If Me.my_interface.itemDict(new_clicked.Item1)(2) <> 0 Then

                    'If mana change, add mana, make it the max mana, if mana is already full dont
allow it to be clicked
                    If Me.mana >= Me.max_mana Then
                        new_clicked.Item3.take_away = False

                    ElseIf Me.mana + Me.my_interface.itemDict(new_clicked.Item1)(2) <= Me.max_mana
Then
                        Me.mana += Me.my_interface.itemDict(new_clicked.Item1)(2)
                        new_clicked.Item3.take_away = True

                    ElseIf Me.mana + Me.my_interface.itemDict(new_clicked.Item1)(2) > max_mana Then
                        Me.mana = Me.max_mana
                        new_clicked.Item3.take_away = True

                    End If

                    'if stackable take an item away
                    If new_clicked.Item3.take_away = True Then
                        new_clicked.Item3.amount -= 1
                        new_clicked.Item3.take_away = False

                    End If

                End If

            End If
        End If


    End Sub

    Public Sub update_armour()

        Dim temp_armour As Integer = 0
```

```
        For Each arm In Me.equipped

            If (arm <> "0" And arm <> "") AndAlso Me.my_interface.itemDict(arm)(0) = "3" Then
                temp_armour += Me.my_interface.itemDict(arm)(4)
            End If

        Next

        If temp_armour > 100 Then temp_armour = 100

        For block = 0 To Me.my_interface.all_equip_boxes.Count()
            If temp_armour > (block + 1) * 10 Then
                Me.my_interface.all_equip_boxes(block).box.BackColor = Color.White
            End If
        Next

        Me.armour = temp_armour


    End Sub

    Public Sub update_mission()
        'if the npc has been clicked add to player missions
        For Each Npc In Me.npc_lisz

            If Npc.already_clicked And Not Npc.dead And Npc.textbox_open Then

                Dim val_lis As New List(Of Integer)

                val_lis = Me.all_missions(Npc.mission_txt_list)
                val_lis.Add(0)
                Npc.mission_stats = val_lis

                Dim mission_text As String = Npc.mission_txt_list(0) & " " & val_lis(2) & "/" &
val_lis(0)
                Npc.mission_txt = New text(Me.game, 950, 0, 15, mission_text, Color.Yellow)

                AddHandler Npc.mission_txt.box.MouseClick, AddressOf Npc.mission_txt.click_handler
                Me.my_interface.missions_npcs.Add(Npc)

                Npc.destroy()

            End If

        Next
        '------------------------------------------------

        'check if any missions have been completed and therefore make them green
        For Each mis In Me.my_interface.missions_npcs

            If Me.killed_enemy_amounts.ContainsKey(mis.mission_txt_list(1)) Then

                mis.mission_stats(2) = Me.killed_enemy_amounts(mis.mission_txt_list(1))
                mis.mission_txt.my_text = mis.mission_txt_list(0) & " " & mis.mission_stats(2) & "/"
& mis.mission_stats(0)

            End If

            'if there are more or equal kills to the number needed
            If mis.mission_stats(2) >= mis.mission_stats(0) Then
                mis.mission_txt.colour = Color.Green
            End If

            'if cliked give reward and tak away from missions
```

```
            If mis.mission_txt.button_clicked And mis.mission_stats(2) >= mis.mission_stats(0) Then

                For num = 0 To mis.mission_stats(1) - 1
                    Me.my_interface.add(mis.mission_txt_list(2))
                Next

                mis.done_mis = True

                mis.mission_txt.button_clicked = False
                mis.mission_txt.destroy()

            End If

        Next
        '-----------------------------------------------------------------
    End Sub


    Public Sub load_slash()
        Dim xer As Integer = Me.x
        Dim yer As Integer = Me.y


        ' Calculate direction
        Dim dx As Integer = mousex - Me.x
        Dim dy As Integer = mousey - Me.y


        ' Normalize and extend
        Dim scale As Double = 0.3 ' length of the line
        xer = CInt(Me.x + dx * scale)
        yer = CInt(Me.y + dy * scale)


        Dim atk As New base_attack(Me.game, 60, 30, xer, yer, 4, chosen_attack, New
Bitmap(Me.attack_dict("slash")(2)), Me.attack_dict("slash")(1) * (1 + Me.all_stats(0) / 10),
get_angle_to_mouse(mousex, mousey))

        atk.spawn_time = Me.tick
        atk.load()

        live_attack.Add(atk)

    End Sub

    Public Sub load_fireball()

        Dim atk As New base_attack(Me.game, 30, 30, Me.x, Me.y, 20, chosen_attack, New
Bitmap(Me.attack_dict("fireball")(2)), Me.attack_dict("fireball")(1) * (1 + Me.all_stats(2) / 10),
get_angle_to_mouse(mousex, mousey))

        atk.move(mousex, mousey, Me, 18)
        atk.spawn_time = Me.tick

        atk.load()

        live_attack.Add(atk)
    End Sub

    Public Sub load_lightning()
        'load lightning into a circle of 6 around the player
        Dim lightning_radius As Integer = 125

        For i = 0 To 6
```

```
            Dim angle As Integer = i * 60
            Dim radian As Double = angle * Math.PI / 180

            Dim xer As Integer = Me.x + lightning_radius * Math.Cos(radian)
            Dim yer As Integer = Me.y + lightning_radius * Math.Sin(radian)

            Dim atk As New base_attack(Me.game, 50, 50, xer, yer, 5, chosen_attack, New
Bitmap(Me.attack_dict("lightning")(2)), Me.attack_dict("lightning")(1) * (1 + Me.all_stats(2) / 10),
radian)

            atk.spawn_time = Me.tick

            atk.load()
            live_attack.Add(atk)

        Next

    End Sub

    Public Sub load_attacks()

        'load the atributes for each attack into a dictionary
        Me.attack_dict = Me.util.load_attacks("plren")
        '------------------------------------------------

    End Sub

    Public Sub load_invent()

        If Me.loaded_all Then
            'load the inventory if its not been loaded into memeory
            If Me.open_inv Then
                Me.my_interface.show_equipped()
                Me.game.Size = New Size(1300, 850)

                Me.my_interface.load_inventory()
                Me.my_interface.draw_grid(60, 6, 7)
                Me.my_interface.load_mis()

                'close the inventory
            ElseIf Not Me.open_inv Then
                Me.game.Size = New Size(900, 850)
                Me.my_interface.delete_inventory()

            End If
            'open the ui if its been loaded
            If Me.opened_ui Then
                Me.my_interface.load_ui_again(Me, Me.boss_room)
                'close the ui
            ElseIf Not Me.opened_ui Then
                Me.my_interface.delete_ui()

            End If
            'only do these once per open or close
            Me.loaded_all = False

        End If

    End Sub


    Public Sub collisions()
        'colllided with doors and choose where the door leads
        For Each door In door_lisz
            If door.box.Bounds.IntersectsWith(Me.box.Bounds) Then
```

```
                If door.goes_to = "bossflr" Then
                    touch_door = True
                    which_door = door
                Else
                    touch_door = True
                    which_door = door
                End If

            End If
        Next
        '-------------------------------------------------


        For Each ene In enemy_lisz

            If ene.type_magic Then

                For Each atk In ene.live_attacks

                    'check if the shield blocks the attack or not
                    If Me.my_sheild.sheild_use And ene.attack_dict(atk.typeofattack)(1) <> -1 Then

                        If Me.my_sheild.collision(-ene.attack_dict(atk.typeofattack)(1), atk) Then
                            atk.dead = True
                        ElseIf Me.player_box_collision(-ene.attack_dict(atk.typeofattack)(1), atk)
Then

                            atk.dead = True
                        End If

                    ElseIf Me.my_sheild.sheild_use = False And ene.attack_dict(atk.typeofattack)(1)
<> -1 Then

                        If Me.player_box_collision(-ene.attack_dict(atk.typeofattack)(1), atk) Then
                            atk.dead = True
                        End If

                    End If
                    '-----------------------------------

                    'teleport the player to the invisible boss if hit
                    If atk.typeofattack = "invisi_ball" Then

                        If atk.box.Bounds.IntersectsWith(Me.box.Bounds) Then
                            Me.x = ene.x + (25 + (Me.util.Rand0())) - 75
                            Me.y = ene.y + (25 + (Me.util.Rand0())) - 75
                        End If

                    End If

                Next
            End If

            'collisions for the enemy cubes
            If ene.type_enemy = "invisi" Then

                For Each cube In ene.cubes

                    If cube.box.Bounds.IntersectsWith(Me.box.Bounds) Then
                        Me.hp -= 1
                    End If

                Next

            End If
```

```
            '--------------------------

            'check if the sheild has collided with an enemy or not
            If Me.my_sheild.sheild_use Then

                If Me.my_sheild.collision(-1, ene) Then
                    pushback(ene, Me.my_sheild.x, Me.my_sheild.y)

                ElseIf Me.player_box_collision(-1, ene) Then
                    pushback(ene, Me.x, Me.y)

                End If

            Else

                If Me.player_box_collision(-1, ene) Then
                    pushback(ene, Me.x, Me.y)
                End If

            End If
            '-----------------------------------------------------

        Next


        For Each wal In wall_lisz
            If wal.box.Bounds.IntersectsWith(Me.box.Bounds) Then
                ' Calculate player and wall boundaries
                Dim player_left As Integer = Me.x
                Dim player_right As Integer = Me.x + Me.width
                Dim player_top As Integer = Me.y
                Dim player_bottom As Integer = Me.y + Me.height

                Dim wall_left As Integer = wal.x
                Dim wall_right As Integer = wal.x + wal.width
                Dim wall_top As Integer = wal.y
                Dim wall_bottom As Integer = wal.y + wal.height

                ' Determine overlap distances
                Dim overlap_left As Integer = player_right - wall_left
                Dim overlap_right As Integer = wall_right - player_left
                Dim overlap_top As Integer = player_bottom - wall_top
                Dim overlap_bottom As Integer = wall_bottom - player_top


                Dim min_overlap As Integer = Math.Min(Math.Min(overlap_left, overlap_right),
Math.Min(overlap_top, overlap_bottom))

                ' Push the player out depending on which side is smallest
                If min_overlap = overlap_left Then

                    Me.x -= overlap_left
                    Me.xvel = 0
                ElseIf min_overlap = overlap_right Then

                    Me.x += overlap_right
                    Me.xvel = 0
                ElseIf min_overlap = overlap_top Then

                    Me.y -= overlap_top
                    Me.yvel = 0
                ElseIf min_overlap = overlap_bottom Then

                    Me.y += overlap_bottom
                    Me.yvel = 0
```

```
            End If
        End If
    Next


    'check if the player has collided with a pick up item on the floor

    'update wich item has been collided with (-1 if none)
    Me.collided_flooritem = -1

    For item = 0 To Me.floor_items_lisz.Count() - 1

        If Me.floor_items_lisz(item).box.Bounds.IntersectsWith(Me.box.Bounds) Then

            Me.my_interface.add(Me.floor_items_lisz(item).name)
            Me.collided_flooritem = item

        End If
    Next
    '-----------------------------------------------------------------


End Sub

Public Function player_box_collision(ByVal hit_taken As Integer, ByVal target As Object)
    'if the collision with the player happens
    If target.box.Bounds.IntersectsWith(Me.box.Bounds) Then
        'changes the damage taken depending on the amount of armour used
        Me.hp += hit_taken * (1 - (Me.armour / Me.max_armour))

        Return True
    End If

    Return False
End Function

Public Sub pushback(ByVal entity As base_enemy, ByVal xer As Integer, ByVal yer As Integer)

    If entity.x < xer Then
        entity.x -= entity.width / 2
    End If

    If entity.x > xer Then
        entity.x += entity.width / 2
    End If

    If entity.y < yer Then
        entity.y -= entity.height / 2
    End If

    If entity.y > yer Then
        entity.y += entity.height / 2
    End If

End Sub


Public Sub limb_load()

    leftLeg.Width = legWidth
    leftLeg.Height = legHeight

    leftLeg.Left = x
    leftLeg.Top = y + height
    leftLeg.BackColor = Color.Black
```

```
        rightLeg.Width = legWidth
        rightLeg.Height = legHeight

        rightLeg.Left = x + (width - legWidth)
        rightLeg.Top = y + height
        rightLeg.BackColor = Color.Black


        leftArm.Width = armW
        leftArm.Height = armH

        leftArm.Left = Me.x - leftArm.Width ' To the left of the box
        leftArm.Top = Me.y + Me.height \ 3
        leftArm.BackColor = Color.Black

        rightArm.Width = armW
        rightArm.Height = armH

        rightArm.Left = Me.x + Me.width ' To the right of the box
        rightArm.Top = Me.y + Me.height \ 3
        rightArm.BackColor = Color.Black


        all_limbs.Add(leftLeg)
        all_limbs.Add(rightLeg)
        all_limbs.Add(leftArm)
        all_limbs.Add(rightArm)


        game.Controls.Add(leftLeg)
        game.Controls.Add(rightLeg)
        game.Controls.Add(leftArm)
        game.Controls.Add(rightArm)

    End Sub

    Public Sub update_arms(ByVal x As Integer, ByVal y As Integer)

        Dim dx As Integer = x - Me.x
        Dim dy As Integer = y - Me.y

        Dim length As Double = Math.Sqrt(dx * dx + dy * dy)
        If length = 0 Then length = 1

        ' Normalized direction vector
        Dim nx As Double = dx / length
        Dim ny As Double = dy / length

        ' Perpendicular vector (for arm spacing)
        Dim perpX As Double = -ny
        Dim perpY As Double = nx

        Dim reach As Integer = 40
        Dim separation As Integer = 10

        ' Final positions for each arm
        Dim armBaseX As Integer = Me.x + 25 + CInt(nx * reach)
        Dim armBaseY As Integer = Me.y + 25 + CInt(ny * reach)

        ' Left arm
        leftArm.Left = armBaseX + CInt(perpX * separation) - leftArm.Width \ 2
        leftArm.Top = armBaseY + CInt(perpY * separation) - leftArm.Height \ 2
```

```
        ' Right arm
        rightArm.Left = armBaseX - CInt(perpX * separation) - rightArm.Width \ 2
        rightArm.Top = armBaseY - CInt(perpY * separation) - rightArm.Height \ 2

    End Sub

    Public Sub movement_nums()
        'a max velocity
        If xvel > 6 Then
            Me.xvel = 6

        ElseIf xvel < -6 Then
            Me.xvel = -6

        End If

        If yvel > 6 Then
            Me.yvel = 6

        ElseIf yvel < -6 Then
            Me.yvel = -6

        End If

        'add the user defined stat to the player speed
        Me.x += Me.xvel * (1 + (Me.all_stats(1) / 10))
        Me.y += Me.yvel * (1 + (Me.all_stats(1) / 10))

    End Sub

    Public Sub move_limbs()

        'every 2 seconds change wich leg id move-to simulate walking
        If Me.tick Mod 2 = 0 Then

            If leg_move = "R" Then
                rightLeg.Left = x + (width - legWidth)
                rightLeg.Top = y + height
                Me.leg_move = "L"

            ElseIf leg_move = "L" Then
                leftLeg.Left = x
                leftLeg.Top = y + height
                Me.leg_move = "R"

            End If

        End If

        Me.update_arms(mousex, mousey)

    End Sub


    Public Sub update(ByVal all_wallz As List(Of wall), ByVal timer As Integer, ByVal mouses_x As
Integer, ByVal mouse_y As Integer, ByVal enemies As List(Of base_enemy), ByVal doorz As List(Of
door), ByVal flooritems As List(Of item), ByVal allnpc As List(Of Npc))

        Me.tick = timer

        Me.touch_door = False

        Me.enemy_lisz = enemies
        Me.door_lisz = doorz
```

```
        Me.wall_lisz = all_wallz
        Me.floor_items_lisz = flooritems
        Me.npc_lisz = allnpc

        Me.movement_nums()

        Me.my_sheild.update()

        Me.move_limbs()

        Me.update_mission()

        Me.load_invent()

        Me.update_invent_or_ui()
        Me.update_armour()

        Me.collisions()

        Me.update_attacks()

        Me.update_bars()

        For Each i In Me.live_attack
            i.update()
            i.box.BringToFront()
        Next

        Me.mousex = mouses_x
        Me.mousey = mouse_y

        Me.box.Left = Me.x
        Me.box.Top = Me.y

    End Sub

End Class
```

RAY_CAST:

```
Public Class ray_cast
    Private nodes As New List(Of List(Of Array))
    Private wallz As New List(Of List(Of Array))

    Private box As List(Of String)

    Public Sub New(ByVal string_map As List(Of String))
        Me.box = string_map

        Me.load_wallz()
    End Sub

    Private Sub connect_nodes()
        'connect nodes that are next to each other
        For Each node In nodes


            Dim my_coords As Array = node(0)

            Dim all_wallz As List(Of Integer) = find_wall(my_coords, wallz)

            'node: mycoords, north, south, east, west, nw, ne, sw, se
            Dim south_done As Boolean = False
            Dim east_done As Boolean = False
            Dim sw_done As Boolean = False
            Dim se_done As Boolean = False


            For Each wl In all_wallz

                'vertical
                If wallz(wl)(0)(1) < my_coords(1) And wallz(wl)(0)(0) = my_coords(0) Then

                    Dim new_endpoint As Array

                    If node(2)(0) <> -1 And node(2)(1) <> -1 Then
                        new_endpoint = node(2)
                    Else
                        new_endpoint = my_coords
                    End If

                    wallz(wl)(1) = new_endpoint
                    south_done = True
                End If

                'horizontal
                If wallz(wl)(0)(0) < my_coords(0) And wallz(wl)(0)(1) = my_coords(1) Then

                    Dim new_endpoint As Array

                    If node(3)(0) <> -1 And node(3)(1) <> -1 Then
                        new_endpoint = node(3)
                    Else
                        new_endpoint = my_coords
                    End If

                    wallz(wl)(1) = new_endpoint
                    east_done = True
                End If

                'diagnoll left, sw
                If wallz(wl)(0)(0) > my_coords(0) And wallz(wl)(0)(1) < my_coords(1) Then
```

```
            Dim new_endpoint As Array

            If node(7)(0) <> -1 And node(7)(1) <> -1 Then
                new_endpoint = node(7)
            Else
                new_endpoint = my_coords
            End If

            wallz(wl)(1) = new_endpoint
            sw_done = True
        End If

        'diagnoll right, se
        If wallz(wl)(0)(0) < my_coords(0) And wallz(wl)(0)(1) < my_coords(1) Then

            Dim new_endpoint As Array

            If node(8)(0) <> -1 And node(8)(1) <> -1 Then
                new_endpoint = node(8)
            Else
                new_endpoint = my_coords
            End If

            wallz(wl)(1) = new_endpoint
            se_done = True
        End If
    Next
    'node: mycoords, north, south, east, west, nw, ne, sw, se
    'wallz havent been done

    If all_wallz.Count < 1 Then
        Dim start_coords As Array = my_coords
        Dim collumn As New List(Of Array)

        collumn.Add(start_coords)

        wallz.Add(collumn)
    End If

    If Not south_done And Not empty(node(2)) Then
        Dim start_coords As Array = my_coords
        Dim end_coords As Array = node(2)

        Dim collumn As New List(Of Array)

        collumn.Add(start_coords)
        collumn.Add(end_coords)

        wallz.Add(collumn)
    End If

    If Not east_done And Not empty(node(3)) Then
        Dim start_coords As Array = my_coords
        Dim end_coords As Array = node(3)

        Dim collumn As New List(Of Array)

        collumn.Add(start_coords)
        collumn.Add(end_coords)

        wallz.Add(collumn)
    End If

    If Not sw_done And Not empty(node(7)) Then
```

```
            Dim start_coords As Array = my_coords
            Dim end_coords As Array = node(7)

            Dim collumn As New List(Of Array)

            collumn.Add(start_coords)
            collumn.Add(end_coords)

            wallz.Add(collumn)
        End If

        If Not se_done And Not empty(node(8)) Then
            Dim start_coords As Array = my_coords
            Dim end_coords As Array = node(8)

            Dim collumn As New List(Of Array)

            collumn.Add(start_coords)
            collumn.Add(end_coords)

            wallz.Add(collumn)
        End If

    Next
    '-----------------------------------------------------------------
End Sub

Private Sub create_nodes()
    'create nodes for certain walls
    For y_now = 1 To box.Count() - 1
        For x_now = 0 To box(y_now).Length() - 1
            Dim new_node As New List(Of Array)

            Dim on_coords(2) As Integer
            Dim N_coords(2) As Integer
            Dim E_coords(2) As Integer
            Dim S_coords(2) As Integer
            Dim W_coords(2) As Integer
            Dim NW_coords(2) As Integer
            Dim NE_coords(2) As Integer
            Dim SW_coords(2) As Integer
            Dim SE_coords(2) As Integer

            If box(y_now)(x_now) = "b" Then

                on_coords(0) = x_now
                on_coords(1) = y_now

                ' NORTH
                If y_now - 1 > 0 AndAlso box(y_now - 1)(x_now) = "b" Then
                    N_coords(0) = x_now
                    N_coords(1) = y_now - 1
                Else
                    N_coords(0) = -1
                    N_coords(1) = -1
                End If

                ' SOUTH
                If y_now + 1 < box.Count() - 1 AndAlso box(y_now + 1)(x_now) = "b" Then
                    S_coords(0) = x_now
                    S_coords(1) = y_now + 1
                Else
                    S_coords(0) = -1
                    S_coords(1) = -1
                End If
```

```
                    ' EAST
                    If x_now - 1 > 0 AndAlso box(y_now)(x_now - 1) = "b" Then
                        W_coords(0) = x_now - 1
                        W_coords(1) = y_now
                    Else
                        W_coords(0) = -1
                        W_coords(1) = -1
                    End If

                    ' WEST
                    If x_now + 1 < box(y_now).Length() - 1 AndAlso box(y_now)(x_now + 1) = "b" Then
                        E_coords(0) = x_now + 1
                        E_coords(1) = y_now
                    Else
                        E_coords(0) = -1
                        E_coords(1) = -1
                    End If

                    ' NORTH-WEST
                    If y_now - 1 > 0 AndAlso x_now - 1 >= 0 AndAlso box(y_now - 1)(x_now - 1) = "b"
Then
                        NW_coords(0) = x_now - 1
                        NW_coords(1) = y_now - 1
                    Else
                        NW_coords(0) = -1
                        NW_coords(1) = -1
                    End If

                    ' NORTH-EAST
                    If y_now - 1 > 0 AndAlso x_now + 1 < box(y_now).Length() AndAlso box(y_now -
1)(x_now + 1) = "b" Then
                        NE_coords(0) = x_now + 1
                        NE_coords(1) = y_now - 1
                    Else
                        NE_coords(0) = -1
                        NE_coords(1) = -1
                    End If

                    ' SOUTH-WEST
                    If y_now + 1 < box.Count() AndAlso x_now - 1 >= 0 AndAlso box(y_now + 1)(x_now -
1) = "b" Then
                        SW_coords(0) = x_now - 1
                        SW_coords(1) = y_now + 1
                    Else
                        SW_coords(0) = -1
                        SW_coords(1) = -1
                    End If

                    ' SOUTH-EAST
                    If y_now + 1 < box.Count() AndAlso x_now + 1 < box(y_now).Length() AndAlso
box(y_now + 1)(x_now + 1) = "b" Then
                        SE_coords(0) = x_now + 1
                        SE_coords(1) = y_now + 1
                    Else
                        SE_coords(0) = -1
                        SE_coords(1) = -1
                    End If

                    ' Add coordinates to the node
                    new_node.Add(on_coords)
                    new_node.Add(N_coords)
                    new_node.Add(S_coords)
                    new_node.Add(E_coords)
                    new_node.Add(W_coords)
```

```
                    new_node.Add(NW_coords)
                    new_node.Add(NE_coords)
                    new_node.Add(SW_coords)
                    new_node.Add(SE_coords)

                    ' Add the node to the main list
                    nodes.Add(new_node)
                End If
            Next
        Next
    End Sub

    Function find_wall(ByVal nodeon As Array, ByVal wallz As List(Of List(Of Array)))
        Dim new_wallz As New List(Of Integer)

        For ind = 0 To wallz.Count() - 1

            If wallz(ind).Count() > 1 AndAlso (wallz(ind)(1)(0) = nodeon(0) And wallz(ind)(1)(1) =
nodeon(1)) Then
                new_wallz.Add(ind)
            End If

        Next

        Return new_wallz

    End Function

    Function empty(ByVal coords As Array)
        'the coordd is empty if it is (-1, -1)->they are empty
        If coords(0) = -1 And coords(1) = -1 Then
            Return True
        End If

        Return False
    End Function

    Private Sub normalize_walls()
        Dim cur_walz As New List(Of List(Of Array))

        'turn coord pairs into walls e.g.(1, 1)+(3,3) into a wall
        For Each pair In Me.wallz
            Dim x1 As Integer = 0, y1 As Integer = 0
            Dim x2 As Integer = 0, y2 As Integer = 0

            If pair.Count = 1 Then
                'if the wall is made of 1 coord  create a box(a cross)
                x1 = pair(0)(0) * 50
                y1 = pair(0)(1) * 50

                Dim new_walla As New List(Of Array)

                Dim coorda(2) As Integer
                Dim coordb(2) As Integer

                coorda(0) = x1
                coorda(1) = y1

                coordb(0) = x1 + 50
                coordb(1) = y1 + 50

                new_walla.Add(coorda)
                new_walla.Add(coordb)

                Dim new_wallb As New List(Of Array)
```

```
            Dim coordc(2) As Integer
            Dim coordd(2) As Integer

            coordc(0) = x1 + 50
            coordc(1) = y1

            coordd(0) = x1
            coordd(1) = y1 + 50

            new_wallb.Add(coordc)
            new_wallb.Add(coordd)

            '2 walls creat a cross shape
            cur_walz.Add(new_walla)
            cur_walz.Add(new_wallb)

        ElseIf pair.Count = 2 Then
            ' pair of coords creating a wall
            x1 = pair(0)(0) * 50
            y1 = pair(0)(1) * 50

            x2 = pair(1)(0) * 50
            y2 = pair(1)(1) * 50


            Dim new_walla As New List(Of Array)

            Dim coorda(2) As Integer
            Dim coordb(2) As Integer

            coorda(0) = x1
            coorda(1) = y1

            coordb(0) = x2
            coordb(1) = y2

            new_walla.Add(coorda)
            new_walla.Add(coordb)

            cur_walz.Add(new_walla)

        End If

    Next

    Me.wallz = cur_walz
End Sub

Public Function ray_blocked(ByVal from_coords As Array, ByVal to_coords As Array)
    Dim coorda(2) As Integer
    Dim coordb(2) As Integer

    coorda = from_coords

    coordb = to_coords

    Dim colis As Boolean = False
    For Each wl In Me.wallz
        'uses fancy math to find if an intersection even exists
        Dim x1 = coorda(0)
        Dim x2 = coordb(0)
        Dim x3 = wl(0)(0)
        Dim x4 = wl(1)(0)
```

```
            Dim y1 = coorda(1)
            Dim y2 = coordb(1)
            Dim y3 = wl(0)(1)
            Dim y4 = wl(1)(1)

            Dim den As Double = (x1 - x2) * (y3 - y4) - (y1 - y2) * (x3 - x4)

            'division by 0
            If den = 0 Then
                Exit For
            End If

            Dim t As Double = ((x1 - x3) * (y3 - y4) - (y1 - y3) * (x3 - x4)) / den
            Dim u As Double = -(((x1 - x2) * (y1 - y3) - (y1 - y2) * (x1 - x3)) / den)

            If t >= 0 And t <= 1 And u >= 0 And u <= 1 Then
                colis = True
                Exit For
            End If
            '------------------------------------------------------------
        Next

        Return colis



    End Function

    Private Sub load_wallz()
        Me.create_nodes()
        Me.connect_nodes()
        Me.normalize_walls()
    End Sub

End Class
```

SHEILD_CLASS:

```
Public Class sheild
    Inherits base_block

    Private plr As player

    Private _sheild_hp_bar As bar
    Public WriteOnly Property sheild_hp_bar As bar
        Set(ByVal value As bar)
            _sheild_hp_bar = value
        End Set
    End Property

    Public sheild_use As Boolean = False
    Public loaded As Boolean = False

    Private sheild_maxhp As Integer = 0
    Private sheild_nowhp As Integer = 0

    Public alive As Boolean = False

    Public sheild_img_path As String

    Public Sub New(ByVal game As Form, ByVal playr As player, ByVal height As Integer, ByVal width
As Integer, ByVal x As Integer, ByVal y As Integer, ByVal maxhp As Integer)
        MyBase.New(game, height, width, x, y)

        Me.game = game
        Me.plr = playr

        Me.sheild_maxhp = maxhp

        Me.sheild_nowhp = maxhp

    End Sub

    Public Sub add_img()

        Me.sheild_img_path = "\\cc4sjd8\2024 Intake\24026166\Visual Studio
2010\Projects\practice_courswork_platformer\invent_imgs\" & Me.plr.equipped(6) & ".png"

        Me.box.Image = New Bitmap(Me.sheild_img_path)

        Me.box.SizeMode = PictureBoxSizeMode.StretchImage

    End Sub

    Public Function collision(ByVal hit_taken As Integer, ByVal target As Object)
        'checks for a collision with the sheild to reduce hp
        If target.box.Bounds.IntersectsWith(Me.box.Bounds) Then
            Me.sheild_nowhp += hit_taken
            Return True
        End If

        Return False
    End Function

    Public Sub check_sheild()
        'checks if the sheild needs to be re-loaded
        If Not Me.loaded And Me.alive Then

            If Me.sheild_use Then
```

```
            Me.load()

            Me.add_img()

            Me.game.Controls.Add(Me.box)

            Me._sheild_hp_bar.load_all()


        ElseIf Not Me.sheild_use Then

            Me.game.Controls.Remove(Me.box)

            Me._sheild_hp_bar.destroy()


        End If
        Me.loaded = True
    End If
    '-------------------------------------


    If Me.sheild_nowhp <= 0 Then

        Me.game.Controls.Remove(Me.box)

        Me._sheild_hp_bar.destroy()

        Me.plr.equipped(6) = "0"

        Me.sheild_use = False
        Me.alive = False

    End If

    'if it is equipped it is alive
    If Me.plr.equipped(6) = "shield" Then

        Me.alive = True

    End If


End Sub


Public Sub update()

    Me.check_sheild()

    'move the sheild to follow te players arms and the mouse
    If Me.plr.mousex > Me.plr.x Then
        Me.x = Me.plr.rightArm.Left
        Me.y = Me.plr.rightArm.Top
    Else
        Me.x = Me.plr.leftArm.Left
        Me.y = Me.plr.leftArm.Top
    End If
    '---------------------------------------

    Me._sheild_hp_bar.new_size(Me.sheild_nowhp / Me.sheild_maxhp)

    Me.box.Left = Me.x - 25
    Me.box.Top = Me.y
```

```
    End Sub

End Class
```

TEXT_CLASS:

```
Public Class text
    Private game As Form

    Public box As New Label
    Public textbox As New TextBox

    Protected Friend x As Integer
    Protected Friend y As Integer

    Protected Friend my_text As String
    Private _size As Integer
    Public ReadOnly Property size
        Get
            Return _size
        End Get
    End Property

    Protected Friend colour As Color
    Protected Friend largetext As Boolean
    Protected Friend lines As Integer

    Public button_clicked As Boolean = False

    Private _border As Boolean = False

    Public Property border As Boolean
        Get
            Return _border
        End Get
        Set(ByVal value As Boolean)
            _border = value
        End Set
    End Property

    Public Sub New(ByVal game As Form, ByVal x As Integer, ByVal y As Integer, ByVal size As
Integer, ByVal txt As String, ByVal colr As Color, Optional ByVal largetext As Boolean = False,
Optional ByVal lns As Integer = 0)
        Me.game = game
        Me.x = x
        Me.y = y

        Me._size = size
        Me.my_text = txt

        'has a textbox and many words in it
        Me.largetext = largetext

        'how many lines in the textbox
        Me.lines = lns

        Me.colour = colr
    End Sub

    Public Sub load()

        If Not Me.largetext Then
            box.Text = Me.my_text
            box.Font = New Font("Arial", size, FontStyle.Bold)
            box.ForeColor = Me.colour
            box.AutoSize = True
            box.Location = New Point(Me.x, Me.y) ' Adjust position as needed
            Me.game.Controls.Add(box)
```

```
            box.BringToFront()
        Else
            Me.textbox.Text = Me.my_text
            Me.textbox.Font = New Font("Arial", size, FontStyle.Bold)
            Me.textbox.Location = New Point(Me.x, Me.y)
            Me.textbox.Size = New Size(780, Me.lines * 20)
            If Me.lines > 0 Then
                Me.textbox.ReadOnly = True
                Me.textbox.Size = New Size(780, Me.lines * 20)
            Else
                Me.textbox.Size = New Size(780, 40)
            End If

            Me.game.Controls.Add(Me.textbox)

            Me.textbox.Multiline = True

            Me.textbox.ScrollBars = ScrollBars.Vertical
            Me.textbox.WordWrap = True
            Me.textbox.BringToFront()
        End If
        Me.draw_border()
    End Sub

    Protected Friend Sub draw_border()
        'if a border is wanted, it draw a rectangle around the
        If Me.border Then

            Dim bit_map As New Bitmap(box.Width, box.Height)
            Dim g As Graphics = Graphics.FromImage(bit_map)
            Dim pen As New Pen(Color.LightGray, 5)

            g.DrawLine(pen, 0, 0, box.Width, 0)
            g.DrawLine(pen, 0, box.Height, box.Width, box.Height)
            g.DrawLine(pen, 0, 0, 0, box.Height)
            g.DrawLine(pen, box.Width, 0, box.Width, box.Height)

            box.Image = bit_map

        End If
        '-----------------------------------------------------

    End Sub

    Protected Friend Sub click_handler(ByVal sender As Object, ByVal e As
System.Windows.Forms.MouseEventArgs)

        'if the text can be clicked this is where it is handled
        If e.Button = MouseButtons.Left And Not Me.button_clicked Then
            Me.button_clicked = True
        End If

    End Sub

    Protected Friend Sub destroy()

        'if it is text that can be scrolled(in the instructions menu) it is destroyed differently
        If Not Me.largetext Then
            Me.game.Controls.Remove(Me.box)
        Else
            Me.game.Controls.Remove(Me.textbox)
        End If
```

```
    End Sub

    Public Sub update(ByVal txt)
        Me.my_text = txt

        box.ForeColor = Me.colour
        box.Text = Me.my_text
    End Sub
End Class
```

UTILITIES:

```
Public Class utilities
    'used to load from the load save file and attack file, and save to the load save file

    Private Const slmpath As String = "\\cc4sjd8\2024 Intake\24026166\Visual Studio
2010\Projects\practice_courswork_platformer\code\save_games.txt"
    Private Const aspath As String = "\\cc4sjd8\2024 Intake\24026166\Visual Studio
2010\Projects\practice_courswork_platformer\code\attacks.txt"
    Private Const inpath As String = "\\cc4sjd8\2024 Intake\24026166\Visual Studio
2010\Projects\practice_courswork_platformer\code\inentv_items.txt"
    Private Const mipath As String = "\\cc4sjd8\2024 Intake\24026166\Visual Studio
2010\Projects\practice_courswork_platformer\code\all_missions.txt"
    Private Const svscpath As String = "\\cc4sjd8\2024 Intake\24026166\Visual Studio
2010\Projects\practice_courswork_platformer\code\scores.txt"

    Protected Friend loaded_game As Boolean = False
    Protected Friend rand As New Random()

    Private _mousex As Integer
    Public ReadOnly Property mousex
        Get
            Return _mousex
        End Get
    End Property

    Private _mousey As Integer
    Public ReadOnly Property mousey
        Get
            Return _mousey
        End Get
    End Property

    Public Function file_exisists(ByVal file_path As String, ByVal read_write As String, Optional
ByVal clear As Boolean = False)
        'checks the file exists and loads it in
        If Not System.IO.File.Exists(file_path) Then
            Debug.Print("file does not exist: " & file_path)
            Stop
        Else
            If read_write = "read" Then
                Return System.IO.File.ReadAllLines(file_path)
            ElseIf read_write = "write" Then
                Return New System.IO.StreamWriter(file_path, clear)
            Else
                Debug.Print("cannot " & read_write)
                Stop
            End If

        End If
        Return Nothing
    End Function

    Public Function add_missions(ByVal all_missions As Dictionary(Of List(Of String), List(Of
Integer)))

        Dim all_missions_txt As New List(Of List(Of String))


        'load the title of the mission into a file for random choosing
        For Each mis In all_missions
            all_missions_txt.Add(mis.Key)
        Next
```

```
        Return all_missions_txt
    End Function

    Public Function Rand0(Optional ByVal lower_bound As Integer = 1, Optional ByVal upper_bound As
Integer = 100) As Integer
        Randomize()

        Return rand.Next(lower_bound, upper_bound)
    End Function


    Public Sub Form1_MouseMove(ByVal sender As Object, ByVal e As MouseEventArgs)

        Me._mousex = e.X
        Me._mousey = e.Y

    End Sub


    Public Sub save_map(ByVal string_to_save As List(Of String), ByVal plr1 As player)

        Dim objWriter As System.IO.StreamWriter = Me.file_exisists(utilities.slmpath, "write")


        'write the map to the file
        For Each line In string_to_save

            objWriter.WriteLine(line)

        Next
        '------------------------
        'new line, player stats/cutomization
        objWriter.WriteLine("/")

        'write the stat modifiers to the text file
        For ind = 0 To 2
            objWriter.WriteLine(plr1.all_stats(ind))
        Next

        'Write each limbs rgb colour to the text file, each rgb vslue is seperated by a /
        Dim body_cols As Color = plr1.box.BackColor
        objWriter.WriteLine(body_cols.R & "\" & body_cols.G & "\" & body_cols.B)

        Dim arm_cols As Color = plr1.leftArm.BackColor
        objWriter.WriteLine(arm_cols.R & "\" & arm_cols.G & "\" & arm_cols.B)

        Dim leg_cols As Color = plr1.leftLeg.BackColor
        objWriter.WriteLine(leg_cols.R & "\" & leg_cols.G & "\" & leg_cols.B)

        objWriter.Close()

    End Sub

    Public Sub save_score(ByVal textp As String)
        'write to text file

        Dim objWriter As System.IO.StreamWriter = Me.file_exisists(utilities.svscpath, "write",
True)


        objWriter.WriteLine(textp)


        objWriter.Close()
    End Sub
```

```
Public Function load_map()

    Dim store_list As New List(Of List(Of String))

    Dim player_stats As Boolean = False
    Dim next_line As Boolean = False

    Dim lines() As String = Me.file_exisists(utilities.slmpath, "read")

    'divide a txt file into lines and read each one then store in a list(map, stats)

    Dim map As New List(Of String)
    Dim stats As New List(Of String)


    For Each line As String In lines

        For Each ind In line

            If ind = "/" Then
                'if / found save all read things to the stat file
                player_stats = True
                next_line = True

                Exit For
            End If

        Next

        If Not next_line Then
            If player_stats Then
                stats.Add(line)
            Else
                map.Add(line)
            End If

        Else
            next_line = False

        End If




    Next
    '-----------------------------------------------------------------


    store_list.Add(map)
    store_list.Add(stats)



    Return store_list
End Function

Public Function Load_scores(ByVal load_dict As Dictionary(Of String, String))

    'read , in lines, from the text file
    Dim lines() As String = Me.file_exisists(utilities.svscpath, "read")

    For line = 0 To (lines.Count() / 2) - 1
```

```
            load_dict(lines(line * 2 + 1)) = lines(line * 2)



        Next
        '-----------------------------------------------
        Return load_dict
    End Function

    Public Function load_attacks(ByVal type As String)
        'type=type of entity as enemies ,players and bosses all have different attacks so no point
in loading the uneccessary ones
        Dim atk_dict As New Dictionary(Of String, List(Of String))

        Dim lines() As String = Me.file_exisists(utilities.aspath, "read")


        'load the atributes for each attack into a dictionary
        For Each line As String In lines

            Dim parts() As String = line.Split("/"c)

            If parts.Length > 2 AndAlso (parts(8) = type Or "none" = type) Then

                Dim name As String = parts(0)
                Dim values As New List(Of String)

                For i As Integer = 1 To parts.Length - 1
                    values.Add(parts(i))
                Next

                atk_dict(name) = values

            End If

        Next
        '-----------------------------------------------

        Return atk_dict
    End Function

    Public Function load_player_inteface_items()

        Dim ret_dict As New Dictionary(Of String, List(Of Integer))

        Dim lines() As String = Me.file_exisists(utilities.inpath, "read")

        'divide a txt file into lines and read each one then store in a dict
        For Each line As String In lines
            Dim parts() As String = line.Split("/"c)

            'makes sure line it is one has multiple parts, must have more than 1 "/"
            If parts.Length >= 2 Then
                Dim name As String = parts(0)
                Dim values As New List(Of Integer)

                'get all item stats and add to a list
                For sublines As Integer = 1 To parts.Length - 1
                    Dim return_value As Integer

                    If Integer.TryParse(parts(sublines), return_value) Then
                        values.Add(return_value)
                    End If
```

```
                Next

                'add the list to its related name in a dictionary

                ret_dict(name) = values


            End If

        Next
        '-----------------------------------------------------------------

        Return ret_dict
    End Function

    Public Function load_missions()

        Dim all_missions As New Dictionary(Of List(Of String), List(Of Integer))

        Dim lines() As String = Me.file_exisists(utilities.mipath, "read")
        'read a text file with the missions in it and load into a dictionary in memory
        For Each line As String In lines

            Dim parts() As String = line.Split("/"c)

            If parts.Count() <= 1 Then Continue For

            Dim key As New List(Of String)
            Dim val As New List(Of Integer)

            'splits the line of string from the text file into name/type of enemy/number to
kill/reward for completion/how many rewards
            'this is then loaded into a dictionary for easy access
            key.Add(parts(0))
            key.Add(parts(1))
            key.Add(parts(3))

            val.Add(parts(2))
            val.Add(parts(4))

            all_missions(key) = val

        Next
        '-------------------------------------------------------------------------------
        Return all_missions
    End Function



End Class
```

WALL_CLASS:

```
Public Class wall
    Inherits base_block

    Private _text_mapp_used As List(Of String)

    Public WriteOnly Property text_mapp_used
        Set(ByVal value)
            _text_mapp_used = value
        End Set
    End Property
```

```
    Public Sub New(ByVal game As Form, ByVal height As Integer, ByVal width As Integer, ByVal x As
Integer, ByVal y As Integer)
        MyBase.New(game, height, width, x, y)

    End Sub


    Public Sub change_img(ByVal map As List(Of String), ByVal image_paths As String)
        Dim endof As String = Me.get_ending(map, image_paths)

        'image path is already known all images are just ordered by number to make them loaded in
easily {little code needed}
        'tile_{number}.png
        Dim img As String = image_paths + endof
        img += ".png"

        Dim originalImage As Bitmap = Bitmap.FromFile(img)

        Me.box.Image = originalImage
        Me.box.SizeMode = PictureBoxSizeMode.StretchImage
        '-------------------------------------------------------------------------------------------
------------------------

    End Sub

    Public Function find_neighbours(ByVal map As List(Of String))
        'every wall finds all neighbouring tiles

        Dim result As New List(Of Boolean())

        Dim newy As Integer = 0
        Dim newx As Integer = 0

        'not on the edge
        If Me.x <> 0 Then
            newx = Me.x / 50
        End If

        If Me.y <> 0 Then
            newy = Me.y / 50
        End If
        '---------------

        newy += 1

        If map(newy)(newx) = "b" Then
            Dim neighbors(7) As Boolean ' N, NE, E, SE, S, SW, W, NW
            'all possible directions
            Dim directions As Integer(,) = {
                {0, -1}, {1, -1}, {1, 0}, {1, 1},
                {0, 1}, {-1, 1}, {-1, 0}, {-1, -1}
            }
            'iterates through all possible directions
            For ind As Integer = 0 To 7
                '{x, y}
                Dim dx As Integer = directions(ind, 0)
                Dim dy As Integer = directions(ind, 1)

                Dim nx As Integer = (newx) + dx
                Dim ny As Integer = (newy) + dy
                'checks edges
                If nx < 0 Or ny < 1 Or nx > map(1).Length() - 1 Or ny > map.Count() - 2 Then
                    neighbors(ind) = True
                    Continue For
```

```
                End If

                If ny >= 0 AndAlso ny < map.Count AndAlso nx >= 0 AndAlso nx < map(ny).Length Then
                    If map(ny)(nx) = "b" Then
                        neighbors(ind) = True

                    End If
                End If
            Next

            result.Add(neighbors)

        End If

        '----------------------------------------

        Return result
    End Function

    Private Function get_ending(ByVal map As List(Of String), ByVal image_paths As String) As String
        Dim current_cell As List(Of Boolean()) = Me.find_neighbours(map)

        'goes through the neighbours checking which are full
        If current_cell.Count() > 0 Then

            Dim neighbors As Array = current_cell(0)

            Dim N As Boolean = neighbors(0)
            Dim E As Boolean = neighbors(2)
            Dim S As Boolean = neighbors(4)
            Dim W As Boolean = neighbors(6)

            Dim NE = neighbors(1)
            Dim SE = neighbors(3)
            Dim NW = neighbors(7)
            Dim SW = neighbors(5)

            Dim endof As String = ""
            Dim final_img As String = image_paths

            ' check which image to be choosen based on the neighbours the wall has (not all have
been done as some are rare to occur and having the image doesnt change them much):
            If NW And N And W And Not SE And Not E And Not S Then
                endof = "2"
            ElseIf W And S And SW And Not NE And Not N And Not E Then
                endof = "3"
            ElseIf E And S And SE And Not N And Not W And Not NW Then
                endof = "4"
            ElseIf N And E And NE And Not SW And Not S And Not W Then
                endof = "5"
            ElseIf N And NE And E And SE And S And SW And W And Not NW Then
                endof = "6"
            ElseIf N And NW And W And NE And E And SE And S And Not SW Then
                endof = "7"
            ElseIf N And NW And W And SW And SW And S And SE And E And Not NE Then
                endof = "36"
            ElseIf N And NE And NW And W And SW And S And E And Not SE Then
                endof = "8"
            ElseIf N And S And Not E And NW And W And SW Or (N And S And Not E And NW And W And SW
And (NE Or SE)) Then
                endof = "13"
            ElseIf N And S And NE And E And SE And Not W Or (N And S And NE And E And SE And Not W
And (NW Or SW)) Then
                endof = "15"
```

```
            ElseIf N And E And W And NE And NW And Not S Or (N And E And W And NE And NW And Not S
And (SW Or NE)) Then
                endof = "14"
            ElseIf S And E And W And SW And SE And Not N Or (S And E And W And SW And SE And Not N
And (NE Or NW)) Then
                endof = "12"
            ElseIf NW And N And W And SE And S And E And Not N And Not SW Then
                endof = "17"
            ElseIf N And NE And E And S And SW And W And Not NE And Not SE Then
                endof = "16"
            ElseIf Not E And W And S And N And (NW Or SW) Then
                endof = "13"
            ElseIf Not N And Not S And Not SE And Not NE And Not SW And Not NW And E And W Or (Not N
And Not S And E And W And (NE Or SE Or SW Or NW)) Then
                endof = "38"
            Else
                endof = "37"
            End If
            '-------------------------------------------------------------------------------------------
-------------------------------------------------------------------


            Return endof
        End If
        Return 0
    End Function

    Public Function blocakble(ByVal map, ByVal y, ByVal x) 'g
        'checks if adajaceent tiles are empty for wall images
        Dim empty_tiles() As String = {".", "e", "w", "p", "B", "N", "d"}

        Dim tile_on = map(y)(x)

        For Each pos_tiles In empty_tiles

            If pos_tiles = tile_on Then
                Return True
            End If

        Next

        '-------------------------------------------------
        Return False
    End Function

    Public Function real_wall(ByVal ynew As Integer, ByVal xnew As Integer)

        'checks if any surrounding tiles are empty , for efficiency so the check is not needed.
        'and to differentiate between real walls and fake walls

        For y_add = -1 To 1
            For x_add = -1 To 1

                'check if the wall is, checking somewhere out of range
                If ynew + y_add <= _text_mapp_used.Count() - 2 And xnew + x_add <=
_text_mapp_used(1).Length() - 1 And ynew + y_add >= 1 And xnew + x_add >= 0 Then

                    If blocakble(_text_mapp_used, (_text_mapp_used.Count() - 1) - ((ynew + y_add) -
1), xnew + x_add) Then

                        Return True

                    End If

                End If
```

```
            Next
        Next
        '-------------------------------------------------------------------------------

        Return False

    End Function
End Class
```

WARLOCK_ENEMY:

```
Public Class warlock_enemy
    Inherits base_enemy

    Private Const fireball_path As String = "\\cc4sjd8\2024 Intake\24026166\Visual Studio 2010\Projects\practice_courswork_platformer\fireball-clipart-pixel-sprite-pixel-transparent-fire-ball--m2H7K9d3N4N4d3H7.png"
    Private fireball_img As New Bitmap(warlock_enemy.fireball_path)

    Public Sub New(ByVal game As Form, ByVal height As Integer, ByVal width As Integer, ByVal x As Integer, ByVal y As Integer, ByVal id As Integer, ByVal hp As Integer, ByVal util As utilities, Optional ByVal magic As Boolean = False)
        MyBase.New(game, height, width, x, y, id, hp, util, magic)

        Me.type_enemy = "warlock"

        Me.colour = Color.Brown
        Me.box.BackColor = Color.Brown

        Me.type_magic = magic
    End Sub

    Protected Overrides Sub update_arms(ByVal plr1 As player)

        Dim centerX As Integer = x + width \ 2
        Dim centerY As Integer = y + height \ 2

        ' warlock arms orbit the entity
        Me.angle += 0.05

        ' Left arm moves counter-clockwise
        leftArm.Left = centerX + CInt(Math.Cos(angle) * 50) - (leftArm.Width \ 2)
        leftArm.Top = centerY + CInt(Math.Sin(angle) * 50) - (leftArm.Height \ 2)
        leftArm.BackColor = Color.Red

        ' Right arm moves clockwise
        rightArm.Left = centerX + CInt(Math.Cos(-angle) * 50) - (rightArm.Width \ 2)
        rightArm.Top = centerY + CInt(Math.Sin(-angle) * 50) - (rightArm.Height \ 2)
        rightArm.BackColor = Color.Red

    End Sub

    Public Sub attack(ByVal x As Double, ByVal y As Double, ByVal tick As Integer, ByVal angle As Double)

        'creates the same attack type, with the same attributes
        Dim new_attack = New base_attack(Me.game, 25, 25, Me.x, Me.y, 6, "fireball", New Bitmap(Me.attack_dict("fireball")(2)), Me.attack_dict("fireball")(1), angle)

        new_attack.spawn_time = tick

        Me.live_attacks.Add(new_attack)
        new_attack.box.BringToFront()

        new_attack.load()
        new_attack.move(x, y, Me)
        '-------------------------------------------------------

    End Sub

    Public Sub update_attacks(ByVal Timer As Integer)

        'checks all attacks are dead, to be destroyed
```

```
        For atk = 0 To Me.live_attacks.Count() - 1

            If Me.live_attacks(atk).check_alive(Timer) Or Me.live_attacks(atk).dead Then

                dead_attacks.Add(live_attacks(atk))
                live_attacks.RemoveAt(atk)

                Exit For

            End If

        Next

    End Sub

    Public Sub load_attacks(ByVal tick, ByVal plr1)

        If Me.plr1_see Then

            'so all attacks from all warlocks dont happen at once, the id is used, reducing lag
            If tick Mod 10 * Me.id = 0 Then

                For num_atk = 0 To 6


                    'creat a circle of fireballs around the warlock
                    angle = num_atk * 60
                    Dim radian As Double = angle * Math.PI / 180

                    Dim usex As Integer = Me.x + 25 + base_enemy.see_radius * Math.Cos(radian)
                    Dim usey As Integer = Me.y + 25 + base_enemy.see_radius * Math.Sin(radian)

                    attack(usex, usey, tick, radian)
                    '---------------------------------------------

                Next

            End If
        End If

    End Sub

    Protected Friend Overrides Sub update(ByVal all_objects As List(Of wall), ByVal plr1 As player,
ByVal enemies As List(Of base_enemy), ByVal map As List(Of String), ByVal tick As Integer)

        MyBase.update(all_objects, plr1, enemies, map, tick)

        load_attacks(tick, plr1)
        update_attacks(tick)

        Me.x += Me.xvel
        Me.y += Me.yvel
        Me.box.Left = Me.x
        Me.box.Top = Me.y

    End Sub

End Class
```

DESIGN:

OVERVIEW:

My RPG game shall use the Windows Forms interface, hence using functionality from it as well. This includes forms to allow images to be printed to the screen, picture boxes to create different entities, and event managers to register inputs from the user. Even though VB Forms also offer inbuilt buttons, progression bars, and text, I have decided to create these myself using picture boxes and different event managers, creating a more dynamic feel to such core components of the game.

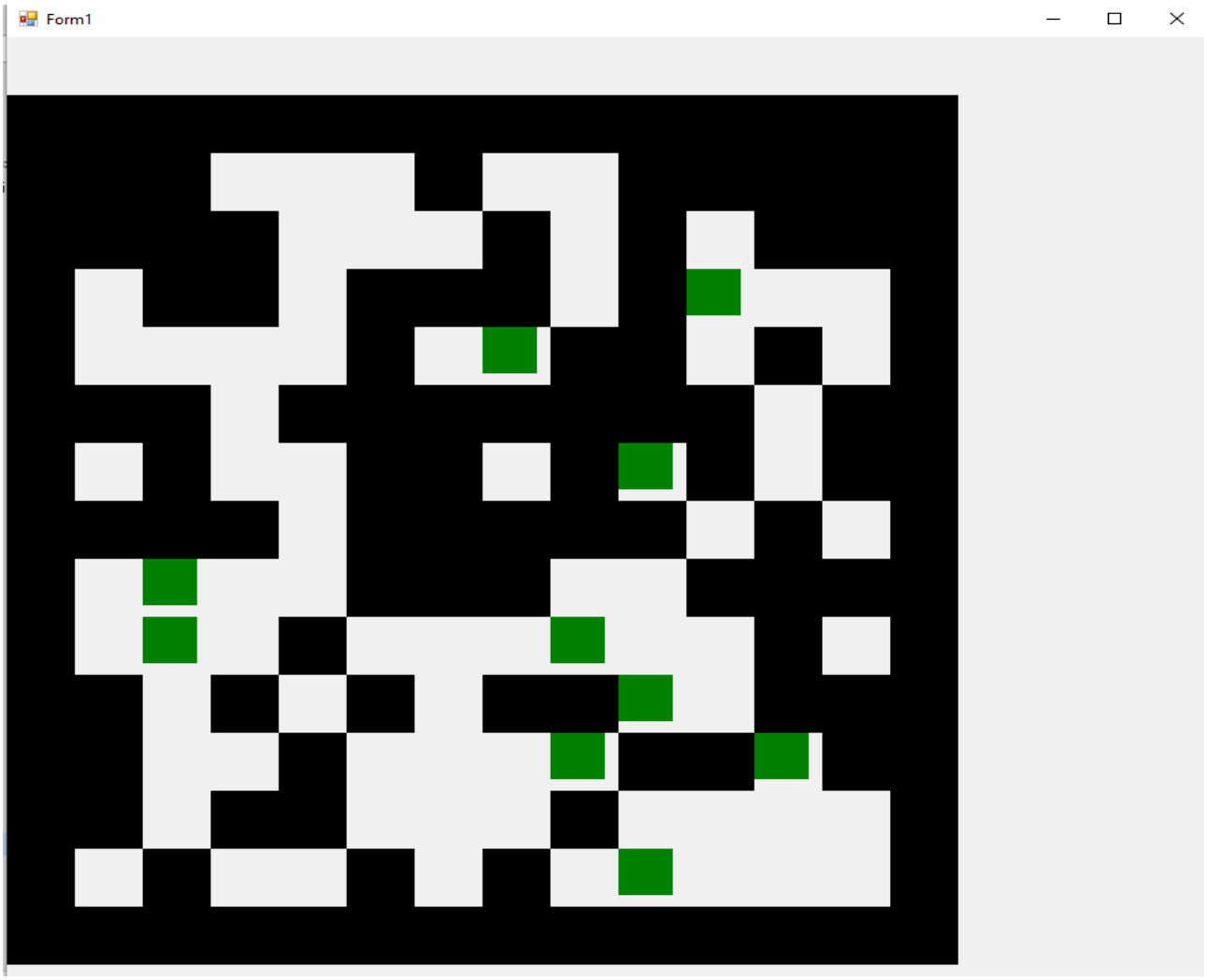
There will be two forms: one to create all the starting pages and the game itself, and one to show the ending screens to the player where previous scores will be ranked. These previous scores will be loaded from a text file, where any new scores are also saved. This strategy is also used for inventory item attributes, different attack attributes, missions and their attributes, and the ability to save the map's 2D array into text so it can be loaded again into a new game.

Generating new maps will be done randomly and dynamically using cellular automata and A* pathfinding to make sure each entity can be moved to from anywhere on the map. Ray casting is also a major algorithm used by nearly all entities to check if there is a wall between them and the player. It uses the coordinates of lines created by checking runs of walls, hence reducing the number of checks and creating the ability to use mathematical formulae to create a more efficient algorithm.

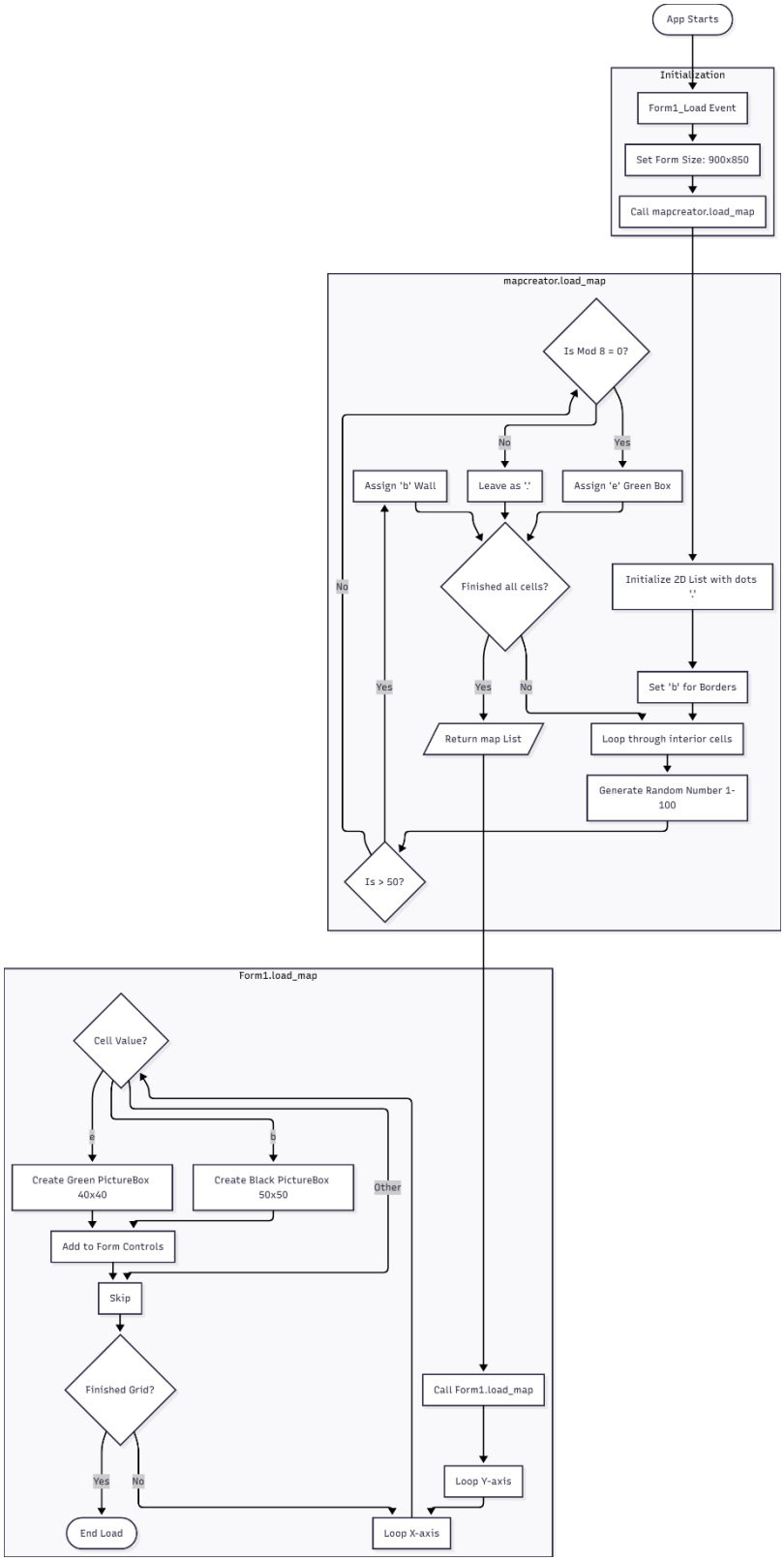
The use of classes greatly simplifies the problem, meaning abstraction by categorization is heavily used, creating large examples of inheritance and polymorphism, with all entities inheriting from the Base_block class and having large trees with nearly all classes having a parent or child.

PROTOTYPE:

I have made a prototype for the map generator and creator which loads the 2D list as a list of lists of strings, where each string (a single character) represents a specific type of entity. However, as it's a prototype, this is completely random except for the edges of the map being made of walls.



FLOWCHART:


App Starts
Initialization
Form1_Load Event
Set Form Size: 900x850
Call mapcreator.load_map
mapcreator.load_map
Is Mod 8 = 0?
No
Yes
Assign 'b' Wall
Leave as '.'
Assign 'e' Green Box
Finished all cells?
No
Yes
Initialize 2D List with dots '.'
Set 'b' for Borders
Yes
Yes
No
Return map List
Loop through interior cells
Generate Random Number 1-100
Is > 50?
Form1.load_map
Cell Value?
e
b
Other
Create Green PictureBox 40x40
Create Black PictureBox 50x50
Add to Form Controls
Skip
Finished Grid?
Yes
No
End Load
Call Form1.load_map
Loop Y-axis
Loop X-axis

CODE:

```
Public Class Form1
    Private width As Integer = 14
    Private height As Integer = 16

    Private mymap As New mapcreator(width, height)

    Private Sub load_map(ByVal mapuse As List(Of List(Of String)))
        For y_add = 0 To Me.height - 1
            For x_add = 0 To Me.width - 1
                'wall
                If mapuse(y_add)(x_add) = "b" Then

                    Dim newcube As New PictureBox()

                    newcube.Size = New Size(50, 50)

                    newcube.Top = y_add * 50
                    newcube.Left = x_add * 50

                    newcube.BackColor = Color.Black

                    Me.Controls.Add(newcube)
                ElseIf mapuse(y_add)(x_add) = "e" Then
Y89
                    Dim newcube As New PictureBox()

                    newcube.Size = New Size(40, 40)

                    newcube.Top = y_add * 50
                    newcube.Left = x_add * 50

                    newcube.BackColor = Color.Green

                    Me.Controls.Add(newcube)
                End If



            Next
        Next
    End Sub
    Private Sub Form1_Load(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles
MyBase.Load
        'loads the start of the game, the entry page
        Me.Size = New Size(900, 850)

        Dim mym As List(Of List(Of String)) = Me.mymap.load_map()

        Me.load_map(mym)
    End Sub



End Class

Class mapcreator
    Private width As Integer
    Private height As Integer

    Private rand As New Random()
    Public Sub New(ByVal width As Integer, ByVal height As Integer)
        Me.width = width - 1
        Me.height = height - 1
```

```
    End Sub

    Public Function Rand0(Optional ByVal lower_bound As Integer = 1, Optional ByVal upper_bound As
Integer = 100) As Integer
        Randomize()

        Return rand.Next(lower_bound, upper_bound)
    End Function

    Public Function load_map()
        Dim newmap As New List(Of List(Of String))

        For i = 0 To Me.height
            Dim newrow As New List(Of String)
            For j = 0 To Me.width
                newrow.Add(".")
            Next
            newmap.Add(newrow)
        Next

        For i As Integer = 0 To Me.width
            newmap(1)(i) = "b"
            newmap(Me.height)(i) = "b"
        Next

        For j As Integer = 1 To Me.height
            newmap(j)(0) = "b"
            newmap(j)(Me.width) = "b"
        Next

        For j = 1 To Me.height - 1
            For i = 1 To Me.width - 1
                Dim rannum As Integer = Me.Rand0()
                If rannum > 50 Then
                    newmap(j)(i) = "b"
                ElseIf rannum Mod 8 = 0 Then
                    newmap(j)(i) = "e"

                End If
            Next
        Next

        Return newmap
    End Function
End Class
```

GRAPHICAL USER INTERFACE:

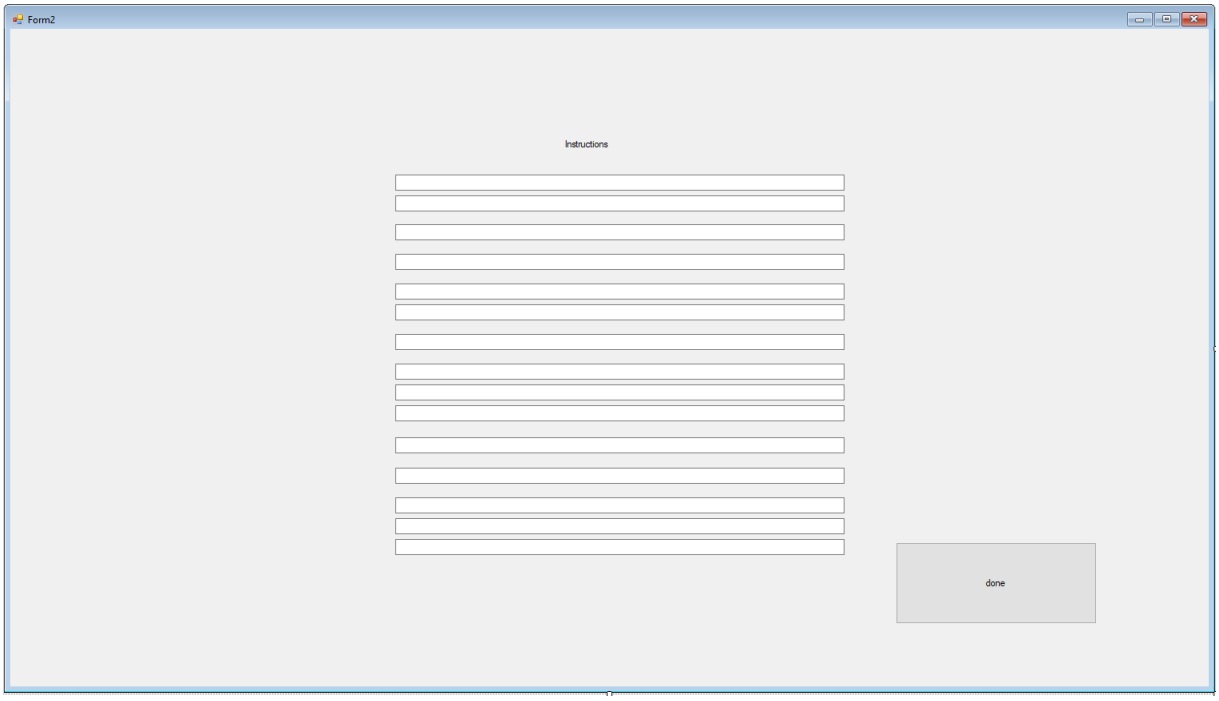
START SCREEN:

The first screen the user meets;



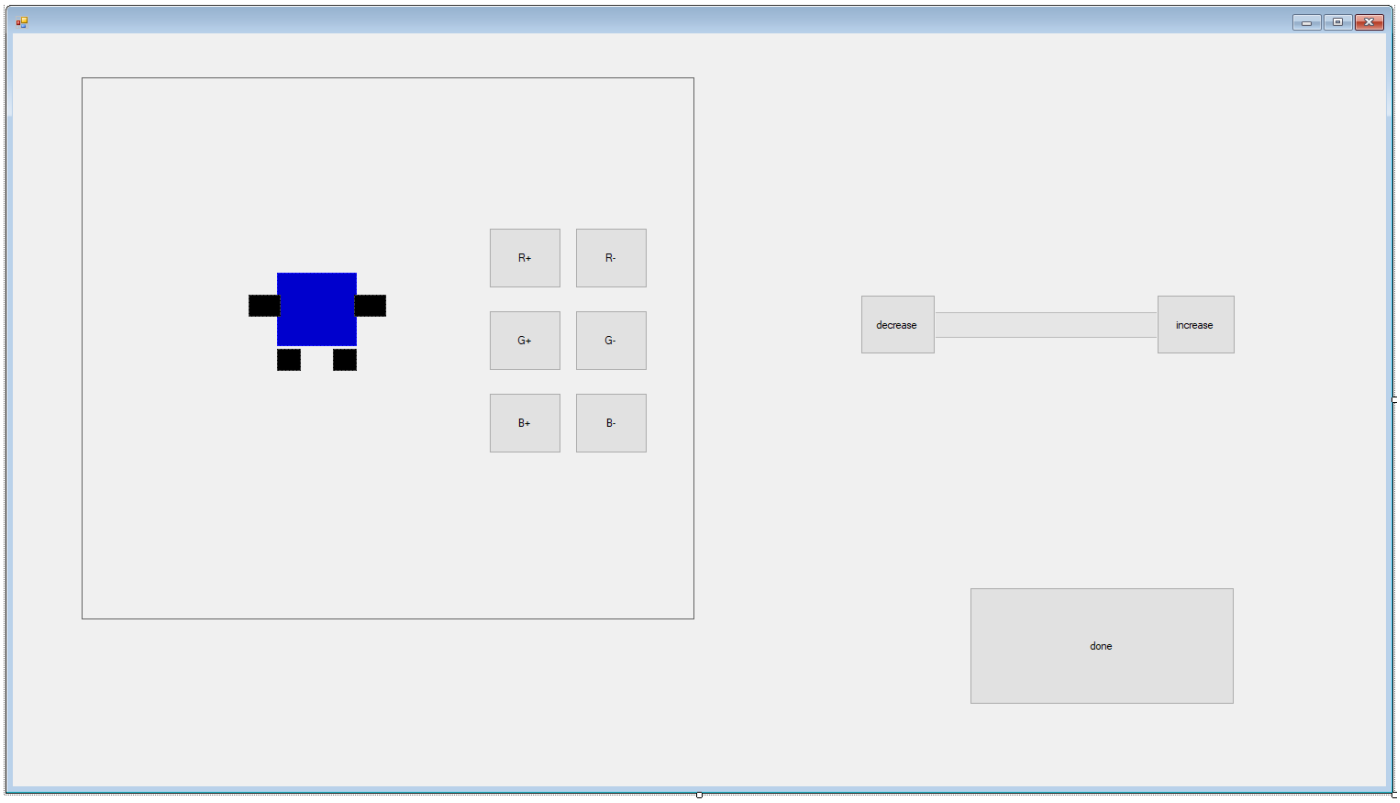
INSTRUCTION SCREEN:

Tells the user how the game is played and how it works;



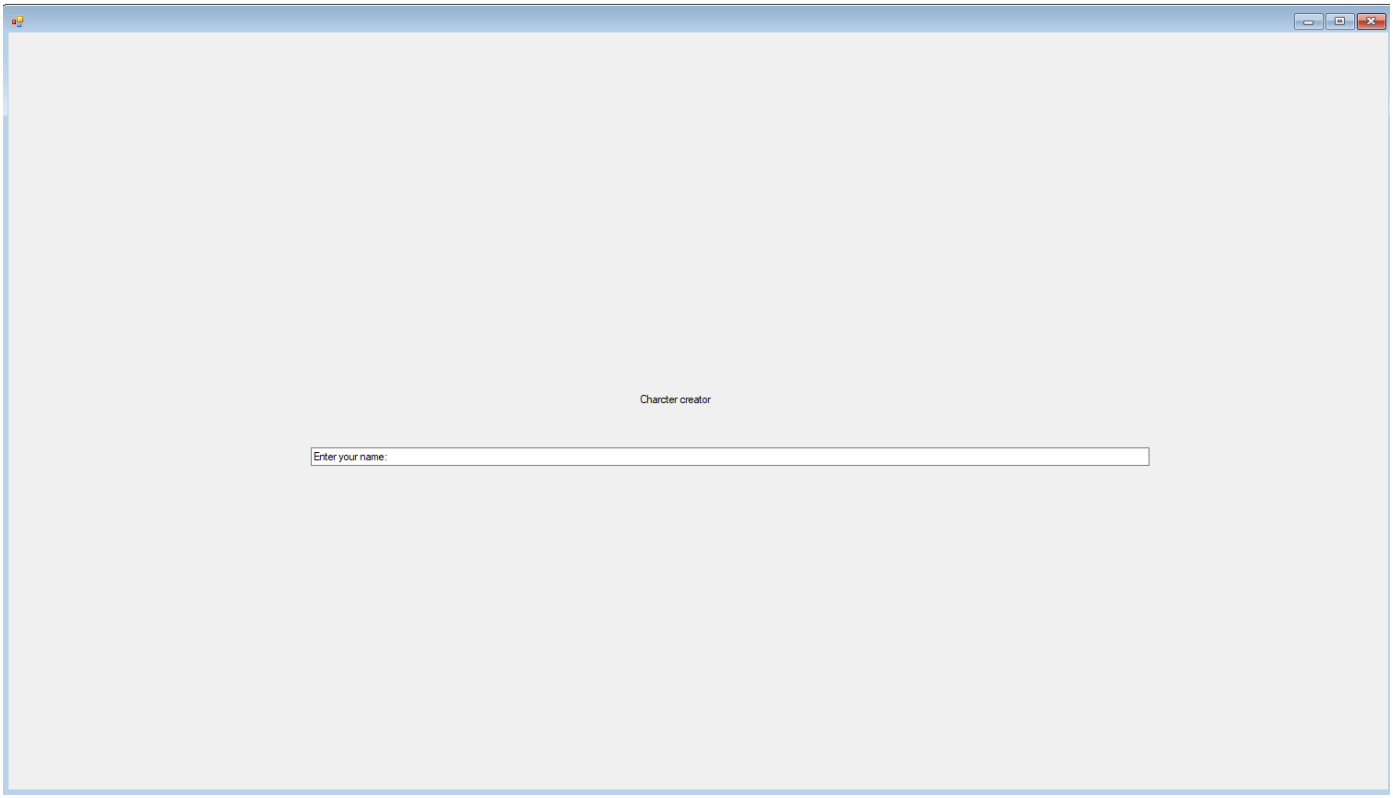
CHARACTER CREATOR SCREEN:

Allows the user to change their character;



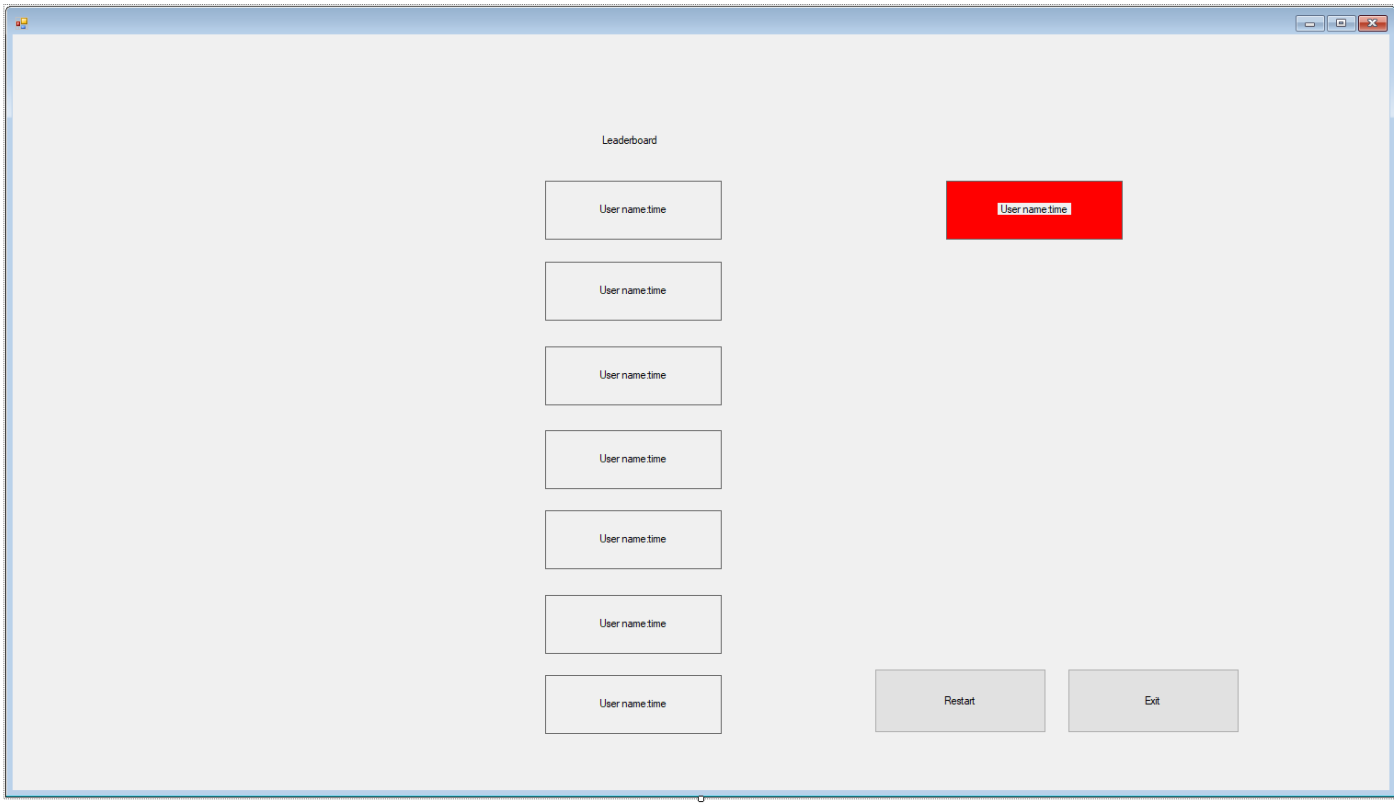
CREATE LEADER BOARD NAME SCREEN:

Allows the user to add their chosen name to the leader board;



LEADER BOARD SCREEN'S

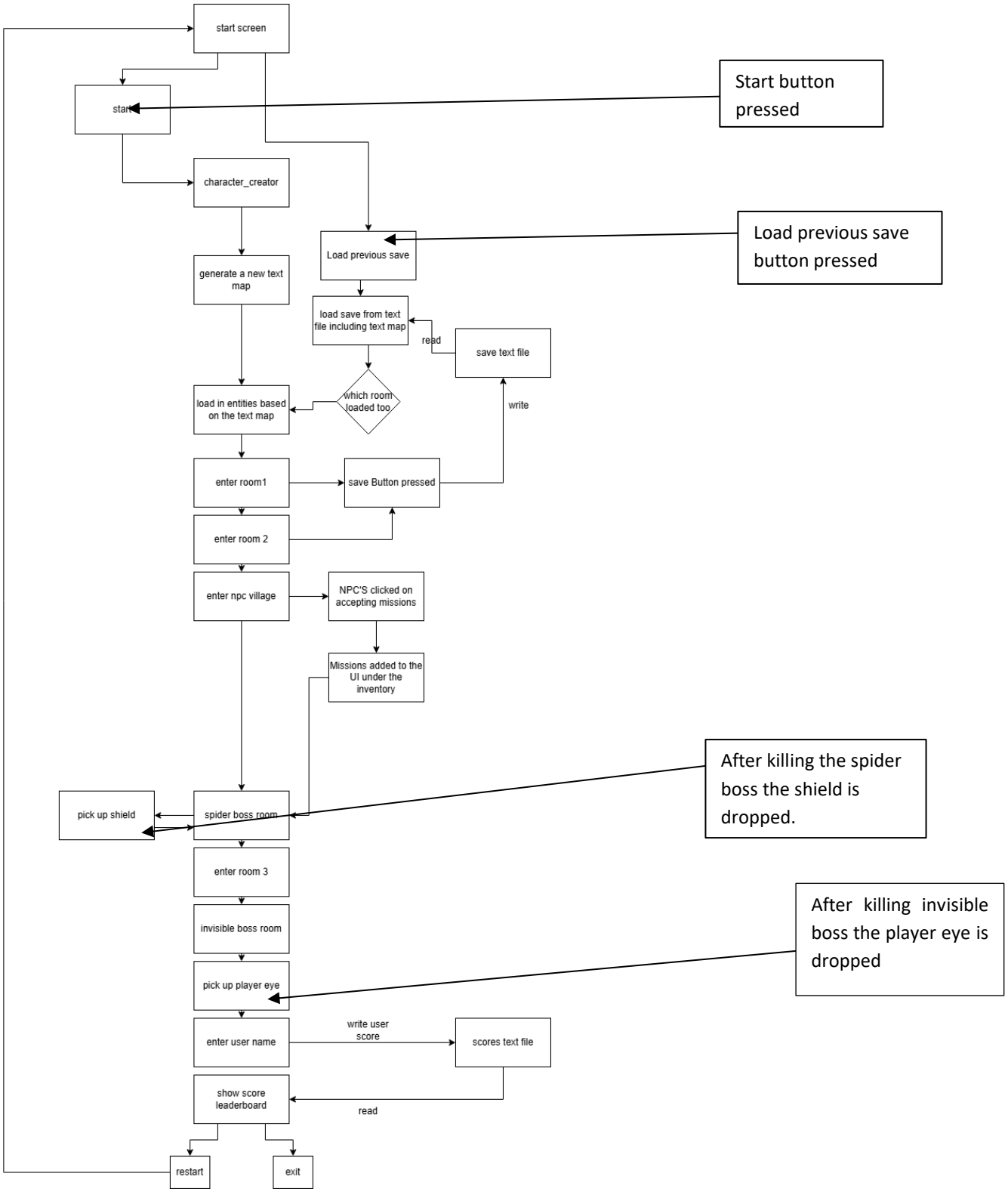
Allows the user to see past scores, ranked and their score(time);



GAME LOOP:

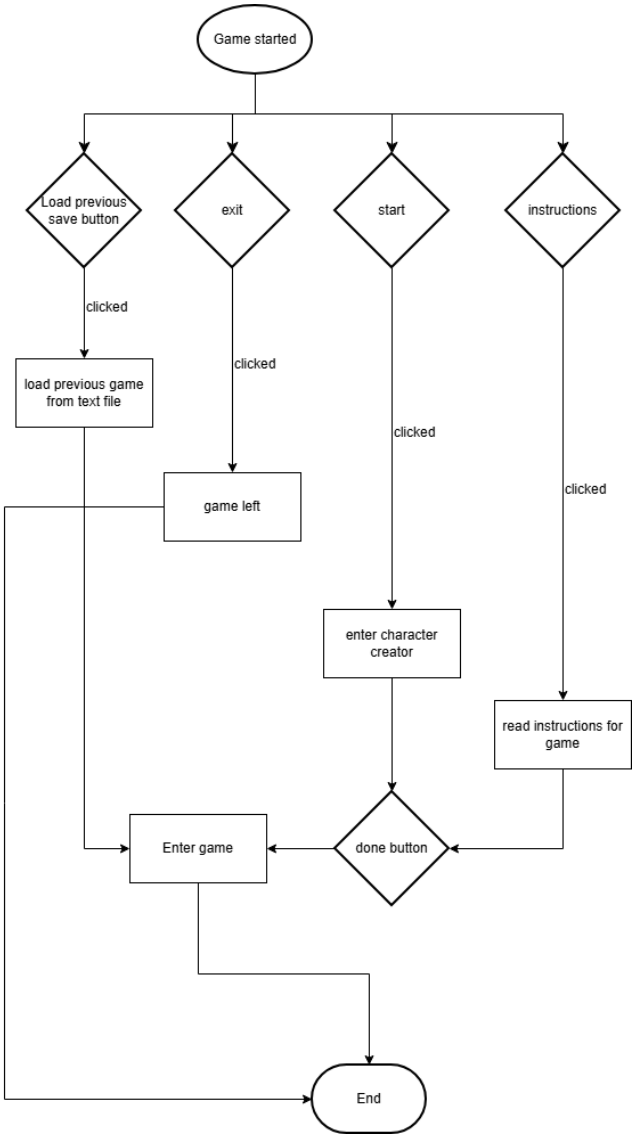
I will draw a flowchart like image for the entire game from when it starts to its end, not going into complexities but giving an overview of the main game loop.

MAIN LOOP:

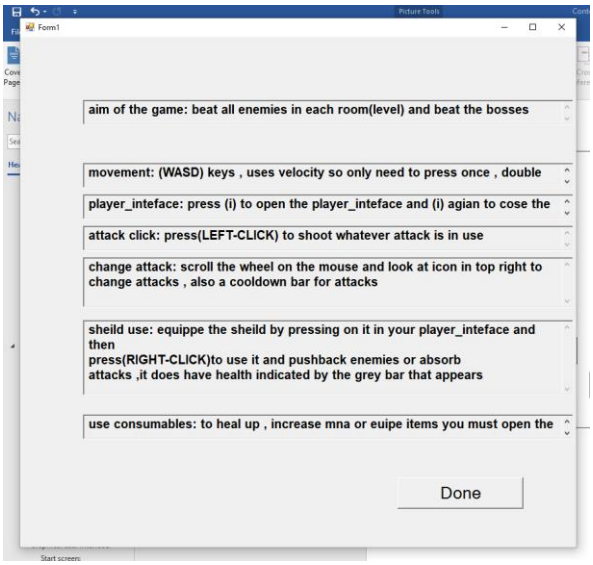


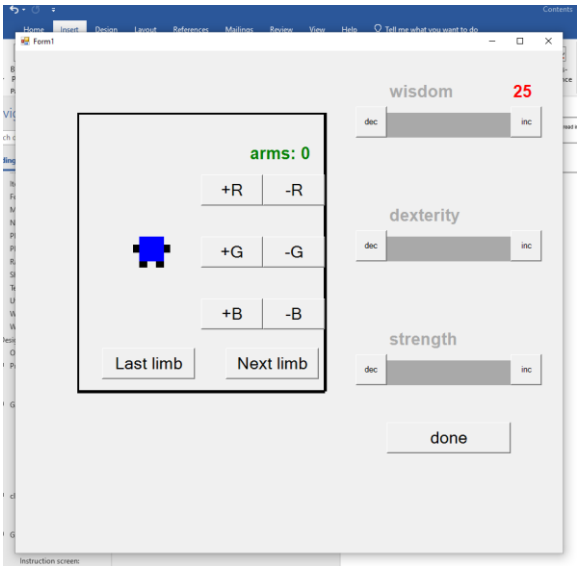
START SCREEN:

Create the start screen with buttons that can change the window to to either the character creator or straight into a game.



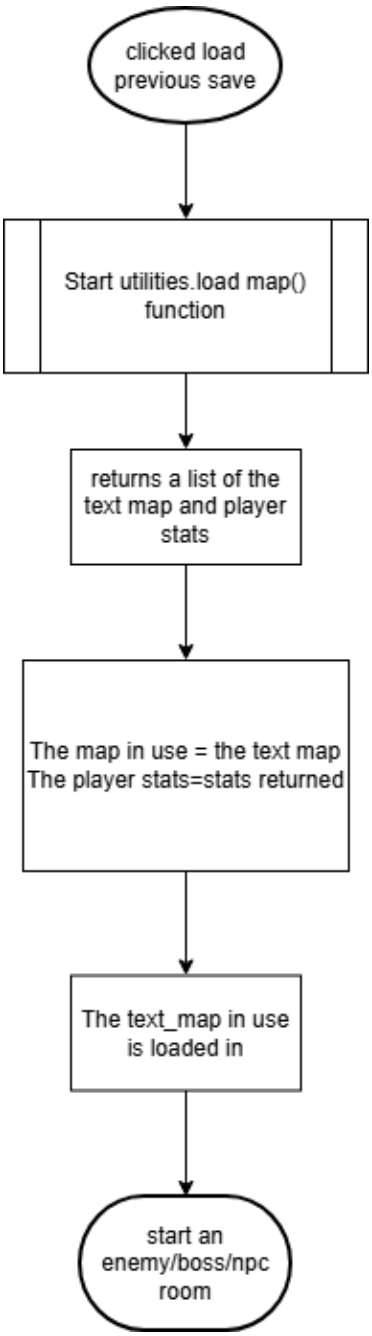


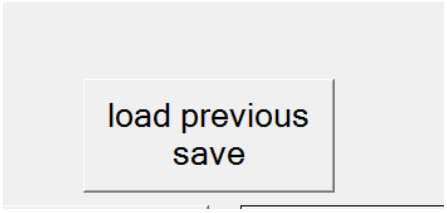




LOAD MAPFROM SAVE FILE:

If load the previous save button is pressed read each line of the text file and load into text file, load the character colour and different attributes.

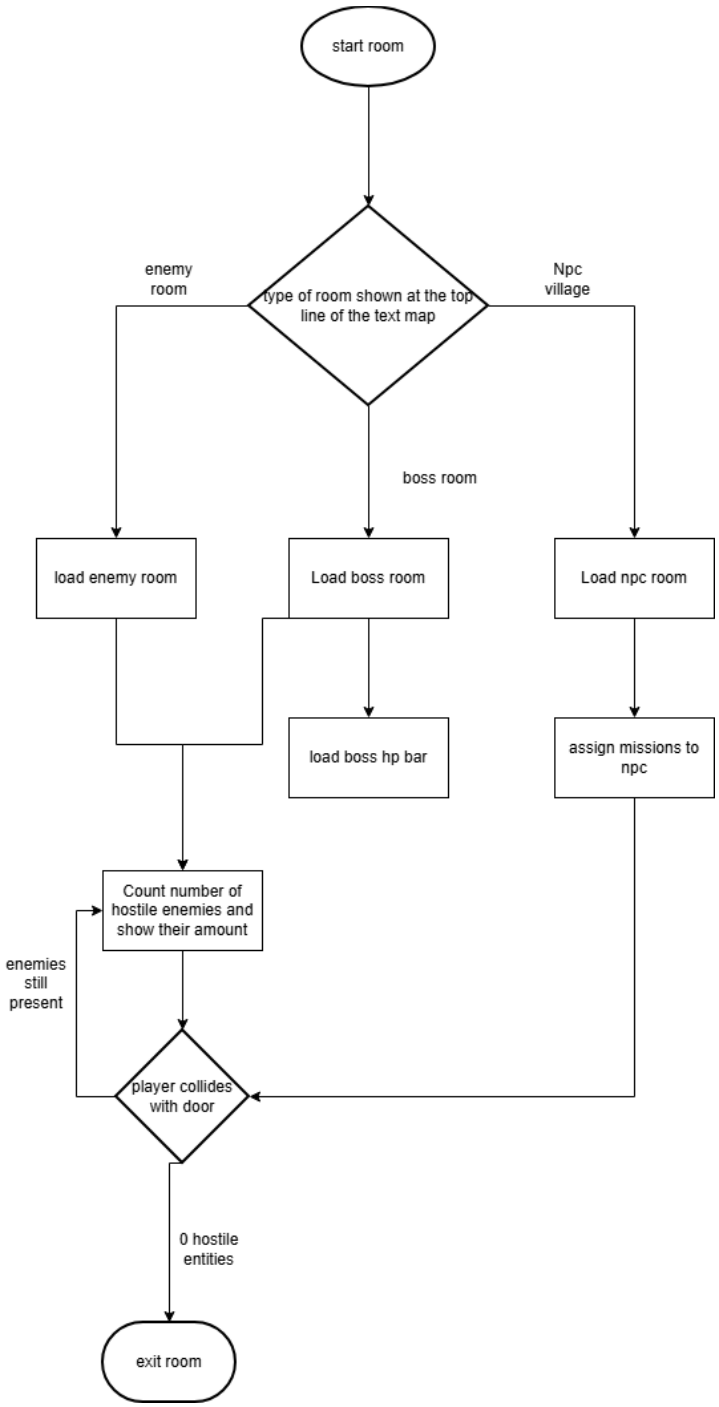




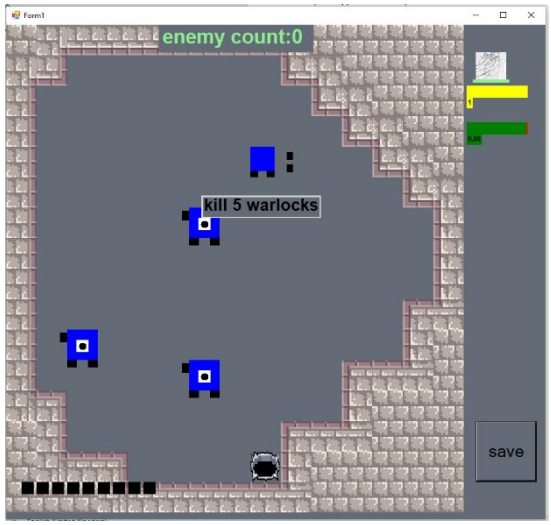

```
floor2
bbbbbbbbbbbbbbb
bbbbb........bb
bbbbbb.......bb
bbbbbb......bbb
bbbbbbbbb.bbbbb
bbbbbbbbbbbbbbb
bbbbbbbbbbbbbbb
bbbbbb.w..bbbbb
bbbbbb...pe.bbb
bbbb.......ebbb
bbbb........bbb
bb.e........bbb
bbb.w.......bbb
bb.........bbbb
bbbb.....dbbbbb
bbbbbbbbbbbbbbb
/
0
20
0
0\0\255
0\0\0
0\0\0
```

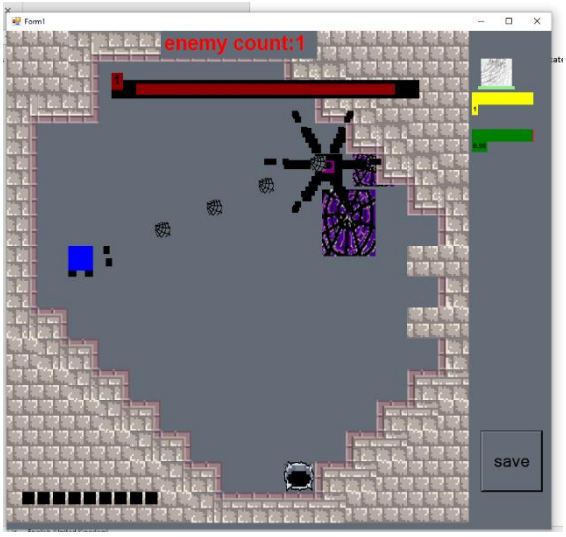
NORMAL/BOSS/NPC ROOM GAME LOOP:

Once the map is created the players is moved to the position dedicated and they must kill all enemies to progress. In boss rooms a boss hp bar appears at the top of the screen.



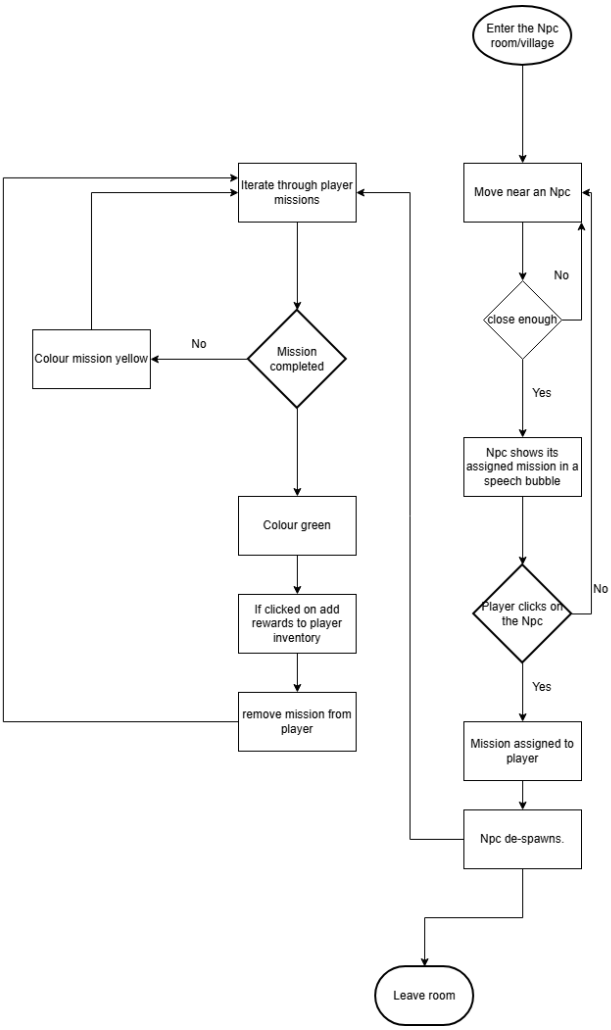


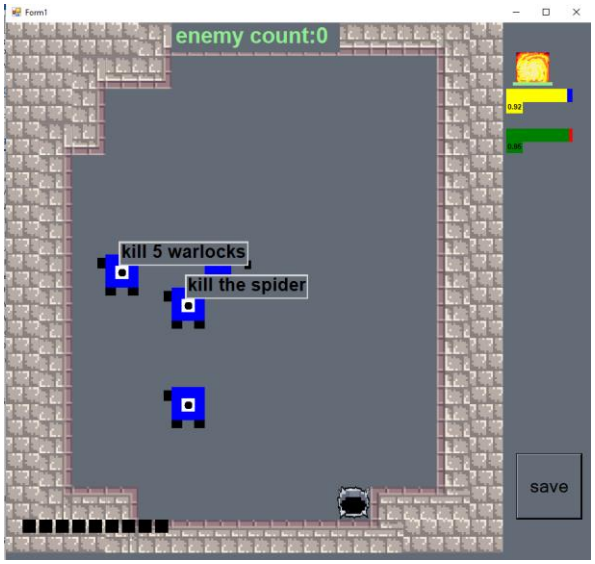


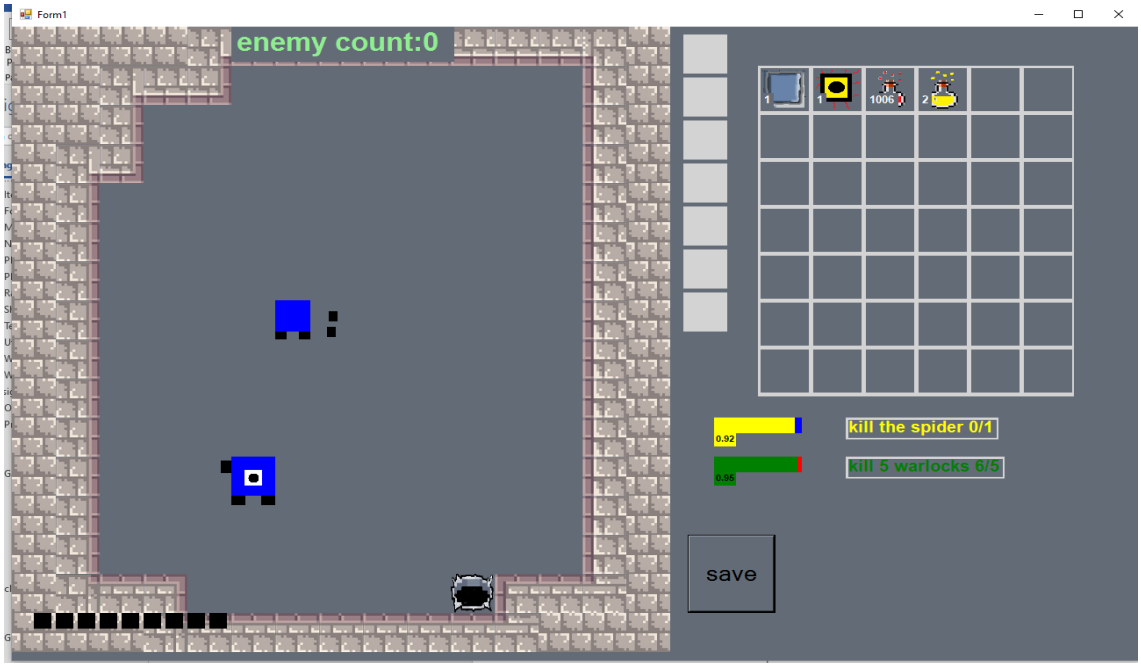


CHOOSING AND COMPLETING MISSIONS:

Th player enters the Npc village where they can click on Npc's when they are close enough choosing their mission andwhich would then appear under the player inventory in yellow and showing which enemy needed to be killed and how many.

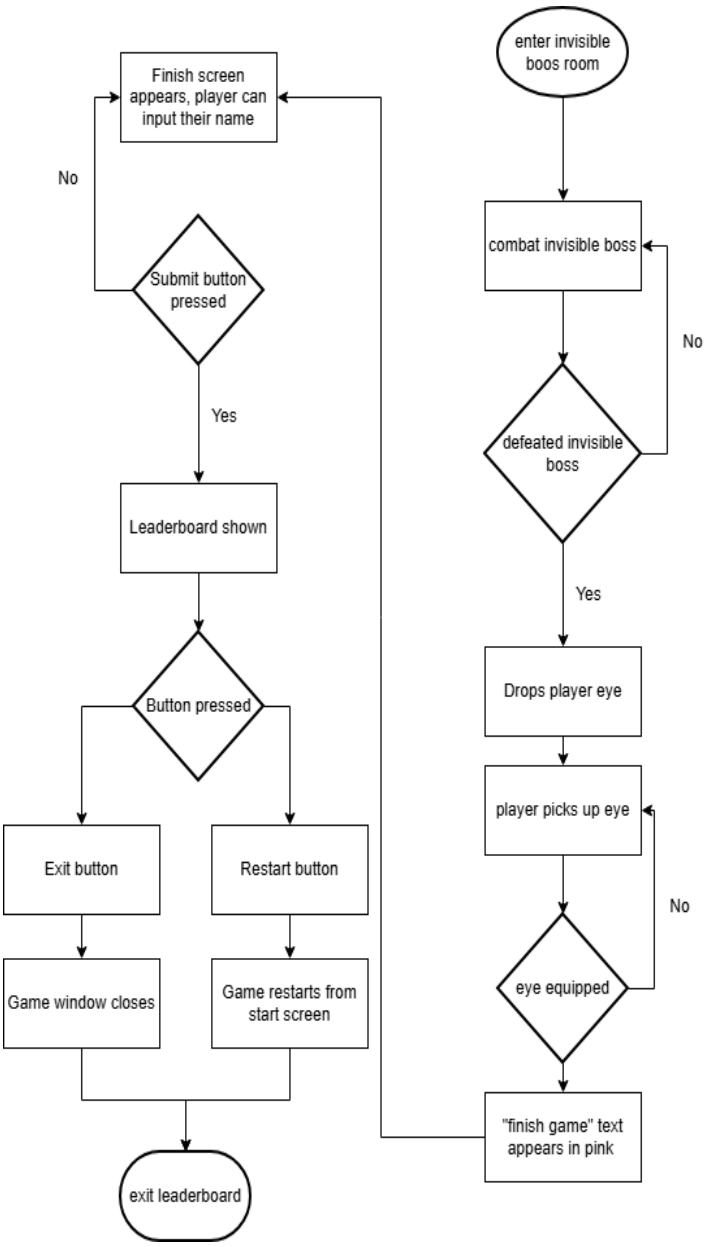


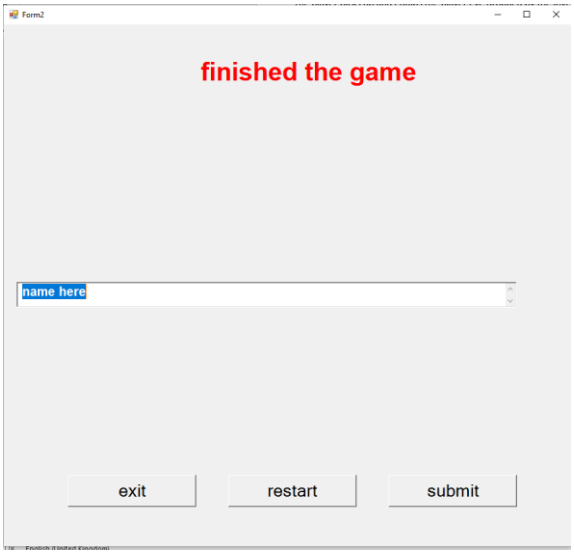


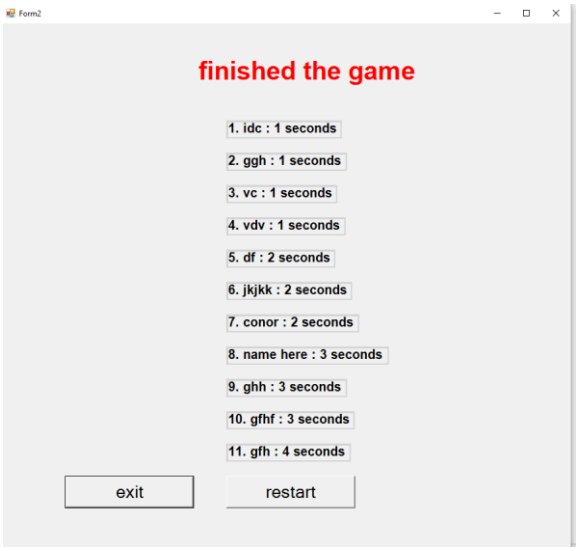


FINISHING SCREEN:

The player picks up and equips the player eye dropped by the invisible boss then the game says “GAME FINISHED” and the finish screen appears. The user inputs their player name and clicks submit where they score is shown next to all others.







CLASS DIAGRAM:

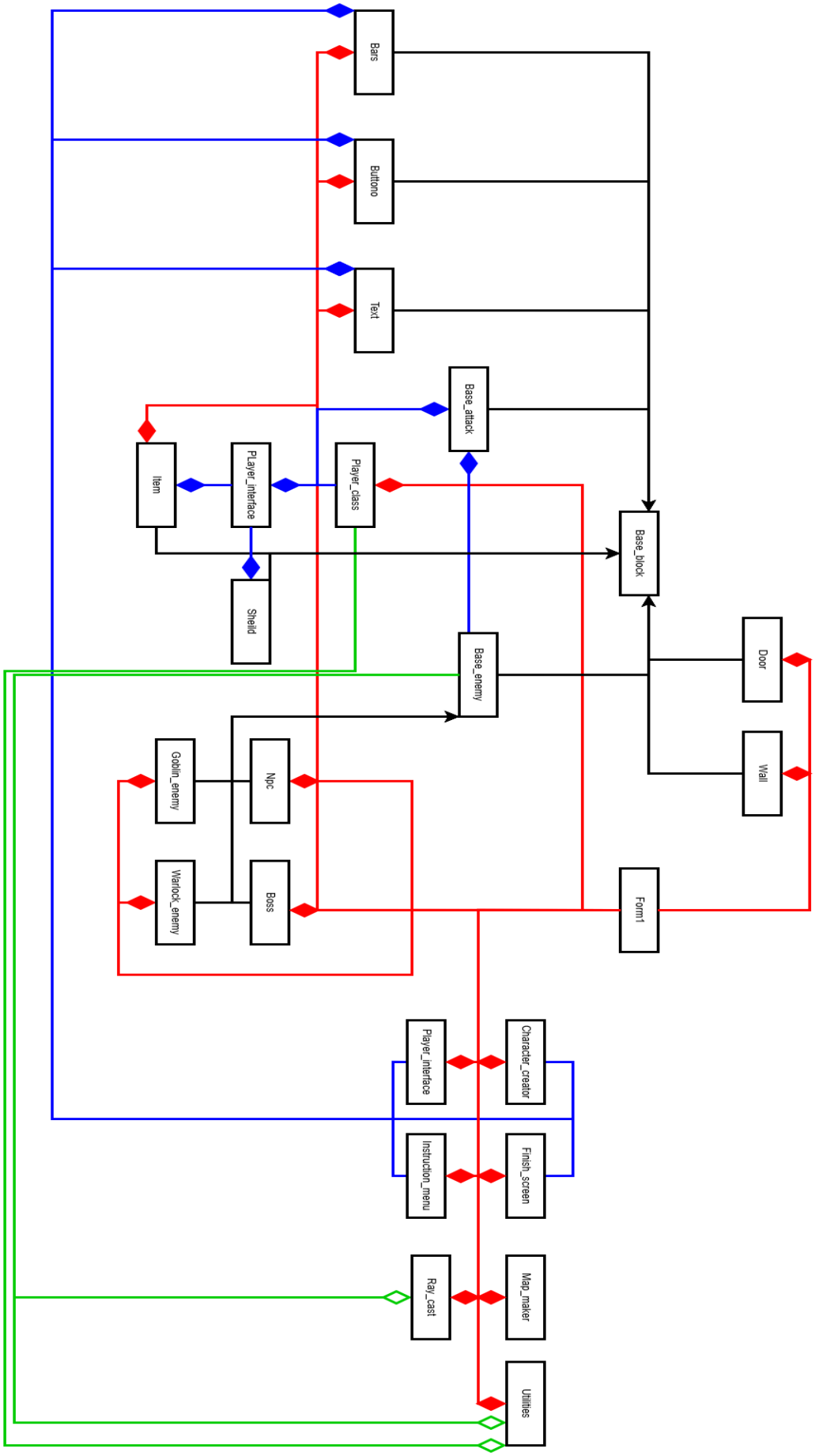
RELATIONSHIP DIAGRAM:

Red: Composition involving form1

Blue: Composition not involving form1

Black: Inheritance

Green: Aggregation



CLASS DEFINITIONS:

Form1

- plr1: player
- char_crea: character_creator
- intru_men: instruction_menu
- enemy_list: List(Of base_enemy)
- boss_list: List(Of boss)
- door_list: List(Of door)
- floor_items: List(Of item)
- NPC_list: List(Of Npc)
- dead_enemys: List(Of base_enemy)
- new_floor_items: List(Of item)
- type_bos: String
- raycaster: ray_cast
- utiliser: utilities
- timera: Timer
- timer_num: Integer
- start_time: Date
- end_time: Date
- finish_scrn: finish_screen
- walls_list: List(Of wall)
- usable_wallz_list: List(Of wall)
- text_map1: List(Of String)
- text_map2: List(Of String)
- text_map3: List(Of String)
- NPC_village: List(Of String)
- boss_floor: List(Of String)
- text_mapp_used: List(Of String)
- all_missions: Dictionary(Of List(Of String), List(Of Integer))
- all_missions_txt: List(Of List(Of String))
- mousex: Integer
- mousey: Integer
- all_buttons: List(Of buttono)
- start_btn: buttono
- exit_btn: buttono
- instruct_btn: buttono
- load_btn: buttono
- save_btn: buttono
- finished_menu: Boolean
- start_text: text
- game_has_start: Boolean
- id_enemy: Integer
- id_npc: Integer
- enemy_count_txt: text

+ create_map()
+ plr_finished()
+ change_room()
+ amount_of_en_txt()
+ update_flr_items()
+ update_npcs()
+ update_enemies()
+ Form1_Load(sender: Object, e: EventArgs)
+ load_game()
+ clear_map()
+ clear_start_menu()
+ load_old_game()
+ game_update()
+ start_screen()
+ Timer1_Tick(sender: Object, e: EventArgs)

utilities

- slmpath: String
- aspath: String
- inpath: String
- mipath: String
- svscpath: String
- loaded_game: Boolean
- rand: Random
- _mousex: Integer
- _mousey: Integer
- mousex: ReadOnly Property(Integer)
- mousey: ReadOnly Property(Integer)

+ file_exisists(file_path: String, read_write: String, Optional clear: Boolean = False)
+ add_missions(all_missions: Dictionary(Of List(Of String), List(Of Integer)))
+ Rand0(Optional lower_bound: Integer = 1, Optional upper_bound: Integer = 100) As Integer
+ Form1_MouseMove(sender: Object, e: MouseEventArgs)
+ save_map(string_to_save: List(Of String), plr1: player)
+ save_score(textp: String)
+ load_map()
+ Load_scores(load_dict: Dictionary(Of String, String))
+ load_attacks(type: String)
+ load_player_inteface_items(
+ load_missions()

instruction_menu

- game: Form
- _created: Boolean
- all_buttons: List(Of buttono)
- done_btn: buttono
- all_text: List(Of text)
- game_aim_txt: text
- move_keys: text
- inv_keys: text
- attack_keys: text
- scroll_keys: text
- sheild_key: text
- use_consumables: text
+ created: Boolean 'Property

+ New(mygame: Form)
+ make_buttons()
+ make_text()
+ load()
+ update()

finish_screen

- timera: Timer
- timer_num: Integer
- txtInput: text
- finiText: text
- all_text: List(Of text)
- submitbtn: buttono
- exitbtn: buttono
- rstbtn: buttono
- all_buttons: List(Of buttono)
- show_all_text: Boolean
- leaderboard_dict: Dictionary(Of String, String)
- util: utilities

+ Form2_Load(sender: Object, e: EventArgs)
+ load_leaderboard()
+ sort_dict()
+ quick_sort(my_list: List(Of Tuple(Of Integer, String))) As List(Of Tuple(Of Integer, String))
+ Timer2_Tick(sender: Object, e: EventArgs)

map_maker

- width: Integer
- height: Integer
- new_map: List(Of String)
- type_room: String
- util: utilities

+ New(utl: utilities, wid: Integer, hei: Integer, Optional typrm: String = "normal")
+ generate_random_map() As List(Of List(Of String))
+ is_navig(map: List(Of List(Of String)), start_pos: Integer(), end_pos: Integer()) As Boolean
+ load_map(Optional bossrm: Boolean = False) As List(Of String)
+ find_path(startx: Integer, starty: Integer, endx: Integer, endy: Integer, map: List(Of List(Of String))) As List(Of Integer())

base_block

+ box: PictureBox
game: Form
+ x: Integer
+ y: Integer
+ xvel: Integer
+ yvel: Integer
+ width: Integer
+ height: Integer
- wall_img_pth: String

+ New(game: Form, height: Integer, width: Integer, x: Integer, y: Integer)
+ load()

ray_cast

- nodes: List(Of List(Of Array))
- wallz: List(Of List(Of Array))
- box: List(Of String)

+ New(string_map: List(Of String))
+ connect_nodes()
+ create_nodes()
+ find_wall(nodeon: Array, wallz: List(Of List(Of Array))): List(Of Integer)
+ empty(coords: Array): Boolean
+ normalize_walls()
+ ray_blocked(from_coords: Array, to_coords: Array): Boolean
+ load_wallz()

+ New(string_map: List(Of String))
+ connect_nodes()
+ create_nodes()
+ find_wall(nodeon: Array, wallz: List(Of List(Of Array))): List(Of Integer)
+ empty(coords: Array): Boolean
+ normalize_walls()
+ ray_blocked(from_coords: Array, to_coords: Array): Boolean
+ load_wallz()

character_creator

+ game: Form
- plr1: player
- str_bar: bar
- str_txt: text
- dex_bar: bar
- dex_txt: text
- wis_bar: bar
- wis_txt: text
- bar_list: List(Of bar)
- txt_list: List(Of text)
- btn_list: List(Of List(Of buttono))
- image_list: List(Of PictureBox)
- lone_btn_list: List(Of buttono)
- _donebtn: buttono
- next_limb: buttono
- bef_limb: buttono
- attribute_pnts: Integer
- attri_txt: text
- colour_limbs: Dictionary(Of Integer, List(Of Integer))
- color_btns: List(Of buttono)
- rgb_vals: List(Of String)
- limb_on: Integer
- limb_on_txt: text
+ created: Boolean
+ ended_char_crea: Boolean

+ New(plr: player, mygame: Form)
- make_bars()
- make_buttons()
- make_txt()
- make_images()
- colr_limbs()
- update_plr_clrs()
- finish_maker()
+ load()
- update()

player

- util: utilities
- max_hp, hp: Integer
- max_mana, mana: Integer
- max_armour, armour: Integer
- all_stats(3): Integer
- enemy_lisz: List(Of base_enemy)
- door_lisz: List(Of door)
- wall_lisz: List(Of wall)
- floor_items_lisz: List(Of item)
- npc_lisz: List(Of Npc)
- live_attack: List(Of base_attack)
- dead_attack: List(Of base_attack)
- attack_dict: Dictionary(Of String, List(Of String))
- chosen_attack: String
- index_on_attack: Integer
- tick: Double
- my_interface: player_inteface
- open_inv, opened_ui, loaded_all: Boolean
- equipped(8): String
- collided_flooritem: Integer
- my_sheild: sheild
- touch_door: Boolean
- which_door: door
- mousex, mousey: Integer
- leftLeg, rightLeg, leftArm, rightArm: PictureBox
- legWidth, legHeight, armW, armH: Integer
- all_limbs: List(Of PictureBox)
- leg_move: String
- attack_img: Bitmap
- all_missions: Dictionary(Of List(Of String), List(Of Integer))
- killed_enemy_amounts: Dictionary(Of String, Integer)
- start_level: Boolean
- boss_room: Boolean
- finished_game: Boolean

+ New(game: Form, height: Integer, width: Integer, x: Integer, y: Integer, util: utilities)
+ key_handler(sender, e: KeyPressEventArgs)
+ get_angle_to_mouse(mx: Double, msy: Double) As Double
+ click_handler(sender, e: MouseEventArgs)
+ scroll_wheel(sender, e: MouseEventArgs)
+ update_bars()
+ update_attacks()
+ update_invent_or_ui()
+ update_armour()
+ update_mission()
+ load_slash()
+ load_fireball()
+ load_lightning()
+ load_attacks()
+ load_invent()
+ collisions()
+ player_box_collision(hit_taken: Integer, target: Object) As Boolean
+ pushback(entity: base_enemy, xer: Integer, yer: Integer)
+ limb_load()
+ update_arms(x: Integer, y: Integer)
+ movement_nums()
+ move_limbs()
+ update(all_wallz: List(Of wall), timer: Integer, mouses_x: Integer, mouse_y: Integer, enemies: List(Of base_enemy), doorz: List(Of door), flooritems: List(Of item), allnpc: List(Of Npc))

base_enemy

util: utilities
+ type_enemy: String
+ type_magic: Boolean
+ colour: Color
see_radius: Integer {Const = 800}
sees_player: Boolean
see_timer: Integer
plr1_see: Boolean
+ dead: Boolean
playerx: Integer
playery: Integer
enemyx: Integer
enemyy: Integer
max_hp: Integer
+ hp: Integer
+ attack_dict: Dictionary(Of String, List(Of String))
+ live_attacks: List(Of base_attack)
+ dead_attacks: List(Of base_attack)
+ cubes: List(Of base_block)
angle: Double
leftLeg: PictureBox
rightLeg: PictureBox
leftArm: PictureBox
rightArm: PictureBox
+ eye: PictureBox
+ eyebox: PictureBox
legWidth: Integer
legHeight: Integer
armWidth: Integer
armHeight: Integer
+ all_limbs: List(Of PictureBox)
+ leg_move: String
- _id: Integer
+ id: Integer {ReadOnly Property}
+ raycast_obj: ray_cast

+ New(game: Form, height: Integer, width: Integer, x: Integer, y: Integer, id: Integer, hp: Integer, utility: utilities, Optional magic: Boolean = False)
+ collisions(wallz: List(Of wall), plr: player, tick: Integer)
- move_to_plr(plr: player, speed: Integer)
limb_load()
update_eye()
update_arms(plr1: player) {MustOverride}
update_legs(tick: Integer)
+ can_see_move(plr1: player, tick: Integer, map: List(Of String))
+ check_in_range(plr1): Boolean
+ move_limbs(tick: Integer, plr1: player)
+ change_col(tick: Integer)
load_attacks_fromtxt(Optional which_atks: String = "plren")
update(all_objects: List(Of wall), plr1: player, enemies: List(Of base_enemy), map: List(Of String), tick: Integer) {Overridable}

Goblin_enemy

- type_enemy: String
- box: PictureBox (inherited from base_enemy)
- colour: Color (inherited from base_enemy)
- leftArm: PictureBox (inherited from base_enemy)
- rightArm: PictureBox (inherited from base_enemy)

+ update_arms(plr1: player) 'Overrides base_enemy
+ New(game: Form, height: Integer, width: Integer, x: Integer, y: Integer, id: Integer, hp: Integer, util: utilities)

warlock_enemy

- fireball_path: String
- fireball_img: Bitmap

+ New(game: Form, height: Integer, width: Integer, x: Integer, y: Integer, id: Integer, hp: Integer, util: utilities, Optional magic: Boolean = False)
+ update_arms(plr1: player)
+ attack(x: Double, y: Double, tick: Integer, angle: Double)
+ update_attacks(Timer: Integer)
+ load_attacks(tick, plr1)
+ update(all_objects: List(Of wall), plr1: player, enemies: List(Of base_enemy), map: List(Of String), tick: Integer)

Boss

thighs: List(Of List(Of PictureBox))
femur: List(Of List(Of PictureBox))
left: Boolean
next_xer: List(Of Double)
next_yer: List(Of Double)
+ radian: Double
+ point: List(Of Double)
+ atk_point: List(Of Double)
+ body_parts: List(Of base_block)
+ invisible: Boolean
- plr1: player

+ New(game: Form, x: Integer, y: Integer, width: Integer, height: Integer, type_of: String, maxhp: Integer, plr: player, utility: utilities, Optional mgic: Boolean = True)
+ update_spider(tick: Integer
update_arms(plr1: player) {Overrides}
+ spider_atks(plr1: player, tick: Integer)
+ load_legs()
+ invisi_atks(plr1: player, tick Integer)
+ update_cubes_bodyparts(tic Integer)
+ load_cubes_bodyparts()
+ update_invisi(tick: Integer)
+ collisions(wallz: List(Of wall), plr: player, tick: Integer {Overrides}
+ update_attacks(Timer: Integer)
+ load() {Overrides}
update(all_objects: List(Of wall), plr1: player, enemies: List(Of base_enemy), map: List(Of String), tick: Integer) {Overrides}

player_interface

- game: Form

'inventory
- all_items: List(Of item)
- itemDict: Dictionary(Of String, List(Of Integer))

'missions
- missions_npcs: List(Of Npc)

'UI
- hp_bar: bar
- mana_bar: bar
- attack_bar: bar
- boss_bar: bar
- ui_lis: List(Of PictureBox)
- all_equip_boxes: List(Of base_block)
- chosen_attack: PictureBox
- grid_box: PictureBox

+ New(form_used: Form, util: utilities)
+ load_bars()
+ check_boss_bar(boss_room Boolean)
+ load_ui(plr: player)
+ load_ui_again(plr: player, boss_room: Boolean)
+ delete_ui()
+ load_equipped()
+ show_equipped()
+ load_armour_stats()
+ load_mis()
+ delete_mis()
+ darken_bitmap(original: Bitmap, darkenfactor: Single) As Bitmap
+ not_in(val1: String, lisz: List(Of item)) As Boolean
+ load_inventory_imgs()
+ load_inventory()
+ draw_grid(cellSize: Integer, cols: Integer, rows: Integer)
+ delete_inventory()
+ add(name: String)
+ clear_inventory()
+ update(tick: Integer, plr: player) As Tuple(Of String,

CLASS EXPLANATIONS:

WINDOW CLASSES:

Form1:

This class is used by Visual Basic as the window the user interacts with, having many inbuilt functions and subroutines in it to do so, e.g. (Load and Timer.Tick). I have also created something of a god class, as if I did not want to do this I would have to create multiple other classes which would not be necessary in a personal project like this one.

This class also handles the page refreshing so items can be moved and player inputs can be registered.

The first page the user is introduced to is the entry page that has 3 buttons (exit, start and instructions).

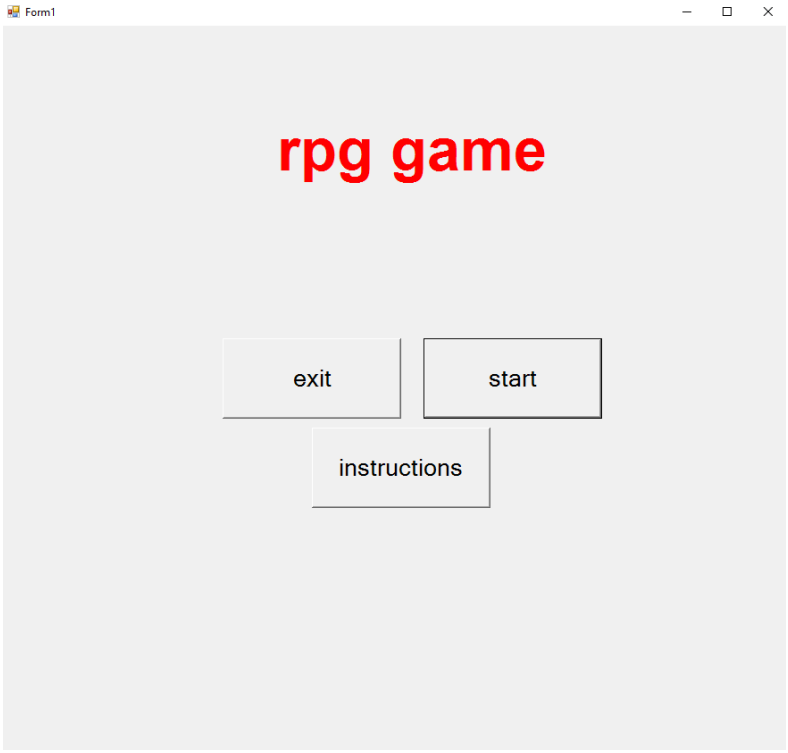


FIGURE 8 THE START PAGE

As you can see, text is also used to portray the name of the game to the user.

Once the game has started, after the character has been created, the loop no longer updates the start screen but begins the gameplay and deletes the previous screen.

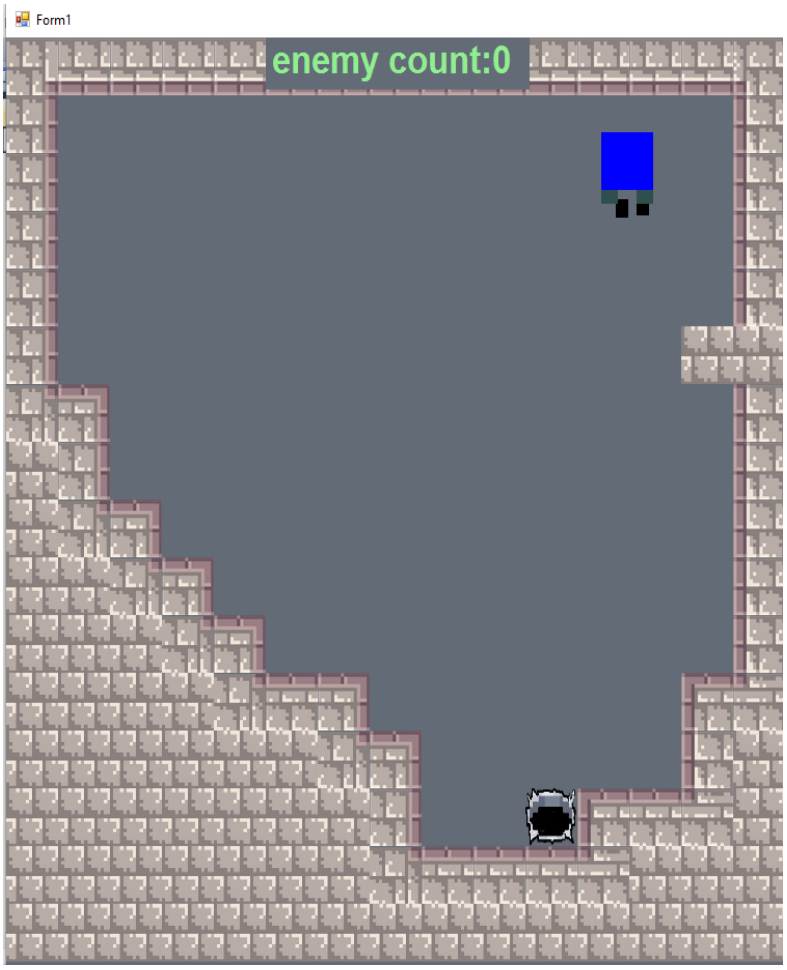
The map is loaded in; this is done through creating the map in the map_generation class. This list of strings is then iterated through, with each character (excluding “.” as it is empty space) being a key for a certain object, e.g. “b” = wall, “w” = warlock. Then each of these objects is added to a list of their type (wall appended to wallz_list and enemy to enemy_listz). These are then iterated through separately and updated (e.g. to check collisions, paths, ray casting, player inputs).

The player class is also initialised with all its various components being loaded in.

The start_screen window is updated until the game_has_start Boolean is assigned a true value, causing the game_update subroutine to be called in a loop by the timer.

This form also controls the event of the player death occurring, which can be concluded if the player health <= 0. The same subroutine also checks whether the game has finished. This is done by checking whether the "player_eye" has been equipped and therefore is in the equipped array in the player class.

Any previously saved games are also loaded in by getting all the information in a 2D list of strings from the utiliser class, then applying those stats to the player — being colours of different limbs and player attribute levels. The main art is the map, which can just have its 2D list of strings saved and then loaded in by the form.



As you can see, this is the generated map. Also, at the top of the window the enemy count is shown, using text to give the user information on how many enemies are left and whether they can leave the room.

Not only are objects updated, but they are also deleted when the character either moves into a new room or they are killed. This destruction is done partly by form1 and partly by the objects themselves, by iterating through the list they are a part of and getting rid of them based on hitpoints, changing to a new room, or time of life (for attacks).

Similar to this, the seconds are roughly counted by the program (I say roughly as Visual Basic loops are not accurate and change depending on the computer used, how well optimised the code is, and the page being used). As a result, I can time certain events (cooldowns, attack death times, and enemy movement).

Character_creator:

This is not its own form and it is updated and loaded by form1.

This is the area for users to create their character. This involves the player's colours and its stats, which consist of wisdom, strength and dexterity, which correlate to health and physical damage, speed, maximum mana and magic damage.

They do this by pressing different buttons which control sliders and allow them to use a certain amount of attribute points. The player character is created in this part, allowing them to change the colour of its arms, legs and body.

The bars, buttons and text are created and loaded in the make_bars(), make_buttons() and make_txt(). The page and all these features are deleted in the finish_maker().

To change the colour of the limbs, when the button is pressed the change it makes is stored in a list to affect integers. These are then used in update_plr_clrs(), where the player_class picturebox colours are changed.

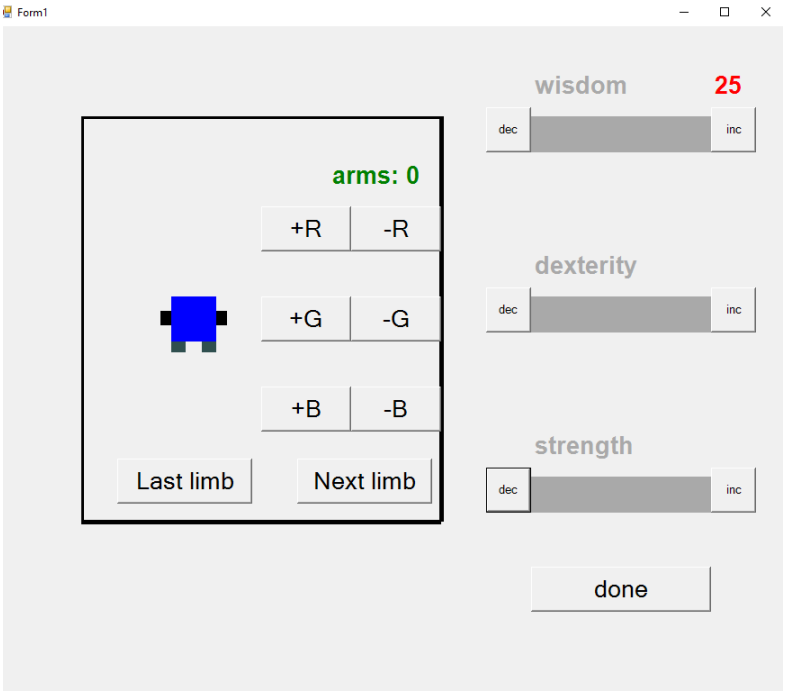


FIGURE 10 THE CHARACTER CREATOR

As you can see, the colour is controlled by adjusting one of the 3 RGB values. Once a limb is done, you can move to the next one. To access this screen, the user must press the start button in the start menu. Once the user is finished, they press done to start the real game. If the load button in the start menu is pressed, then this window is skipped as the player attributes will already have been changed.

Instruct_menu:

This is not its own form and it is updated and loaded by form1.

This page is used to give vital information to the user on how the game works, using text boxes to create paragraphs of writing. Once finished, they can press the done button, restarting the game and sending them back to the start menu.

The text and buttons are created and loaded in the make_buttons() and make_text() subroutines. In the make_text subroutine.the text

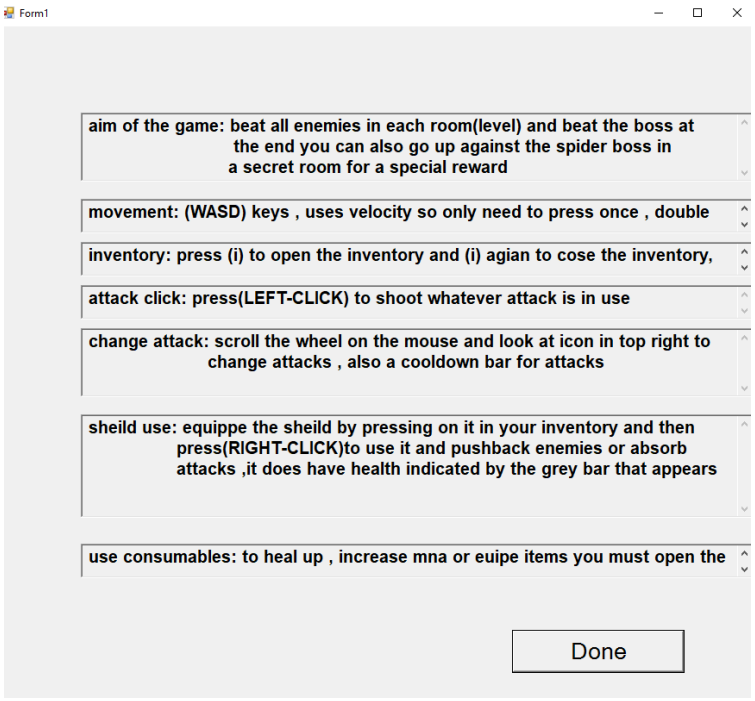


FIGURE 11 INSTRUCTION MENU

To enter this page the user must press the instruction button in the start menu and to exit it the done button is pressed which just restart the program meaning the user is sent back to the main start page.

Finish screen:

This does use its own form and is only opened when form1 is closed.

This page is accessed at the end of the game, once the player has collected their lost eye (in-game lore). Once equipped in the helmet position, this window opens, deleting the old form and showing itself.

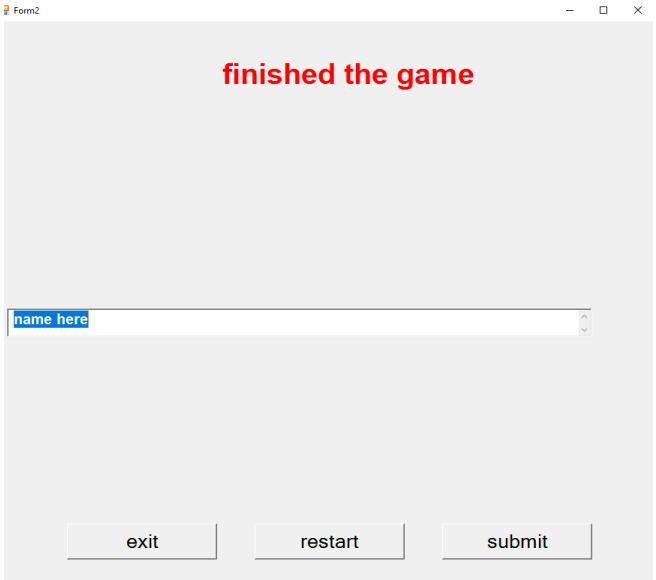


FIGURE 12 THE START OF THE FORM2 WINDOW

This screen allows the user to enter the name, also it contains an exit so they can leave the game, a restart button so they can replay the game and a submit button so they can finish entering their name for the leader board.

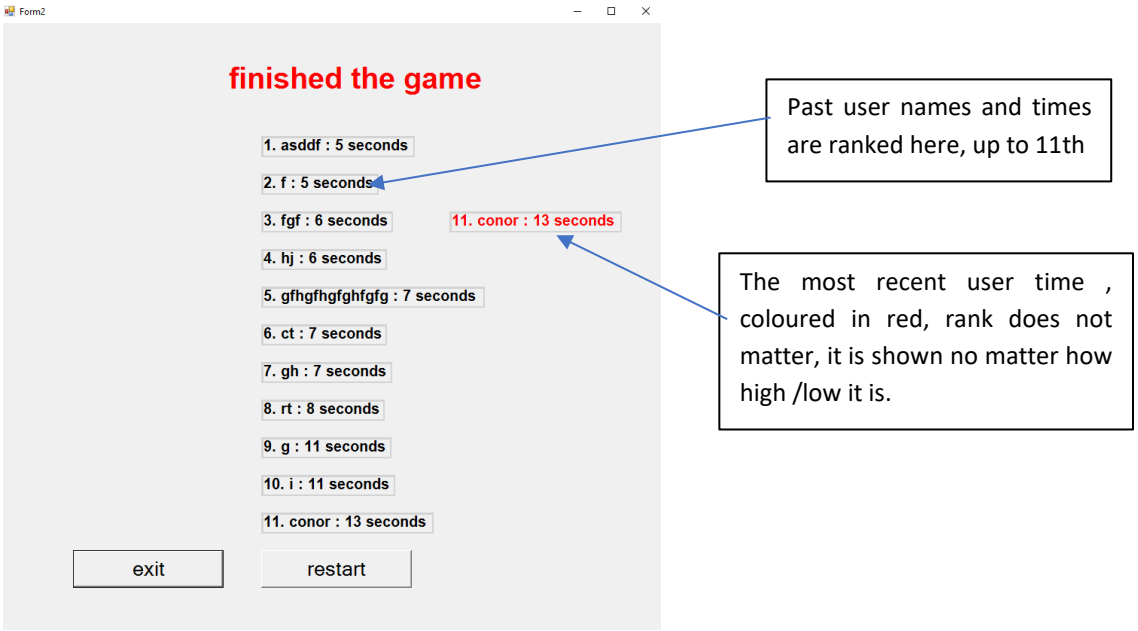


FIGURE 13 END OF THE FINISHED GAME WINDOW, LEADER-BOARD

As you can see once the submit button is pressed the user is taken to this page where previous rankings are shown and ordered depending on times, the name in red signifies the most recent time recorded.

These name and times are saved to a text file so no matter how many times you re-load the game they go no-where, they are loaded in the utiliser class then a dictionary containing them is sent here. Then a quick sort algorithm is used(done in the sort_dict() subroutine), by converting them into list form and ordering them from lowest to highest by time and reverting it back into dictionary form , this is done in the load_leaderboard() subroutine. After place 11 results aren't shown except the most recent score. To create the borders a box is drawn around each place, this makes them stand out and separates them from one another.

MAIN PARENT CLASSES

Base_block:

Does not inherit from any other class being the superclass,

It contains the main attributes of each entity (x, y, width, height xvel, yvel) and the picture box used to represent each. It also contains the form, having an aggregation type of relationship with it, allowing it to load the picture box to the screen

CLASS DEFINITION

Base_block= Class {

Private:

Wall_img_path:string

Protected:

Game:form

Public:

x:integer

y:integer

xvel:integer

yvel:integer

width:integer

height:integer

subroutine:new

subroutine:load(overridable)

}

Base_attack:

Inherits from base block and can be used in implemented in many different ways to create unique attacks

This class also controls the life_time of each attack, deleting it if it has been alive, reducing the amount of lag over time due to many attacks being present. This is implemented by using the variable life_time and spawn_time with the function check_alive which takes tick as a parameter, this variable tracks the time the game has been running since start has been pressed.

As can be seen in fig.14 and fig.15 some attacks move whereas some are stationary this is done through the rotate_img sub and direction_to function

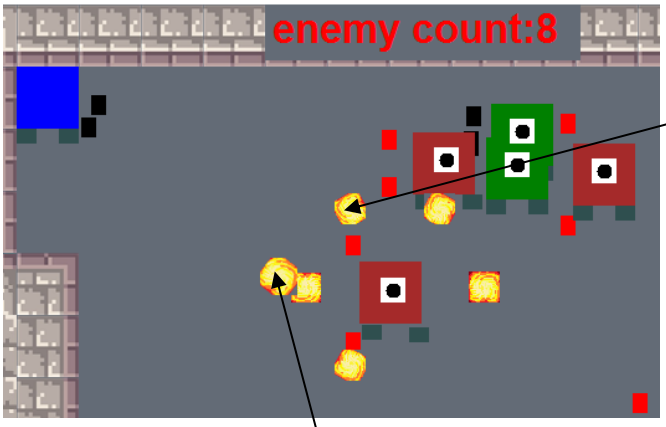
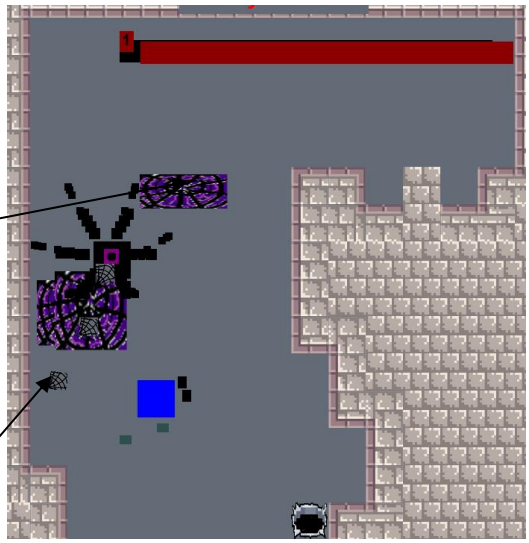


FIGURE 14 ATTACKS BEING USED BY THE PLAYER AND ENEMIES



CLASS DEFINITION

Base_attack= Class(base_block) {

Private:

_typeofattack:string

_damage:integer

Public:

life_time:integer

spawn_time:integer

property:damage

property:typeofattack(readonly)

subroutine:rotate_img

subroutine:move

function:direction_to

}

UI CLASSES:

Buttono (In-built button class is NOT used):

This class is used to create a button like object that can be used in various UI aspects of the game (form1, form2, instruction menu and character creator) having size stats, width and height, location stats, x and y , a been_clicked Boolean and a text attribute to give the buttons meaning to the user.

The been_clicked variable is controlled by the me_onclick subroutine which is a vb forms subroutine, hence it can register button clicks.

The button is deleted with the destroy () subroutine which deletes the object.

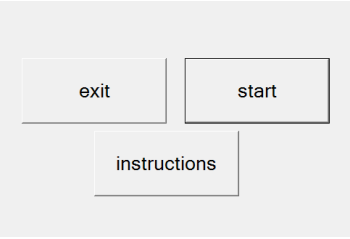


FIGURE 16 AN EXAMPLE OF THE BUTTONS

Once they are clicked the been_clicked variable becomes true letting the program know that they have been clicked, however to stop uncontrolled clicks the Been_clicked variable must be set back to false after it is used by the program!

It is called **Buttono** to not interfere with the in-built button class.

Text:

Location and size stats are used so it can be moved and the actual words are represented by the my_text variable so words can be changed during runtime e.g. (number of enemies, number of items and attribute points)



FIGURE 17 CHANGING NUMBER TEXT



FIGURE 18 CHANGING NUMBER AND WORDS TEXT



FIGURE 19 STATIC TEXT

As well as these borders can be added and text boxes can be used so large amounts of text can be written (instruction menu)

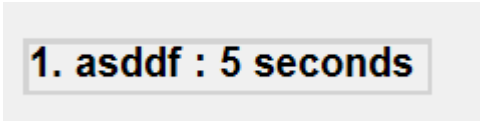


FIGURE 20 BORDERED TEXT

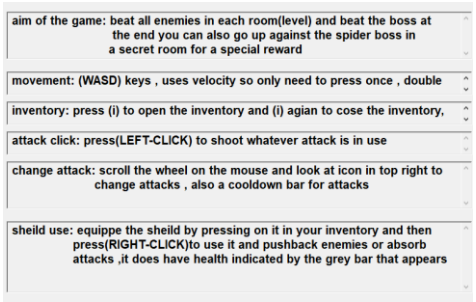


FIGURE 21 TEXTBOXES

Also, colour can be changed before the program is run and during run-time.

Bars:

The bars inherit baseblock and also uses a picturebox.

These bars are used to show ratios between stats therefore giving more visual information to the user, this is done by having to picture boxes of different colours on top of each other so when 1 goes down the other can be seen. This means their size must be changed in run-time which can be done through a subroutine that uses the total e.g. (maximum hp, mana or shield health) and the instantaneous hp, mana, or shield health then divides the second by the first to find a decimal <1 which the set size is multiplied by to show a change in one of these stats and visualizes this to the user.



FIGURE 22 THE DEXTERITY STAT BAR

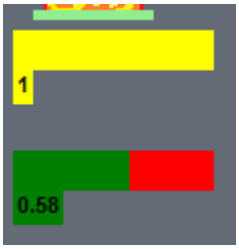


FIGURE 23 THE COOLDOWN, MANA AND HP BAR

As you can see in fig.21 there are also numbers used to give more in depth information of the stats, this is a variable in the bar class that allows text to be toggled on and off, showing the ratio of max stat to instantaneous stat in decimal form.

ENEMY CLASSES:

Base_enemy:

This class inherits a base block for the main body of each enemy.

This class is used by all others in the enemy classes, as a result it holds a lot of functions, subroutines and variables used by each. This includes the size, position, radius in which it can see, the state of its limbs and the position of the player. First, I shall go over how limbs work at their base functionality, the arms follow the main body of the player and point towards the player, the legs also follow the body but only move every few frames to simulate walking or footsteps.

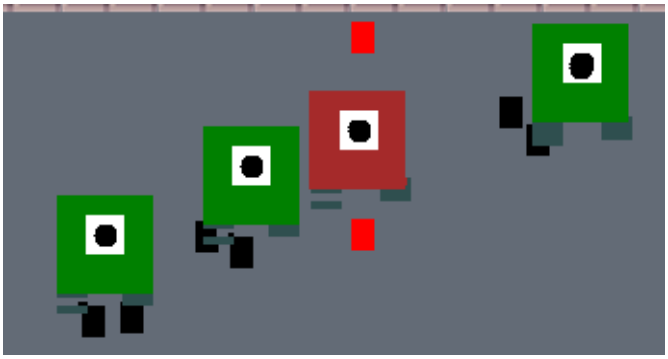


FIGURE 24 THE GOBLIN ENEMY AND WARLOCK ENEMY WALKING TOWARDS THE ENEMY

As you can see the enemies also have an eye that directly follows the main body and is made of an “eyebox” and eye part that uses the graphics function to draw the black circle in the centre. The ways in which the legs move one after the other is by using a variable called **leg_move** which is a string that can take the value “R” or “L” which changes every few seconds to allow either the left or right leg to move.

Next, I will go over how movement is done, it does NOT use pathfinding unlike the map_making class, instead it uses ray casting (a simplified version for performance) that creates a line between the enemy and player then check if any walls are in-between to check whether the enemy can see the player.

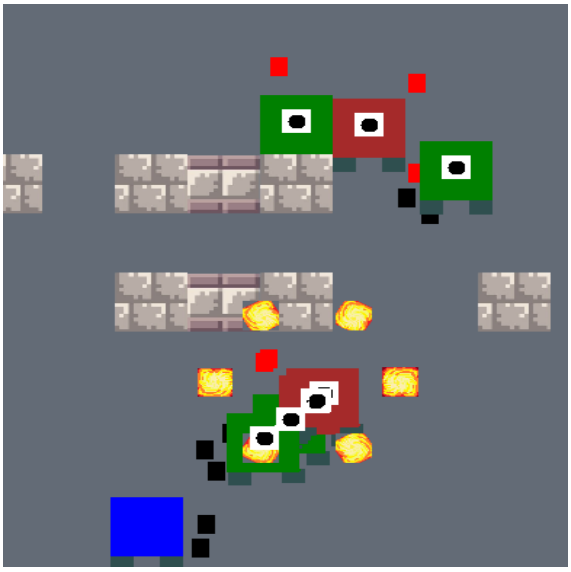


FIGURE 25 ENEMIES (IN GREEN AND RED) MOVING TOWARDS THE PLAYER (IN BLUE) IF THEY CAN SEE THEM

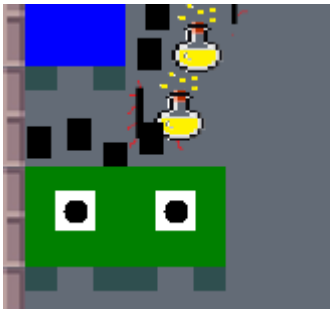


FIGURE 26 DROPPED ITEMS FROM ENEMIES

Attacks are the way enemies interact with the player, however due to the large variety of attacks only the list collecting live and dead attacks can be found in the base_enemy class, with the form1 class handling time of death of attacks that aren't instant.

If killed they can drop a variety of items that can be picked up by the player and stored in their inventory for later use.

Finally, collisions are also used to check if the enemy touches a wall, player or magic attack sent from the player. to check this all the walls and the player are used as arguments in the subroutine **collisions,** the walls are then iterated through to check for a collision then the player and all the attacks that are alive are iterated through to check if they touch so they can take push-back, take damage or apply damage to the player. Lastly, they also have a max hp and hp which is different between objects.

Warlock:

This inherits the base enemy class using it as a template for its extra features.

This entity inherits from the base_enemy and is portrayed as having a dark red body and red arms that rotate around the main body. Its main difference is the fact that it shoots a circle of fireballs in every direction when near the player every few seconds.

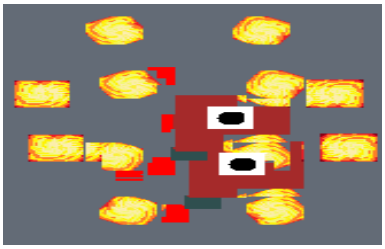


FIGURE 27 WARLOCK ENEMY

To deal with lag the warlock has an id which is used to order the time at which the fireballs are shot, as when they are all shot from each entity at once it creates a lot of picture boxes which visual basic 2010 cannot handle hence by spacing them out by id make it less lag inducing.

Goblin:

This inherits the base enemy class.

This entity is the least unique, only being able to attack through contact with the player and then getting push backed as to not create a large amount of attacks whilst touching the player. The hp of this entity is also very low as if to create an introductory enemy to the player.

It is portrayed as having a green body, grey legs (as normal) and black arms that follow the player.

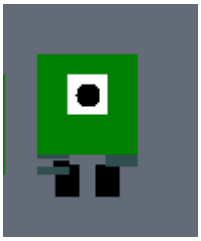


FIGURE 28 GOBLIN ENEMY

Boss:

This class also inherits the base enemy giving each boss a main body that can interact with other entities in the game(collisions, attacks the player)

These are the most unique enemies, they also have their own rooms and health bars that give the player information that is very visible. There are two types: the spider and the invisible enemy.

First, I will talk about the spider, it has a door that appears on the 4th floor that gives the player an option of fighting it, however, to increase the chance of a fight it drops a shield that can be equipped to push away enemies and absorb attacks. It has six legs and a main body, which is used for movement and followed by the legs, these move like the legs with each one alternating movement every few seconds attaching to points that surround the main body simulating crawling. The attacks that can be utilised by the spider are shooting a line of spider webs towards the player and creating sticky webs nearby slowing down the player, if they touch the spider damage is caused (bite).

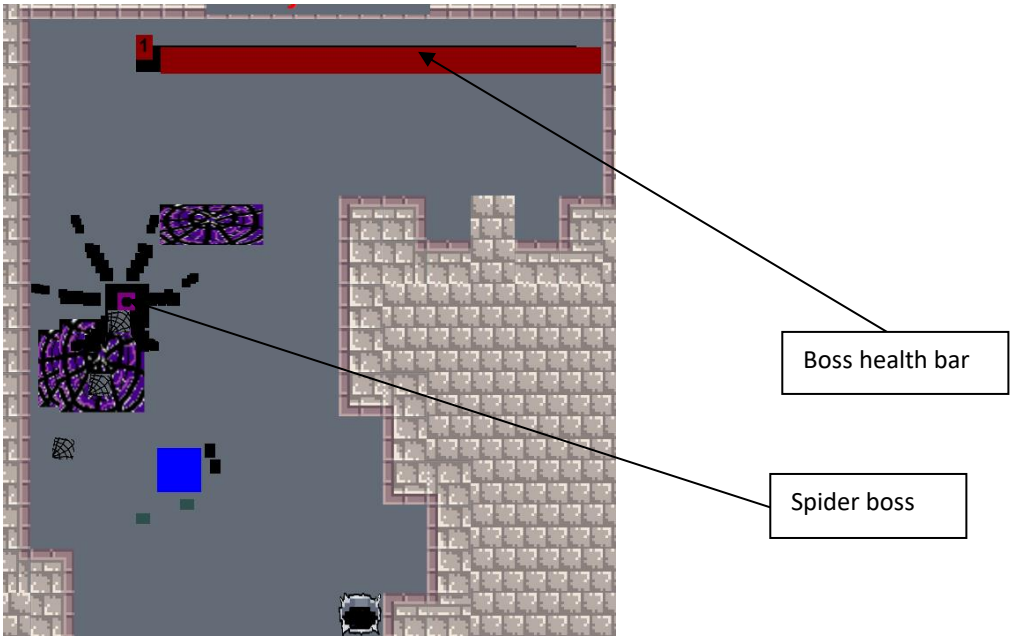


FIGURE 29 SPIDER BOSS ROOM

Secondly, the invisible boss is the final boss of the game, as a result it is very hard to beat and if done so it drops the players eye finishing the game. However, to deal damage to it, it must not be invisible and the best way is to use either the lightning spell or hitting it directly with the slash attack as it has small cubes floating around it blocking incoming attacks and doing damage to the player. These cubes are stored in a list that is iterated through and is made up of base_blocks with 2 different colours to signify if it blocks or deals damage (red and black). It goes invisible every few seconds or when its damage is dealt to it by the player, whilst invisible it stays in the state for a while and can only be tracked by the cubes surrounding it. As well as the cubes floating around it, there are also picture boxes around it to create a circle like image hence signifying an eyeball.

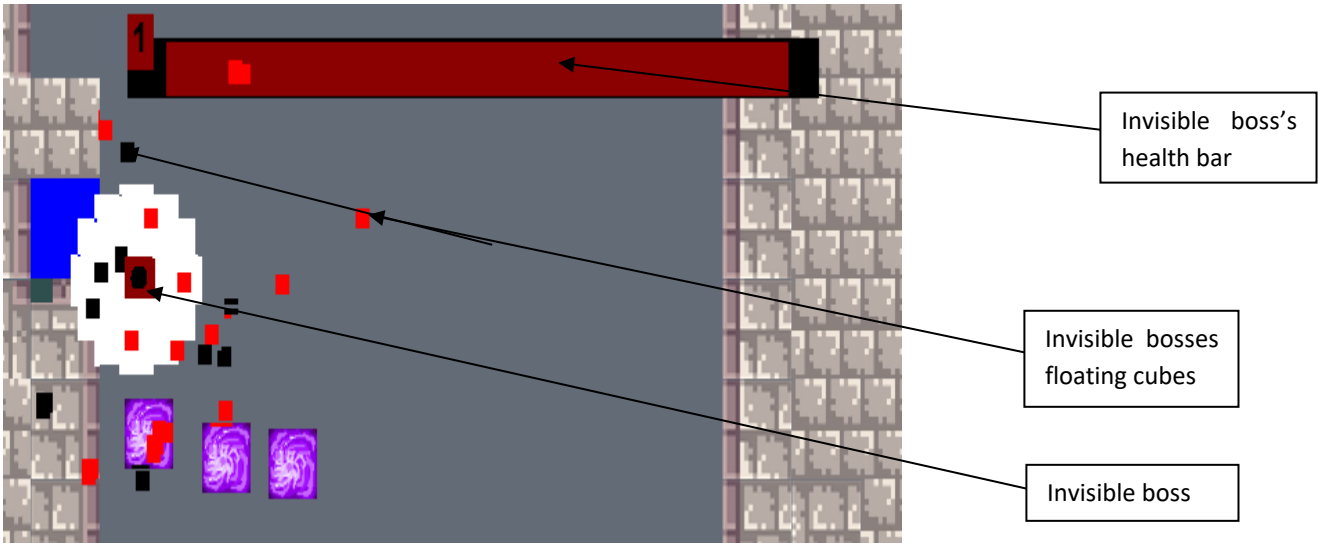


FIGURE 30 INVISIBLE BOSS ROOM

There is also an attack it has (shown by the purple orbs) that can only be done when it is visible first it sends a ray to the player then fills that ray with the purple orbs, if the player touches these orbs it is teleported to the boss and take damage from the floating cubes, although the cubes are sparsely scatted along the ray so the user is able to avoid these orbs.

Npc

Inherits from the base enemy, however, is a passive entity that only uses the base enemy for its collision and limb features.

These are not hostile entities and only inherit from the base_enemy class to create limbs and have pre-built ways to update and load itself. They share the same colour as the player to show they are off the same kind (in-game lore), they also give missions to the player by showing a speech bubble when they get close and if they are clicked on the mission they are taken by the player and they are destroyed. Missions are stored beneath the inventory UI and can be seen if the player opens the inventory by pressing (I).

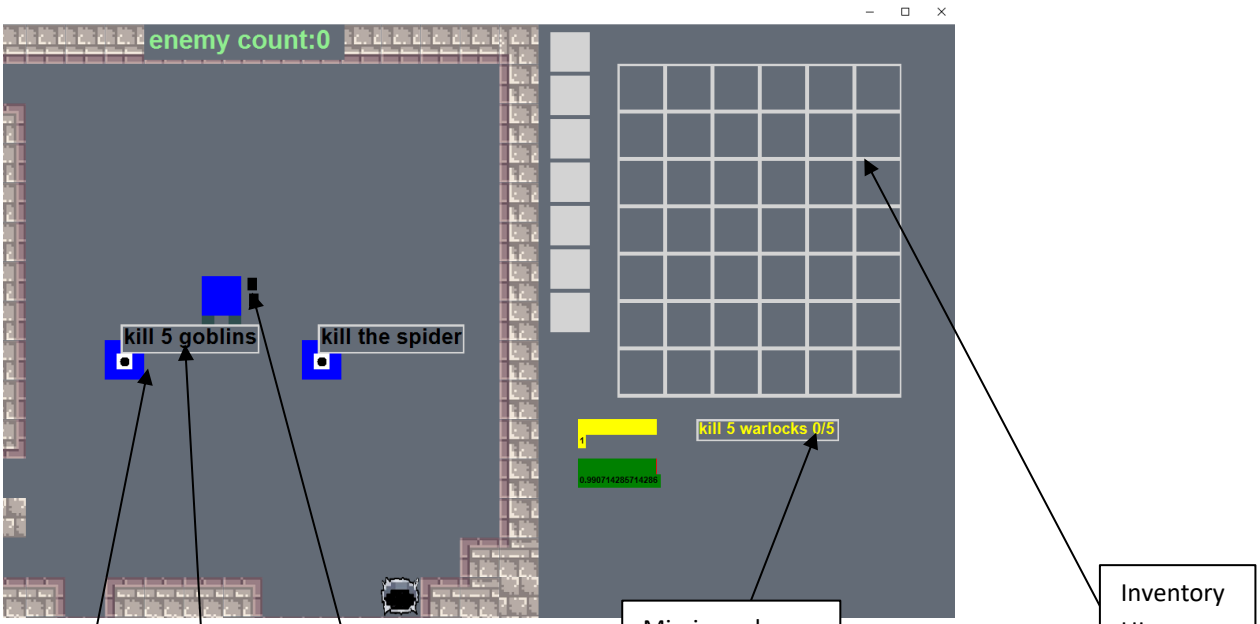


FIGURE 31 NPC ROOM, MISSION DIPLAYED FOR PLAYER

As you can see the mission chosen has a count, once this is satisfied it will turn green, then it can be clicked giving the player a reward and deleting itself from chosen missions, more than one mission can be chosen at a time.

HELPER CLASSES:

Map_maker:

Does not inherit from anything, its only link is through aggregation with the main form where it is instantiated and then used to create maps for different levels.

The map maker is used to randomly generate rooms, and make sure they are playable (the player can access the door) and fun (have a reasonable number of enemies). This is done through many strategies, the first one is the use of cellular automata to generate rooms, this algorithm is used in Conway's game of life and many other simulations, it involves simple rules applied to a grid of cell where each one is either alive or dead depending on rule e.g. (having 3 live neighbours or less than 4 dead neighbours) this grid is then iterated through a set amount of time (I used 4) to create a somewhat random order, before the grid is iterated through ,using a random number generator, live cells are added randomly therefore starting the algorithm differently each time. Set rules are done at the start and end to make it look more room-like, before it starts a border is added so the player and enemies cannot escape, after it is run the door is added and enemies are randomly added. As well as this, so there is enough space to move and attack comfortably I check the number of walls there are if this is less than the width*length/2 then the algorithm is run again.

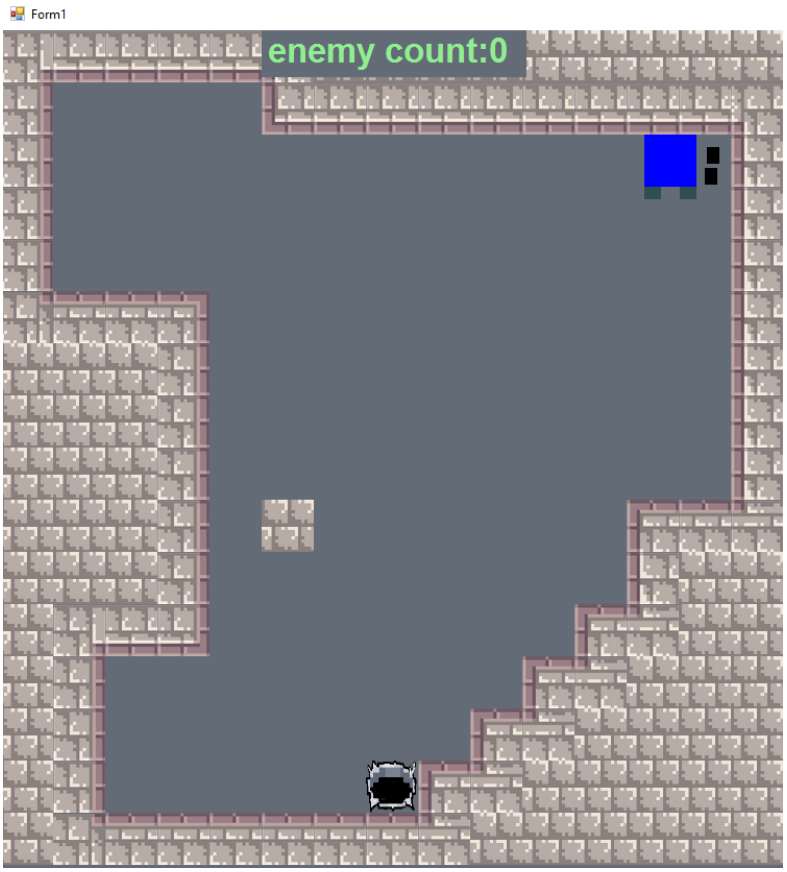


FIGURE 32 RANDOM MAP GENERATION WITHOUT ENEMIES

As can be seen the door is accessible by the player and at the end of the room forcing movement throughout the room, this is done through an A* pathfinding algorithm being used to check whether there is a navigable path available.

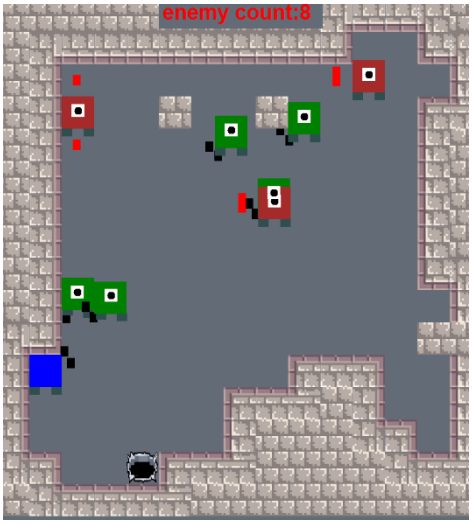


FIGURE 33 RANDOM MAP GENERATION WITH ENEMIES

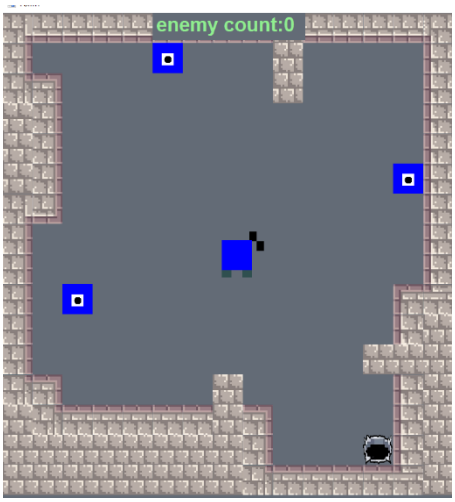


FIGURE 34 RANDOM MAP GENERATION WITH NPC'S

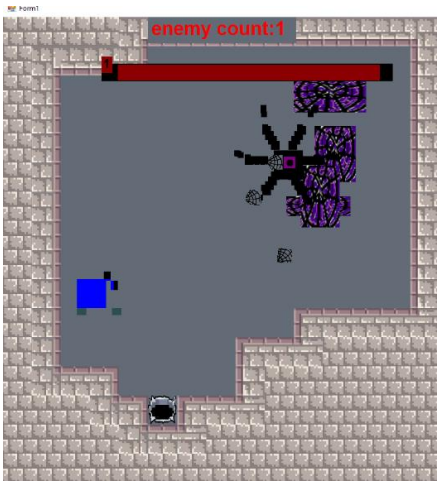


FIGURE 35 RANDOM MAP GENERATION WITH SPIDER BOSS

Portrayed in these images a variety of rooms can be created as the map_maker class has a variety of variable than can be changed to true to create this wide variety of rooms, this is done by changing the text map returned by the map_maker class to add different symbol for entities therefore only spawning in a different type of enemy (“e”=goblin, “w”=warlock, “N”=NPC, “E”=boss).

Ray cast:

The ray cast class does not inherit anything or have dependencies on any other code in the project.

The first task it is in charge of is creating the lines that block rays (Blockers for short) this is to increase the optimisation, instead of each ray having to check collisions with every wall line from one end of a run of walls to the other a created the 2 coordinates of these lines are then stored in an array which is stored in a list. Then using maths all the algorithm needs are these 2 coordinates to check if there will be a collision with it, hence meaning only one calculation must be done for multiple walls.

Once these blockers are created the algorithm for checking collision of rays and the blockers can be done. The sender and target of the ray must also be sent into the ray caster as parameters so only one ray must be sent to check if they can view

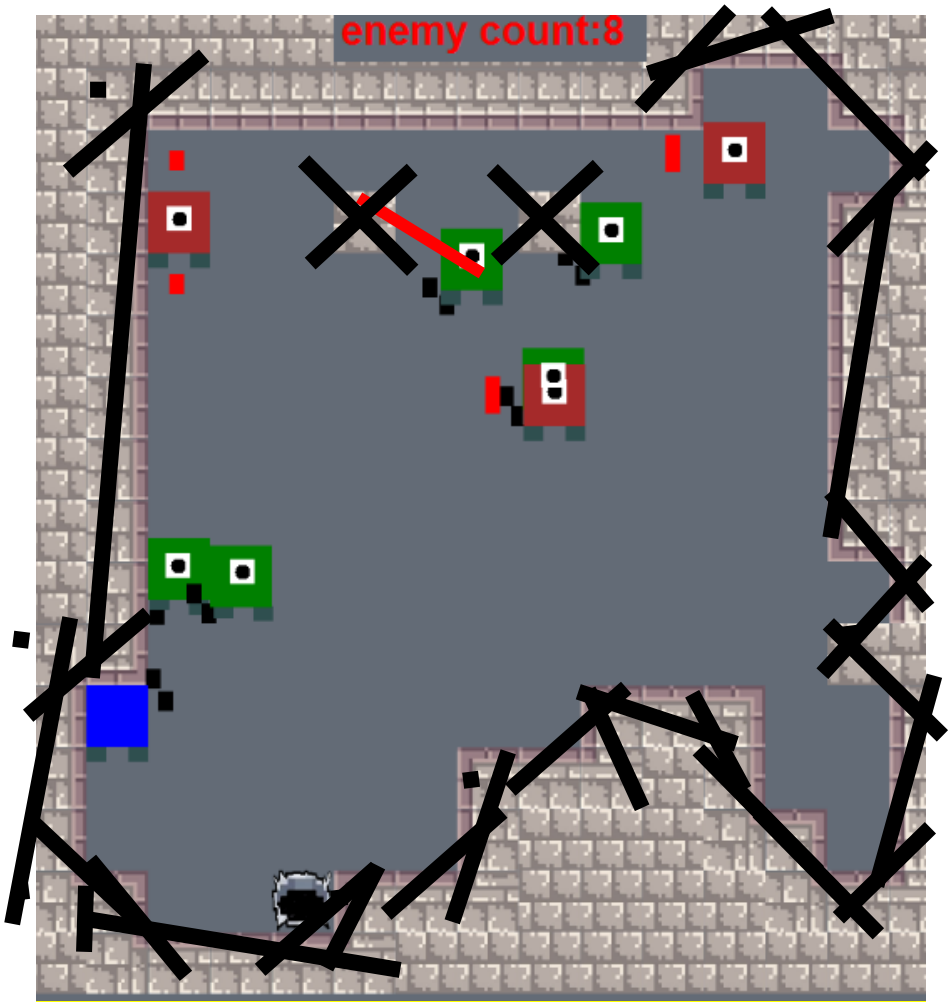


FIGURE 36 RAY CASTER AND BLOCKERS

Utilities:

This class also inherits from no other class and has no outside dependencies.

It has various helper jobs, file loading, file saving, random number generation and mouse tracking, these are all used in various other classes therefore making them smaller and reducing the amount of code that must be repeated.

The loading is always from a text file and is usually for attributes of items, attacks and missions. However, loading and saving current games is also done in the utilities class.

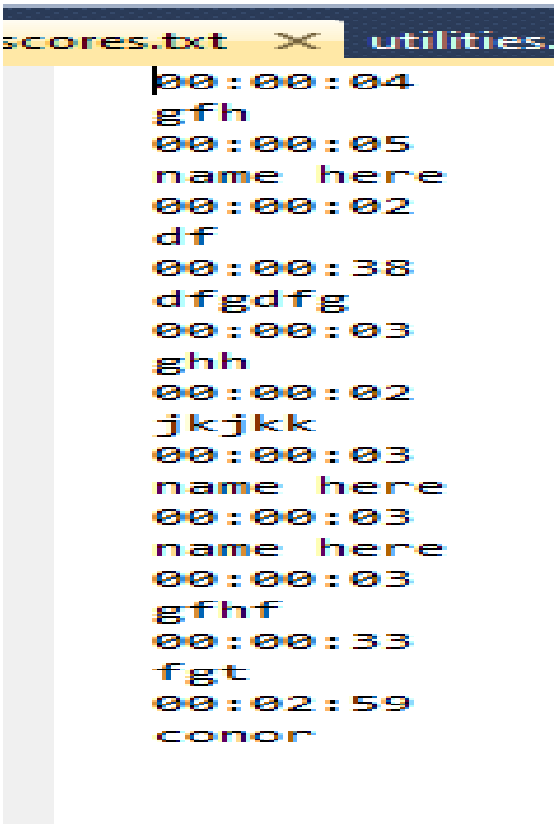


FIGURE 37 SCORES TEXT FILE

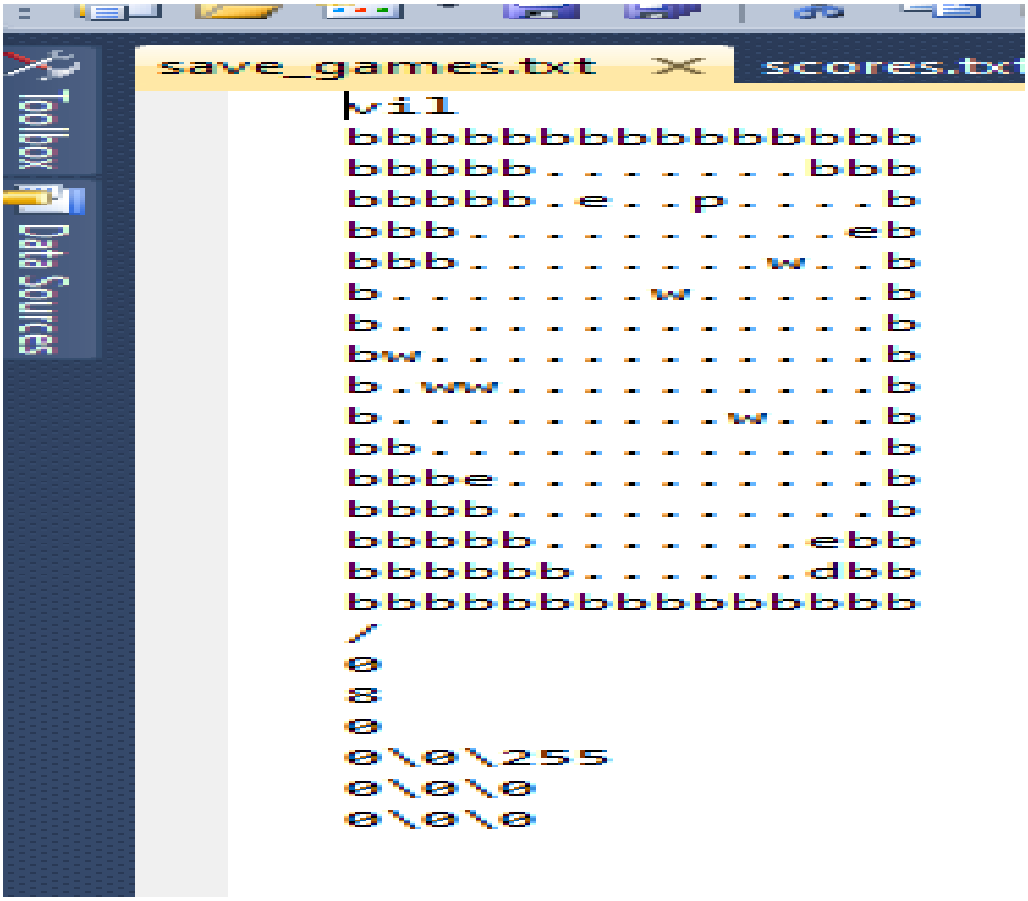


FIGURE 38 SAVED GAMES TEXT FILE

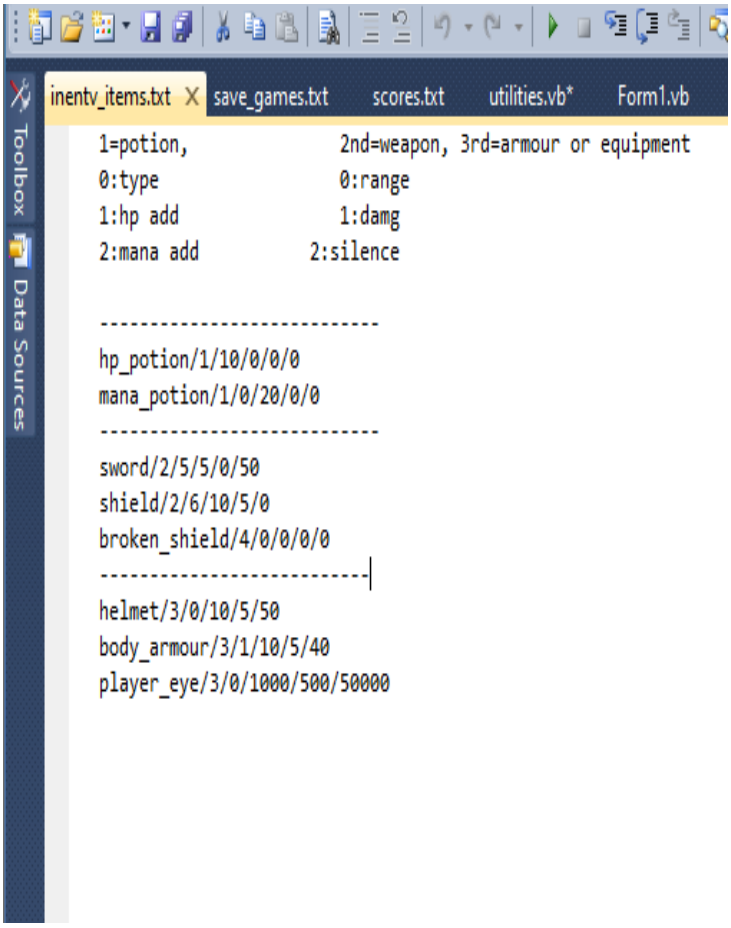


FIGURE 39INVENTORY ITEMS ATTRIBUTES TEXT FILE

MAIN CLASSES

Wall:



The wall object inherits from the base_base block object,

It can change the image portrayed depending on the neighbours it has, this is done in the change_img subroutine which uses the find_neighbours function and get_ending function both of which take the text map as a parameter, creating a more room looking level for the user.

FIGURE 40 THE WALLS HAVING DIFFERENT IMAGES

Door:

This class is very similar to base block inheriting from it with only one extra variable- “goes_to”.

Which is used to connect the next level, so when a door I collide with by the player it goes to a for loop which generates the next room, and changes it depending on what type it is.

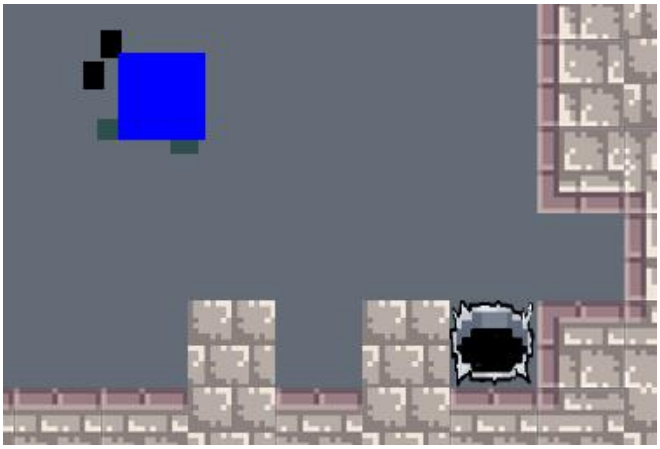


FIGURE 41 PLAYER ENTERING A DOOR TO THE NEXT ROOM



FIGURE 42PLAYERED HAS JUST ENTERED A DOOR AND BEEN SPAWNED IN THE NEXT ROOM

PLAYER CLASSES:

Player class:

This class inherits from Base_block and has limbs for more feedback and for looks.

It also handles all key-presses for movement and to open and close the inventory. It also handles clicks to use attacks and the shield.

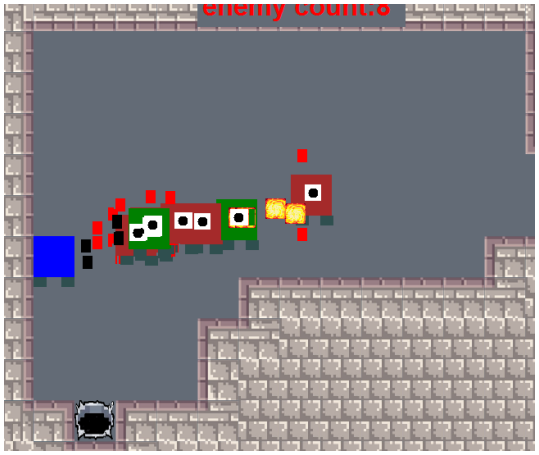


FIGURE 43 PLAYER LIMBS

The arms of the player points towards the mouse to make it seem as if they are spawning different attacks. Speaking of which there are 3: fireball, slash and lightning. Fireball and lightning are both magical therefore use mana, the fireball shoots an orb in the direction the mouse is pointed at, the lightning creates 6 cubes of lightning to be spawned by the player both of which do damage if they collide with an enemy. The slash is not magical so can be used indefinitely, although all attacks have a cooldown to reduce the player constantly using them making the game very easy to play. This cooldown is different for each attack and shown by a small bar under the attack image showing which attack is in use. To change attacks the scroll wheel must be scrolled to iterate between different attacks.

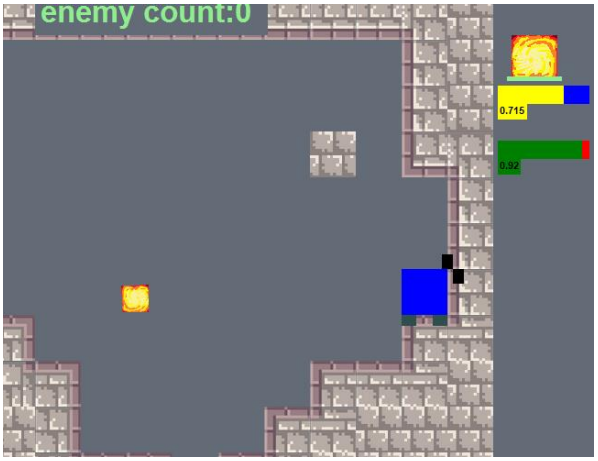


FIGURE 44 FIREBALL ATTACK

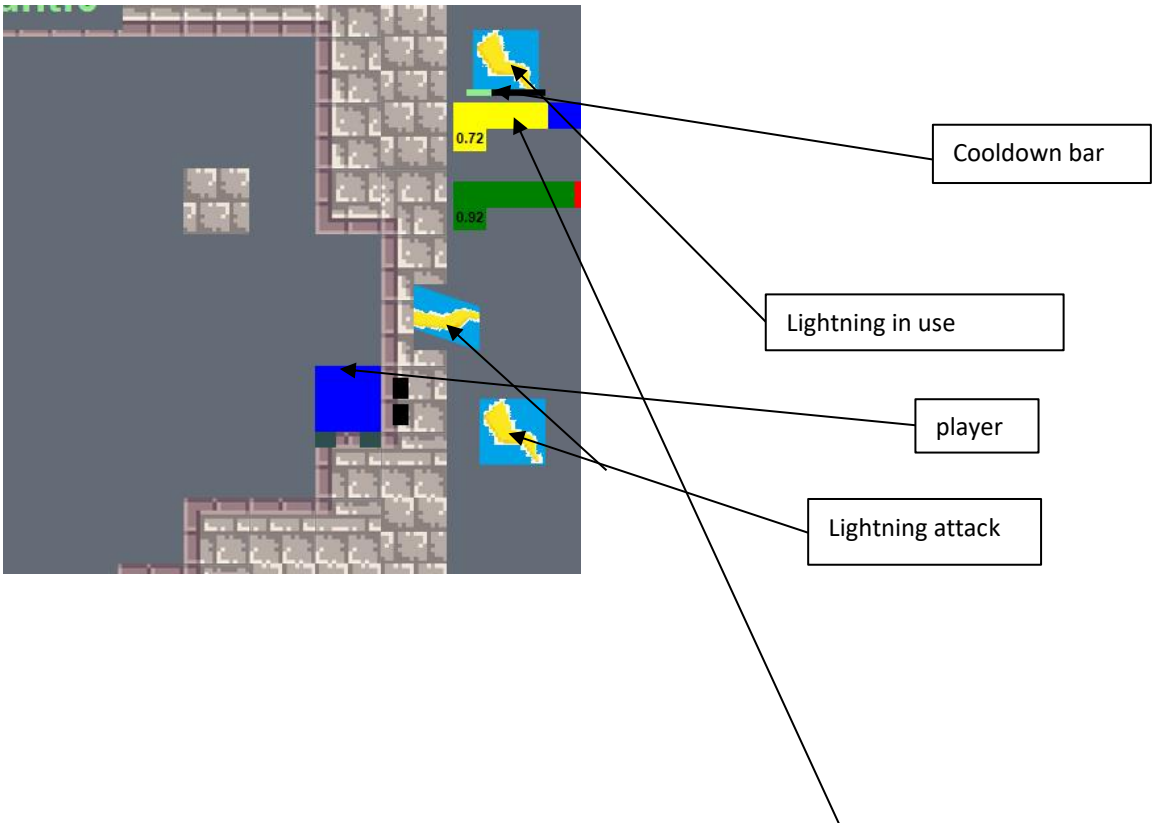


FIGURE 45 LIGHTNING ATTACK

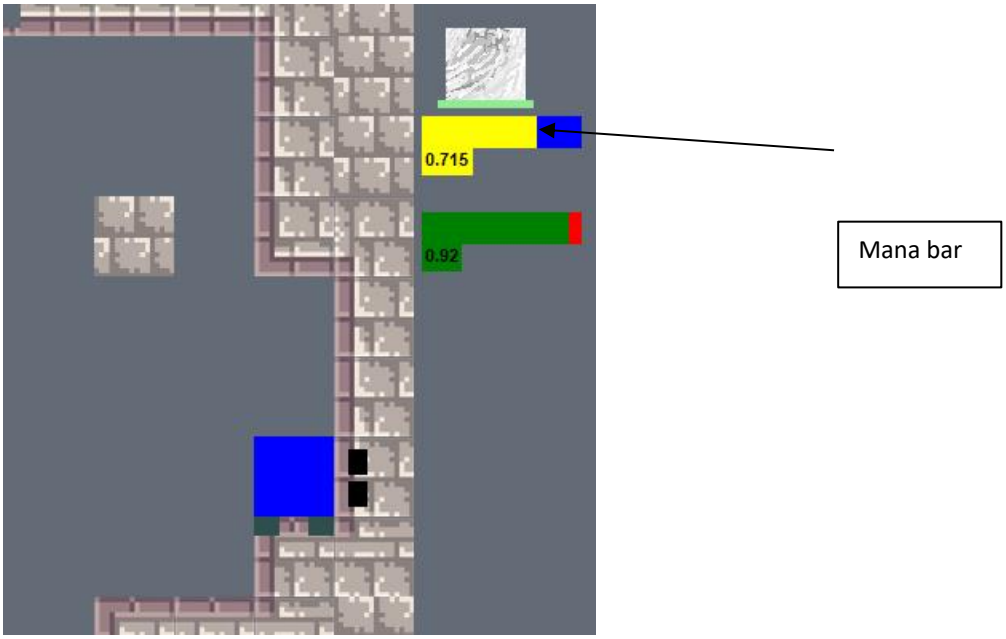


FIGURE 46 SLASH ATTACK

Different attacks have different times of life; therefore, they stay for more or less than others. The player class also contains the functionality for the shield and the inventory class, which also controls the UI like the bars or items equipped which are stored as an array in the player. Missions selected by the user are all also stored in a list in the player.

CLASS DEFINITION

Player = Class (base_block) {

Private:

Util:utilities

_max_hp:integer

_max_mana:integer

_boss_room:boolean

Public:

Property:hp

Property:max_hp

Property:max_mana

Subroutine:update

Subroutine:collisions

Function:player_box_collision

}

Player interface:

The player interface does not inherit from any other classes, however is tightly linked to the player class therefore has a composition type of relationship.

The inventory not only controls how items are stored and equipped but also the UI which includes the hp and mana bars, the chosen attack and the missions in use, as these are all parts only the player uses. Firstly , the inventory system, this stores items picked up or rewarded to the player, it has an inbuilt function to easily add new items by just giving their name (string) , when the inventory is first loaded it reads a text file that stores the stats of all possible item and stores them in a dictionary as a result, when a new item is added the inventory can easily use the dictionary to see all the stats of the item. If the item is already present in the inventory and is stackable (armour is NOT) then the item's amount increases by 1 shown in the bottom left of the item.

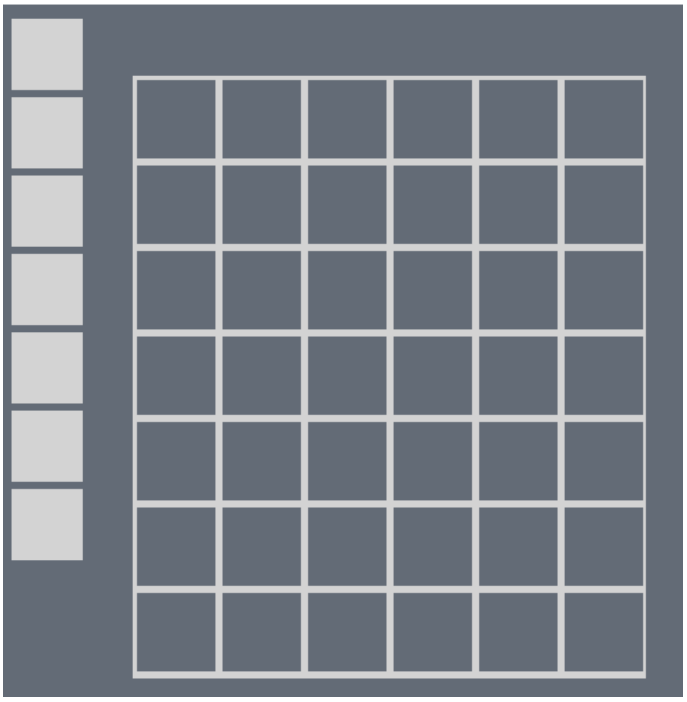


FIGURE 47 THIS IS THE INVENTORY GRID AND SHOWS ITEMS THAT ARE EQUIPPED

On the left you can see a column of grey boxes that are used for equipped items (armour/equipment) an item can be equipped by clicking on it.

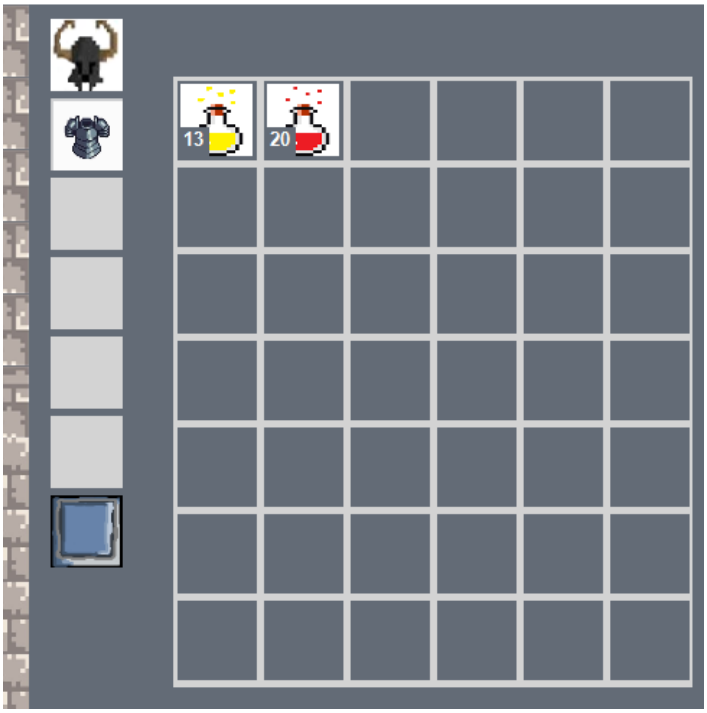


FIGURE 48 INVENTORY EITH ITEMS IN IT

To use items, you must click on the box they are in, to equip different items, press on the item you want to use and the item currently in use will be stored in the inventory.

Secondly, the UI system changes depending if the inventory is opened moving out the way, with the bars moving under the inventory and the chosen attack disappearing entirely

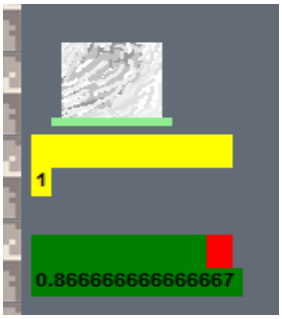


FIGURE 49 INVENTORY CLOSED UI

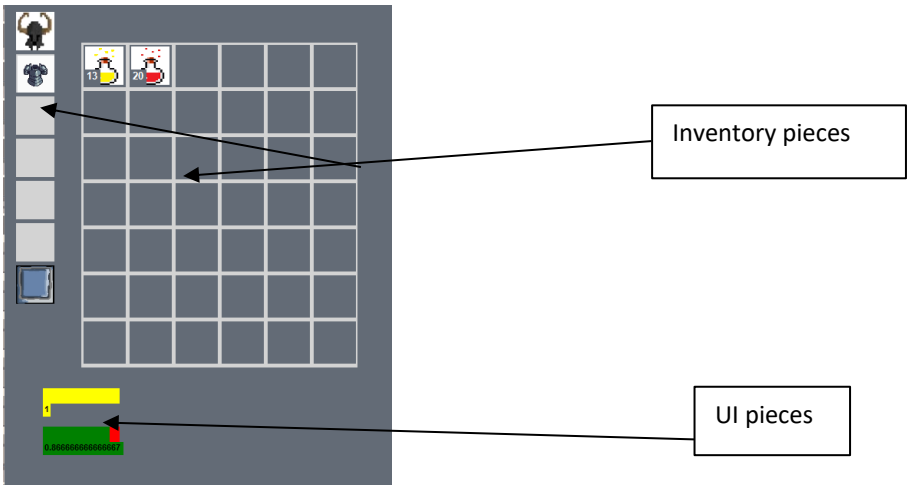


FIGURE 50 INVENTORY OPEN UI

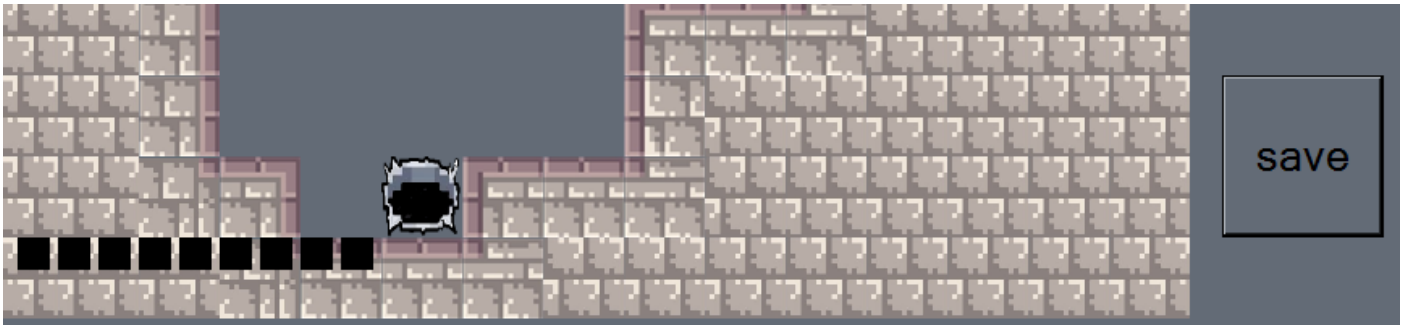


FIGURE 51 THE AMOUR COUNT AND SAVE BUTTON

As armour is equipped it is added to the total armour count in the player class which is then represented by these boxes where if the total armour count is above certain markers these boxes turn white.

The save button saves the current map and player attributes to a text file, this is done in the utilities class which is linked by an aggregation relationship to the player interface.

Item:

This class is used in the inventory and also in objects picked up off the ground, it acts as a button instead of using **button_clicked** it uses the variable named **touched** to check for clicks. To be picked up it checks for a collision with the player entity and if there is it adds itself to the players inventory.

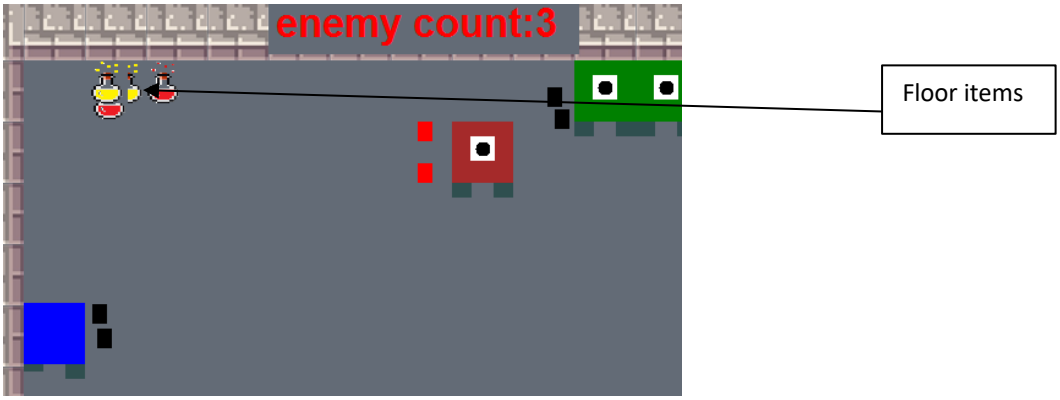


FIGURE 52 FLOOR ITEMS

The item class also uses text to show the amount of something there is, it therefore stores this amount in a variable.

Shield class:

This class inherits from the base block class, it does this to allow the use of collisions which is the main feature of the shield

This class allows the shield to check whether it is equipped and has been opened, however to increase the difficulty of the game the shield and the inventory cannot be open/used at the same time. When the shield is used a shield hp bar appears which uses composition to create a relationship between it and the shield.

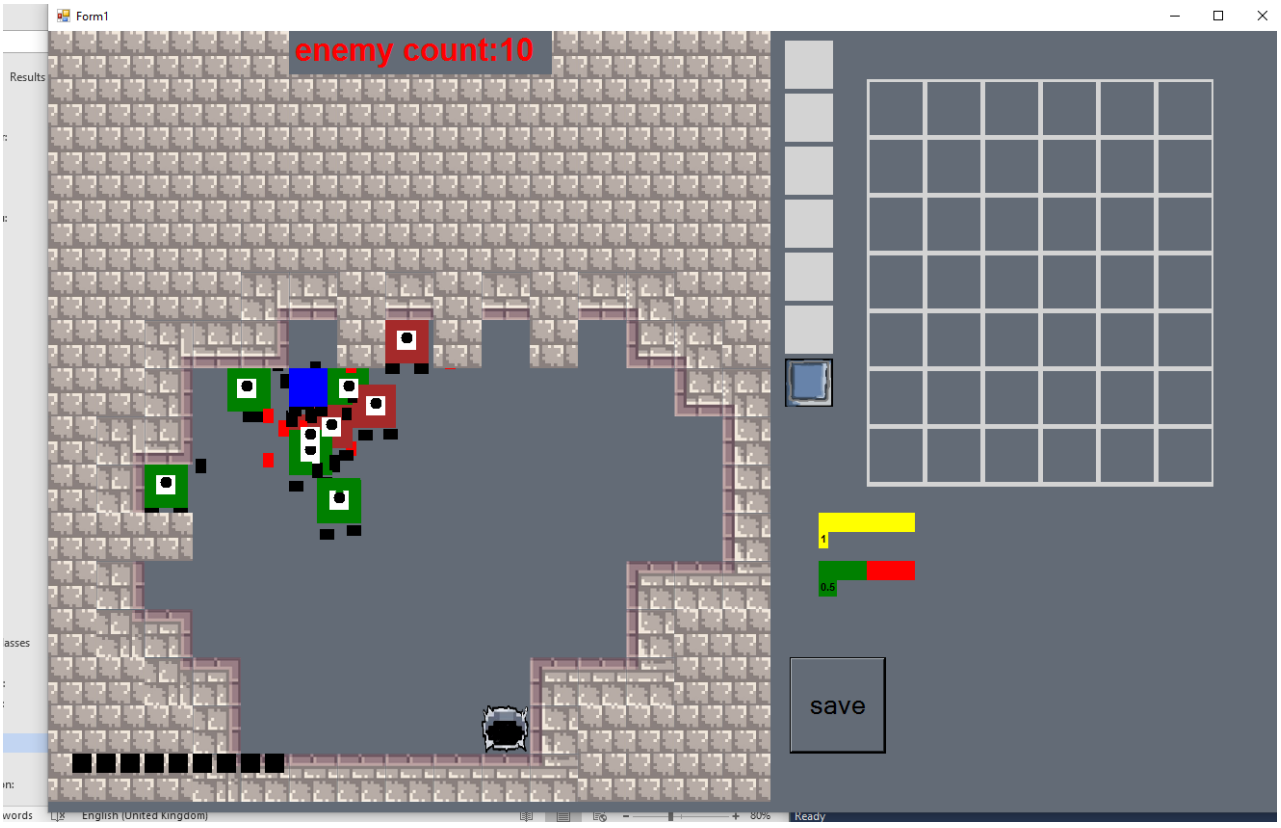


FIGURE 53 THE SHIELD HAS BEEN EQUIPPED

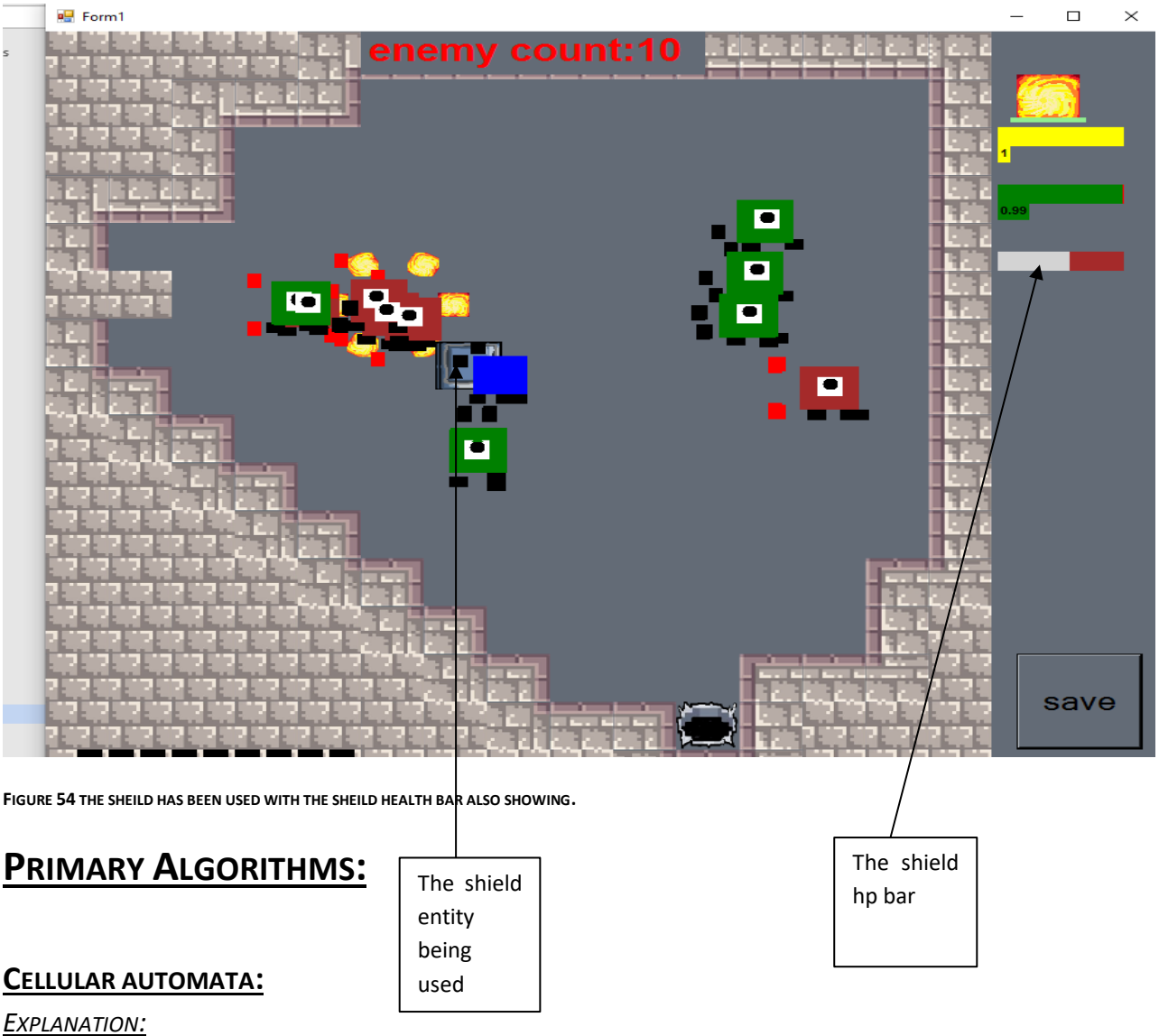


FIGURE 54 THE SHEILD HAS BEEN USED WITH THE SHEILD HEALTH BAR ALSO SHOWING.

PRIMARY ALGORITHMS:

CELLULAR AUTOMATA:

EXPLANATION:

A 2D list of strings is made where each string represents a block/entity. Strings are used instead of characters as once the character has been added it is read-only, so it cannot be changed. First, an empty map is created populated by "." — these represent empty space. Secondly, this grid is put through the generate_random_map() function that returns the same-size map, but each empty space has a random chance of being a wall. Then borders are added so the player cannot leave the designated area. These walls are represented by a "b".

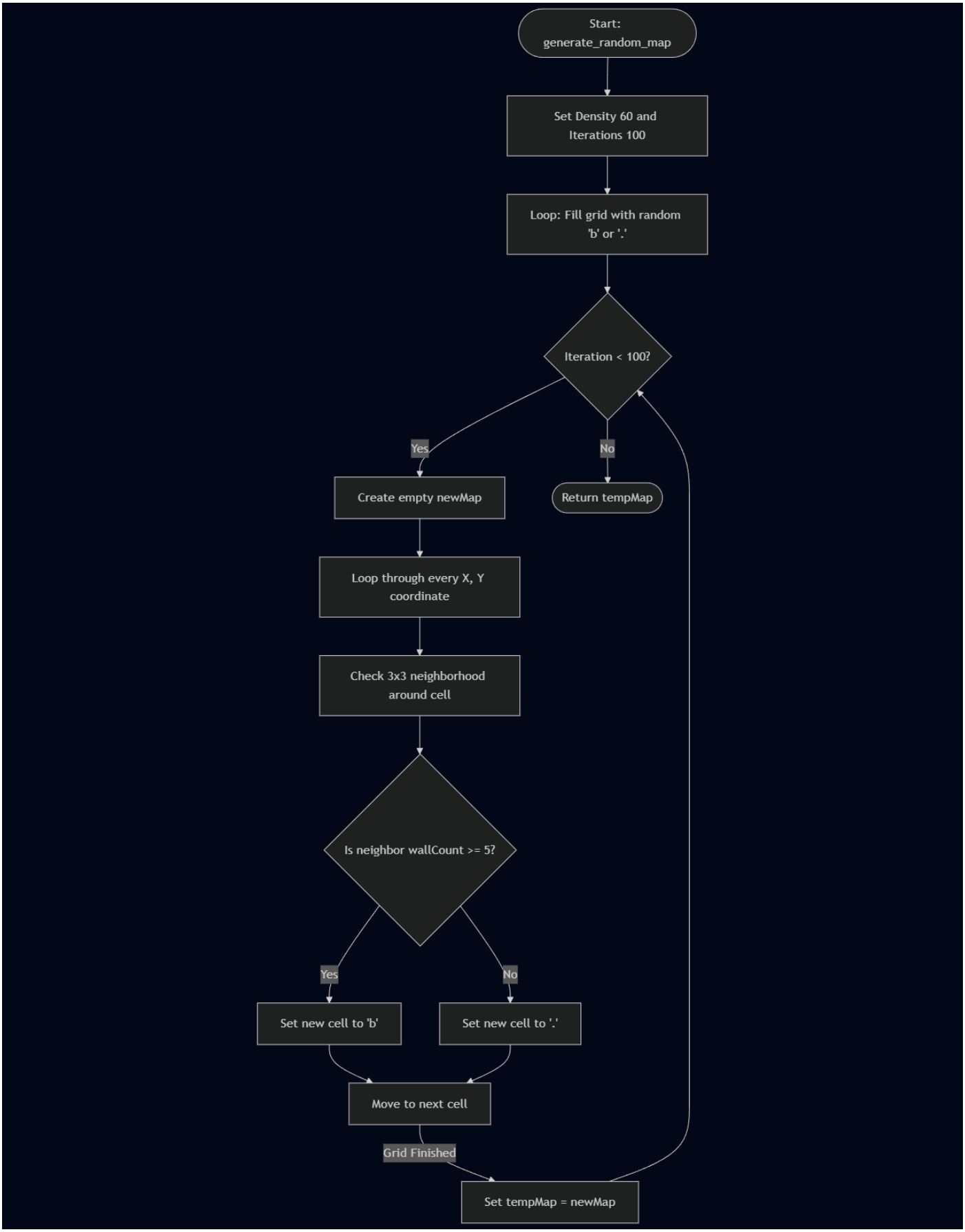
Once this is done, the cellular automata algorithm is used. This algorithm applies simple rules to each cell in the map, e.g. if more than 2 neighbours are walls, become a wall. These types of rules cause changes that are repeatedly applied until a suitable map is created.

Once this is done, different entities (like the player, enemies or door) are added to the text map, with the A* pathfinding being used to make sure a path is connected to the player character. If there is not, they may either be placed in a new position or, if there is no position left, the entire algorithm is run again.

Complexity:

O(I * W * H) where "I" is the number of iterations of the cellular automata algorithm, "W" is the width and "H" is the height of the map.

FLOWCHART:


Start: generate_random_map
Set Density 60 and Iterations 100
Loop: Fill grid with random 'b' or '.'
Iteration < 100?
Yes
No
Create empty newMap
Return tempMap
Loop through every X, Y coordinate
Check 3x3 neighborhood around cell
Is neighbor wallCount >= 5?
Yes
No
Set new cell to 'b'
Set new cell to '.'
Move to next cell
Grid Finished
Set tempMap = newMap

PSUEDOCODE:

```
FUNCTION generate_random_map()

  SET density = 60

  SET iterations = 100

  CREATE tempMap as empty 2D list

  ------------------------------------------------------------

  -- Step 1: Create initial random map

  ------------------------------------------------------------

  FOR y FROM 0 TO height

    CREATE row as empty list

    FOR x FROM 0 TO width

      IF random number between 1 and 100 < density THEN

        ADD "b" to row       // wall

      ELSE

        ADD "." to row       // empty space

      ENDIF

    END FOR

    ADD row to tempMap

  END FOR

  ------------------------------------------------------------

  -- Step 2: Run cellular automata smoothing iterations

  ------------------------------------------------------------

  FOR iteration FROM 1 TO iterations
```

```
CREATE newMap as empty 2D list

FOR y FROM 0 TO number of rows in tempMap - 1
  CREATE newRow as empty list

  FOR x FROM 0 TO number of columns in tempMap - 1

    SET wallCount = 0

    ------------------------------------------------------------
    -- Count neighboring walls (8 neighbors)
    ------------------------------------------------------------
    FOR dy FROM -1 TO 1
      FOR dx FROM -1 TO 1

        ny = y + dy
        nx = x + dx

        IF ny and nx are inside map bounds THEN
          IF (dy != 0 OR dx != 0) AND tempMap[ny][nx] == "b" THEN
            wallCount = wallCount + 1
          ENDIF
        ELSE
          -- Treat out-of-bounds as walls
          wallCount = wallCount + 1
        ENDIF

      END FOR
    END FOR
```

```
        ------------------------------------------------------

        -- Apply rule: cell becomes wall if enough neighbors

        ------------------------------------------------------

        IF wallCount >= 5 THEN

          ADD "b" to newRow

        ELSE

          ADD "." to newRow

        ENDIF


      END FOR


      ADD newRow to newMap

    END FOR


    tempMap = newMap


  END FOR


  RETURN tempMap


END FUNCTION
```

CODE:

```
    Public Function generate_random_map() As List(Of List(Of String))

        Dim density As Integer = 60
        Dim iterations As Integer = 100 'higher the number is the less unique rooms may be

        Dim tempMap As New List(Of List(Of String))

        ' randomly add walls to the empty grid----------------
        For y As Integer = 0 To height

            Dim row As New List(Of String)
            For x As Integer = 0 To width

                'add walls depending on density requested:
                If Me.util.Rand0(1, 100) < density Then
                    row.Add("b")
                Else
                    row.Add(".")
```

```
                End If

            Next

            tempMap.Add(row)

        Next
        '-------------------------------------------------

        'amount of iterations:
        For iteration As Integer = 0 To iterations - 1

            Dim newMap As New List(Of List(Of String))

            ' use cellular automata to make the grid look more room-like----------------------------
------------------
            For y As Integer = 0 To tempMap.Count - 1

                Dim newRow As New List(Of String)
                For x As Integer = 0 To tempMap(0).Count - 1

                    Dim wallCount As Integer = 0

                    'count how many neibouring cells are walls-------------------------------------
------------------
                    For dy As Integer = -1 To 1
                        For dx As Integer = -1 To 1

                            Dim ny As Integer = y + dy
                            Dim nx As Integer = x + dx

                            If ny >= 0 AndAlso ny < tempMap.Count AndAlso nx >= 0 AndAlso nx <
tempMap(0).Count Then
                                If (dy <> 0 OrElse dx <> 0) And tempMap(ny)(nx) = "b" Then
                                    wallCount += 1
                                End If
                            Else
                                wallCount += 1
                            End If

                        Next
                    Next
                    '-----------------------------------------------------------------------------
------------------

                    'add rules so only cells with enough neighbours live--------
                    If wallCount >= 5 Then
                        newRow.Add("b")
                    Else
                        newRow.Add(".")
                    End If
                    '-----------------------------------------------------------
                Next

                newMap.Add(newRow)

            Next

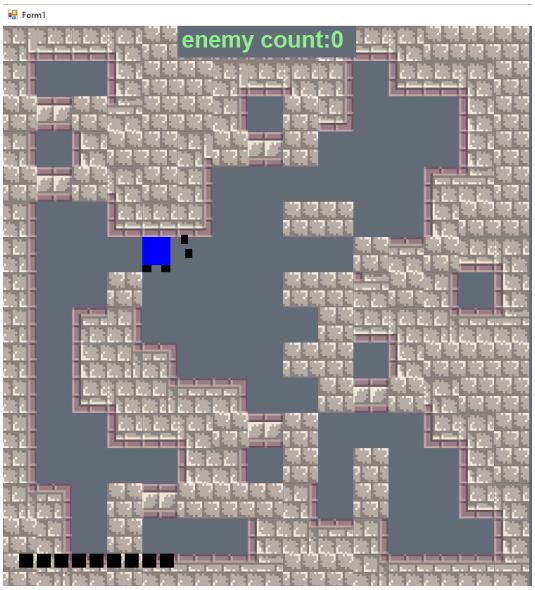
            tempMap = newMap

        Next
        '-------------------------------------------------------------------------------------------
---------------

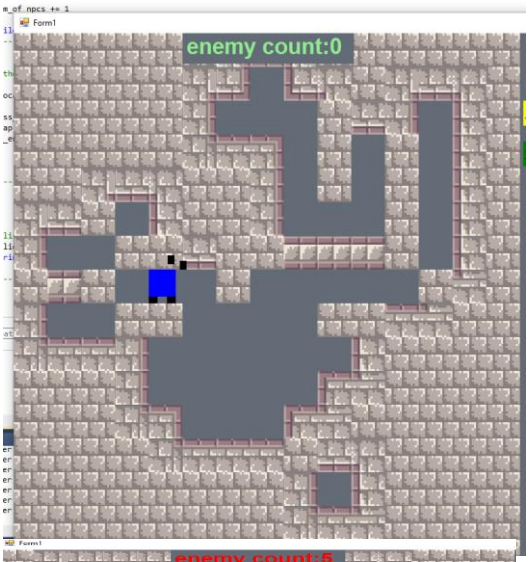
        Return tempMap
```

```
    End Function
```

IMAGES:



0 Iterations, just the randomised map



2 iterations, a room is now forming



1000 iterations, a large room has been created and can be filled with entities

A* PATHFINDING:

EXPLANATION:

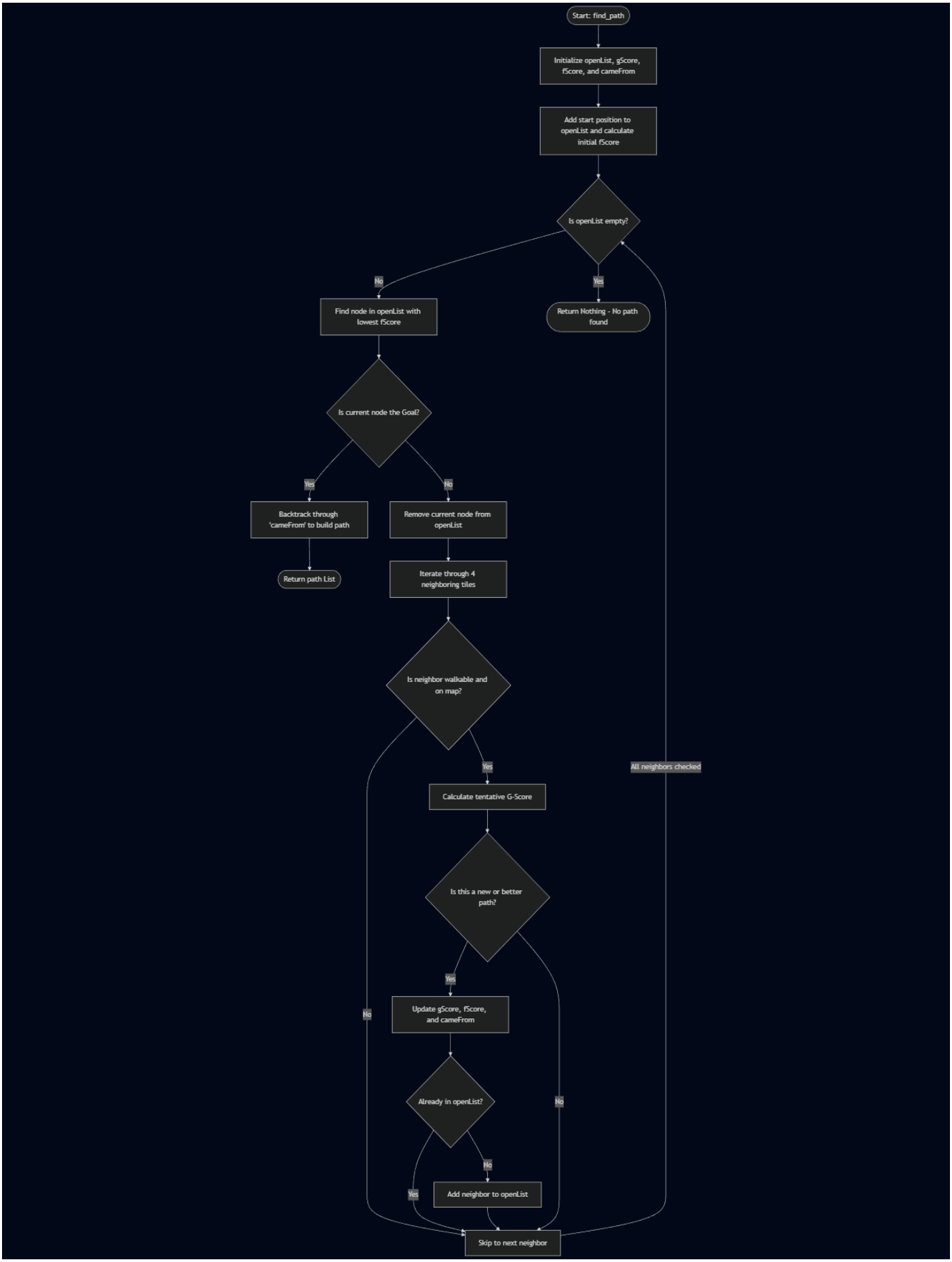
The A* pathfinding algorithm is a heuristic approach to finding the shortest path in a graph. To set up the graph, I used the already-made text map. Then, as the algorithm goes through the text map, different cells are added into the cameFrom dictionary and given an f-score and g-score by also adding them to the gScore and fScore dictionaries.

These scores are added up in the program, creating an h-score which is used to rank cells. Then, at the end of the algorithm, these scores can be ordered and turned into a path.

Complexity:

O(E log(V)) where “E” is the edges and “V” is the vertices/grid squares.

FLOWCHART:


Start: find_path
Initialize openList, gScore, fScore, and cameFrom
Add start position to openList and calculate initial fScore
Is openList empty?
No
Yes
Find node in openList with lowest fScore
Return Nothing - No path found
Is current node the Goal?
Yes
No
Backtrack through 'cameFrom' to build path
Remove current node from openList
Return path List
Iterate through 4 neighboring tiles
Is neighbor walkable and on map?
Yes
All neighbors checked
Calculate tentative G-Score
Is this a new or better path?
Yes
No
Update gScore, fScore, and cameFrom
Already in openList?
No
Yes
No
Add neighbor to openList
Skip to next neighbor

PSUEDOCODE:

```
FUNCTION find_path(startX, startY, endX, endY, map)

  CREATE openList as empty list
  CREATE cameFrom as empty dictionary
  CREATE gScore as empty dictionary
  CREATE fScore as empty dictionary
  CREATE path as empty list

  startKey = "startX,startY"

  ADD (startX, startY) to openList
  gScore[startKey] = 0
  fScore[startKey] = |startX - endX| + |startY - endY|   // Manhattan distance

  WHILE openList is not empty

    ------------------------------------------------------------
    -- Step 1: Pick node in openList with lowest fScore
    ------------------------------------------------------------
    current = openList[0]
    currentKey = "current.x,current.y"

    FOR each node in openList starting from index 1
      nodeKey = "node.x,node.y"

      IF fScore[nodeKey] exists AND fScore[nodeKey] < fScore[currentKey]
        current = node
        currentKey = nodeKey
      ENDIF
```

```
END FOR

------------------------------------------------------------

-- Step 2: If goal reached, reconstruct path

------------------------------------------------------------

IF current.x == endX AND current.y == endY

   ADD current to path


   WHILE cameFrom contains currentKey

      current = cameFrom[currentKey]

      INSERT current at beginning of path

      currentKey = "current.x,current.y"

   END WHILE


   RETURN path

ENDIF


REMOVE current from openList


------------------------------------------------------------

-- Step 3: Check all 4 neighbors (up, down, left, right)

------------------------------------------------------------

OFFSETS = [(1,0), (-1,0), (0,1), (0,-1)]


FOR each offset in OFFSETS


   nx = current.x + offset.x

   ny = current.y + offset.y


   -- Skip if out of bounds
```

```
    IF nx < 0 OR nx >= map.width OR ny < 0 OR ny >= map.height
        CONTINUE
    ENDIF

    -- Skip if wall
    IF map[ny][nx] == "b"
        CONTINUE
    ENDIF

    nKey = "nx,ny"
    tentativeG = gScore[currentKey] + 1

    ------------------------------------------------------------
    -- Step 4: Update scores if this path is better
    ------------------------------------------------------------
    IF nKey not in gScore OR tentativeG < gScore[nKey]

        cameFrom[nKey] = current
        gScore[nKey] = tentativeG
        fScore[nKey] = tentativeG + |nx - endX| + |ny - endY|

        -- Add to openList if not already present
        IF (nx, ny) not in openList
            ADD (nx, ny) to openList
        ENDIF

    ENDIF

END FOR
```

END WHILE

RETURN null // No path found

END FUNCTION

CODE:

```
Public Function find_path(ByVal startx As Integer, ByVal starty As Integer, ByVal endx As Integer,
ByVal endy As Integer, ByVal map As List(Of List(Of String)))
        Dim openList As New List(Of Integer())

        Dim cameFrom As New Dictionary(Of String, Integer())
        'arrays to check whether cells have scores and if they are higher than the new scores
        Dim gScore As New Dictionary(Of String, Integer)
        Dim fScore As New Dictionary(Of String, Integer)
        '-------------------------------------------------------------------------------
        Dim path As New List(Of Integer())
        Dim startKey As String = startx.ToString() & "," & starty.ToString()

        openList.Add(New Integer() {startx, starty})
        gScore(startKey) = 0
        fScore(startKey) = Math.Abs(startx - endx) + Math.Abs(starty - endy)

        While openList.Count > 0

            Dim current As Integer() = openList(0)
            Dim currentKey As String = current(0).ToString() & "," & current(1).ToString()

            'scores all cells that have not got a score but are being checked-------------------
            For num As Integer = 1 To openList.Count - 1
                Dim node As Integer() = openList(num)
                Dim nodeKey As String = node(0).ToString() & "," & node(1).ToString()

                If fScore.ContainsKey(nodeKey) AndAlso fScore(nodeKey) < fScore(currentKey) Then
                    current = node
                    currentKey = nodeKey
                End If

            Next
            '-------------------------------------------------------------------------------

            'checks if the end node is in the the list of checked cells, so it ends the program and
finds the best path depending on scores
            If current(0) = endx AndAlso current(1) = endy Then
                path.Add(current)

                While cameFrom.ContainsKey(currentKey)
                    current = cameFrom(currentKey)
                    path.Insert(0, current)
                    currentKey = current(0).ToString() & "," & current(1).ToString()
                End While

                Return path
            End If
            '-------------------------------------------------------------------------------
---------------------------------------
            openList.Remove(current)
            'list of all neighbouring cels to one being checked:
```

```
            Dim offsets As Integer()() = {New Integer() {1, 0}, New Integer() {-1, 0}, New Integer()
{0, 1}, New Integer() {0, -1}}
            'iterates between all neighbours changing their scores----------------------------
            For j As Integer = 0 To offsets.Length - 1

                Dim nx As Integer = current(0) + offsets(j)(0)
                Dim ny As Integer = current(1) + offsets(j)(1)

                'checks the cell being checked is empty(not a wall) and in the array:
                If nx < 0 Or nx >= map(0).Count Or ny < 0 Or ny >= map.Count Then Continue For
                If map(ny)(nx) = "b" Then Continue For

                Dim nKey As String = nx.ToString() & "," & ny.ToString()
                Dim tentativeG As Integer = gScore(currentKey) + 1

                'changes the score of cells already been checked, or adds a score to cells that have
not been checked---
                If Not gScore.ContainsKey(nKey) OrElse tentativeG < gScore(nKey) Then
                    cameFrom(nKey) = current
                    gScore(nKey) = tentativeG
                    fScore(nKey) = tentativeG + Math.Abs(nx - endx) + Math.Abs(ny - endy)

                    Dim found As Boolean = False

                    For k As Integer = 0 To openList.Count - 1
                        If openList(k)(0) = nx AndAlso openList(k)(1) = ny Then
                            found = True
                            Exit For
                        End If
                    Next

                    If Not found Then
                        openList.Add(New Integer() {nx, ny})
                    End If

                End If
                '-------------------------------------------------------------------------------
-------------------
            Next
            '-------------------------------------------------------------------------------
        End While

        Return Nothing
    End Function
End Class
```

BLOCKER MAKER IN RAY CASTING:

EXPLANATION:

First, I create a nodemap that yet again uses the text map. Nodes have neighbours surrounding them and are only connected to neighbours if they are a wall (“b”).

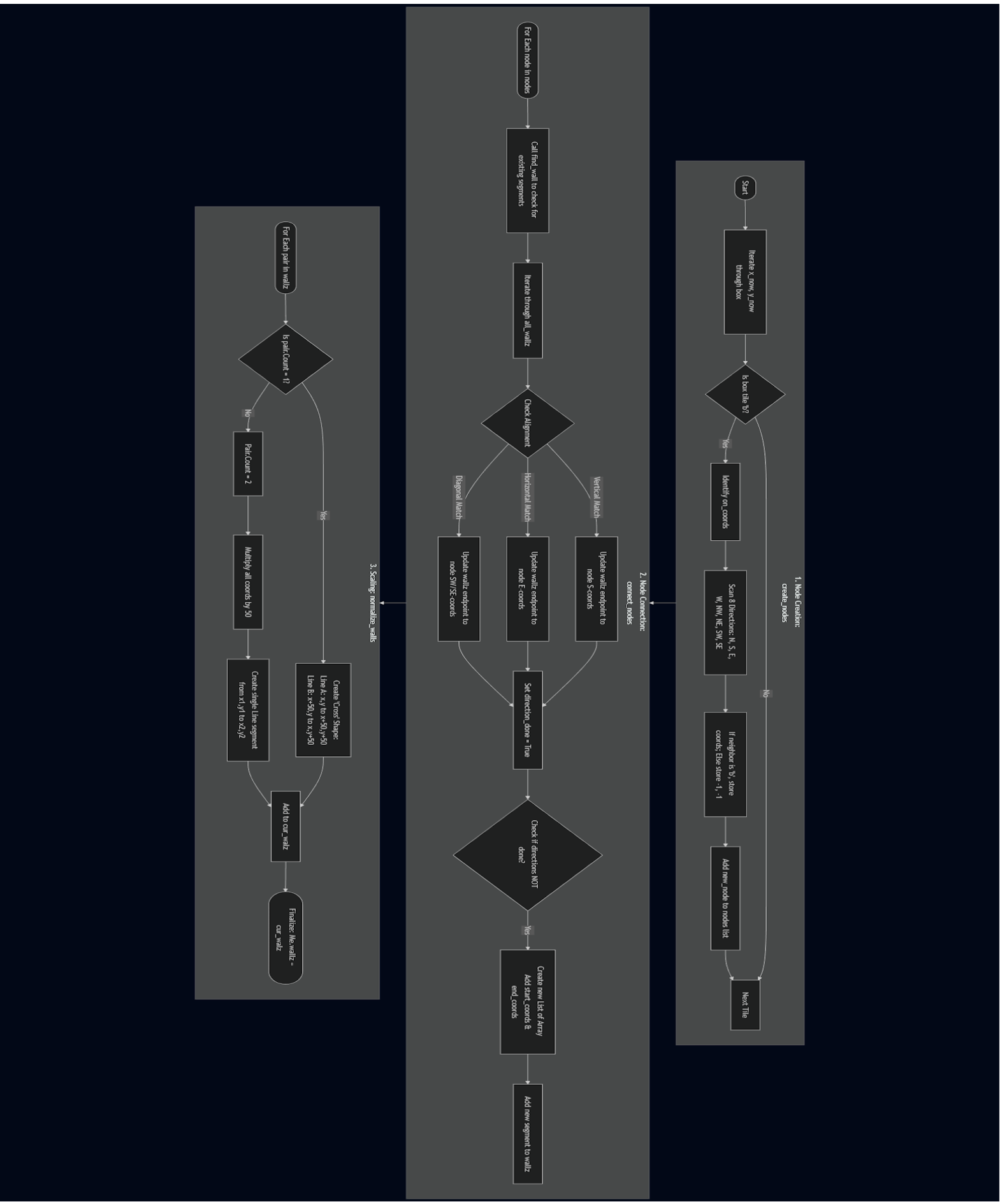
Once the nodemap has been created, I connect the nodes to create the lines (blockers). This is done by going along the walls and checking if there is a wall in each direction.

I then normalise these walls by turning the pair of coordinates into a line by pairing them up. If a coordinate has no partner, it will just be represented as two lines that create a cross. When normalising, all coordinates are multiplied by 50 as that is the true size of the map.

Complexity:

O(N * W) where “N” is the number of walls that connect and “W” is the number of wall segments.

FLOWCHART:


1. Node Creation: create_nodes
Start
Iterate x_now, y_now through box
Is box tile 'b'?
Yes
No
Identify on_coords
Scan 8 Directions: N, S, E, W, NW, NE, SW, SE
If neighbor is 'b', store coords; Else store -1, -1
Add new_node to nodes list
Next Tile
2. Node Connection: connect_nodes
For Each node in nodes
Call find_wall to check for existing segments
Iterate through all_wallz
Check Alignment
Vertical Match
Horizontal Match
Diagonal Match
Update wallz endpoint to node S coords
Update wallz endpoint to node E coords
Update wallz endpoint to node SW/SE coords
Set direction_done = True
Check if directions NOT done?
Yes
Create new List of Array
Add start_coords & end_coords
Add new segment to wallz
3. Scaling: normalize_walls
For Each pair in wallz
Is pair.Count = 1?
No
Yes
PairCount = 2
Multiply all coords by 50
Create single Line segment from x1,y1 to x2,y2
Create 'Cross' Shape:
Line A: x,y to x+50, y+50
Line B: x+50,y to x,y+50
Add to cur_wallz
Finalize: Me.wallz = cur_wallz

PSEUDOCODE:

```
FUNCTION create_nodes()


  FOR y FROM 1 TO box.height - 1

    FOR x FROM 0 TO box[y].length - 1


      IF box[y][x] == "b" THEN


        node = empty list


        on  = (x, y)


        north = IF box[y-1][x] == "b" THEN (x, y-1) ELSE (-1, -1)

        south = IF box[y+1][x] == "b" THEN (x, y+1) ELSE (-1, -1)

        west  = IF box[y][x-1] == "b" THEN (x-1, y) ELSE (-1, -1)

        east  = IF box[y][x+1] == "b" THEN (x+1, y) ELSE (-1, -1)


        nw = IF box[y-1][x-1] == "b" THEN (x-1, y-1) ELSE (-1, -1)

        ne = IF box[y-1][x+1] == "b" THEN (x+1, y-1) ELSE (-1, -1)

        sw = IF box[y+1][x-1] == "b" THEN (x-1, y+1) ELSE (-1, -1)

        se = IF box[y+1][x+1] == "b" THEN (x+1, y+1) ELSE (-1, -1)


        ADD on, north, south, east, west, nw, ne, sw, se TO node

        ADD node TO nodes


      ENDIF


    END FOR

  END FOR
```

```
END FUNCTION



FUNCTION find_wall(nodeCoords, wallList)

    result = empty list

    FOR i FROM 0 TO wallList.count - 1
        IF wallList[i].count == 2 AND wallList[i][1] == nodeCoords THEN
            ADD i TO result
        ENDIF
    END FOR

    RETURN result

END FUNCTION



FUNCTION empty(coords)
    RETURN coords == (-1, -1)
END FUNCTION



PROCEDURE connect_nodes()

    FOR EACH node IN nodes

        my = node[0]
        connected = find_wall(my, wallz)
```

```
southDone = FALSE
eastDone  = FALSE
swDone    = FALSE
seDone    = FALSE

FOR EACH index IN connected

  start = wallz[index][0]

  IF start.x == my.x AND start.y < my.y THEN
    newEnd = IF NOT empty(node[2]) THEN node[2] ELSE my
    wallz[index][1] = newEnd
    southDone = TRUE
  ENDIF

  IF start.y == my.y AND start.x < my.x THEN
    newEnd = IF NOT empty(node[3]) THEN node[3] ELSE my
    wallz[index][1] = newEnd
    eastDone = TRUE
  ENDIF

  IF start.x > my.x AND start.y < my.y THEN
    newEnd = IF NOT empty(node[7]) THEN node[7] ELSE my
    wallz[index][1] = newEnd
    swDone = TRUE
  ENDIF

  IF start.x < my.x AND start.y < my.y THEN
    newEnd = IF NOT empty(node[8]) THEN node[8] ELSE my
    wallz[index][1] = newEnd
```

```
            seDone = TRUE
        ENDIF

    END FOR

    IF connected.count == 0 THEN
        ADD [my] TO wallz
    ENDIF

    IF NOT southDone AND NOT empty(node[2]) THEN
        ADD [my, node[2]] TO wallz
    ENDIF

    IF NOT eastDone AND NOT empty(node[3]) THEN
        ADD [my, node[3]] TO wallz
    ENDIF

    IF NOT swDone AND NOT empty(node[7]) THEN
        ADD [my, node[7]] TO wallz
    ENDIF

    IF NOT seDone AND NOT empty(node[8]) THEN
        ADD [my, node[8]] TO wallz
    ENDIF

  END FOR

END PROCEDURE
```

```
PROCEDURE normalize_walls()

  newList = empty list

  FOR EACH wall IN wallz

    IF wall.count == 1 THEN

      base = wall[0] * 50

      ADD [(base.x, base.y), (base.x+50, base.y+50)] TO newList
      ADD [(base.x+50, base.y), (base.x, base.y+50)] TO newList

    ELSE IF wall.count == 2 THEN

      start = wall[0] * 50
      end   = wall[1] * 50

      ADD [start, end] TO newList

    ENDIF

  END FOR

  wallz = newList

END PROCEDURE
```

CODE:

```
Private nodes As New List(Of List(Of Array))
    Private wallz As New List(Of List(Of Array))

    Private box As List(Of String)
```

```
Private Sub connect_nodes()
    'connect nodes that are next to each other
    For Each node In nodes


        Dim my_coords As Array = node(0)

        Dim all_wallz As List(Of Integer) = find_wall(my_coords, wallz)

        'node: mycoords, north, south, east, west, nw, ne, sw, se
        Dim south_done As Boolean = False
        Dim east_done As Boolean = False
        Dim sw_done As Boolean = False
        Dim se_done As Boolean = False


        For Each wl In all_wallz

            'vertical
            If wallz(wl)(0)(1) < my_coords(1) And wallz(wl)(0)(0) = my_coords(0) Then

                Dim new_endpoint As Array

                If node(2)(0) <> -1 And node(2)(1) <> -1 Then
                    new_endpoint = node(2)
                Else
                    new_endpoint = my_coords
                End If

                wallz(wl)(1) = new_endpoint
                south_done = True
            End If

            'horizontal
            If wallz(wl)(0)(0) < my_coords(0) And wallz(wl)(0)(1) = my_coords(1) Then

                Dim new_endpoint As Array

                If node(3)(0) <> -1 And node(3)(1) <> -1 Then
                    new_endpoint = node(3)
                Else
                    new_endpoint = my_coords
                End If

                wallz(wl)(1) = new_endpoint
                east_done = True
            End If

            'diagnoll left, sw
            If wallz(wl)(0)(0) > my_coords(0) And wallz(wl)(0)(1) < my_coords(1) Then

                Dim new_endpoint As Array

                If node(7)(0) <> -1 And node(7)(1) <> -1 Then
                    new_endpoint = node(7)
                Else
                    new_endpoint = my_coords
                End If

                wallz(wl)(1) = new_endpoint
                sw_done = True
            End If

            'diagnoll right, se
```

```
    If wallz(wl)(0)(0) < my_coords(0) And wallz(wl)(0)(1) < my_coords(1) Then

        Dim new_endpoint As Array

        If node(8)(0) <> -1 And node(8)(1) <> -1 Then
            new_endpoint = node(8)
        Else
            new_endpoint = my_coords
        End If

        wallz(wl)(1) = new_endpoint
        se_done = True
    End If
Next
'node: mycoords, north, south, east, west, nw, ne, sw, se
'wallz havent been done

If all_wallz.Count < 1 Then
    Dim start_coords As Array = my_coords
    Dim collumn As New List(Of Array)

    collumn.Add(start_coords)

    wallz.Add(collumn)
End If

If Not south_done And Not empty(node(2)) Then
    Dim start_coords As Array = my_coords
    Dim end_coords As Array = node(2)

    Dim collumn As New List(Of Array)

    collumn.Add(start_coords)
    collumn.Add(end_coords)

    wallz.Add(collumn)
End If

If Not east_done And Not empty(node(3)) Then
    Dim start_coords As Array = my_coords
    Dim end_coords As Array = node(3)

    Dim collumn As New List(Of Array)

    collumn.Add(start_coords)
    collumn.Add(end_coords)

    wallz.Add(collumn)
End If

If Not sw_done And Not empty(node(7)) Then
    Dim start_coords As Array = my_coords
    Dim end_coords As Array = node(7)

    Dim collumn As New List(Of Array)

    collumn.Add(start_coords)
    collumn.Add(end_coords)

    wallz.Add(collumn)
End If

If Not se_done And Not empty(node(8)) Then
    Dim start_coords As Array = my_coords
    Dim end_coords As Array = node(8)
```

```
            Dim collumn As New List(Of Array)

            collumn.Add(start_coords)
            collumn.Add(end_coords)

            wallz.Add(collumn)
        End If

    Next
    '-----------------------------------------------------------------
End Sub

Private Sub create_nodes()
    'create nodes for certain walls
    For y_now = 1 To box.Count() - 1
        For x_now = 0 To box(y_now).Length() - 1
            Dim new_node As New List(Of Array)

            Dim on_coords(2) As Integer
            Dim N_coords(2) As Integer
            Dim E_coords(2) As Integer
            Dim S_coords(2) As Integer
            Dim W_coords(2) As Integer
            Dim NW_coords(2) As Integer
            Dim NE_coords(2) As Integer
            Dim SW_coords(2) As Integer
            Dim SE_coords(2) As Integer

            If box(y_now)(x_now) = "b" Then

                on_coords(0) = x_now
                on_coords(1) = y_now

                ' NORTH
                If y_now - 1 > 0 AndAlso box(y_now - 1)(x_now) = "b" Then
                    N_coords(0) = x_now
                    N_coords(1) = y_now - 1
                Else
                    N_coords(0) = -1
                    N_coords(1) = -1
                End If

                ' SOUTH
                If y_now + 1 < box.Count() - 1 AndAlso box(y_now + 1)(x_now) = "b" Then
                    S_coords(0) = x_now
                    S_coords(1) = y_now + 1
                Else
                    S_coords(0) = -1
                    S_coords(1) = -1
                End If

                ' EAST
                If x_now - 1 > 0 AndAlso box(y_now)(x_now - 1) = "b" Then
                    W_coords(0) = x_now - 1
                    W_coords(1) = y_now
                Else
                    W_coords(0) = -1
                    W_coords(1) = -1
                End If

                ' WEST
                If x_now + 1 < box(y_now).Length() - 1 AndAlso box(y_now)(x_now + 1) = "b" Then
                    E_coords(0) = x_now + 1
                    E_coords(1) = y_now
```

```
                    Else
                        E_coords(0) = -1
                        E_coords(1) = -1
                    End If

                    ' NORTH-WEST
                    If y_now - 1 > 0 AndAlso x_now - 1 >= 0 AndAlso box(y_now - 1)(x_now - 1) = "b"
Then
                        NW_coords(0) = x_now - 1
                        NW_coords(1) = y_now - 1
                    Else
                        NW_coords(0) = -1
                        NW_coords(1) = -1
                    End If

                    ' NORTH-EAST
                    If y_now - 1 > 0 AndAlso x_now + 1 < box(y_now).Length() AndAlso box(y_now -
1)(x_now + 1) = "b" Then
                        NE_coords(0) = x_now + 1
                        NE_coords(1) = y_now - 1
                    Else
                        NE_coords(0) = -1
                        NE_coords(1) = -1
                    End If

                    ' SOUTH-WEST
                    If y_now + 1 < box.Count() AndAlso x_now - 1 >= 0 AndAlso box(y_now + 1)(x_now -
1) = "b" Then
                        SW_coords(0) = x_now - 1
                        SW_coords(1) = y_now + 1
                    Else
                        SW_coords(0) = -1
                        SW_coords(1) = -1
                    End If

                    ' SOUTH-EAST
                    If y_now + 1 < box.Count() AndAlso x_now + 1 < box(y_now).Length() AndAlso
box(y_now + 1)(x_now + 1) = "b" Then
                        SE_coords(0) = x_now + 1
                        SE_coords(1) = y_now + 1
                    Else
                        SE_coords(0) = -1
                        SE_coords(1) = -1
                    End If

                    ' Add coordinates to the node
                    new_node.Add(on_coords)
                    new_node.Add(N_coords)
                    new_node.Add(S_coords)
                    new_node.Add(E_coords)
                    new_node.Add(W_coords)
                    new_node.Add(NW_coords)
                    new_node.Add(NE_coords)
                    new_node.Add(SW_coords)
                    new_node.Add(SE_coords)

                    ' Add the node to the main list
                    nodes.Add(new_node)
                End If
            Next
        Next
    End Sub

    Function find_wall(ByVal nodeon As Array, ByVal wallz As List(Of List(Of Array)))
        Dim new_wallz As New List(Of Integer)
```

```
        For ind = 0 To wallz.Count() - 1

            If wallz(ind).Count() > 1 AndAlso (wallz(ind)(1)(0) = nodeon(0) And wallz(ind)(1)(1) =
nodeon(1)) Then
                new_wallz.Add(ind)
            End If

        Next

        Return new_wallz

    End Function

    Function empty(ByVal coords As Array)
        'the coordd is empty if it is (-1, -1)->they are empty
        If coords(0) = -1 And coords(1) = -1 Then
            Return True
        End If

        Return False
    End Function

    Private Sub normalize_walls()
        Dim cur_walz As New List(Of List(Of Array))

        'turn coord pairs into walls e.g.(1, 1)+(3,3) into a wall
        For Each pair In Me.wallz
            Dim x1 As Integer = 0, y1 As Integer = 0
            Dim x2 As Integer = 0, y2 As Integer = 0

            If pair.Count = 1 Then
                'if the wall is made of 1 coord  create a box(a cross)
                x1 = pair(0)(0) * 50
                y1 = pair(0)(1) * 50

                Dim new_walla As New List(Of Array)

                Dim coorda(2) As Integer
                Dim coordb(2) As Integer

                coorda(0) = x1
                coorda(1) = y1

                coordb(0) = x1 + 50
                coordb(1) = y1 + 50

                new_walla.Add(coorda)
                new_walla.Add(coordb)

                Dim new_wallb As New List(Of Array)

                Dim coordc(2) As Integer
                Dim coordd(2) As Integer

                coordc(0) = x1 + 50
                coordc(1) = y1

                coordd(0) = x1
                coordd(1) = y1 + 50

                new_wallb.Add(coordc)
                new_wallb.Add(coordd)

                '2 walls creat a cross shape
```

```
            cur_walz.Add(new_walla)
            cur_walz.Add(new_wallb)

        ElseIf pair.Count = 2 Then
            ' pair of coords creating a wall
            x1 = pair(0)(0) * 50
            y1 = pair(0)(1) * 50

            x2 = pair(1)(0) * 50
            y2 = pair(1)(1) * 50


            Dim new_walla As New List(Of Array)

            Dim coorda(2) As Integer
            Dim coordb(2) As Integer

            coorda(0) = x1
            coorda(1) = y1

            coordb(0) = x2
            coordb(1) = y2

            new_walla.Add(coorda)
            new_walla.Add(coordb)

            cur_walz.Add(new_walla)

        End If

    Next

    Me.wallz = cur_walz
End Sub
```

RAY CASTING:

EXPLANATION:

Using the list of blockers created in the blocker-making algorithm, this algorithm checks if there is an intersection between a ray and multiple lines. The ray is just a line between two specified points set up by me in other parts of the code; both points have an x and y component, creating an array known as the coord.

Then, using math, I can check if there has been an intersection between this line and other lines with a small amount of math:

respect to the line segments, we can define lines L_1 and L_2 in terms of first degree Bezier parameters:

$$L_1 = \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} + t \begin{bmatrix} x_2 - x_1 \\ y_2 - y_1 \end{bmatrix}, \qquad L_2 = \begin{bmatrix} x_3 \\ y_3 \end{bmatrix} + u \begin{bmatrix} x_4 - x_3 \\ y_4 - y_3 \end{bmatrix}$$

(where t and u are real numbers). The intersection point of the lines is found with one of the following values of t or u, where

$$t = \frac{\begin{vmatrix} x_1 - x_3 & x_3 - x_4 \\ y_1 - y_3 & y_3 - y_4 \end{vmatrix}}{\begin{vmatrix} x_1 - x_2 & x_3 - x_4 \\ y_1 - y_2 & y_3 - y_4 \end{vmatrix}} = \frac{(x_1 - x_3)(y_3 - y_4) - (y_1 - y_3)(x_3 - x_4)}{(x_1 - x_2)(y_3 - y_4) - (y_1 - y_2)(x_3 - x_4)}$$

and

$$u = -\frac{\begin{vmatrix} x_1 - x_2 & x_1 - x_3 \\ y_1 - y_2 & y_1 - y_3 \end{vmatrix}}{\begin{vmatrix} x_1 - x_2 & x_3 - x_4 \\ y_1 - y_2 & y_3 - y_4 \end{vmatrix}} = -\frac{(x_1 - x_2)(y_1 - y_3) - (y_1 - y_2)(x_1 - x_3)}{(x_1 - x_2)(y_3 - y_4) - (y_1 - y_2)(x_3 - x_4)},$$

with

$$(P_x, P_y) = \big(x_1 + t(x_2 - x_1),\ y_1 + t(y_2 - y_1)\big) \quad \text{or} \quad (P_x, P_y) = \big(x_3 + u(x_4 - x_3),\ y_3 + u(y_4 - y_3)\big)$$

There will be an intersection if $0 \le t \le 1$ and $0 \le u \le 1$. The intersection point falls within the first line segment if $0 \le t \le 1$, and it falls within the second line segment if $0 \le u \le 1$. These inequalities can be tested without the need for division, allowing rapid determination of the existence of any line segment intersection before calculating its exact point.[3]

In the case where the two line segments share an x axis and $x_2 = x_1 + 1$, t and u simplify to

$$t = u = \frac{y_1 - y_3}{y_1 - y_2 - y_3 + y_4},$$

with

$$(P_x, P_y) = \big(x_1 + t,\ y_1 + t(y_2 - y_1)\big) \quad \text{or} \quad (P_x, P_y) = \big(x_1 + t,\ y_3 + t(y_4 - y_3)\big).$$

Given two line equations [illegible]

FIGURE 55 THE MATH FROM WIKIPEDIA

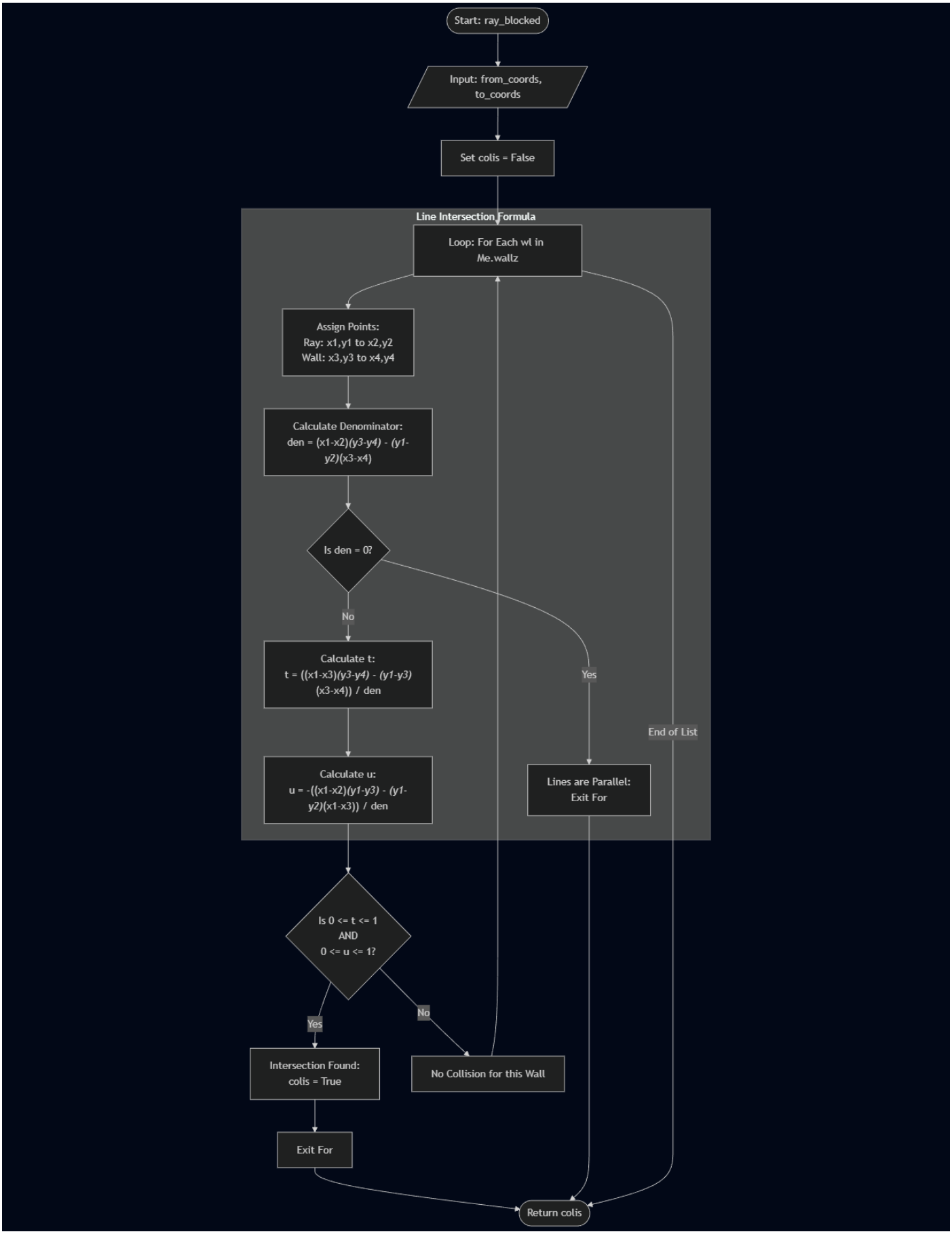
This maths can be found on the **Line–line intersection page**

It works by adding the different x and y values of the blockers and the ray creating a t and u value, if the t and u value are both in the range of 0->1 then there is an intersection and the program terminates and returns true.

Complexity:

O(W) where “W” is the number of walls

FLOWCHART:


Start: ray_blocked
Input: from_coords, to_coords
Set colis = False
Line Intersection Formula
Loop: For Each wl in Me.wallz
Assign Points: Ray: x1,y1 to x2,y2 Wall: x3,y3 to x4,y4
Calculate Denominator: den = (x1-x2)(y3-y4) - (y1-y2)(x3-x4)
Is den = 0?
No
Yes
Calculate t: t = ((x1-x3)(y3-y4) - (y1-y3)(x3-x4)) / den
Calculate u: u = -((x1-x2)(y1-y3) - (y1-y2)(x1-x3)) / den
Lines are Parallel: Exit For
End of List
Is 0 <= t <= 1 AND 0 <= u <= 1?
Yes
No
Intersection Found: colis = True
No Collision for this Wall
Exit For
Return colis

PSEUDOCODE:

```
FUNCTION ray_blocked(fromCoords, toCoords)

    a = fromCoords

    b = toCoords


    collision = FALSE


    FOR EACH wall IN wallz


        x1 = a.x

        y1 = a.y

        x2 = b.x

        y2 = b.y


        x3 = wall.start.x

        y3 = wall.start.y

        x4 = wall.end.x

        y4 = wall.end.y


        den = (x1 - x2) * (y3 - y4) - (y1 - y2) * (x3 - x4)


        IF den == 0 THEN

            EXIT FOR

        ENDIF


        t = ((x1 - x3) * (y3 - y4) - (y1 - y3) * (x3 - x4)) / den

        u = -(((x1 - x2) * (y1 - y3) - (y1 - y2) * (x1 - x3)) / den)


        IF t >= 0 AND t <= 1 AND u >= 0 AND u <= 1 THEN
```

```
        collision = TRUE

        EXIT FOR

      ENDIF


  END FOR


  RETURN collision


END FUNCTION
```

CODE:

```
  Public Function ray_blocked(ByVal from_coords As Array, ByVal to_coords As Array)
        Dim coorda(2) As Integer
        Dim coordb(2) As Integer

        coorda = from_coords

        coordb = to_coords

        Dim colis As Boolean = False
        For Each wl In Me.wallz
            'uses fancy math to find if an intersection even exists
            Dim x1 = coorda(0)
            Dim x2 = coordb(0)
            Dim x3 = wl(0)(0)
            Dim x4 = wl(1)(0)

            Dim y1 = coorda(1)
            Dim y2 = coordb(1)
            Dim y3 = wl(0)(1)
            Dim y4 = wl(1)(1)

            Dim den As Double = (x1 - x2) * (y3 - y4) - (y1 - y2) * (x3 - x4)

            'division by 0
            If den = 0 Then
                Exit For
            End If

            Dim t As Double = ((x1 - x3) * (y3 - y4) - (y1 - y3) * (x3 - x4)) / den
            Dim u As Double = -(((x1 - x2) * (y1 - y3) - (y1 - y2) * (x1 - x3)) / den)

            If t >= 0 And t <= 1 And u >= 0 And u <= 1 Then
                colis = True
                Exit For
            End If
            '-----------------------------------------------------------
        Next

        Return colis


    End Function
```

SPIDER BOSS LIMBS:

EXPLANATION:

The purpose of this algorithm is to simulate crawling for the spider boss's legs; therefore, legs cannot move at the same time but must move near to the main body so the spider doesn't look unnatural.

First, the legs are initialised in the load_legs() subroutine. Here I filled two lists, thighs and femurs, both of which are 2D lists of pictureboxes. The reason they are 2D is so each thigh is not just a rectangle but instead made of multiple boxes, giving it more detail and character.

Next, the legs are updated in update_spider(). Points are created in a circle surrounding the spider's main body; these can be called anchor points, and they are generated using trigonometry.

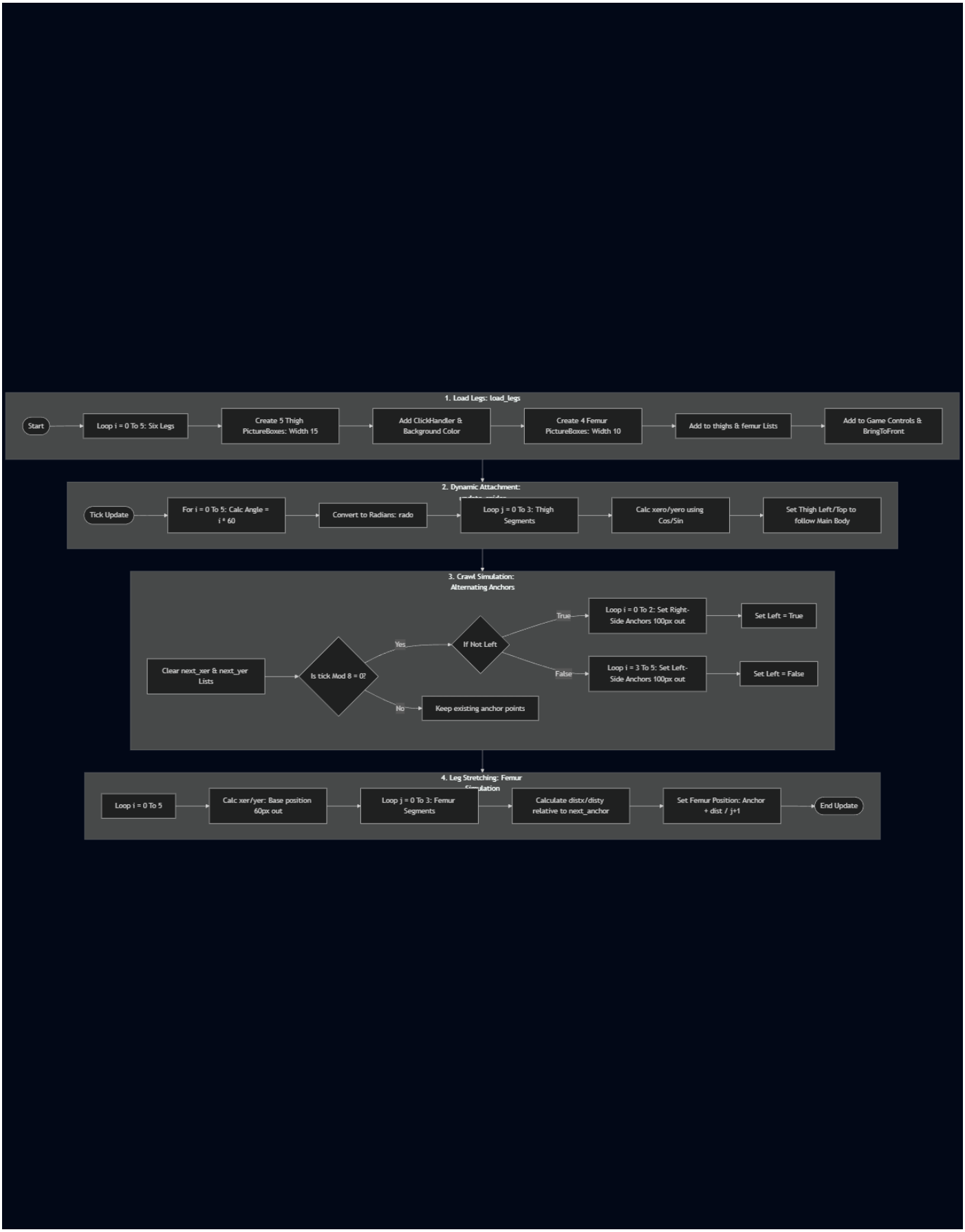
Then, every few ticks (8), these anchor points are moved to new positions. A variable that indicates whether the left legs or right legs are being changed is used. Depending on which legs (defined by their index in a list), the femurs are connected from the anchor point to the first thigh.

The thighs are always connected to the spider's main body and move in the same direction as it.

Complexity:

O(L * S) where "L" is the number of legs and "S" is the number of segments. This includes the combination of thighs and femurs. This can be simplified to O(1) as it is run once per frame and depends on the small number of legs the spider has (around 6).

FLOWCHART:


1. Load Legs: load_legs
Start
Loop i = 0 To 5: Six Legs
Create 5 Thigh PictureBoxes: Width 15
Add ClickHandler & Background Color
Create 4 Femur PictureBoxes: Width 10
Add to thighs & femur Lists
Add to Game Controls & BringToFront
2. Dynamic Attachment:
Tick Update
For i = 0 To 5: Calc Angle = i * 60
Convert to Radians: rado
Loop j = 0 To 3: Thigh Segments
Calc xero/yero using Cos/Sin
Set Thigh Left/Top to follow Main Body
3. Crawl Simulation: Alternating Anchors
Clear next_xer & next_yer Lists
Is tick Mod 8 = 0?
Yes
If Not Left
True
Loop i = 0 To 2: Set Right-Side Anchors 100px out
Set Left = True
False
Loop i = 3 To 5: Set Left-Side Anchors 100px out
Set Left = False
No
Keep existing anchor points
4. Leg Stretching: Femur
Loop i = 0 To 5
Calc xer/yer: Base position 60px out
Loop j = 0 To 3: Femur Segments
Calculate distx/disty relative to next_anchor
Set Femur Position: Anchor + dist / j+1
End Update

PSEUDOCODE:

```
PROCEDURE update_spider(tick)


  FOR i FROM 0 TO 5
    angle = i * 60
    rad = angle * PI/180

    FOR j FROM 0 TO 3
      x = (xPos + 10) + (j+3) * 10 * COS(rad)
      y = (yPos + 10) + (j+3) * 10 * SIN(rad)

      thighs[i][j].Left = x
      thighs[i][j].Top  = y
    END FOR
  END FOR

  FOR i FROM 0 TO 5
    next_xer.ADD(0)
    next_yer.ADD(0)
  END FOR

  IF tick MOD 8 == 0 THEN

    IF NOT left THEN
      FOR i FROM 0 TO 2
        angle = i * 60
        rad = angle * PI/180
        next_xer[i] = (xPos + 8) + 100 * COS(rad)
        next_yer[i] = (yPos + 8) + 100 * SIN(rad)
      END FOR
```

```
      left = TRUE
    ELSE
      FOR i FROM 3 TO 5
        angle = i * 60
        rad = angle * PI/180
        next_xer[i] = (xPos + 8) + 100 * COS(rad)
        next_yer[i] = (yPos + 8) + 100 * SIN(rad)
      END FOR
      left = FALSE
    ENDIF

ENDIF

FOR i FROM 0 TO 5

    angle = i * 60
    rad = angle * PI/180

    baseX = (xPos + 8) + 60 * COS(rad)
    baseY = (yPos + 8) + 60 * SIN(rad)

    FOR j FROM 0 TO 3
      dx = baseX - next_xer[i]
      dy = baseY - next_yer[i]

      femur[i][j].Left = next_xer[i] + dx/(j+1)
      femur[i][j].Top  = next_yer[i] + dy/(j+1)
    END FOR

END FOR
```

```
END PROCEDURE



PROCEDURE load_legs()


  FOR i FROM 0 TO 5
    newList = empty list
    FOR j FROM 0 TO 4
      piece = PictureBox()
      piece.Width = 15
      piece.Height = 15
      piece.BackColor = BLACK
      newList.ADD(piece)
      game.Controls.ADD(piece)
      piece.BringToFront()
    END FOR
    thighs.ADD(newList)
  END FOR


  FOR i FROM 0 TO 5
    newList = empty list
    FOR j FROM 0 TO 3
      piece = PictureBox()
      piece.Width = 10
      piece.Height = 10
      piece.BackColor = BLACK
      newList.ADD(piece)
      game.Controls.ADD(piece)
      piece.BringToFront()
```

```
    END FOR

    femur.ADD(newList)

  END FOR


END PROCEDURE
```

CODE:

```
    Public Sub update_spider(ByVal tick As Integer)


        'the spider legs attach themselves to an anchor point then every few ticks the hav a new
anchor closer to the main body set, simulating crawling

        'moves the upper leg with the spider body, as if its connected to it
        For i = 0 To 5

            angle = i * 60
            Dim rado As Double = angle * Math.PI / 180


            For j = 0 To 3

                Dim xero As Integer = (Me.x + 10) + (j + 3) * 10 * Math.Cos(rado)
                Dim yero As Integer = (Me.y + 10) + (j + 3) * 10 * Math.Sin(rado)

                Me.thighs(i)(j).Left = xero
                Me.thighs(i)(j).Top = yero
            Next
        Next
        '-----------------------------------------------------------------

        'empties the next_list of y and x coords for new ones
        For i = 0 To 5
            Me.next_xer.Add(0)
            Me.next_yer.Add(0)
        Next
        '-----------------------------------------------------------------

        'every 8 ticks either the left lower legs or right lower create new anchors to go to
        If tick Mod 8 = 0 Then

            If Not left Then

                For i = 0 To 2
                    angle = i * 60
                    radian = angle * Math.PI / 180

                    Me.next_xer(i) = (Me.x + 8) + 100 * Math.Cos(radian)
                    Me.next_yer(i) = (Me.y + 8) + 100 * Math.Sin(radian)
                Next

                Me.left = True

            Else

                For i = 3 To 5
                    angle = i * 60
                    radian = angle * Math.PI / 180
```

```
                    Me.next_xer(i) = (Me.x + 8) + 100 * Math.Cos(radian)
                    Me.next_yer(i) = (Me.y + 8) + 100 * Math.Sin(radian)
                Next

                Me.left = False

            End If

        End If
        '-------------------------------------------------------------------------------------------
--------------

        'use the new anchor created to simulate moving, if new anchors havent been made stretch the
lower leg fom body so the legs still looks alive and not still
        For i = 0 To 5

            angle = i * 60
            radian = angle * Math.PI / 180

            Dim xer As Double = (Me.x + 8) + 60 * Math.Cos(radian)
            Dim yer As Double = (Me.y + 8) + 60 * Math.Sin(radian)

            For j = 0 To 3

                Dim distx As Double = xer - Me.next_xer(i)
                Dim disty As Double = yer - Me.next_yer(i)

                Me.femur(i)(j).Left = Me.next_xer(i) + (distx / (j + 1))
                Me.femur(i)(j).Top = Me.next_yer(i) + (disty / (j + 1))

            Next

        Next

    End Sub


Public Sub load_legs()
        'add upper leg peices to the spider
        For i = 0 To 5

            Dim new_list As New List(Of PictureBox)

            For j = 0 To 4

                Dim new_add As New PictureBox

                AddHandler new_add.MouseClick, AddressOf Me.plr1.click_handler

                new_add.Width = 15
                new_add.Height = 15
                new_add.BackColor = Color.Black



                new_list.Add(new_add)
                new_add.BringToFront()
                Me.game.Controls.Add(new_add)

            Next

            Me.thighs.Add(new_list)
        Next
        '-----------------------------------
```

```
        'add lower leg peices to the spider
        For i = 0 To 5

            Dim new_list As New List(Of PictureBox)

            For j = 0 To 3

                Dim new_add As New PictureBox

                new_add.Width = 10
                new_add.Height = 10
                new_add.BackColor = Color.Black

                new_list.Add(new_add)
                Me.game.Controls.Add(new_add)
                new_add.BringToFront()

            Next

            Me.femur.Add(new_list)
        Next
        '-----------------------------------
    End Sub
```

MAP CREATION:

EXPLANATION:

This algorithm is used in the main form1 class with the cellular automata algorithm and A* which creates a text map that is then changed for each new room.

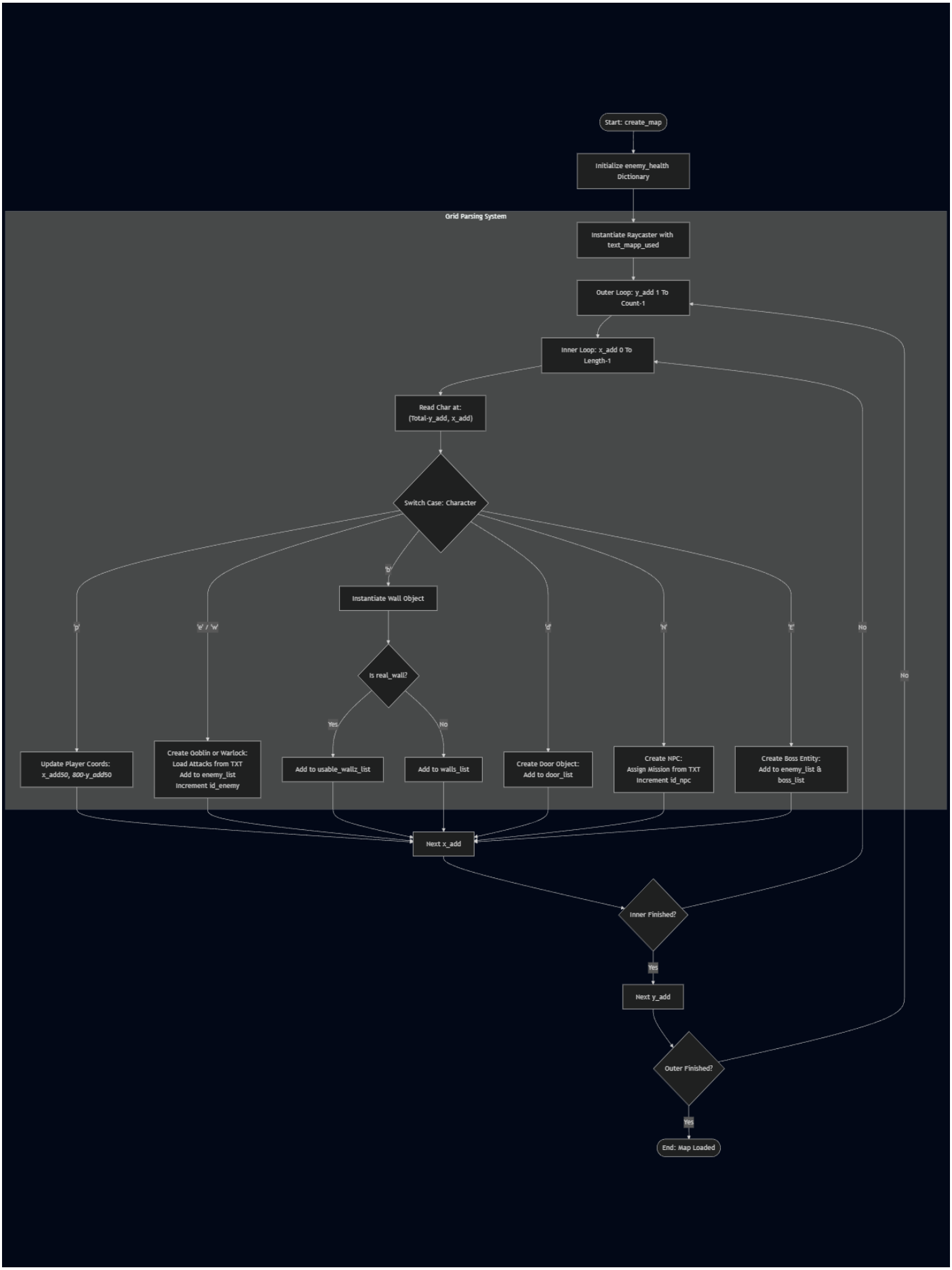
The map is a 1D list made of strings that are also iterated through; hence, it is used as a 2D list (as strings are just arrays of characters).

First, the 2D list is iterated through upside down, due to the y-axis being inverted in Visual Basic Forms. As a result, this is more intuitive when adding entity characters. Next, I read each character and then decide what should be spawned at that location.

Thirdly, I then instantiate that entity, multiplying the x and y coordinates by 50 as that is the size of the screen. The player object is not instantiated, as I use the same class throughout each level, only changing its location and making the y and x velocity 0 so it does not clip into the walls while the map is loading. This means I do not have to offer processing power and duplicate data to create a new player object in each room.

There are also two different types of wall: ones that are neighbouring empty tiles and ones that are fully closed off from the player. If I decided to check all of them for every collision, the game would be much slower. As a result, only the ones neighbouring an empty cell are put in a list used for collision detection between entities.

FLOWCHART:


Start: create_map
Initialize enemy_health Dictionary
Grid Parsing System
Instantiate Raycaster with text_mapp_used
Outer Loop: y_add 1 To Count-1
Inner Loop: x_add 0 To Length-1
Read Char at: (Total-y_add, x_add)
Switch Case: Character
'b'
Instantiate Wall Object
'p'
'e' / 'w'
'd'
'n'
'c'
No
Is real_wall?
Yes
No
No
Update Player Coords: x_add50, 800-y_add50
Create Goblin or Warlock: Load Attacks from TXT Add to enemy_list Increment id_enemy
Add to usable_wallz_list
Add to walls_list
Create Door Object: Add to door_list
Create NPC: Assign Mission from TXT Increment id_npc
Create Boss Entity: Add to enemy_list & boss_list
Next x_add
Inner Finished?
Yes
Next y_add
Outer Finished?
Yes
End: Map Loaded

PSEUDOCODE:

```
PROCEDURE create_map()


  enemyHealth = {
    "spider": 150,
    "invisi": 200,
    "goblin": 2,
    "warlock": 4
  }


  raycaster = ray_cast(textMap)


  FOR y FROM 1 TO textMap.height - 1
    FOR x FROM 0 TO textMap[y].length - 1


      tile = textMap[(textMap.height - 1) - (y - 1)][x]


      IF tile == "p" THEN
        player.x = x * 50
        player.y = 800 - (y * 50)


      ELSE IF tile == "e" THEN
        g = goblin_enemy(game, 45, 45, x*50, 800 - y*50, enemyID, enemyHealth["goblin"], util)
        g.load_attacks_fromtxt()
        enemyList.ADD(g)
        enemyID += 1


      ELSE IF tile == "b" THEN
        w = wall(game, 50, 50, x*50, 800 - y*50)
        w.textMap = textMap
```

```
    IF w.real_wall(y, x) THEN
        usableWalls.ADD(w)
    ELSE
        walls.ADD(w)
    ENDIF

ELSE IF tile == "w" THEN
    wlk = warlock_enemy(game, 45, 45, x*50, 800 - y*50, enemyID, enemyHealth["warlock"], util, TRUE)
    wlk.load_attacks_fromtxt()
    enemyList.ADD(wlk)
    enemyID += 1

ELSE IF tile == "d" THEN
    d = door(game, 50, 50, x*50, 800 - y*50, textMap[0])
    doorList.ADD(d)

ELSE IF tile == "N" THEN
    npc = Npc(game, player, x*50, 800 - y*50, 50, 50, allMissions[npcID], util)
    NPCList.ADD(npc)
    npcID += 1

ELSE IF tile == "E" THEN
    bossEnt = boss(game, x*50, 800 - y*50, 45, 45, bossType, enemyHealth[bossType], player, util)
    bossEnt.load_attacks_fromtxt("none")
    enemyList.ADD(bossEnt)
    bossList.ADD(bossEnt)

ENDIF
```

END FOR

END FOR

END PROCEDURE

CODE:

```
Private Sub create_map()

        Dim enemy_health As New Dictionary(Of String, Integer) From {
                {"spider", 150},
                {"invisi", 200},
                {"goblin", 2},
                {"warlock", 4}
        }

        'create coordinates for walls for ray_casting, to make the ray casting more optimizes
instead of checking each wall seperately

        Me.raycaster = New ray_cast(Me.text_mapp_used)


        'loads the text map into visualized the player can interact with.
        'whilst also making terrain generation and adding in different structure randomly incredibly
easy

        For y_add = 1 To text_mapp_used.Count() - 1
            For x_add = 0 To text_mapp_used(y_add).Length() - 1
                If text_mapp_used((text_mapp_used.Count() - 1) - (y_add - 1))(x_add) = "p" Then
                    ' moves the already loaded player image to the starting locations, so i dont
need to re-load the player each new map
                    plr1.x = x_add * 50
                    plr1.y = 800 - (y_add * 50)

                ElseIf text_mapp_used((text_mapp_used.Count() - 1) - (y_add - 1))(x_add) = "e" Then

                    Dim new_goblin As New goblin_enemy(Me, 45, 45, x_add * 50, 800 - (y_add * 50),
Me.id_enemy, enemy_health("goblin"), Me.utiliser)

                    new_goblin.load_attacks_fromtxt()

                    enemy_list.Add(new_goblin)

                    Me.id_enemy += 1
                ElseIf text_mapp_used((text_mapp_used.Count() - 1) - (y_add - 1))(x_add) = "b" Then
                    Dim new_wall As New wall(Me, 50, 50, x_add * 50, 800 - (y_add * 50))
                    new_wall.text_mapp_used = Me.text_mapp_used

                    If new_wall.real_wall(y_add, x_add) Then

                        Me.usable_wallz_list.Add(new_wall)

                    Else

                        Me.walls_list.Add(new_wall)

                    End If

                ElseIf text_mapp_used((text_mapp_used.Count() - 1) - (y_add - 1))(x_add) = "w" Then
```

```
                    Dim new_warlock As New warlock_enemy(Me, 45, 45, x_add * 50, 800 - (y_add * 50),
Me.id_enemy, enemy_health("warlock"), Me.utiliser, True)

                    new_warlock.load_attacks_fromtxt()

                    enemy_list.Add(new_warlock)

                    Me.id_enemy += 1
                ElseIf text_mapp_used((text_mapp_used.Count() - 1) - (y_add - 1))(x_add) = "d" Then

                    Dim new_box As New door(Me, 50, 50, x_add * 50, 800 - (y_add * 50),
text_mapp_used(0))

                    door_list.Add(new_box)

                ElseIf text_mapp_used((text_mapp_used.Count() - 1) - (y_add - 1))(x_add) = "N" Then

                    Dim new_box As New Npc(Me, Me.plr1, x_add * 50, 800 - (y_add * 50), 50, 50,
Me.all_missions_txt(Me.id_npc), Me.utiliser)

                    Me.NPC_list.Add(new_box)

                    Me.id_npc += 1


                ElseIf text_mapp_used((text_mapp_used.Count() - 1) - (y_add - 1))(x_add) = "E" Then
                    'a boss entity
                    Dim new_boss As New boss(Me, x_add * 50, 800 - (y_add * 50), 45, 45,
Me.type_bos, enemy_health(Me.type_bos), Me.plr1, Me.utiliser)

                    new_boss.load_attacks_fromtxt("none")

                    Me.enemy_list.Add(new_boss)
                    Me.boss_list.Add(new_boss)

                End If


            Next
        Next
```

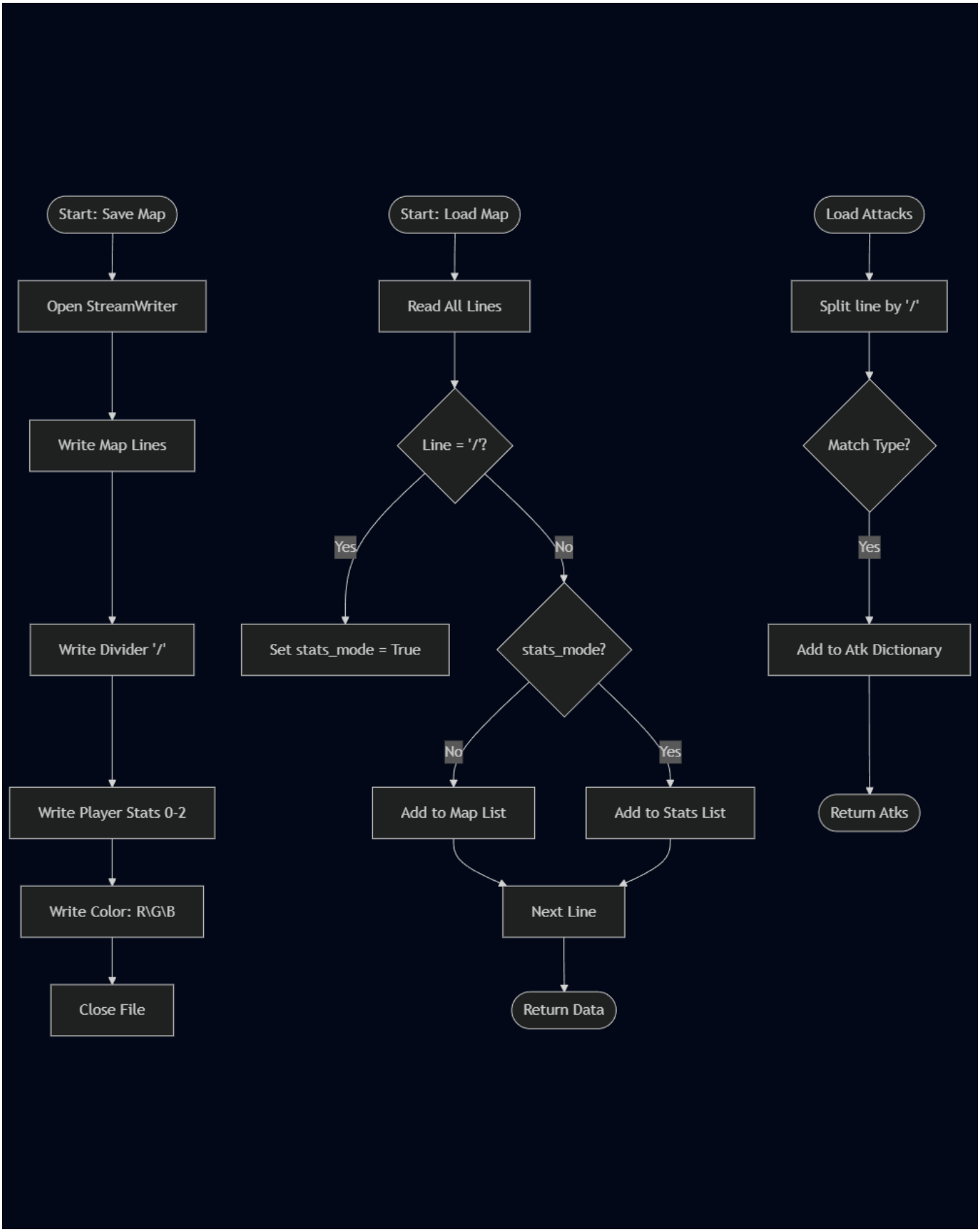
SAVING AND LOADING:

EXPLANATION:

When loading or saving files, first I check if the file path exists which, if it does not, is shown to the user with a debug.print statement; otherwise the program continues running. The text file is then loaded in, where each line is read and the string on it is appended to a list. Files that store statistics about in-game objects usually have stats stored in a line separated by "/", which I use to split the string and return the stats in, usually, a dictionary. In the game saving part of the utility class I save the map currently in use and then on the lines below the current player attributes are written.

Complexity: O(L) where "L" is the number of lines in the file

FLOWCHART:


Start: Save Map
Open StreamWriter
Write Map Lines
Write Divider '/'
Write Player Stats 0-2
Write Color: R\G\B
Close File
Start: Load Map
Read All Lines
Line = '/'?
Yes
No
Set stats_mode = True
stats_mode?
No
Yes
Add to Map List
Add to Stats List
Next Line
Return Data
Load Attacks
Split line by '/'
Match Type?
Yes
Add to Atk Dictionary
Return Atks

PSEUDOCODE:

```
FUNCTION file_exists(path, mode, clear = FALSE)

    IF NOT FILE_EXISTS(path) THEN
        STOP
    ELSE
        IF mode == "read" THEN
            RETURN READ_ALL_LINES(path)
        ELSE IF mode == "write" THEN
            RETURN STREAM_WRITER(path, clear)
        ELSE
            STOP
        ENDIF
    ENDIF

    RETURN NULL

END FUNCTION



PROCEDURE save_map(linesToSave, player)

    writer = file_exists(saveMapPath, "write")

    FOR EACH line IN linesToSave
        writer.WRITE_LINE(line)
    END FOR

    writer.WRITE_LINE("/")
```

```
    FOR i FROM 0 TO 2
        writer.WRITE_LINE(player.stats[i])
    END FOR

    body = player.bodyColor
    writer.WRITE_LINE(body.R + "\" + body.G + "\" + body.B)

    arm = player.armColor
    writer.WRITE_LINE(arm.R + "\" + arm.G + "\" + arm.B)

    leg = player.legColor
    writer.WRITE_LINE(leg.R + "\" + leg.G + "\" + leg.B)

    writer.CLOSE()

END PROCEDURE


PROCEDURE save_score(text)

    writer = file_exists(scorePath, "write", TRUE)
    writer.WRITE_LINE(text)
    writer.CLOSE()

END PROCEDURE


FUNCTION load_map()

    store = empty list
```

```
statsMode = FALSE
skipNext = FALSE

lines = file_exists(saveMapPath, "read")

map = empty list
stats = empty list

FOR EACH line IN lines

  FOR EACH char IN line
    IF char == "/" THEN
      statsMode = TRUE
      skipNext = TRUE
      BREAK
    ENDIF
  END FOR

  IF NOT skipNext THEN
    IF statsMode THEN
      stats.ADD(line)
    ELSE
      map.ADD(line)
    ENDIF
  ELSE
    skipNext = FALSE
  ENDIF

END FOR
```

```
    store.ADD(map)
    store.ADD(stats)

    RETURN store

END FUNCTION



FUNCTION load_scores(dict)

    lines = file_exists(scorePath, "read")

    FOR i FROM 0 TO (lines.count / 2) - 1
        key = lines[i*2 + 1]
        val = lines[i*2]
        dict[key] = val
    END FOR

    RETURN dict

END FUNCTION



FUNCTION load_attacks(type)

    atkDict = empty dictionary
    lines = file_exists(attacksPath, "read")

    FOR EACH line IN lines
```

```
    parts = SPLIT(line, "/")

    IF parts.length > 2 AND (parts[8] == type OR type == "none") THEN

      name = parts[0]
      values = empty list

      FOR i FROM 1 TO parts.length - 1
        values.ADD(parts[i])
      END FOR

      atkDict[name] = values

    ENDIF

  END FOR

  RETURN atkDict

END FUNCTION
```

CODE (REAL USE EXAMPLES):

```
    Public Function file_exisists(ByVal file_path As String, ByVal read_write As String, Optional
ByVal clear As Boolean = False)
        'checks the file exists and loads it in
        If Not System.IO.File.Exists(file_path) Then
            Debug.Print("file does not exist: " & file_path)
            Stop
        Else
            If read_write = "read" Then
                Return System.IO.File.ReadAllLines(file_path)
            ElseIf read_write = "write" Then
                Return New System.IO.StreamWriter(file_path, clear)
            Else
                Debug.Print("cannot " & read_write)
                Stop
            End If

        End If
        Return Nothing
    End Function
Public Sub save_map(ByVal string_to_save As List(Of String), ByVal plr1 As player)
```

```
        Dim objWriter As System.IO.StreamWriter = Me.file_exisists(utilities.slmpath, "write")


        'write the map to the file
        For Each line In string_to_save

            objWriter.WriteLine(line)

        Next
        '------------------------
        'new line, player stats/cutomization
        objWriter.WriteLine("/")

        'write the stat modifiers to the text file
        For ind = 0 To 2
            objWriter.WriteLine(plr1.all_stats(ind))
        Next

        'Write each limbs rgb colour to the text file, each rgb vslue is seperated by a /
        Dim body_cols As Color = plr1.box.BackColor
        objWriter.WriteLine(body_cols.R & "\" & body_cols.G & "\" & body_cols.B)

        Dim arm_cols As Color = plr1.leftArm.BackColor
        objWriter.WriteLine(arm_cols.R & "\" & arm_cols.G & "\" & arm_cols.B)

        Dim leg_cols As Color = plr1.leftLeg.BackColor
        objWriter.WriteLine(leg_cols.R & "\" & leg_cols.G & "\" & leg_cols.B)

        objWriter.Close()

    End Sub

    Public Sub save_score(ByVal textp As String)
        'write to text file

        Dim objWriter As System.IO.StreamWriter = Me.file_exisists(utilities.svscpath, "write",
True)


        objWriter.WriteLine(textp)


        objWriter.Close()
    End Sub


    Public Function load_map()

        Dim store_list As New List(Of List(Of String))

        Dim player_stats As Boolean = False
        Dim next_line As Boolean = False

        Dim lines() As String = Me.file_exisists(utilities.slmpath, "read")

        'divide a txt file into lines and read each one then store in a list(map, stats)

        Dim map As New List(Of String)
        Dim stats As New List(Of String)


        For Each line As String In lines

            For Each ind In line
```

```
                If ind = "/" Then
                    'if / found save all read things to the stat file
                    player_stats = True
                    next_line = True

                    Exit For
                End If

            Next

            If Not next_line Then
                If player_stats Then
                    stats.Add(line)
                Else
                    map.Add(line)
                End If

            Else
                next_line = False

            End If



        Next
        '-----------------------------------------------------------------


        store_list.Add(map)
        store_list.Add(stats)



        Return store_list
    End Function

    Public Function Load_scores(ByVal load_dict As Dictionary(Of String, String))

        'read , in lines, from the text file
        Dim lines() As String = Me.file_exisists(utilities.svscpath, "read")

        For line = 0 To (lines.Count() / 2) - 1


            load_dict(lines(line * 2 + 1)) = lines(line * 2)



        Next
        '-----------------------------------------------
        Return load_dict
End Function

    Public Function load_attacks(ByVal type As String)
        'type=type of entity as enemies ,players and bosses all have different attacks so no point
in loading the uneccessary ones
        Dim atk_dict As New Dictionary(Of String, List(Of String))

        Dim lines() As String = Me.file_exisists(utilities.aspath, "read")


        'load the atributes for each attack into a dictionary
        For Each line As String In lines
```

```
        Dim parts() As String = line.Split("/"c)

        If parts.Length > 2 AndAlso (parts(8) = type Or "none" = type) Then

            Dim name As String = parts(0)
            Dim values As New List(Of String)

            For i As Integer = 1 To parts.Length - 1
                values.Add(parts(i))
            Next

            atk_dict(name) = values

        End If

    Next
    '------------------------------------------------

    Return atk_dict
End Function
```

QUICK SORT AND LEADERBOARD CREATION:

EXPLANATION:

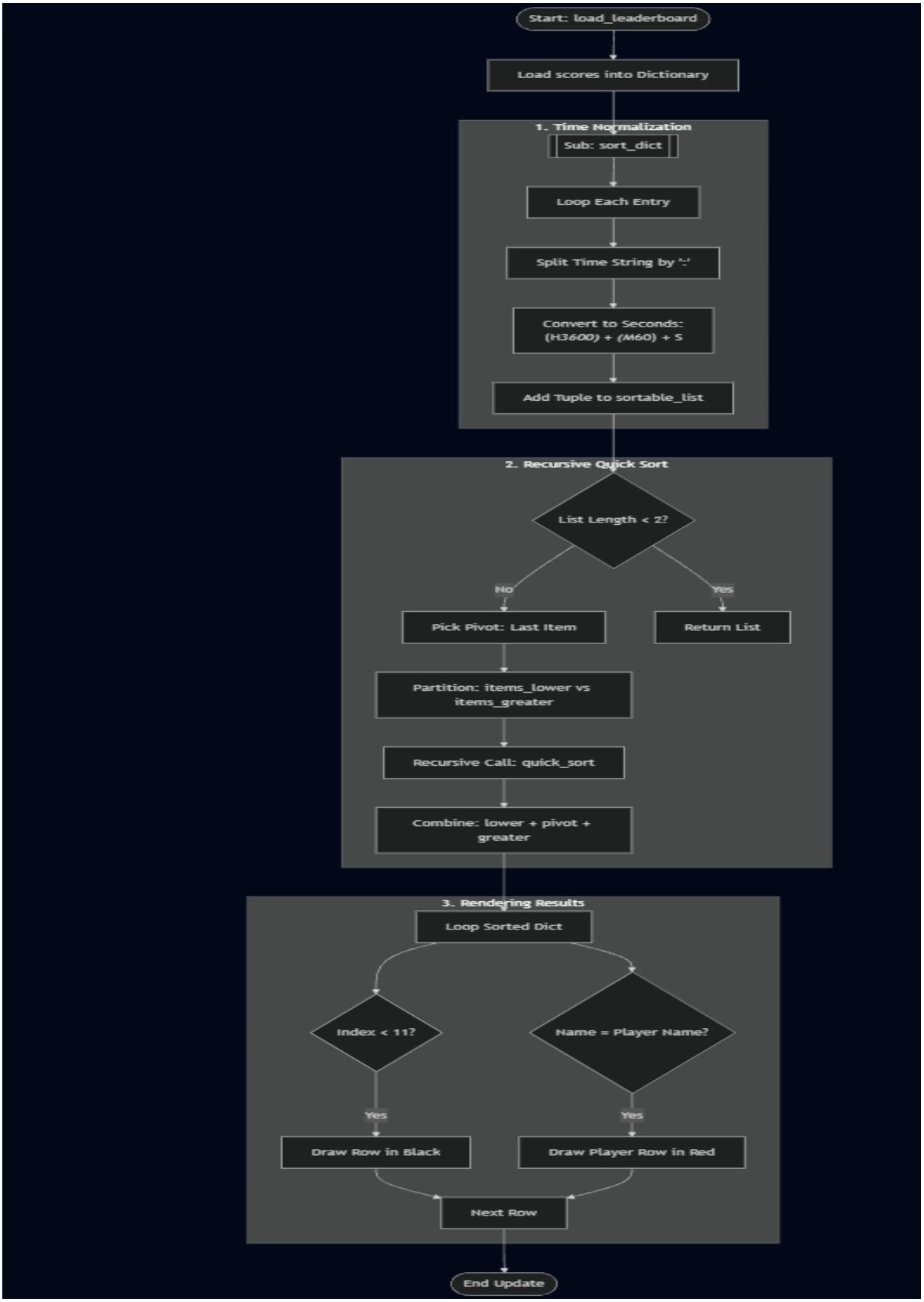
This algorithm takes place in the Finish_screen form and, due to how linked these algorithms are, I have decided to show them together.

In the leaderboard: A dictionary is created that stores the name of the user and the score they achieved. This is loaded from a text file where each name has its corresponding score below it. The score is just the time they took to finish the game. This is now a full unordered dictionary; hence, quicksort is now used.

In the quicksort algorithm: In the first subroutine sort_dict() a new dictionary is created and a list of a tuple made of an integer (which is the time converted into seconds) and a string (which is the name corresponding to that time). Then the recursive algorithm is put to work in the quick_sort function. A pivot is made which is in the center of the list; all items greater than this are put in a greater-items list and lower are put in a lower-than list. This is done multiple times until every item has been a pivot and had any smaller values put below it and larger above it. Due to this being done recursively, each iteration returns a value into another list which adds all lower-than values at its start, the pivot in the middle, and all higher-than at the end. This sorted list of tuples is then returned into the sort_dict() subroutine where it is converted back into a dictionary and is assigned to the leaderboard_dict variable.

Quicksort complexity: Most cases O(Nlog(N)), worst case O(N^2)

FLOWCHART:


Start: load_leaderboard
Load scores into Dictionary
1. Time Normalization
Sub: sort_dict
Loop Each Entry
Split Time String by "-"
Convert to Seconds: (H3600) + (M60) + S
Add Tuple to sortable_list
2. Recursive Quick Sort
List Length < 2?
No
Yes
Pick Pivot: Last Item
Return List
Partition: items_lower vs items_greater
Recursive Call: quick_sort
Combine: lower + pivot + greater
3. Rendering Results
Loop Sorted Dict
Index < 11?
Name = Player Name?
Yes
Yes
Draw Row in Black
Draw Player Row in Red
Next Row
End Update

PSEUDOCODE:

```
PROCEDURE load_leaderboard()

  leaderboard = util.Load_scores(leaderboard)

  sort_dict()

  index = 0

  FOR EACH entry IN leaderboard

    IF index < 11 THEN
      row = text(game, 350, 150 + index*50, 15,
            (index+1) + ". " + entry.key + " : " + leaderboard(entry.key) + " seconds",
            BLACK)
      row.border = TRUE
      row.load()
    ENDIF

    IF entry.key == inputBox.text THEN
      playerRow = text(game, 600, 150, 15,
                (index+1) + ". " + entry.key + " : " + leaderboard(entry.key) + " seconds",
                RED)
      playerRow.border = TRUE
      playerRow.load()
    ENDIF

    index += 1

  END FOR
```

```
END PROCEDURE



PROCEDURE sort_dict()

    sortable = empty list
    newDict = empty dictionary

    FOR EACH entry IN leaderboard

        parts = SPLIT(leaderboard(entry.key), ":")
        total = parts[0]*3600 + parts[1]*60 + parts[2]

        sortable.ADD( (total, entry.key) )

    END FOR

    FOR EACH item IN quick_sort(sortable)
        newDict[item.key] = item.value
    END FOR

    leaderboard = newDict

END PROCEDURE



FUNCTION quick_sort(list)

    n = list.count
```

```
  IF n < 2 THEN
    RETURN list
  ENDIF

  pivot = list[n-1]
  REMOVE list[n-1]

  greater = empty list
  lower   = empty list

  FOR EACH item IN list
    IF item.value > pivot.value THEN
      greater.ADD(item)
    ELSE
      lower.ADD(item)
    ENDIF
  END FOR

  result = empty list
  result.ADD_RANGE( quick_sort(lower) )
  result.ADD(pivot)
  result.ADD_RANGE( quick_sort(greater) )

  RETURN result

END FUNCTION
```

CODE:

```
Private Sub load_leaderboard()
        'loads lines from saved scores file
        Me.leaderboard_dict = Me.util.Load_scores(Me.leaderboard_dict)

        'sort the dictionary
```

```
        Me.sort_dict()

        Dim ind As Integer = 0
        'print all the rows in the sorted dictionary
        For Each scr In Me.leaderboard_dict

            'only show the first 10
            If ind < 11 Then

                Dim new_row As New text(Me, 350, 150 + (ind * 50), 15, ind + 1 & ". " & scr.Key & "
: " & Me.leaderboard_dict(scr.Key) & " seconds", Color.Black)

                new_row.border = True
                new_row.load()

            End If
            '--------------------

            'show the player score/time and where they rank
            If scr.Key = Me.txtInput.textbox.Text Then

                Dim player_score As New text(Me, 600, 150, 15, ind + 1 & ". " & scr.Key & " : " &
Me.leaderboard_dict(scr.Key) & " seconds", Color.Red)

                player_score.border = True
                player_score.load()
            End If
            '--------------------------------------------

            ind += 1

        Next
        '-------------------------------------------


    End Sub

    Private Sub sort_dict()

        Dim sortable_list As New List(Of Tuple(Of Integer, String))
        Dim new_dict As New Dictionary(Of String, String)

        'change the dictionary to a list thaat can be sorted
        For Each scr In Me.leaderboard_dict

            Dim val As Integer = 0
            Dim parts() As String = Me.leaderboard_dict(scr.Key).Split(":"c)

            'change hours and minutes to seconds, for ordering
            val = CUInt(parts(0)) * 3600 + CUInt(parts(1)) * 60 + CUInt(parts(2))

            sortable_list.Add(Tuple.Create(val, scr.Key))

        Next
        '-------------------------------------------------

        'sort the new list and store the result as a dictionary
        For Each scr In quick_sort(sortable_list)
            new_dict((scr.item2).ToString()) = scr.item1
        Next

        Me.leaderboard_dict = new_dict
        '-----------------------------------------------------

    End Sub
```

```
Private Function quick_sort(ByVal my_lizt As List(Of Tuple(Of Integer, String)))
    'performs a quick sort to a list

    Dim length As Integer = my_lizt.Count()

    'makes sure the list is big enough : bigger then 1/not smaller then 2
    If length < 2 Then
        Return my_lizt
    End If


    'split the list in halves
    Dim pivot As Tuple(Of Integer, String) = my_lizt(length - 1)

    my_lizt.RemoveAt(length - 1)
    '--------------------

    'order the split list
    Dim items_greater As New List(Of Tuple(Of Integer, String))
    Dim items_lower As New List(Of Tuple(Of Integer, String))

    For Each scr In my_lizt

        If scr.Item1 > pivot.Item1 Then

            items_greater.Add(scr)
        Else

            items_lower.Add(scr)
        End If

    Next
    '----------------------

    'recursively order it
    Dim return_my_lizt As New List(Of Tuple(Of Integer, String))

    return_my_lizt.AddRange(quick_sort(items_lower))
    return_my_lizt.Add(pivot)
    return_my_lizt.AddRange(quick_sort(items_greater))
    '--------------------

    Return return_my_lizt
End Function
```

SECONDARY ALGORITHMS:

EUCLIDEAN VECTOR NORMALIZATION FOR CONSTANT VELOCITY:

EXPLANATION:

This algorithm is used by moving attacks so they can keep a constant velocity and direction towards either the player mouse or enemy-defined positions.

The parameters are the mousex, mousey, entity and speed. Mousex and y are just the point the projectile is travelling to, the entity is just the object producing the attack (start position), and speed is just the speed of the attack.

```
Dim dx As Double = mousex - entity.x
```

```
Dim dist As Double = Math.Sqrt(dx ^ 2 + dy ^ 2)
```

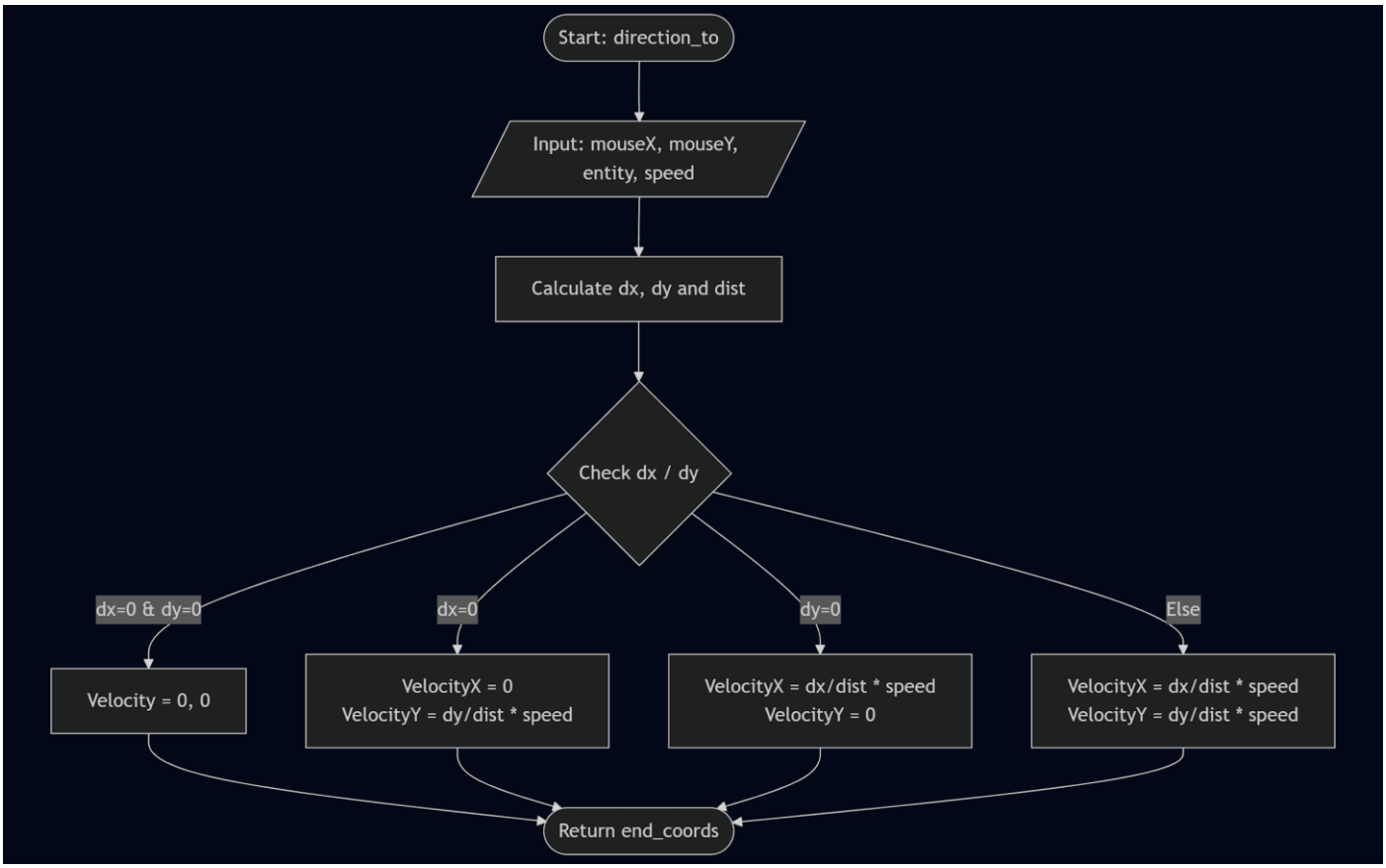
```
(dx / dist) * speed
```

The final pair of velocities (x and y) are then added to an array called end_coords where end_coords(0) is x and end_coords(1) is y.

This is then returned for the attack to use.

```
ElseIf dy = 0 Then, ElseIf dx = 0 Then …….
```

FLOWCHART:



CODE:

```
Function direction_to(ByVal mousex As Integer, ByVal mousey As Integer, ByVal entity As base_block, ByVal speed As Integer)
        ' if the attack is moving towards the player(like firebal) it uses this

        'find the change in x and change in y
```

```
    Dim dx As Double = mousex - entity.x
    Dim dy As Double = mousey - entity.y
    Dim dist As Double = Math.Sqrt(dx ^ 2 + dy ^ 2)

    Dim end_coords(2) As Double

    ' Avoid division by zero
    If dx = 0 And dy = 0 Then
        end_coords(0) = 0
        end_coords(1) = 0
        Return end_coords

    ElseIf dx = 0 Then
        end_coords(0) = 0
        end_coords(1) = (dy / dist) * speed
        Return end_coords

    ElseIf dy = 0 Then
        end_coords(1) = 0
        end_coords(0) = (dx / dist) * speed
        Return end_coords
    Else
        end_coords(0) = (dx / dist) * speed
        end_coords(1) = (dy / dist) * speed
        Return end_coords

    End If

    'normalise and multiply by the speed

    '-----------------------------------------------------------------------
    Return end_coords
End Function
```

THE "PERPENDICULAR VECTOR" POSITIONING ALGORITHM:

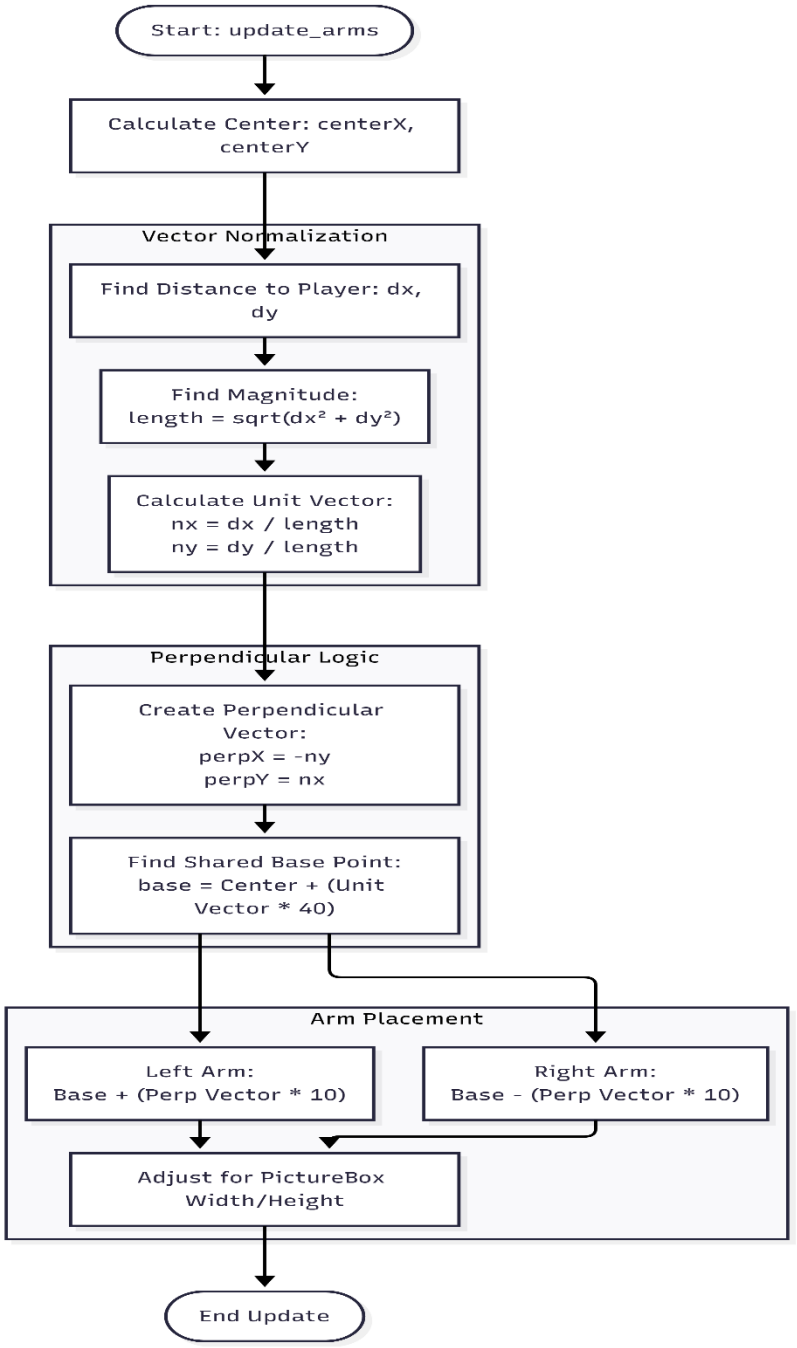
EXPLANATION:

This is used to give the enemies arms that move towards the enemy, hence giving them further fluidity and procedural animation.

First the center of the entity is found. Next, the change in y and x is found between the player and the entity using the algorithm. Thirdly, I find the length which can be used in normalising the vectors now created. This normalisation is done by dividing the dx and dy by the length found.

Then all these vectors are combined with a preset separation and reach value to determine where each arm will be located. Each arm is then assigned to where each vector points, making it seem as if the arms are following the player.

FLOWCHART:



CODE:

```
Protected Overrides Sub update_arms(ByVal plr1 As player)

        Dim centerX As Integer = x + width \ 2
```

```
        Dim centerY As Integer = y + height \ 2

        'the goblin arms follow the player
        Dim dx As Integer = plr1.x - centerX
        Dim dy As Integer = plr1.y - centerY

        Dim length As Double = Math.Sqrt(dx * dx + dy * dy)
        If length = 0 Then length = 1

        ' Normalized direction vector
        Dim nx As Double = dx / length
        Dim ny As Double = dy / length

        ' Perpendicular vector (for arm spacing)
        Dim perpX As Double = -ny
        Dim perpY As Double = nx

        Dim reach As Integer = 40
        Dim separation As Integer = 10

        ' Final positions for each arm
        Dim armBaseX As Integer = centerX + CInt(nx * reach)
        Dim armBaseY As Integer = centerY + CInt(ny * reach)

        ' Left arm
        leftArm.Left = armBaseX + CInt(perpX * separation) - leftArm.Width \ 2
        leftArm.Top = armBaseY + CInt(perpY * separation) - leftArm.Height \ 2

        ' Right arm
        rightArm.Left = armBaseX - CInt(perpX * separation) - rightArm.Width \ 2
        rightArm.Top = armBaseY - CInt(perpY * separation) - rightArm.Height \ 2

    End Sub
```

IMAGE PROCESSING (MANUAL PIXEL MANIPULATION):

EXPLANATION:

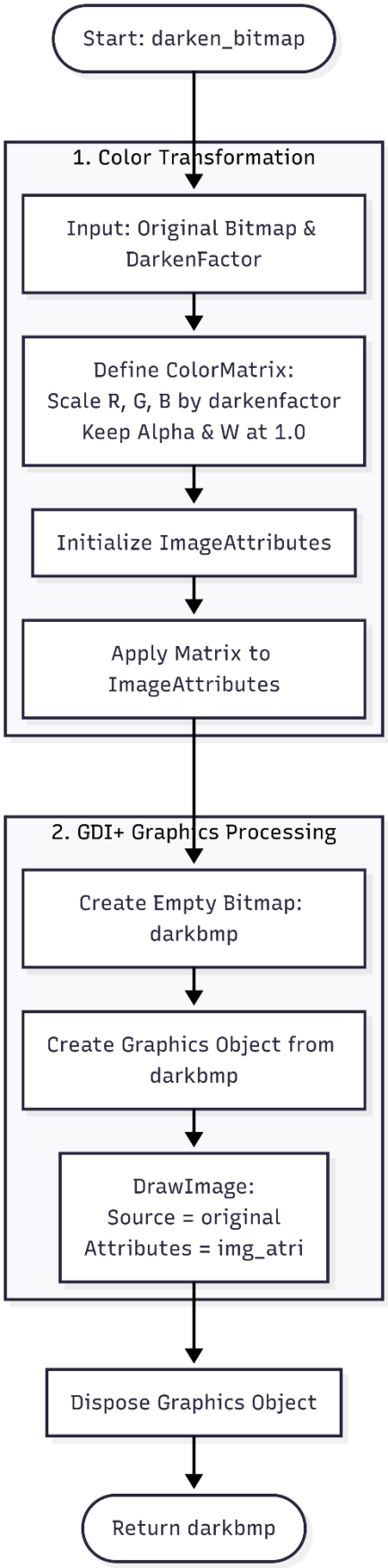
This algorithm takes a bitmapped image and returns a darker version of it, it is used in the inventory to give the user feedback when they click on an item. The parameters are the original image and the darken factor which is in a range from 0->1 and controls the shading of images.

A matrix is used to change the colour of each pixel in the bitmap:

```
    'the matrix to darken an image
    Dim col_mat As New ColorMatrix(New Single()()
        New Single() {darkenfactor, 0, 0, 0, 0},
        New Single() {0, darkenfactor, 0, 0, 0},
        New Single() {0, 0, darkenfactor, 0, 0},
        New Single() {0, 0, 0, 1, 0},
        New Single() {0, 0, 0, 0, 1}
    })
```

The bitmap is then returned to the program and shown.

FLOWCHART:



CODE:

```
Public Function darken_bitmap(ByVal original As Bitmap, ByVal darkenfactor As Single)
    'darken the item picture when clicked on, giving feedback to the user
```

```
        'the matrix to darken an image
        Dim col_mat As New ColorMatrix(New Single()() {
            New Single() {darkenfactor, 0, 0, 0, 0},
            New Single() {0, darkenfactor, 0, 0, 0},
            New Single() {0, 0, darkenfactor, 0, 0},
            New Single() {0, 0, 0, 1, 0},
            New Single() {0, 0, 0, 0, 1}
        })

        Dim img_atri As New ImageAttributes()
        img_atri.SetColorMatrix(col_mat)

        Dim darkbmp As New Bitmap(original.Width, original.Height)

        Using g As Graphics = Graphics.FromImage(darkbmp)

            g.DrawImage(original, New Rectangle(0, 0, darkbmp.Width, darkbmp.Height), 0, 0,
original.Width, original.Height, GraphicsUnit.Pixel, img_atri)

        End Using
        '-----------------------------------------------------------------------
        Return darkbmp
    End Function
```

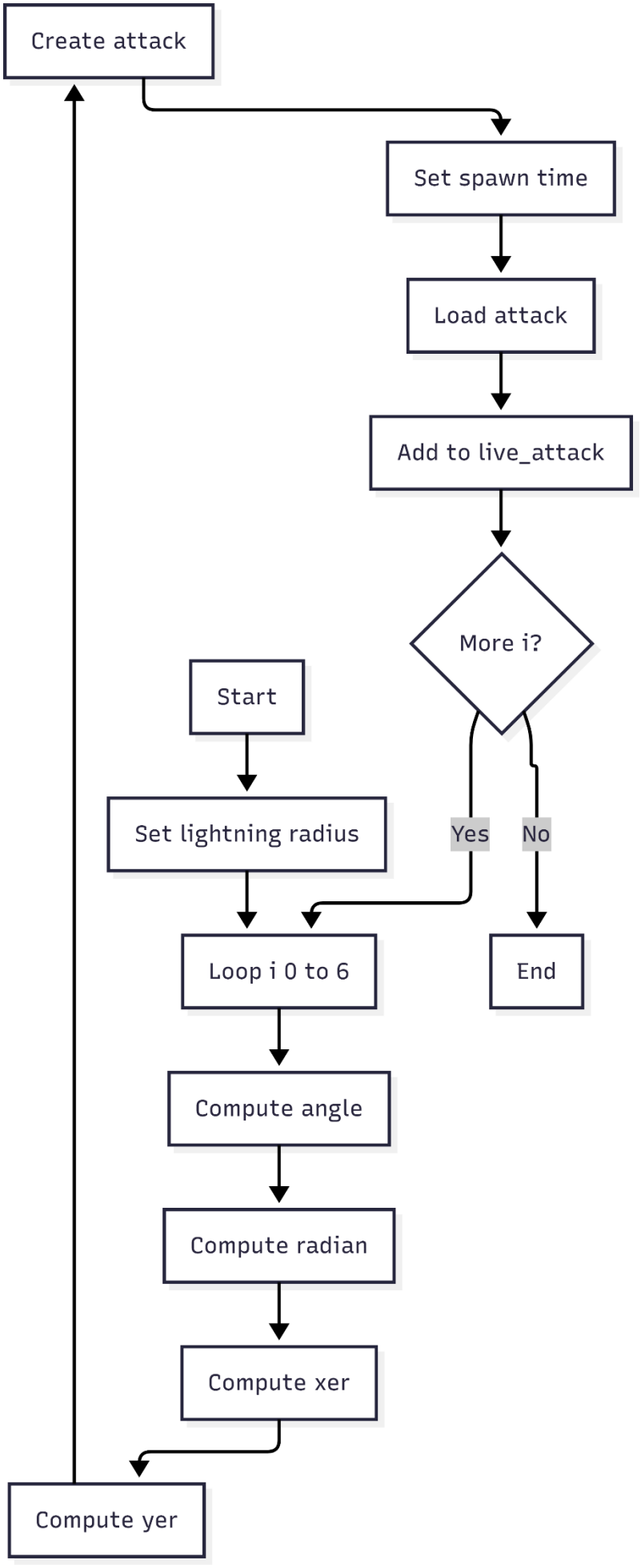
RADIAL SPAWNING:

EXPLANATION:

It is used to spawn the attacks around an entity, this gives them a range and another way they can be utilised in the game.

It does this by creating a circle of points around the enemy then assigning each attack to one of these points. The subroutine is implemented in a few different classes to create different iterations of each; e.g. in the warlock class fireball attacks move towards the points whereas in the player class lightning attacks just spawn on the points.

FLOWCHART:


Create attack
Set spawn time
Load attack
Add to live_attack
More i?
Start
Set lightning radius
Yes
No
Loop i 0 to 6
End
Compute angle
Compute radian
Compute xer
Compute yer

CODE:

```
        'load lightning into a circle of 6 around the player
        Dim lightning_radius As Integer = 125

        For i = 0 To 6

            Dim angle As Integer = i * 60
            Dim radian As Double = angle * Math.PI / 180

            Dim xer As Integer = Me.x + lightning_radius * Math.Cos(radian)
            Dim yer As Integer = Me.y + lightning_radius * Math.Sin(radian)

            Dim atk As New base_attack(Me.game, 50, 50, xer, yer, 5, chosen_attack, New
Bitmap(Me.attack_dict("lightning")(2)), Me.attack_dict("lightning")(1) * (1 + Me.all_stats(2) / 10),
radian)

            atk.spawn_time = Me.tick

            atk.load()
            live_attack.Add(atk)

        Next
```

DATA STRUCTURES:

These are examples of each different type of data structure used(normal and abstract ones) and why they are used.

ARRAYS:

The Text_mapp_used variable in the main form , is assigned the text_map in use currently and is made of a list and strings(which can are treated as arrays that store character)

It is sent to many different classes and methods to either be created or used for map data.

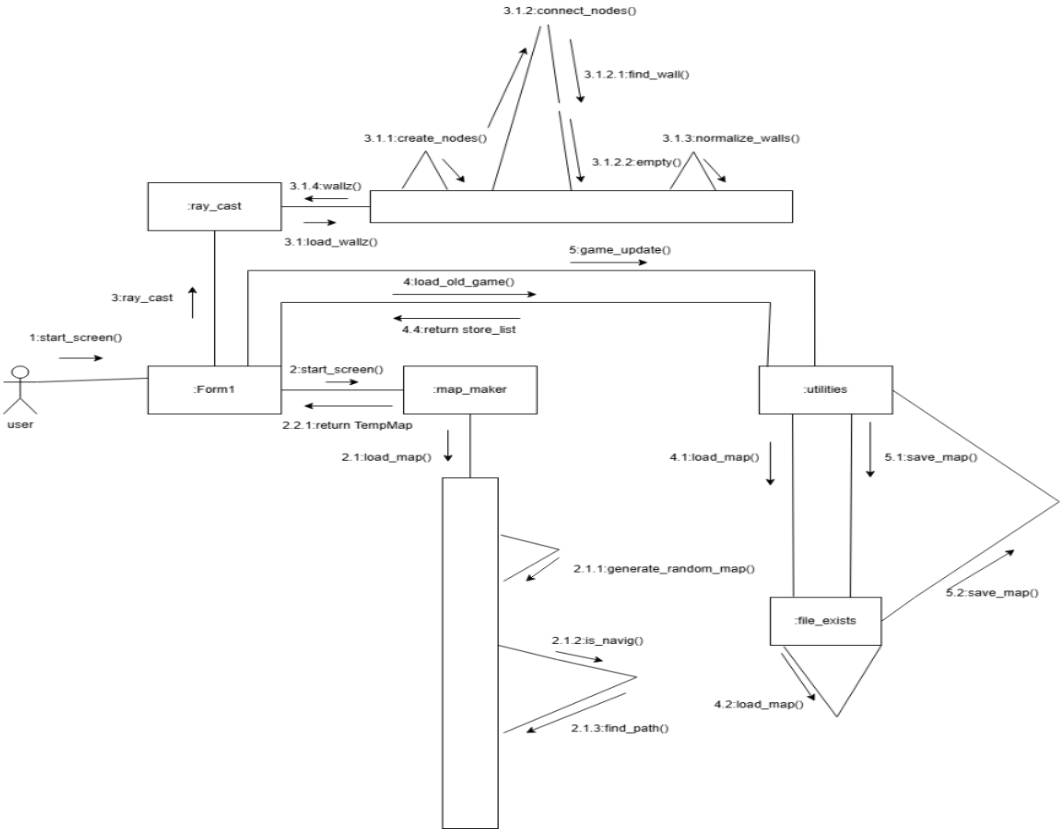


FIGURE 56 THE TEXT_MAPP_USED DATA FLOW

```
Private text_map1 As New List(Of String)
Private text_map2 As New List(Of String
Private text_map3 As New List(Of String)

Private NPC_village As New List(Of String)
Private boss_floor As New List(Of String)

Private text_mapp_used As List(Of String)
```

MAP MAKER:

This class uses a large amount of array this can be seen in:

TempMap, this is a 2d list that conceptually is used as an array as it is iterated through using fixed variable width and height. Therefore it does not change. Hence this 2d array is used as a grid for a 2d map.

```
Dim tempMap As New List(Of List(Of String))
```

- [[“b”, “b”,” b”],

[“b”, “.”, “b”],

[“b”, “b”, “b”]]-

newMap, this is used to temporarily make changes to the text map without changing the main tempMap as a result it is also a 2d list that is used as if it were an array

```
Dim newMap As New List(Of List(Of String))
```

path, this is a list however it uses an array of integer inside it making it a 2d combination of both, the reason there is an array is as it acts as 2 coordinates so cells can be shown as coordinates in the pathfinder

```
Dim path As List(Of Integer())
```

RAY CAST:

This class is used to check ray casting and create walls, as a result nodes are used therefore creating the need for arrays:

Walls, this is also a somewhat 2d array made up of lists with definite ends and arrays that make up the coordinates which are linked to create lines that are used as “blockers”.

```
Private wallz As New List(Of List(Of Array))
```

- [[0, 100], [20, 50], [23, 2]]-

LISTS:

PLAYER INTERFACE:

Mission_npcs, This is a list of NPC objects, it is dynamic so more NPC’s can be added and iterated through easily.

```
Public missions_npcs As New List(Of Npc)
```

All_items, This is another 1d list of item objects that can also be dynamically changed to fit more items, this is necessary as there is no set amount of items the player may pick up.

```
Protected Friend all_items As New List(Of item)
```

```
Example:
```

- [item, item, item, item, item]-

FORM1:

Enemy_list, This is also a 1d list containing the base_enemy objects, however, the base_enemy class is a parent class to goblin, warlock and boss classes hence they can all be added to this list but are processed differently depending on class. This is an exemplary case of polymorphism in action.

```
Private enemy_list As New List(Of base_enemy)
```

All_mission_txt, This is a 2d list that contains the different missions with all their attributes which are represented as strings.

```
Private all_missions_txt As New List(Of List(Of String))
```

```
Example:
```

- [[“kill goblins”, “goblins”, “5”],

[“kill warlocks”, “warlocks”, “5”]

[“kill spider boss”, “spider”, “1”]]-

DICTIONARIES:

UTILITIES:

Atk_dict, this is a local dictionary in the load_attacks function used to store the name of attacks and their attributes. This is a string key list to a list(of string) value.This is created to store data read from the “attacks.txt”

```
Dim atk_dict As New Dictionary(Of String, List(Of String))
```

Ret_dict, This too, is a local dictionary created in the load_player_interface_items function ehich stores the name of the item and then various attributes that are integers. The key is a string and the value is a list(of integer). It too is created to store data from the “inventv_items.txt”.

```
Dim ret_dict As New Dictionary(Of String, List(Of Integer))
```

Example:

- {“hp_potion”: [1, 2, 6, 5],

“shield”: [3, 5, 6, 1],

“mana_potion”: [1, 2, 4, 5]}-

BASE_ENEMY:

Attack_dict, this dictionary is used to store the various possible attacks they can come into contact with so they are able to interact with them. It holds the name of the attack and various other attributes. The key is a string and the value is a list(of string).

```
Public attack_dict As New Dictionary(Of String, List(Of String))
```

Example:

- {“fireball”: [“1”, “2”, “10”],

“lightning”: [“1”, “2”, “3”]}-

PLAYER:

Killed_enemy_amounts, this dictionary is used to check the amount of each type of enemy killed, it is used by the missions to check whether they have been completed. The key is a string and value is an integer.

```
Public killed_enemy_amounts As New Dictionary(Of String, Integer)
```

Example:

- {“goblin”:10,

“warlock”:3,

“spider”:0,

“invisi”:0}-

PRIORITY QUEUE:

MAP MAKER:

Openlist, This is used in the A* pathfinding algorithm to order the fscore of nodes and ranking them so the A* is able to create the shortest path. The actual structure of the "priority qeue" is as a list of arrays where each array holds an array representing each node.

```
Dim openList As New List(Of Integer())
```

Example:

-[[1, 32], [2, 12], [3, 90]]

GRAPHS:

RAY CAST:

Nodes, this is a 3d list with 2 dimensions used as the x and y with the 3 data structure being an array (with the N, NE, NW and so on, components) Therefore acting as a node which links to other neighbouring nodes hence creating a graph.

```
Private nodes As New List(Of List(Of Array))
```

Example:

-[[node on index, neighbour: N, NE, NW, E, S, SE, SW, W]]

TEXT FILES:

These types of files are used to store data that is non-volatile so I do not need to hard-code values into the program itself. These make different attributes, e.g. item stats, attack stats and mission stats, much more contained and easier to change without ruining other code.

They are also used to save information from previous playthroughs, allowing the user to affect what is saved; therefore, they are dynamic and different data can be stored in them.

LOADING TEXT FILES:

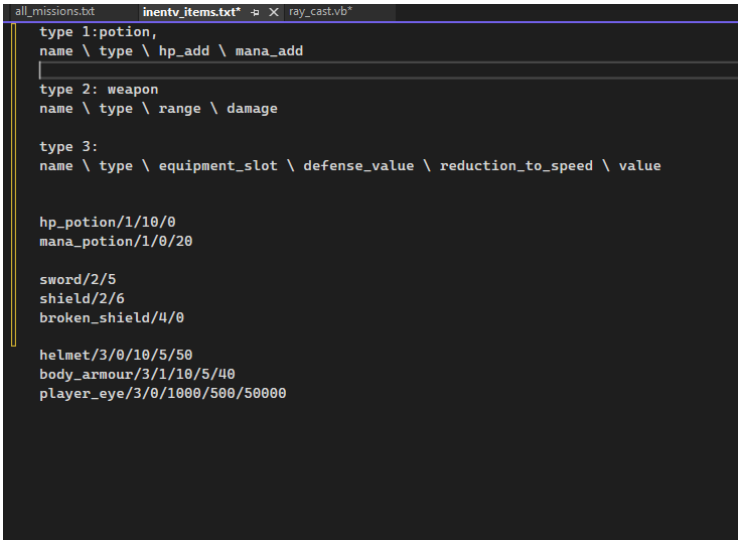


FIGURE 57 INVENTORY_ITEMS.TXT

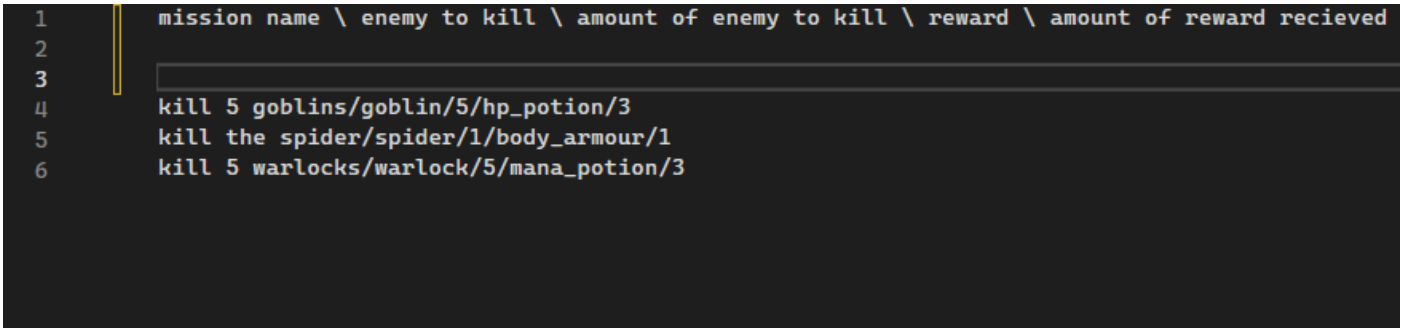


FIGURE 58 ALL_MISSIONS.TXT

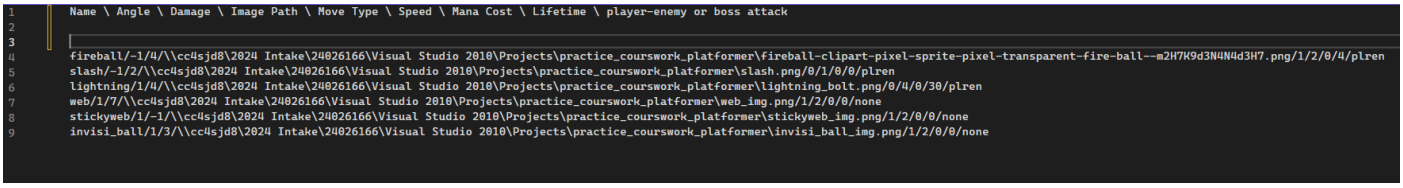


FIGURE 59 ATTACKS.TXT

SAVING TEXT FILES:

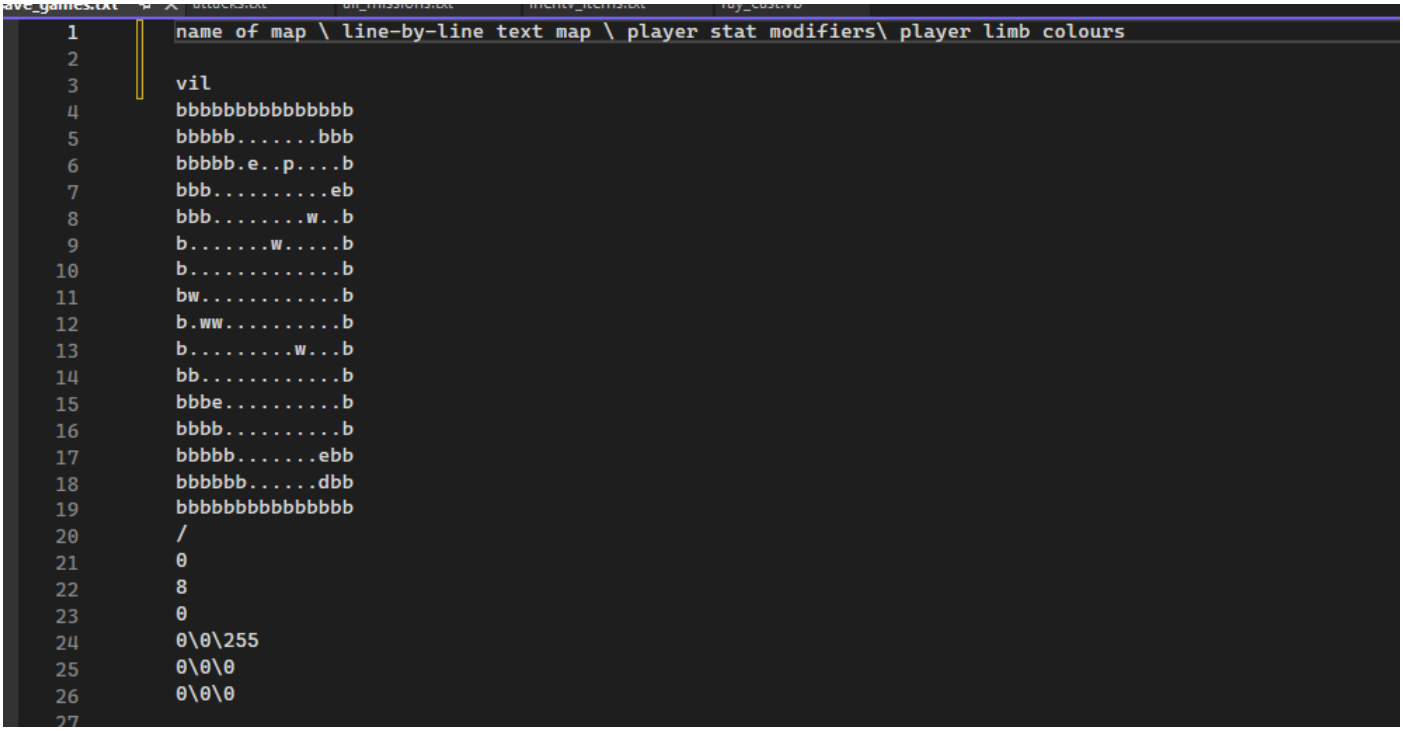


FIGURE 60 SAVE_GAME.TXT

These are each read line-by-line to be returned as dictionaries to the main game.

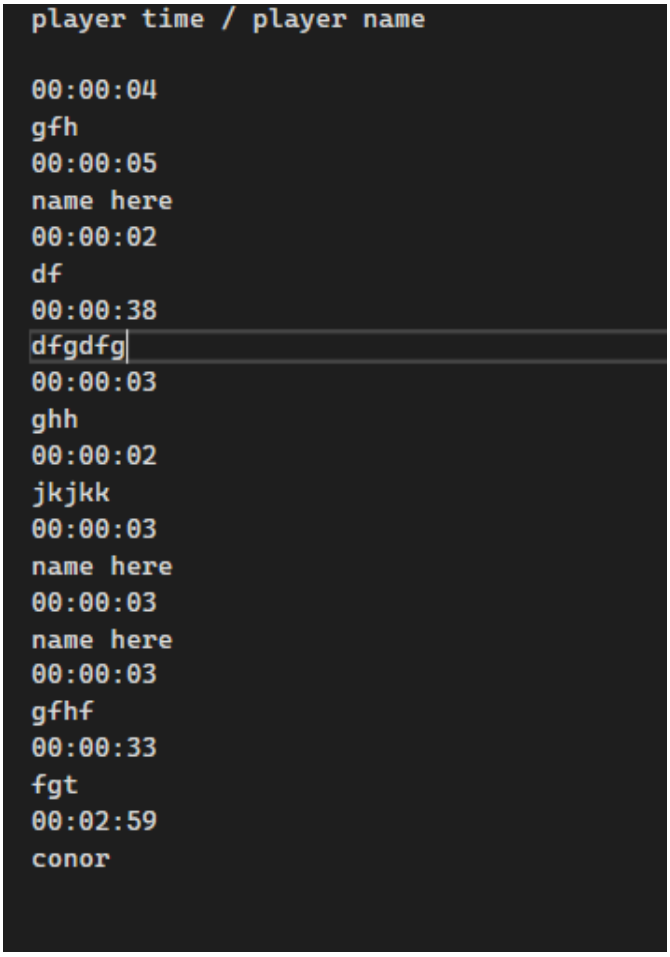


FIGURE 61 SCORES.TXT

TESTING:

OBJECTIVE TESTING:

Intro: 0:00:00

Explaining bugs: 0:48:40

Outro: 1:00:50

Test No.	Objective/s tested.	Test.	Test procedure.	Expected result.	Passed. TRUE/FALSE	Evidence/ timestamp.
1.	Objects:1. A. B. C.	Checking various UI features like the button, progression bar and text.	Check the various buttons and make sure they have an effect on the game. Check if the progression bars can change, and that text can also change dynamically.	The progression bar expands and shrinks, the main enemy text changes its count for each enemy that dies	True	0:00:52
2.	Objects:3. 4.	The various forms and their ability to show images. Show the player object can take various user inputs that have an effect in the game	Show both the first form and final form. Show myself inputting different actions and their events being registered.	The forms can be seen, all inputs create different actions (movement, attacks)	True	0:01:42
3.	Objects:5. A. B. 6.	Show the different uses of the base_enemy class and the different bosses in the game. Show the base_block class and how it is used in nearly all classes.	Show the player character interacting with different enemies and bosses. Show the code for the base_block and how the main body can be seen in many entities.	All different types of enemies appear. The base_block main body can be seen to be in many different entities.	True	0:02:38
4.	Objects:7. A.	Show the base_attack class being used to create a	Show the player using different attacks and the enemies	The player character creates a variety of attacks and all different attacks are created	True	0:04:21

		variety of different attacks.	using those same attacks also show where and how attacks are created.	using the base_attack class.		
5.	Entry screen:1. 2.	Check the exit button can be used and exit the game. Check the start button can be used and loaded in the character creator.	Click the start and exit button and see their effects.	When the exit button is clicked the game is closed. When the start button is pressed the user enters the character creator.	True	0:05:24
6.	Entry screen:3. A. B. 4.	Check when the instruction button is clicked a new page opens. Check if the instruction page shows the various rules of the game. Check when the done button is pressed the user goes back to the main page. There is text present in the main page.	Click on the instruction button, read the rules, click on the done button. Read the text on the main page.	The instruction button leads the instruction page and the done button leads to the main page. See the text on the main page.	True	0:05:46
7.	Customisation menu: 1. A. B. C. 2. 3. 4. 5.A	Check if the indicators are present on the character creator page. The progression bars that link to each then increase and decrease buttons that change the progression bars. Once all attribute points are used no changes can be made.	Load In to the character creator page, check that all the strength, stamina and wisdom text and corresponding bars can be seen. Check the attribute text and the progression bars change with the increase and decrease buttons.	All progression bars change, the attribute text changes and all button actions can be noticed. Once the attribute points are used there can be no changes unless the points are increased.	True	0:06:18
8.	Customisation menu:6. 7. 8. A. B. 9.	The player character is shown in a box. There are r, g, b buttons with increase and decrease. A button for	Check all the various buttons can be clicked, check the colour of each limb can be changed. Check the limb	Different limbs colours are changed, text changes and when the done button is clicked the game starts with the	True	0:07:30

		changing limbs to change their colour. A piece of test that changes depending on limbs and loop through limbs. A done button that starts the game.	being customised can be changed along with the text. Check the done button can be clicked and open the game.	customised character.		
9.	Instruction menu: 1. 2. 3. 4. 5. 6.	The instructions of the game are all shown and give in-depth detail on how to play the game.	Read all the instructions present.	All the information is given.	True	0:08:36
10.	Map generation:1. A. B. C. 2. 3	Show that the map is represented as text in a text map (“b”, “e”, “B”, “N”). The cellular automata algorithm creates the random rooms.	Show the text map in the code and that all each piece of text corresponds to an entity. Show the automata algorithm creates random rooms.	Show the code, use the text save file to show the maps text file and then load it in to show all corresponding entities. Multiple random rooms are generated.	True	0:09:10
11.	Map generation:4. 5. 6. A.	Apply rules to the newly generated rooms so that at least 50 cell empty rooms are created. Use A* pathfinding to make sure all entities have a path to the players (enemies, doors, bosses). Only walls next to empty cells can be collided with.	Show the code for this and show the rooms abiding by these rules and A* pathfinding. Collided with the interior walls.	The code is shown all entities have a path to the player and all interior walls cause collisions.	True	0:11:22
12.	Player movement and attacks: 1. 2. 3. 4.	Movement can be done with the W, A, S, D keys and create a velocity in that direction that can be increased. However, this can be infinitely increased.	Show the player moving with key presses, show the player getting faster and show that they can't get infinitely fast.	The player moves with a velocity, can speed up but has a ceiling to which this speed increases.	True	0:13:30

13.	Player movement and attacks: 5.A. B. 6. 7. 8. A. B.	Attacks can be used with the left click, they can be switched between with the scroll wheel and a cooldown bar for each attack can be seen. The attack attributes are stored in a text file.	Show the attacks being used, show the attacks being looped through, show the cooldown bar changing. Change to the text files and show how they affect the attacks.	The attacks are used and can be seen moving. The cooldown bar can be seen changing and stopping the user from using an attack. Scrolling between attacks is seen and the text files are also seen.	true	0:14:56
14.	Ray casting: 1. A. B. 2. 3. 4.	A map of lines(blockers) is created so when rays from entities are shot they must get through them if they don't have a wall between them. Enemies have a see timer so they keep going in the direction the player was last.	Show the enemy being out of view from the player so does not move towards them. Show the enemy still moving to the player for a time after the player goes out of sight.	The enemies stop when behind walls. Enemies still move for a short amount of time after seeing the player. A view of the code.	True	0:18:22
15.	Collisions: 1.2.3. A. B. 4.	Collisions between all entities. When the door is collided with the player moves to the next room if all enemies in the current room are dead. Bosses are collided with moving their limbs accordingly.	Have the player collide with the walls, enemies and bosses. Show their reaction and show what happens when the player collides with the door. Finally show the enemies when they collide (are pushed) into a wall.	The enemies should bounce into the wall and vibrate. The player should bounce and get pushed back. The bosses should move around and their limbs should too.	True	0:22:32
16.	Inventory/Game UI: 1. A. 3. A. B. 4.	When the I key is pressed the inventory is opened. Items are used when clicked on and certain items are stackable and are not.	Have the user press I to open and close the inventory, have the user use consumables and equipment. Show count for the items changing accordingly.	Items are used and have effects on the player. The inventory opens and closes and the items have numbers which show their size.	True	0:24:40

17.	Inventory/Game UI: 2. 5. 6.	Progression bars that show the players health and mana. & separate boxes that indicate what has been equipped. All the attributes for all items are saved on a text file.	Have the player take damage and use mana to change the progression bars. Show items being equipped into the boxes and show the text file being used.	The progression bars should change size showing different colours. The equipment boxes should fill and the text file should be seen.	True	0:27:03
18.	Shield: 1. 2. A. B. 3. 4. 5. A. 6.	The shield item is dropped when the spider boss is killed and can be picked up. The shield when used has a progression bar showing its health. If the health runs out the shield disappears. The shield causes pushback on enemies and absorbs magic attacks (fireball, spider webb). The shield follows the user cursor and can be turned on and off. Change whether the inventory can be opened if the shield is being used.	Have the player kill the spider boss, equip the shield, use it by moving their cursor and colliding with enemies to show its health can be seen to change. Enemies should get pushed back and magic attacks from enemies will be absorbed. When the shield is being used the inventory should be attempted to be opened.	The shield will be picked up and should move with the cursor. When colliding with enemies and attacks they should either be pushed back or absorbed. The inventory can be opened when the shield is in use.	True	0:31:39
19.	Enemy: 1. 2. 4. 5. A. 7. 8.	In the map maker they are represented by their own character. There will be a goblin and warlock enemy. A base enemy class can be seen also with a ray caster object connected to the enemy as seen in the code. If the ray caster shows there is	Show the code of the enemy class inheriting from the base enemy and the functionality that it brings. Show the enemy having different variations of itself and show the ray caster with enemies moving in straight lines	The code with the inheritance will be seen. Different types of enemy will be seen and they all can move in a straight line towards the player when no walls are present.	True	0:34:56

		no obstacles it will allow the enemy to move in a straight line to the player	towards the player.			
20.	Enemy:3. A. B. 6. 9.	Attacks can be used by enemies with different enemies using different attacks. These can be ranged when in contact with the player damage is caused. Limbs can be seen that move with the enemies that also are different for different enemies. When enemies die an item is dropped based on rarity of the enemy (this mainly applies to bosses)	Show the warlock shooting fireballs in a circular pattern. Show the player taking damage from the goblin and warlock enemy. Show limbs for each enemy following the enemy main body around. The warlock has red arms surrounding it unlike the goblin which has just black boxes. Kill multiple enemies to show different items are always dropped.	All enemies will attack the player with the warlock firing fireballs. All enemies will have limbs that follow the main box with different types of limbs for the warlock. When enemies are killed items are dropped which can be picked up.	True	0:37:16
21.	Bosses: 1. A. 2. A. B. C. 3. 4. 10.	There will be a special character in the text map for bosses. A boss hp progression bar is created on entering a room and deleted as the room is exited. There are 2 bosses, spider and invisible. When the player leaves the room all boss parts are deleted. When killed an object is dropped.	Show in code the boss character. enter the spider room to see the progression bar for boss hp. Show all the bosses by going to the invisible boss room too. Kill the spider boss to drop an item. Leave the boss room to show all of the boss artefacts being deleted including the bar.	The boss character can be seen in the code. The boss hp progression bar will change depending on boss health and be deleted when leaving. Both bosses can be seen. When the room is left all boss artefacts are destroyed from the screen. When killed, show all the boss objects dropping their items.	True	0:39:54

22.	Bosses: 5. 6. 7. 8. 9. A. B.	The spider has a multitude of limbs which move procedurally. The invisible boss is a circle of square picture boxes. The invisible boss has a swarm of cubes that deflect and do damage to the player. The invisible boss shoots rays of orbs that teleport the player. The invisible boss becomes invisible every few ticks so cannot attack and lights up grey if attacked.	Show the limbs of the spider when the player sees it and show it moving towards the player. Show the invisible boss being circular and its orbiting cubes. Also show the invisible boss shooting a ray of orbs at the player teleporting the player closer to it. Finally show the invisible boss going inviable and being attacked.	The limbs can be seen to be cubes that move procedurally no matter what happens to the main body. The invisible boss is seen to be a sort of circle and is surrounded by black and red cubes which do damage and attacks. Show this boss also going invisible and lighting up to attacks.	True	0:46:40
23.	Npc's: 1. 2. A. B. 3. 6.	There is a class for Npc's and a special character on the test map for them. The Npc's can create speech bubbles with text in them when the player is close enough. The missions are randomly distributed to Npc's. There is a room with only Npc's in it that all can be accessed by the player.	Show the code for the Npc class and show its text map character in code. Show a speech bubble being created when the player is close enough. Enter the room with only Npc's that can all be reached by the player.	The Npc character can be seen in the text file and its class can be seen. A text bubble can be seen when the Npc is approached close enough. There is a room with only Npc's that can all be reached.	True	0:50:05
24.	Npc's: 4. A. 5. 7. 8.	The user clicks on the Npc speech bubble to accept it, once done the Npc disappears. If the mission is accepted it goes into their UI coloured yellow, if complete it turns green and	Click on multiple Npc's to collect the mission and see it appear under the inventory in blue. Complete a mission by killing enemies in the next	The Npc characters are deleted once clicked on, the mission appears in yellow (or green if already completed) when an uncompleted mission is finished it turns yellow. A text file with various Npc	True	0:53:37

		can be clicked on for a reward. A text file is used that stores the various attributes of the mission.	room so it turns green and can be accepted. Show the txt file with the information about missions.	information can be seen.		
25.	End screen: 1. 2. A 3. 4.	When the invisible boss is killed an eye is dropped when picked up and equipped the game ends. A new form is created where the user can input their text and then press submit.	Finish the game and see the invisible boss dropping and eye pick it up and enter the new form then input the user's name.	The invisible boss will die and eyes will be dropped and the player will pick it up, equip it, then go into a new form and input their name and press submit.	True	0:55:14
26.	End screen: 5. 6. 7. 8. A. B. C.	When the submit button is pressed a new window will be opened with all previous times and names ordered. The player's time will be in a red box at the top. A restart and exit button can be pressed to exit. To order these names a quicksort algorithm is used on the name and scores which are stored in a text file.	Press the submit button then see the names ordered and the user name in the top right. Press exit to exit the game. Show the text file for names and scores and show the code for the quicksort, also show it has worked by ordering names on the screen.	Pressing the submit button will load a new page with name and time in a leaderboards with the user name in the top right corner. The name and scores will be ordered and when the exit button is pressed the user will exit the window. The user will show the text file with names and timers in a list. The code for the quicksort will also be seen and the leaderboards will be ordered.	true	0:56:29

ROBUSTNESS TESTING:

<table>
<tr><th>Test No.</th><th>Test explanation:</th><th>Pass/Fail:</th><th>Timestamp:</th></tr>
<tr><td>1.</td><td>On the main page check if the button can be pressed too many times, crashing the game.</td><td>Pass</td><td>00:35</td></tr>
<tr><td>2.</td><td>The character creator checks if attribute points or R, G, B values can be increased or decreased too far, crashing the game.</td><td>Pass</td><td>01:12</td></tr>
<tr><td>3.</td><td>Check if random button presses on the keyboard during gameplay cause unwanted effects.</td><td>Pass</td><td>02:17</td></tr>
<tr><td>4.</td><td>Checking how well collisions are handled for the player character and other entities.</td><td>Pass</td><td>02:59</td></tr>
<tr><td>5.</td><td>Check if the attacks can be used in quick succession without cooldown and if attacks can be used without mana.</td><td>Pass</td><td>03:45</td></tr>
<tr><td>6.</td><td>Check what happens if the player tries using health or mana potions at max and what happens at their minimums.</td><td>Pass</td><td>04:50</td></tr>
<tr><td>7.</td><td>Check if armour can be equipped multiple times and if it disappears once used.</td><td>Fail</td><td>05:40</td></tr>
<tr><td>8.</td><td>Check if once the inventory is full new items cannot be added</td><td>Fail</td><td>06:09</td></tr>
<tr><td>9.</td><td>Check if there are no saved games the player will enter a new game and if there are no scores a new one will be created.</td><td>Pass</td><td>07:08</td></tr>
<tr><td>10.</td><td>Check if inputting a name with special character or no characters will be accepted and check for length of name inputted. So, entering normal, boundary and erroneous data.</td><td><table>
<tr><th>input:</th><th>data:</th><th>Pass/Fail:</th></tr>
<tr><td>Normal data:</td><td>“conor”</td><td>Pass</td></tr>
<tr><td>Erroneous data:</td><td>“123/’;.,;’ @##sdadf aASDWE4 £”</td><td>Pass</td></tr>
<tr><td>Edge case data:</td><td>“”, “----------

----------”</td><td>Pass</td></tr>
</table></td><td>09:15</td></tr>
<tr><td>11.</td><td>Check if the player character can escape the main game zone through collisions.</td><td>Pass</td><td>11:08</td></tr>
<tr><td>12.</td><td>Check if the saving button can be pressed repeatedly without errors.</td><td>Pass</td><td>11:46</td></tr>
</table>

EVALUATION:

To determine whether this project has achieved or previously set goal I shall evaluate different systems and give the positives, negatives and any possible improvements that could have been made during the programming process.

SELF-REFLECTIONS:

OBJECTIVES:

OBJECTS:

I believe I have completed all of my objectives. In objects, I had a great amount of inheritance between the base_class, base_enemy, and polymorphism through base_attack. I also used a great deal of encapsulation throughout the code. This is most prevalent in the utility class, which handles all saving and reading to text files, as well as many other widely used methods like random number generation, tracking the mouse location, and many others. This reduced the need for repeated code and made debugging much easier, as I only had to scrutinize one self-contained class instead of searching through multiple classes for bugs.

ENTRY SCREEN, CUSTOMISATION MENU, INSTRUCTION MENU:

In the entry screen, I created buttons that all work and can be used by the player on entering the game without drowning them in information. Partnered with the customisation menu and instruction menu, I have fully utilised UI elements (buttons, text, and progression bars), meaning users do not have to read large amounts of text or input vast amounts of data to create a customised game. The instruction menu creates a single condensed page for the user to easily understand the entire game. All of these contain exit and done buttons to make navigation smooth and easy.

MAP GENERATION:

The map generation is one of the key parts of the game, as it allows for random maps for each room loaded into. This is something that I believe is one of, if not the main system that creates the dynamic feel of the game. I fully completed this objective by creating a cellular automaton to generate rooms and A* pathfinding to make sure that each entity in the rooms can reach the player.

PLAYER MOVEMENT AND ATTACKS, INVENTORY/GAME UI, SHIELD:

The player movement makes the game feel fast-paced, with attacks utilising the entire mouse. However, due to VB Forms having a single key press at a time, velocity was needed, making movement unusual compared to other games. Therefore, this is one of the negatives that I would have liked to improve. The game UI and inventory are very dynamic, as items can be added and removed easily, and all UI pieces—such as progression bars, equipment images, and attack images—have the functionality to change during the game, meaning they do not feel like static placeholders. This is something I think I accomplished, as there are nearly no UI objects that do not change during runtime. However, the inventory does have a few bugs, including not having a max capacity and causing certain equipment to disappear when used twice. This is due to a lack of game testing on the inventory once created and is therefore an improvement I would like to make.

The shield is also a piece of equipment that I believe is perfectly dynamic, allowing the user to use their own skills to block attacks and move it in certain directions. Unfortunately, creating multiple different images to portray it being broken was not implemented, nor was the ability to pick up a new one once it was destroyed. Therefore, these are definitely changes I would have liked to make, but overall, I think the shield is well done and increases the interactiveness of the game.

RAY CASTING, COLLISIONS:

My ray casting is another key system as it gives the enemies the feeling of artificial intelligence. I believe that the system was implemented quite well, with enemies stopping and moving in accordance with where the player is. It is also very optimised, with all enemies using ray casting with the same "blocker" map for walls and using extremely efficient maths to make all checks incredibly fast. Although, due to the use of other entities being boxes and the ray casting using a direct line from the centres of each, sometimes ray casting does not seem to work. Therefore, it can be said that it is not fully robust.

The collisions work for nearly all situations; they are also greatly optimised, with only walls next to empty grid cells being checked, reducing the collisions needed to check. That said, collisions can break down if the player is going at a fast speed, meaning I could either hard-cap speed or increase the efficiency of collisions.

ENEMY, BOSS, NPC:

Enemies use a great amount of inheritance and can be very unique with a wide range of attacks. This is the same with bosses but on a greater level. Therefore, I think I implemented this nearly perfectly, with enemies creating the competitiveness of the game and forcing the user to change their play style for each different type, meaning the game does not get as boring as fast. Yet there are downsides: for bosses, I did not create each class separately for different types like the enemies; therefore, polymorphism had to be used. The reason was to save programming time; however, it did make the boss class much larger and harder to debug. Meaning if I had more time, I would have encapsulated the bosses into their own classes to increase the difference between them and reduce the code needed.

The NPC class is also greatly done, with the ability to create them anywhere and their ability to offer missions to the player. As a result, it is my belief that I accomplished this objective greatly, as there are no apparent bugs and the rewards give a side mission for the user, urging them to get into more battles and use more systems in the game. Hence, I think I would not change them at all, as they work exactly as presented in the objectives.

REASONS FOR THE REDUCED USE OF ENCAPSULATION:

As can already be seen, this project is very large; hence not all characteristics that could be encapsulated—e.g. different attacks (fireball, lightning, slash…), missions, radial burst attack pattern, bosses (spider, invisible)—were encapsulated. The reason for this is that a data-driven approach was taken. Instead of creating a class for each type, a text file is read and then a base class is used (attack) which changes its behaviour based on this. For missions without a base class, a more subtle encapsulation took place using a dictionary; therefore, I focused on efficiency, where a dictionary lookup time is constant time complexity, O(1), whereas a list of mission objects takes a time complexity of O(n). The reason the boss class was not separated is due to trying to avoid over-engineering the project, where I would only create one entity of each class in the game, so creating an entire class would be a waste of resources efficiency-wise and time to program them.

USER-INTERVEIW:

HOW DID YOU FIND THE OVERALL GAMEPLAY EXPERIENCE, INCLUDING MOVEMENT, COMBAT, AND THE PACE OF THE GAME?

I found the overall gameplay experience very enjoyable as it is centered around fast-paced gameplay, and most interactable objects are buttons and clicking, so I was never bored. I really enjoyed movement and how the user had to make sure the player character was not hit by the many attacks sent out by enemies. This meant my reflexes were put on show, giving it a more skill-based feel. This feeling was the same with attacks, as they required aim and timing. The variation of attacks meant I never got bored with the game; as well as this, it also gave the feel of problem-solving as to when to use certain attacks against certain enemies and bosses. However, due to the quick pace, I would have liked a quicker way to change direction or stop instead of having to quickly press the button, as this was tiring for my fingers and made the game much harder on outdated software that registered clicks slower.

As you can see, the user enjoyed the game very much and fully embraced how quick-paced it was. This had the wanted effect of making it more skill-based and reliant on user reaction time. However, unexpectedly, it made movement rely on multiple button presses, which was less reliable for older hardware. This means that speed needs to be changed according to hardware and user reaction time.

HOW SMOOTHLY DID THE GAME RUN ON YOUR DEVICE, AND DID YOU NOTICE ANY LAG OR PERFORMANCE ISSUES WHILE PLAYING?

No, the lag was unnoticeable and all gameplay was smooth even with the older hardware of my device. This was clearly due to the intervals between warlock attacks and probably tricks used whilst programming. Although, there was noticeable visual lag causing jitter of entities, especially during fast movement or impactful collisions.

Reading this response, I can see my optimisations to various algorithms and gameplay meant that the logic ran very well. This means that my use of text maps for generating maps, collision checks with only outer walls, ray-casting using line-line intersection maths, and intervals between warlock attacks clearly greatly contributed to optimisation. However, graphical lag creating jitters was not dealt with as well. The reason for this is due to the use of VB Forms PictureBoxes, which have large overhead in the Windows operating system, therefore creating jitters when they are moved. I could have used Graphics Paint, which is possible with Forms; this creates much more efficient drawing but results in much less satisfying UI objects (buttons don't have the nice click visuals).

WHAT ASPECTS OF THE GAME DID YOU ENJOY THE MOST, AND WHAT WOULD YOU IMPROVE IF YOU PLAYED AGAIN?

My favourite aspect was how dynamic the gameplay felt, with a large variety of randomly spawned enemies and bosses with very different attacks. As well as this, every time I played, a new map was generated, and each one worked perfectly, creating no soft locks (trapping the player but not crashing the game). Furthermore, all attacks seemed different, as the enemies did not have set movements or attacks, so the gameplay flowed much better. My final praise is the spider boss having legs that moved dynamically with its main body even with collisions, which truthfully scared me the first time I saw it. My improvements would be a greater range of items and more missions; as well as this, more boss types and attacks would have been great so I could play the game longer.

This final response shows my dedication to randomness clearly paid off, meaning the user was not bored even after multiple playthroughs. This is greatly attributed to my map generation algorithm creating random maps, random entity placements, and making the journey through levels different every time. The recommendation for more items and more is something I largely agree with; however, due to this being coursework and having time restraints, it meant that adding more variation of all the

things stated would have increased the time, either causing me to run out of time or forcing me to sacrifice other systems, which is something I could not do.

IMPROVEMENTS:

NPC'S:

1) I would have liked to create more missions that were not based solely on killing enemies - for example retrieving items, escorting NPCs, or protecting them during events. Create additional mission types in the NPC class, including item-retrieval missions, escort missions, and timed protection missions. Store mission data in a text file and load it dynamically.

2) I would have liked to create a shopping system run by an NPC, which would also require a money system - enemies could drop currency. Add a Merchant_NPC class with a shop UI. Introduce a currency variable in the player class and allow enemies to drop money on death.

OPTIMISATION:

3) I would have liked to use graphics drawing instead of PictureBoxes to reduce lag. Replace PictureBoxes with VB graphics drawing functions to reduce memory usage and improve performance.

4) I would have liked to optimise the ray-caster so it only checks walls next to empty grid cells, as ray-casting against unreachable walls is unnecessary. Instead of using the full test map, create a list of "active walls" (walls next to walkable tiles) and only ray-cast against those.

SAVING GAMES:

5) Saving should include the player's current attributes, not just their base stats — this includes HP, mana, inventory items, and active missions. Extend the save file format to store current HP, mana, inventory contents, mission progress, and equipped items.

6) Saving should include all future maps that may appear, not just the current map. When saving, write the current map at the top of the text file and append all other generated maps afterwards.

7) The game should allow multiple save files and let the user choose between them. Use a single text file but separate each save with a line containing "-" and a save-slot number so the user can select between them.

MAP GENERATION:

8) I should have added random variation in map sizes to increase replayability. Randomly choose map width and height within a defined range before generating the map.

9) A new feature could have been maze generation to give the user puzzle-based rooms. Use a maze-generation algorithm such as Prim's or Kruskal's to create puzzle-style rooms and include them in the map rotation.

ATTACKS:

10) Having different types of attacks, such as traps or bomb-style throwable attacks. Create a BombAttack class with an arc trajectory and an explosion radius, and add trap entities that activate on timers or proximity.

11) Having attacks that cause damage over a duration of time. Add damage-over-time effects (poison, burn, bleed) by storing a timer and periodic damage in the enemy or player class.

12) Having attacks that act like a laser beam. Use ray-casting to detect instant hits and draw a temporary beam along the ray path.

PLAYER CONTROLS AND MOVEMENT:

13) Adding a dodge-roll or dash mechanic to increase movement skill and reaction time. Bind Spacebar to a short burst of movement toward the mouse, with a cooldown and brief invincibility frames.

14) Adding active abilities mapped to keys (Q, E, R, F) to increase button usage. Create an ability system where each key triggers a unique effect such as a speed boost, stun wave, teleport strike, or temporary shield buff.

15) Adding a stamina system to limit dodging, sprinting, and heavy attacks. Add a stamina variable that decreases when using special movement or abilities and regenerates over time, displayed as a UI bar.

NEW ENTITY TYPES:

16) Adding trap entities such as arrow turrets, spike floors, and flame jets. Create a Trap class with timers or proximity triggers, using ray-casting for turrets and collision checks for floor traps.

17) Adding summoner-type enemies that spawn weaker minions. Extend the Base_enemy class to include a spawn timer and minion list, and use A* to keep distance from the player.

18) Adding environmental interactables such as shrines, levers, and totems. Create an Interactable class activated with the F key, triggering effects like opening doors, granting buffs, or disabling traps.

19) Adding elite enemies - stronger variants of normal enemies. Apply stat multipliers and recoloured sprites on spawn, and give elites unique abilities or behaviours.

20) Adding companion entities such as pets, drones, or summoned spirits. Create a Companion class that follows the player using A* and assists in combat with simple attacks or buffs.

UI AND INVENTORY:

21) Adding hotkeys for consumables (Z, X, C, V). Bind each key to a consumable slot and reduce inventory count when used.

22) Adding weapon swapping using number keys (1, 2, 3). Store multiple weapon types in the player class and switch the active weapon on key press.

ENVIRONMENT AND ROOM VARIETY:

23) Adding environmental hazards such as wind rooms, dark rooms, or slippery ice floors. Modify movement physics depending on room type and add special tiles to the map generator.

24) Adding moving platforms or shifting walls to increase movement complexity. Create entities that move along predefined paths and update collision boundaries dynamically.